

05

Cross-engine Consistency.

The symptom appears in one engine and the cause lives in another. In the SQL endpoint, the Warehouse or a report sees something different.

ENGINES THIS PART TOUCHES

WRITE	PIPELINES Not in this part	DATAFLOW Not in this part	SPARK 8 of 9
STORE	ONELAKE 4 of 9		
READ	SQL ENDPOINT 7 of 9	WAREHOUSE 4 of 9	
GOVERN	SECURITY 1 of 9		

NO. PATTERN

- 5.01 Spark wrote the rows. The SQL endpoint does not show them yet.
- 5.02 The Warehouse shows today's load. Spark shows the previous day's load.
- 5.03 The same T-SQL query returns different results.
- 5.04 A Spark change to a shared table succeeds.

Sanjay Chandra · Global CoE Community

104

II · INGESTION AND ORCHESTRATION

2.10 Limit

2.10 Lookup returned 5,000 rows. The source has 12,000.

PIPELINES DATAFLOW SPARK ONELAKE WAREHOUSE SQLEP DIRECT LAKE

Lookup returns at most 5,000 rows and **drops the rest**. The ForEach after it processes a partial list and the run succeeds.

5,000

Maximum rows a Lookup activity returns. Larger results return 5,000 rows with no error.

Microsoft Learn, Lookup activity, Sep 2026

HOW YOU NOTICE

The ForEach runs exactly 5,000 iterations.

```
@Length(activity('Lookup tables').output.value)
// 5000 means truncated
```

NEIGHBOURING LIMITS

50

ForEach batch count maximum. Default 20.

100,000

ForEach items maximum.

120

Activity

Sanjay Chandra · Global CoE Community

V · CROSS-ENGINE CONSISTENCY

5.01 Between engines

5.01 Spark wrote the rows. The SQL endpoint does not show them yet.

PIPELINES DATAFLOW SPARK ONELAKE WAREHOUSE SQLEP DIRECT LAKE

The SQL analytics endpoint picks up Delta commits through a background metadata sync, so queries before the sync see the old table.

Notebook

Write commits a new version

New version in delta_log

T-SQL check reads the old table

Background sync applies it

Refresh API first: counts match

Next step

SQL endpoint

Pipeline

TIME

CAUSE LIVES IN

The sync gap after the Spark commit

SEEN IN
Endpoint queries right after a load

FIX

Call the Refresh SQL analytics endpoint metadata API before any step that reads through the endpoint.

```
r = fabric.FabricRestClient().post(
  f"/v1/workspaces/{ws}/sqlEndpoints/{ep}/refreshMetadata")
```

Sanjay Chandra · Global CoE Community

105

Fabric Practitioner's Handbook 4.0

What each Microsoft Fabric feature is for, what breaks, and how to design systems that don't.

FREE

www.ssanjaychandra.com
github.com/ssanjaychandra123
linkedin.com/in/ssanjaychandra

Sanjay Chandra

GLOBAL COE COMMUNITY

19

COMPARISONS

123

PATTERNS

30

SCENARIOS

7

PATTERN TYPES

Pieces, failures and designs.

How to use this book 03

From five books to one 263

ENGINE

PIPE

DF

SPARK

ONELAKE

WH

SQL EP

DL

SEC

CAP

PATTERNS

PAGE

SECTION 1 Know the pieces

CCommonly Confused

19 groups of features people mix up, one question each

04

SECTION 2 What breaks

WHERE THE DATA LIVES

1Storage, OneLake and Data Placement

1

6

6

3

6

1

2

8

33

HOW IT ARRIVES

2Ingestion and Orchestration

25

6

3

14

2

2

4

30

42

HOW EACH ENGINE TREATS IT

3Lakehouse, Spark and Delta

3

1

12

8

2

3

12

74

4Warehouse and SQL Endpoint

1

2

17

5

1

2

17

87

BETWEEN ENGINES

5Cross-engine Consistency

8

4

4

7

5

1

9

104

HOW IT IS SERVED

6Direct Lake and Power BI

2

1

5

3

9

2

2

9

115

GOVERNING AND RUNNING IT

7Security and Identity

7

2

3

9

2

6

6

15

15

124

8Capacity and Cost

3

8

3

2

4

12

13

141

9CI/CD and Operations

5

2

2

5

1

4

3

10

155

SECTION 3 Design it

SSystem Design Scenarios

30 design briefs in six themes, each linked to the patterns

167

PATTERN TYPE

SILENT FAILURE

SLOWDOWN

WORKS HERE, FAILS THERE

BETWEEN ENGINES

DESIGN DECISION

LIMIT

HABIT

1Storage, OneLake and Data Placement

2

2

1

3

4

1

2Ingestion and Orchestration

12

4

6

1

3

4

3Lakehouse, Spark and Delta

2

5

2

1

1

1

4Warehouse and SQL Endpoint

3

5

2

1

1

5

5Cross-engine Consistency

1

1

7

6Direct Lake and Power BI

1

2

3

1

7Security and Identity

1

7

2

5

8Capacity and Cost

3

1

1

5

3

9CI/CD and Operations

2

3

2

3

Cell number = patterns in the part that involve the engine (top) or follow the pattern type (bottom); darker = a larger share of the part.

BEFORE YOU START

How to use this book.

The book has three sections. Start with the one that matches what you are doing today.

SECTION 1

Know the pieces.

19 comparisons of features people mix up. Start here when you are learning Fabric or choosing between two features.

Page 04

SECTION 2

What breaks.

123 patterns in nine parts, by layer. Start here when something fails, runs slow or shows the wrong numbers.

Page 33

SECTION 3

Design it.

30 design briefs, answered step by step. Start here when you plan a platform, a migration or a load.

Page 167

A COMPARISON GROUP

The **question** that separates the features, then one card per feature: what it is, its **answer** to the question, the usual **mix-up** and when to pick it. Numbered C.01 to C.19.

A PATTERN

Number in the part colour, then the **title**: the symptom as you meet it. **Engines**: the strip highlights every Fabric engine involved. **Short answer** in one or two sentences, with **cost of failure**, status and the date the facts were verified. The body follows one of seven types, and **Related** patterns close it.

TYPE	LAYOUT	PATTERNS
1 Silent failure	Diagnostic path	22
2 Slowdown	Measure first	23
3 Works here, fails there	Diff	25
4 Between engines	Swim lanes	17
5 Design decision	Decision sheet	21
6 Limit	Number led	7
7 Habit that does not carry over	Myth cards	8

A DESIGN SCENARIO

The brief is the title. Then a **short answer**, the questions to ask first, the data path at a glance, the **build steps** in order with the trap at each step, security, checks and follow-ups. Numbered S.01 to S.30.

TAB COLOURS

 C Compare  1 Storage  2 Ingestion  3 Lakehouse  4 Warehouse  5 Cross-engine  6 Power BI
 7 Security  8 Capacity  9 CI/CD  S Scenarios

FACTS AND SOURCES

Every default, limit and behaviour was checked against Microsoft Learn in September 2026. Only generally available behaviour is stated as fact; preview features are named as preview. Fabric changes monthly, so check the date on each pattern.



Commonly Confused.

Nineteen groups of Fabric features that people mix up. Each group starts with the one question that tells them apart and answers it for every feature. Then it names the usual mix-up and says when to pick which.

NO.	THE GROUP, AND THE QUESTION THAT SEPARATES IT	PAGE
C.01	Lakehouse, Warehouse and SQL analytics endpoint Who can write to the tables, and in which language?	05
C.02	Mirroring, shortcuts, materialized lake views and Delta Sharing Does the data move, and who owns the copy?	06
C.03	Capacities, workspaces, domains and endorsement What does each one control: compute, access, grouping or trust?	07
C.04	Copy activity, Copy job, Dataflow Gen2 and mirroring How much do you build and run yourself?	09
C.05	Notebook activity, Spark job definition and runMultiple Who schedules it, and does it share a Spark session?	10
C.06	Eventstream, Eventhouse, KQL database, Real-Time hub and Activator Does it move events, store them, list them, or act on them?	11
C.07	Starter pool, custom pool, high concurrency mode and environments What does each decide: start time, size, sharing, or libraries and settings?	13
C.08	V-Order, Z-order, liquid clustering and OPTIMIZE Does it change how a file is written, where rows sit, or how many files there are?	14
C.09	Materialized lake views, KQL materialized views, Databricks materialized views and SQL Server indexed views What keeps the stored result up to date, and when does that happen?	16
C.10	Delta time travel, Warehouse time travel, Warehouse snapshots, restore points and table clones How far back can you go, and what do you get back?	17
C.11	Semantic model refresh, Direct Lake framing and SQL analytics endpoint metadata sync What has to happen before a new row shows up in the report?	19
C.12	Import, DirectQuery, Direct Lake on SQL and Direct Lake on OneLake Where does the query run, and what happens when a limit is hit?	20
C.13	Workspace identity, service principal, managed identity and the last modified user Whose identity does the job use, and what happens when that person leaves?	21
C.14	On-premises data gateway, virtual network data gateway, managed private endpoints and trusted workspace access Which direction is the connection, and which Fabric items can use it?	23
C.15	Workspace roles, item permissions, OneLake security roles and SQL GRANT with row-level security Which engine enforces it, and who gets around it?	24
C.16	Sensitivity labels, DLP policies, protection policies and OneLake security Does it classify, warn, or block access?	26
C.17	External data sharing, Delta Sharing, cross-tenant shortcuts and B2B guest access Who signs in where, and whose capacity pays for the reads?	27
C.18	Smoothing, bursting, throttling, surge protection, capacity overage and autoscale billing for Spark When demand runs past the SKU, does it spread the cost, stop the work, or add to the bill?	29
C.19	Deployment pipelines, Git integration, fabric-cicd and variable libraries What moves between stages: definitions, data or settings?	31

C.01 Lakehouse, Warehouse and SQL analytics endpoint.

All three hold tables in OneLake, and all three answer T-SQL queries. Every Lakehouse comes with its own SQL analytics endpoint, so people treat the endpoint as a small Warehouse. The real split is which engine may change the data.

THE QUESTION THAT SEPARATES THEM

Who can write to the tables, and in which language?

Lakehouse

A Fabric item that keeps Delta tables in a Tables area and any other files in a Files area. Notebooks, Spark job definitions, pipelines and dataflows load it.

Answer. Spark writes, in Python, Scala, R or Spark SQL, and pipelines and dataflows write too. T-SQL cannot change its tables.

The mix-up. People expect a Spark job that updates several tables to commit them together. A Lakehouse has no multi-table transaction, so a failure halfway leaves some tables changed and others not. **5.04**

Pick it when the people who load the data write Spark, or when raw files must sit beside the tables.

SQL analytics endpoint

A read-only T-SQL layer that Fabric creates for every Lakehouse and every mirrored database. It shows the Delta tables in the Tables area as SQL tables.

Answer. Nobody writes table data through it. You can add views, functions, stored procedures and SQL security, but you cannot insert, update or delete rows.

The mix-up. People query it straight after a Spark write and see the old table. A background metadata sync picks up commits, usually in under a minute, and it stops after 15 minutes without queries. **5.01 · 1.01**

Use it for T-SQL reads over tables that Spark maintains, and call the refresh metadata API after loads that reports depend on.

Warehouse

A Fabric item where the SQL engine owns the tables and stores them in OneLake as Parquet files with published Delta logs. It supports multi-table transactions.

Answer. T-SQL writes, through statements such as INSERT, COPY INTO and CREATE TABLE AS SELECT, and pipelines and dataflows load it too. Other engines read its tables directly but cannot write them.

The mix-up. People move every table here and then find notebooks cannot write back. The Spark connector can append to or overwrite a table by staging data and running COPY INTO. Learn documents no mode for updating rows. **1.02 · 4.12**

Pick it when the team writes T-SQL, or when several tables must change in one transaction.

WHICH ONE, WHEN

- If the people who load the data write Spark, use a Lakehouse.
- If they write T-SQL, or several tables must commit together, use a Warehouse.
- If readers only need T-SQL over Lakehouse tables, use the SQL analytics endpoint and skip the Warehouse.
- If two teams want to write one table, give it one owner and let the other engine read it.

Watch out. A Warehouse transaction can read Lakehouse tables in the same workspace through their SQL analytics endpoints. The endpoint still takes no writes, so a Warehouse procedure cannot update a Lakehouse table.

SCENARIOS **S.03** A 3 billion row fact read by date and region **S.06** Synapse dedicated SQL pool to Fabric
S.20 Silver without half-finished loads

C.02 Mirroring, shortcuts, materialized lake views and Delta Sharing.

All four put data from somewhere else in front of a Fabric or Databricks reader, so the names get swapped in design talks. They differ in whether data is copied, who runs the work, and which way the data travels.

THE QUESTION THAT SEPARATES THEM

Does the data move, and who owns the copy?

Shortcut

A pointer in OneLake to data stored elsewhere, such as another Lakehouse, ADLS Gen2 or Amazon S3. It appears as a folder or table in your Lakehouse.

Answer. Nothing is copied, and the source owner keeps the only copy. Reads go to the source, apart from an optional cache for S3, S3 compatible, Google Cloud Storage and gateway sources.

The mix-up. People treat it like a copy they own. Deleting the shortcut leaves the target alone, but deleting files inside it removes them at the source when you have delete rights there. **1.06 · 1.05**

Pick it when the data already sits in OneLake or open storage and you only read it in place.

Mirroring

A managed replica of a database, such as Azure SQL Database, Oracle or Snowflake, kept in OneLake as Delta tables. You query it through a read-only SQL analytics endpoint.

Answer. The data is copied into OneLake, and Fabric runs and owns the replica. Replication compute is free, and each capacity unit you buy brings 1 TB of free mirroring storage.

The mix-up. People expect it to clean or reshape data. It replicates the source tables as they are, so transformation still needs a notebook, a view or a materialized lake view. **2.03**

Pick it when you need a continuously updated copy of a supported database without building a pipeline.

Materialized lake view

A Spark SQL query whose result Fabric saves as a Delta table in a Lakehouse. It is GA, and authoring one in PySpark is in preview.

Answer. The data moves inside OneLake into a new Delta table that you own. A refresh can process only changes when source tables have change data feed on and receive only appends.

The mix-up. The name suggests a SQL Server indexed view that stays current by itself. It refreshes only when a schedule or a manual run starts it, and each refresh uses capacity. **3.12**

Pick it when you want Silver or Gold tables defined in SQL and a schedule is fresh enough.

Delta Sharing (now OpenSharing)

An open protocol from Databricks for sharing tables with other organisations and platforms. Databricks renamed Delta Sharing to OpenSharing in June 2026.

Answer. Recipients read the provider's data in place and read-only, and the provider owns it. In Fabric, the connector for shares keeps the old name, Delta Sharing, and works only in Dataflow Gen2, which copies the rows into your destination.

The mix-up. People reach for it to bring their own Databricks data into Fabric. Inside one company, mirroring the Unity Catalog or a shortcut to the storage reads the data with no copy.

Pick it when Databricks data must go out to a partner on another platform.

WHICH ONE, WHEN

- If the data sits in OneLake or open storage and should stay there, use a shortcut.
- If the source is an operational database, use mirroring.
- If you are shaping tables that are already in OneLake, use a materialized lake view or a notebook.
- If Databricks data leaves the company, use Delta Sharing, now called OpenSharing.
- If Fabric data goes to another Fabric tenant, use external data sharing, which gives the recipient a shortcut back to your data.

Watch out. Mirroring an Azure Databricks catalog is the odd one out. It copies only the catalog structure and reads the tables where they sit through shortcuts.

SCENARIOS **S.08** Databricks stays, Fabric reads in place **S.05** Three business units, one customer list
S.23 External partners read part of Gold **S.16** Ten minutes behind a busy table

C.03 Capacities, workspaces, domains and endorsement.

All four show up in the admin portal and in workspace settings, and each seems to organise items. People expect a domain to grant access, or a certified badge to make data safe. Each one controls something different.

THE QUESTION THAT SEPARATES THEM

What does each one control: compute, access, grouping or trust?

Capacity

A pool of compute bought as an F SKU and measured in capacity units, or CUs. It lives in one region, and its workspaces store their content there.

Answer. It controls compute. Every workspace assigned to it shares its CUs, and throttling applies to the whole capacity.

The mix-up. People put many teams on one capacity and expect each to keep its own share. One heavy job slows every workspace on it, and background work is smoothed over the next 24 hours. **8.04 · 8.08**

Pick a separate capacity when one workload must not be slowed by the others, or data must stay in another region.

Workspace

A container for items such as Lakehouses, pipelines and reports, assigned to one capacity. It also holds settings such as Git integration.

Answer. It controls access. Its Admin, Member, Contributor and Viewer roles set what people can do with the items in it.

The mix-up. People put dev and prod items in one workspace and rely on naming. A Contributor can write or delete Lakehouses, pipelines and notebooks there, prod items included. **9.06 · 7.09**

Use it when a set of items needs one group of people and one release stage.

Domain

A logical grouping of workspaces by business area, such as Finance or Sales, with optional subdomains. A workspace belongs to one domain at a time.

Answer. It controls grouping and some governance. It grants no access to data, but it filters the OneLake catalog and lets domain admins override delegated tenant settings.

The mix-up. People assign a workspace to the Finance domain and expect only Finance staff to see it. Access still comes only from workspace roles and item permissions.

Use it when you want data found and governed by business area across many workspaces.

Endorsement

A badge on an item: Promoted, Certified or Master data. Endorsed items carry the badge and are listed first in some searches.

Answer. It controls trust signals only. Anyone with write permission can promote an item, and only users the Fabric admin names can certify it or mark it as master data.

The mix-up. People read Certified as a security or quality guarantee. The badge changes no permissions and runs no checks, so it means only what your review process puts behind it.

Use it when readers must tell the approved table or model apart from the copies around it.

WHICH ONE, WHEN

- If one workload slows another, look at capacities.
- If the wrong people can read or change items, look at workspace roles and item permissions.
- If people cannot find data by business area, group workspaces into domains.
- If people cannot tell which copy to trust, certify it or mark it as master data.

Watch out. A Fabric admin can delegate certification settings to a domain, so domain admins choose the certifiers. That ties grouping to trust, yet neither one changes who can open an item.

SCENARIOS **S.05** Three business units, one customer list **S.26** One capacity, three workloads

S.27 Dev, test and prod for two teams

C.04 Copy activity, Copy job, Dataflow Gen2 and mirroring.

All four bring data from a source into OneLake, and several share the same connectors. They differ in how much of the load you design, schedule and repair yourself.

THE QUESTION THAT SEPARATES THEM

How much do you build and run yourself?

Copy activity

A step inside a **Data Factory** pipeline that reads from one source and writes to one destination. You set column mapping, parallelism, fault tolerance and staging.

Answer. You build most of it. Incremental logic, schedules, retries and error handling are pipeline work you design and maintain.

The mix-up. People build a pipeline for a plain table copy and then write watermark lookups by hand. A Copy job tracks incremental state natively, with less to maintain. **2.06 · 2.04**

Pick it when the copy is one step in a larger flow, or a metadata table drives many sources.

Copy job

A standalone Fabric item that copies many tables from a source to a destination, with its own schedules. It needs no pipeline.

Answer. You choose the source, tables, copy mode and write method, and Fabric runs the rest. It tracks watermarks itself, and change data capture replication is in preview.

The mix-up. People treat its incremental state as permanent. A reset to full copy, for the job or one table, reloads every row. With Append, those rows land twice unless you set the job to clear the destination first. **2.02**

Pick it when you need tables copied fully or incrementally on a schedule, with no transformation.

Dataflow Gen2

A Power Query item that reads from hundreds of sources and applies over 300 transformations without code. It writes to destinations such as a Lakehouse or Warehouse.

Answer. You build the transformation steps, and Fabric runs them in its own engine. It has its own schedule, up to 48 refreshes a day, or a pipeline can run it.

The mix-up. People load large raw tables through it because the editor is easy. It bills capacity for every second each query runs, and staging adds a second meter. **2.21 · 2.22**

Pick it when data needs low-code shaping on the way in and volumes are modest.

Mirroring

A Fabric item that replicates a supported database, such as Azure SQL Database, SQL Server or Snowflake, into OneLake as Delta tables. It keeps the copy in sync continuously.

Answer. You build almost nothing. You pick the database and tables, and Fabric runs the replication with free compute and no schedule to manage.

The mix-up. People expect to shape data on the way in. The mirror is read-only and holds the source tables as they are, so transformation happens afterwards. **2.03**

Pick it when a supported database must stay current in OneLake and you want no load logic to maintain.

WHICH ONE, WHEN

- If a supported database must stay current with no load code, use mirroring.
- If you need tables copied on a schedule, fully or incrementally, use a Copy job.
- If the data needs shaping on the way in and volumes are modest, use Dataflow Gen2.
- If the copy is one step among many, or metadata drives it, use a copy activity in a pipeline.

Watch out. Copy job with change data capture and mirroring both keep tables in sync. Mirroring compute is free and its copy is read-only. Copy job change data capture is in preview, bills capacity and writes to a destination you choose.

SCENARIOS **S.10** Oracle ERP, 800 tables, no CDC **S.16** Ten minutes behind a busy table

S.02 Metadata-driven load, twelve sources **S.14** Replacing ADF mapping data flows

C.05 Notebook activity, Spark job definition and runMultiple.

All three run Spark code without anyone pressing Run in a notebook. They differ in what decides when the code starts, and in whether several pieces of code share one Spark session. Session sharing drives both start time and capacity use.

THE QUESTION THAT SEPARATES THEM

Who schedules it, and does it share a Spark session?

Notebook activity

A pipeline activity that runs a Fabric notebook as one step. It passes base parameters into the notebook's parameter cell.

Answer. The pipeline schedules it, through the pipeline's own schedule or trigger. Notebooks share a session only when high concurrency for pipelines is on in the workspace. An optional session tag groups them.

The mix-up. People run many notebook activities in parallel and expect them to be cheap. Without shared sessions each one asks the capacity for its own cores, and the extra runs wait in the Spark job queue. **2.07 · 8.02**

Pick it when a notebook is one step among copies, checks and other activities that a pipeline already orders.

Spark job definition

A Fabric item that submits a main file, such as a .py, .jar or .r file, as a batch or streaming Spark job. Reference files, command-line arguments and a default lakehouse go with it.

Answer. Its own schedule or a pipeline's Spark job definition activity starts it. Learn documents session sharing for notebooks only, so plan for each run to start its own session.

The mix-up. People expect a scheduled job to run under a service account. A scheduled run submits as the user who created the schedule, so check that this user keeps access. **8.02**

Pick it when the code is packaged and tested outside Fabric and runs as a batch or streaming job.

runMultiple

The call `notebookutils.notebook.runMultiple`, made inside a notebook. It runs several other notebooks in parallel, or in an order you describe as a DAG, a dependency graph written in JSON.

Answer. The calling notebook schedules it, so whatever starts that notebook starts the whole set. Every child runs in the caller's Spark session and shares its compute.

The mix-up. People treat each child as its own job. All children compete for one session's executors, so one heavy child slows the rest. By default up to three times the available core count run at once.

Pick it when many small notebooks should run under one session, such as a loop over a list of tables.

WHICH ONE, WHEN

- If the Spark step sits among copies and other activities, use a notebook activity in a pipeline.
- If many notebook activities run together, turn on high concurrency for pipelines. Add one session tag to group them.
- If the code is a packaged .py or .jar file, use a Spark job definition.
- If one notebook should fan out to many small notebooks in one session, use `runMultiple`.

Watch out. Only pipeline, scheduler and job definition runs join the Spark job queue. A notebook you run by hand to call `runMultiple` is not queued. On a busy capacity it fails with HTTP 430.

SCENARIOS **S.14** Replacing ADF mapping data flows **S.15** ADF orchestrates Fabric jobs

C.06 Eventstream, Eventhouse, KQL database, Real-Time hub and Activator.

All five sit under Real-Time Intelligence and appear in the same streaming design. Their names do not say which one holds the data. Teams then look for stored history in an item that only passes events on.

THE QUESTION THAT SEPARATES THEM

Does it move events, store them, list them, or act on them?

Eventstream

A no-code item that captures events from sources such as Azure Event Hubs, Kafka or database change feeds. It filters and shapes them, then routes them to destinations.

Answer. It moves events. It keeps them only for a retention period, one day by default and at most 90 days, and then they expire.

The mix-up. People treat the eventstream as the store and add no destination. After the retention period the events are gone.

Pick it when events must be filtered, shaped or split before they land in an eventhouse, a lakehouse or Activator.

Eventhouse

A container for one or more KQL databases that share its compute and storage. Creating an eventhouse also creates a default KQL database with the same name.

Answer. It stores events, through the KQL databases inside it. Compute is managed at the eventhouse, which suspends when idle unless you configure it to stay active.

The mix-up. People create a new eventhouse for every database. Each one then needs its own compute settings and monitoring, while databases in one eventhouse share both.

Pick it when you need one home for streaming data that several related databases will share.

KQL database

A database inside an eventhouse whose tables you query with KQL, the Kusto Query Language, or a subset of T-SQL. Retention and caching policies decide how long data stays and how much sits in fast cache.

Answer. It stores events in tables. Ingested data lives here, and queries run here.

The mix-up. People expect other engines to read these tables straight away. OneLake availability has to be turned on for the database or table before Spark or other Fabric engines can read them.

Pick it when you need to query recent event data by time and keep it for a set retention period.

Real-Time hub

A tenant-wide catalogue of streaming data that every Fabric tenant gets without setup. It lists eventstream outputs, KQL tables you can access, and Fabric and Azure events.

Answer. It lists events. Learn calls it a logical place for streaming data, and the streams it lists come from eventstreams, databases and Azure sources.

The mix-up. People look for a Real-Time hub item to create or secure. There is none to create, and each person sees only the streams and tables their own access reaches.

Use it when you need to find a stream, or start an eventstream or alert from Fabric or Azure events.

Activator

A no-code engine that watches eventstreams, Power BI reports, Real-Time dashboards and Fabric events. Rules on objects, such as a freezer or a customer account, fire when a condition holds.

Answer. It acts on events. A rule sends an email or Teams message, starts a Power Automate flow, or runs a Fabric item such as a pipeline or notebook.

The mix-up. People expect Activator to keep event history for reports. Rules look back over a set period only, so the KQL database stays the record. **2.14**

Pick it when a condition in the data should raise an alert or start a job.

WHICH ONE, WHEN

- If events need filtering or routing, use an eventstream.
- If events must be kept and queried, land them in a KQL database inside an eventhouse.
- If you need to find which streams exist in the tenant, open the Real-Time hub.
- If a condition should send an alert or start a job, use Activator.

Watch out. An eventstream with 90 days of retention still holds no tables. Queryable history exists only once a destination such as a KQL database stores the events.

C.07 Starter pool, custom pool, high concurrency mode and environments.

All four live in the same Spark settings pages, and all four change how a session starts. People switch pools to fix slow starts that come from libraries, or edit an environment to change node size. Each one controls a different thing.

THE QUESTION THAT SEPARATES THEM

What does each decide: start time, size, sharing, or libraries and settings?

Starter pool

The default Spark pool in every workspace. It uses medium nodes from pre-warmed clusters that Microsoft manages on a best-effort basis.

Answer. It decides start time. A session usually starts in 5 to 10 seconds, because a warm cluster is waiting.

The mix-up. People expect seconds every time. Custom libraries add 30 seconds to 5 minutes. Private links or managed virtual networks add 2 to 5 minutes. **3.11**

Pick it when medium nodes are big enough and a fast start matters more than node size.

Custom pool

A pool a workspace admin creates with a chosen node family, a node size from Small to XX-Large, autoscale limits and dynamic executor allocation.

Answer. It decides size. A standard custom pool starts on demand in about 3 minutes. A custom live pool keeps clusters warm on a schedule, for starts of about 5 seconds.

The mix-up. People move to a custom pool to speed up starts and get slower ones. Only a custom live pool, generally available as of September 2026, stays warm, and billing runs while its clusters are allocated. **3.11 · 8.01**

Pick it when jobs need bigger or memory-optimised nodes, or a fixed ceiling on node count.

High concurrency mode

A workspace setting that lets several notebooks share one running Spark session. Each notebook gets its own REPL, an isolated interpreter, so variables do not collide.

Answer. It decides sharing. Notebooks share when they belong to one user and have the same default lakehouse and Spark settings. Only the notebook that started the session is billed.

The mix-up. People expect every notebook to join the session. A different default lakehouse or environment starts a new session, and one session holds 5 notebooks by default.

Pick it when one person runs several notebooks together, or a pipeline runs many notebook activities.

Environment

A workspace item that bundles a Spark runtime, compute settings, public and custom libraries, Spark properties and small resource files. You attach it to a notebook or job definition, or make it the workspace default.

Answer. It decides libraries and settings. Library and compute changes take effect only after you publish the environment.

The mix-up. People add libraries and then blame the pool for slow starts. Full mode takes 3 to 6 minutes to publish and adds 1 to 3 minutes to each session start without a custom live pool. **3.06 · 3.11**

Use it when several notebooks or jobs need the same runtime, libraries and Spark properties.

WHICH ONE, WHEN

- If sessions start slowly, check the environment's libraries and the workspace networking before changing the pool.
- If jobs need larger nodes, create a custom pool.
- If a custom pool must start fast during known hours, make it a custom live pool with a schedule.
- If several notebooks run together for one user or one pipeline, turn on high concurrency.
- If notebooks need shared libraries or Spark properties, put them in one environment.

Watch out. Notebooks attached to environments with different libraries or compute settings do not share a high concurrency session. Attach the same environment and default lakehouse when you want them to share.

SCENARIOS **S.26** One capacity, three workloads

C.08 V-Order, Z-order, liquid clustering and OPTIMIZE.

All four are table layout terms, and three of them can appear in one OPTIMIZE statement. One acts inside each file, two decide which rows share a file, and one sets the file count. Mixing them up leads to maintenance jobs that rewrite tables for no gain.

THE QUESTION THAT SEPARATES THEM

Does it change how a file is written, where rows sit, or how many files there are?

V-Order

A Microsoft write-time optimisation for Parquet files. It changes row-group layout, encoding and compression, and the files stay standard Parquet.

Answer. It changes how each file is written. It does not decide which rows share a file, and it does not change the file count.

The mix-up. People turn it on everywhere. Writes take about 15 percent longer, and Learn says to enable it for Spark tables only when Direct Lake is a main reader. New workspaces default to the writeHeavy profile with V-Order off. **3.09 · 5.08**

Pick it when Direct Lake reads a Gold table, or set the readHeavyForPBI resource profile instead.

Z-order

A clause on OPTIMIZE, written ZORDER BY, that rewrites active files so rows with similar values in the chosen columns sit in the same files.

Answer. It changes where rows sit. Queries that filter on those columns can skip files whose value ranges do not match.

The mix-up. People add it to a nightly job on every table. Azure Databricks documents Z-ordering as not idempotent, so repeat runs may rewrite data again. It cannot be combined with liquid clustering.

Pick it when an existing partitioned table is filtered on the same columns within each partition.

Liquid clustering

A Delta table setting, declared with CLUSTER BY, that groups rows by the chosen key columns. Learn advises 1 to 4 columns, and ALTER TABLE can change them later.

Answer. It changes where rows sit. New rows are clustered only when OPTIMIZE or auto compaction runs. Runtime 1.3 supports it, and Runtime 2.0 clusters incrementally by default.

The mix-up. People adopt it on Runtime 1.3 and run OPTIMIZE nightly. That runtime rewrites every file in a Z-cube, a group of clustered files, each run, so tables under 100 GB are rewritten in full. **3.04**

Pick it when a new table on Runtime 2.0 is filtered on a few columns, in place of partitioning or Z-order.

OPTIMIZE

A Delta command that compacts many small files into fewer large ones. Clauses add Z-order or V-Order to the same run.

Answer. It changes how many files there are. Spark applies bin compaction first, then Z-order, then V-Order.

The mix-up. People expect Fabric to compact tables on its own. Auto compaction is off by default, so small files pile up until you enable it or run OPTIMIZE. **3.01 · 8.12**

Use it when a backlog of small files needs clearing once, then enable auto compaction for ongoing writes.

WHICH ONE, WHEN

- If Direct Lake reads a Spark-written table, write it with V-Order or the readHeavyForPBI profile.
- If a new table is filtered on a few columns, use liquid clustering on Runtime 2.0.
- If an existing partitioned table is filtered on other columns inside each partition, use Z-order.
- If a table has many small files, run OPTIMIZE once and then turn on auto compaction.

Watch out. Learn lists liquid clustering as optional for Direct Lake, for better compression inside Parquet files. It does not say the SQL analytics endpoint skips files by clustering keys, so measure query times before and after.

SCENARIOS **S.03** A 3 billion row fact read by date and region **S.09** Premium import models to Direct Lake

C.09 Materialized lake views, KQL materialized views, Databricks materialized views and SQL Server indexed views.

All four store the result of a query so that reads are fast. All four are called views. They differ in what keeps that stored result current and how stale it can get. Fabric Warehouse supports none of them.

THE QUESTION THAT SEPARATES THEM

What keeps the stored result up to date, and when does that happen?

Materialized lake view

A Spark SQL query in a Lakehouse whose result Fabric saves as a Delta table. SQL authoring is GA, and PySpark authoring is in preview.

Answer. A refresh run keeps it current. Runs start on a time schedule, on demand, or from an event trigger in preview. Each run skips, refreshes incrementally or rebuilds in full.

The mix-up. People expect it to follow its sources by itself. Between runs it serves the last result, and incremental refresh needs change data feed and append-only sources. **3.12**

Pick it when Lakehouse Silver or Gold tables can be written in SQL and a scheduled or event-driven refresh is fresh enough.

KQL materialized view

An aggregation over an Eventhouse table, written as a single KQL summarize statement. A background process folds new rows into a stored result, called the materialized part.

Answer. The background materialization process keeps the stored part current as rows arrive. A query by the view's name adds the rows not yet processed, so the result is always current.

The mix-up. People think it lags like a scheduled job. Lag only shows when you query with `materialized_view()`, which reads the stored part alone for speed.

Pick it when streaming data in an Eventhouse needs fast, current aggregates, such as the latest record per device.

Databricks materialized view

A Unity Catalog managed table that stores a query result. You define it in Databricks SQL or inside a Lakeflow Spark Declarative Pipeline.

Answer. A refresh on serverless pipeline compute keeps it current. In a pipeline it refreshes when the pipeline update runs. A standalone one refreshes on REFRESH, a schedule or TRIGGER ON UPDATE.

The mix-up. People assume queries see the latest source rows. Between refreshes the view returns the last result, and incremental refresh needs row tracking on the Delta sources.

Pick it when the data is governed in Unity Catalog and Databricks should decide between incremental and full refresh.

SQL Server indexed view

A view created WITH SCHEMABINDING that has a unique clustered index. SQL Server then stores its rows the way it stores a table.

Answer. The database engine updates it inside every INSERT, UPDATE and DELETE on the base tables. It is never stale, and each write pays the cost.

The mix-up. Teams moving to Fabric Warehouse look for it there. Warehouse lists indexed views and materialized views as unsupported, so the logic becomes a table the load maintains. **4.02**

Use it when you stay on SQL Server or Azure SQL and a heavy aggregate must never be stale.

WHICH ONE, WHEN

- If the data is in a Lakehouse and a refresh run is fresh enough, use a materialized lake view.
- If events stream into an Eventhouse and aggregates must be current at query time, use a KQL materialized view.
- If the data is governed in Unity Catalog on Databricks, use a Databricks materialized view.
- If you move to Fabric Warehouse, rebuild each indexed view as a table that the load procedure writes.

Watch out. Materialized lake views and Databricks standalone views can both refresh when a source changes. That is still a separate run, so readers see the old result until it finishes.

C.10 Delta time travel, Warehouse time travel, Warehouse snapshots, restore points and table clones.

All five reach into retained history, and the four Warehouse features share one retention setting. They differ in scope, from one query to a whole warehouse. They also differ in whether you get read-only results or live tables back.

THE QUESTION THAT SEPARATES THEM

How far back can you go, and what do you get back?

Delta time travel

A query on a Lakehouse Delta table with `VERSION AS OF` or `TIMESTAMP AS OF`. `RESTORE TABLE` makes a chosen version current again as a new commit.

Answer. You can go back while both the Delta log and the data files remain. Logs are kept 30 days by default, and `VACUUM` removes old files after 7. You get one table, read-only or restored.

The mix-up. People count on 30 days because the history lists them. Once `VACUUM` has removed the files, reads and `RESTORE` for those versions fail.

3.10 · 8.11

Pick it when one Lakehouse table needs a look at, or a return to, a version from the last few days.

Warehouse time travel

A T-SQL `SELECT` with `OPTION (FOR TIMESTAMP AS OF)`, with the time given in UTC. On a SQL analytics endpoint it needs the new metadata sync, which is in preview.

Answer. You can go back as far as the retention period, 30 days by default. You get read-only results for every table in that one statement.

The mix-up. People put the clause in a view definition, or query a table that was dropped and recreated. Neither works, and columns added after the timestamp cannot be referenced.

Pick it when you need to read past Warehouse data without changing anything.

Warehouse snapshot

A read-only child item of a warehouse that shows the whole warehouse at one point in time. It stores no data files of its own.

Answer. You can set it anywhere in the retention period. That is 30 days by default, and up to 120 days with the preview retention setting. You get a stable, read-only view of every table.

The mix-up. People point a Direct Lake model at it. Snapshots need DirectQuery or Import, and a snapshot cannot be a shortcut source. **4.13 · 4.07**

Pick it when reports must read a consistent warehouse while loads write to the parent, and you roll it forward in one step afterwards.

Restore point

A saved point in time for a warehouse. Fabric creates one every 8 hours while the warehouse is active. You can add your own before risky changes.

Answer. You can go back as far as the retention period, 30 days by default. A workspace admin restores the whole warehouse in place, and every change after that point is lost.

The mix-up. People use it to recover one table. The restore overwrites every table, so good loads after the point disappear too. **9.10**

Pick it when a release damaged many tables and losing everything since the restore point is acceptable.

Table clone

A zero-copy table made with CREATE TABLE AS CLONE OF. It copies metadata only and references the same Parquet files in OneLake.

Answer. You can clone the current state or any point in the retention period, 30 days by default. You get a new writable table that is independent of its source.

The mix-up. People try to clone Lakehouse tables or clone into another warehouse. Clones work only inside one warehouse, and SQL analytics endpoint tables cannot be cloned. **9.10**

Pick it when one Warehouse table must come back as it was, beside the damaged one, for comparison or repair.

WHICH ONE, WHEN

- If a Lakehouse table went wrong in the last few days, check it with time travel, then RESTORE it.
- If you only need to read past Warehouse data, add Warehouse time travel to the query.
- If one Warehouse table needs repair, clone it at a good time and fix forward from the clone.
- If the whole warehouse is damaged, restore it in place from a restore point.
- If reports need a stable picture during loads, point them at a Warehouse snapshot.

Watch out. Table clones cannot go back past a table's last schema change. Warehouse time travel can, but only for columns that existed at that time.

SCENARIOS **S.29** Recovering from a bad Monday release **S.04** Month-end figures frozen while loads run

C.11 Semantic model refresh, Direct Lake framing and SQL analytics endpoint metadata sync.

All three sit between a committed row and a report visual. Each can be called a refresh. Any one of them can be the step that has not happened yet.

THE QUESTION THAT SEPARATES THEM

What has to happen before a new row shows up in the report?

Semantic model refresh (Import)

A refresh of an Import model reads the source and copies the data into the model. On Premium or Fabric capacity you can schedule up to 48 a day. Premium allows five hours per refresh.

Answer. A refresh must run after the row is committed and visible at the source. Until then the report shows the old copy.

The mix-up. People refresh straight after a Spark load, reading through the SQL analytics endpoint. If the endpoint has not synced yet, the refresh may read the old table. **8.07**

Use it when the model is in Import mode, and start it from the pipeline after the load completes.

Direct Lake framing

Framing is what a refresh does in a Direct Lake model. It points the model at the latest Delta table version and copies no data.

Answer. The table must be reframed after the Delta commit. With Keep your Direct Lake data up to date on, the default, that happens when a change is detected. Otherwise a manual, scheduled or API refresh does it.

The mix-up. People leave automatic updates on during a load that writes several tables. Each detected change triggers framing, so reports can show a half-finished load. **6.04 · 6.05**

Use it when the load is finished, with automatic updates off, so reports move from one complete state to the next.

SQL analytics endpoint metadata sync

A background process that reads the Delta logs of a Lakehouse and updates the SQL tables of its SQL analytics endpoint. The lag is usually under a minute.

Answer. The sync must pick up the commit before any T-SQL reader sees the row. DirectQuery and Import models that read through the endpoint may see the old table until then.

The mix-up. People assume the endpoint reads Delta live. New tables and rows stay missing until the sync runs, which can take minutes after large loads. **5.01**

Use it when a T-SQL reader needs the data next, and force it from the portal or REST API at pipeline end.

WHICH ONE, WHEN

- If the model is Import and reads a Lakehouse SQL analytics endpoint, force the sync first and refresh the model after it.
- If the model is Direct Lake, the row appears once its table is reframed.
- If a load writes several related tables, turn off automatic updates and frame once at the end.
- If T-SQL users or DirectQuery reports read a Lakehouse, force the metadata sync before you call the data ready.

Watch out. A new metadata sync, in preview for new endpoints, also checks freshness when a query arrives. Existing endpoints keep the background sync, so test which one you have.

SCENARIOS **S.09** Premium import models to Direct Lake **S.16** Ten minutes behind a busy table

C.12 Import, DirectQuery, Direct Lake on SQL and Direct Lake on OneLake.

All four are storage modes of a Power BI semantic model, and a report looks the same on each. They differ in which engine answers the query and in what breaks when data grows past a limit.

THE QUESTION THAT SEPARATES THEM

Where does the query run, and what happens when a limit is hit?

Import

The model holds a compressed copy of the data in memory, filled by scheduled or on-demand refresh.

Answer. Queries run in the model's in-memory engine. The size limit is 1 GB by default, or the capacity limit with large format. A big model can run out of memory during refresh.

The mix-up. People choose it for data that cannot refresh in time. Refresh limits still apply on capacity, such as five hours per refresh on Premium. **8.07**

Pick it when the data fits in memory, the source is outside OneLake, and a few refreshes a day are fresh enough.

DirectQuery

The model stores no data. Every visual sends one or more queries to the source, such as a Warehouse or a SQL database.

Answer. Queries run at the source. A query that returns more than 1,000,000 intermediate rows fails. So does one that passes the four-minute service timeout.

The mix-up. People expect Import speed. Each visual waits on the source. Large intermediate results fail unless a capacity admin raises the row limit.

Pick it when results must match the source at query time and the source can answer quickly.

Direct Lake on SQL

A Direct Lake model bound to one Lakehouse or Warehouse through its SQL analytics endpoint. It loads Delta columns from OneLake into memory when a query first needs them.

Answer. Queries run in memory, like Import. Past a guardrail, such as 5,000 Parquet files per table on F64, the query falls back to DirectQuery. It then runs slower on the endpoint.

The mix-up. People see no error and assume Direct Lake is working. SQL views and SQL endpoint security also cause fallback, and TABLETRAITS() shows which tables fall back. **6.07 · 5.07**

Pick it when the model reads one Lakehouse or Warehouse and you accept slower answers during fallback.

Direct Lake on OneLake

A Direct Lake model that reads Delta tables straight from OneLake, from one or more Fabric items. It can combine Direct Lake tables with Import tables.

Answer. Queries run in memory, and columns load from OneLake. It has no DirectQuery fallback, so past a guardrail the refresh fails and the model cannot be queried.

The mix-up. People move from Direct Lake on SQL and expect the same safety net. Tables that fell back quietly before now fail until someone optimises them. **6.01 · 6.03**

Pick it when tables are well maintained, come from several items, or need Import tables beside them.

WHICH ONE, WHEN

- If the source is outside OneLake and fits in memory, use Import.
- If results must match the source at query time, use DirectQuery and keep visuals simple.
- If the data sits in one Lakehouse or Warehouse and a slow fallback is acceptable, use Direct Lake on SQL.
- If tables span several Fabric items or you need Import tables in the same model, use Direct Lake on OneLake.
- If you use either Direct Lake mode, maintain the tables so they stay inside the guardrails.

Watch out. Max memory is not a guardrail in Direct Lake. A model that needs more memory pages columns in and out, so queries slow down without falling back.

SCENARIOS **S.09** Premium import models to Direct Lake **S.03** A 3 billion row fact read by date and region

C.13 Workspace identity, service principal, managed identity and the last modified user.

Every scheduled job in Fabric shows some identity to its sources. Four options come up in design talks, and three of them sound alike. They differ in who owns the identity, who looks after its secret, and what breaks when a person leaves.

THE QUESTION THAT SEPARATES THEM

Whose identity does the job use, and what happens when that person leaves?

Workspace identity

A service principal that Fabric creates for one workspace and manages itself. A workspace admin creates it in workspace settings, and Fabric handles its credentials.

Answer. The job uses the workspace's own identity, so a person leaving changes nothing. It lasts until an admin deletes it or the workspace is permanently deleted.

The mix-up. People expect it to work everywhere. It works in ADLS Gen2 shortcuts, pipeline Copy, Lookup, Get Metadata and Notebook activities, import models and CI/CD Dataflows Gen2. It never works through a gateway. **7.03 · 7.05**

Pick it when a production workspace reads Entra-secured Azure sources such as ADLS Gen2 and nobody should rotate a secret.

Service principal

An application identity registered in Microsoft Entra ID, the directory behind Microsoft sign-in. You hold its secret or certificate and add it to workspaces like a user.

Answer. The job uses the app's identity, so a leaver does not break it. The risk moves to its secret or certificate, which an app owner or admin must renew before it expires.

The mix-up. Teams create one service principal for every workspace. One leaked secret then opens all of them, and one expired secret stops every job on the same day. **7.04 · 7.02**

Pick it when a tool outside Azure calls Fabric REST APIs, or when a pipeline needs a last modifier that is not a person.

Managed identity

An Entra identity that Azure attaches to a resource, such as a **Data Factory**. Azure manages its credentials, and nobody ever sees a secret.

Answer. The job uses the Azure resource's identity, so people leaving does not affect it. A system-assigned one is deleted with its resource, and a user-assigned one lasts until someone deletes it.

The mix-up. People look for it in Fabric item settings. Inside Fabric the workspace identity plays that part, so a managed identity belongs to the Azure service making the call.

Pick it when an Azure service calls Fabric, for example the **Data Factory** Fabric Lakehouse connector, with at least Contributor on the workspace.

Last modified user

The person who last edited a pipeline. A notebook that the pipeline calls runs as that user, and a scheduled notebook runs as whoever created or last updated its schedule.

Answer. The job uses a person's identity. If that person skips Fabric sign-in for 90 days, their schedules expire. If they leave, items they own can stop working.

The mix-up. People assume the owner is the identity. A small edit by a colleague changes the run identity, and their permissions then decide what the run reads. **7.01 · 2.11**

Use it when a job is still in development, and make a service principal the last modifier through the REST API before production.

WHICH ONE, WHEN

- If the source is Entra-secured Azure storage and the item supports it, use the workspace identity.
- If a tool outside Azure calls Fabric REST APIs, or a pipeline needs a last modifier that is not a person, use a service principal. Renew its secret on a calendar.
- If an Azure service such as **Data Factory** calls into Fabric, use that service's managed identity.
- If a pipeline still runs as a person, move the last modifier to a service principal before that person leaves.
- If the source sits behind a gateway, pick a credential other than the workspace identity, which gateway connections do not support.

Watch out. A connection decides how one activity signs in to a source. The pipeline run itself belongs to the last modified user. A notebook activity runs as that person unless you set a connection on the activity, such as the workspace identity.

SCENARIOS **S.30** Handover when the only engineer leaves **S.15** ADF orchestrates Fabric jobs

C.14 On-premises data gateway, virtual network data gateway, managed private endpoints and trusted workspace access.

All four let Fabric reach data that the public internet cannot. They get mixed up because each is sold as private connectivity. They differ in which side opens the connection and which Fabric items can use it.

THE QUESTION THAT SEPARATES THEM

Which direction is the connection, and which Fabric items can use it?

On-premises data gateway

Software you install on a Windows machine inside your network. It relays queries from Fabric to sources the cloud cannot reach, such as an on-premises SQL Server.

Answer. The gateway opens outbound connections to Azure and polls for work, so no inbound port is opened. Dataflow Gen2, pipelines, copy jobs, semantic models and S3, S3-compatible and GCS shortcuts can use it.

The mix-up. People reuse a workspace identity connection through it. Gateway connections do not support workspace identity, so the run fails until the connection uses another credential. **2.16**

Pick it when the source sits on premises or in another network you control and the reader is a dataflow, pipeline, copy job or model.

Virtual network data gateway

A gateway that Microsoft runs for you inside a subnet of your Azure virtual network. You delegate the subnet to Microsoft.PowerPlatform/vnetaccesslinks, and no machine is needed.

Answer. Queries start from your own subnet, so sources behind private endpoints or network rules answer them. Dataflow Gen2, pipelines, copy jobs, mirroring, semantic models and paginated reports can use it.

The mix-up. People expect OAuth connections to keep working as they do elsewhere. This gateway does not refresh OAuth tokens. They usually expire about an hour after a refresh starts, so long refreshes fail.

Pick it when Azure sources sit behind private endpoints and you do not want to patch and monitor gateway machines.

Managed private endpoint

A private endpoint that Fabric creates from the workspace's managed virtual network to one Azure resource. A workspace admin requests it, and the resource owner approves it.

Answer. The connection goes out from Fabric to the resource over Private Link. Notebooks, Spark job definitions and Lakehouse Spark operations use it, and Eventstream support is in preview.

The mix-up. People add one and then wonder why notebooks start slowly. Once the managed network exists, starter pools are off and a session can take 3 to 5 minutes to start. **3.11 · 7.06**

Pick it when Spark must read an Azure PaaS source whose public network access is switched off.

Trusted workspace access

A firewall exception on an Azure Storage account for one named Fabric workspace. It needs a resource instance rule, a workspace identity and an F SKU capacity.

Answer. Fabric calls the storage account, and the storage firewall lets that workspace in. OneLake shortcuts, pipelines, the Warehouse COPY statement and import semantic models can use it, and Spark cannot use it directly.

The mix-up. People move the workspace to a trial or non-F capacity. Trusted access stops after about an hour, and every shortcut and copy that relied on it fails. **7.05**

Pick it when the source is firewalled ADLS Gen2 or Blob Storage and the reader is a shortcut, pipeline or COPY statement.

WHICH ONE, WHEN

- If the source is on premises, use the on-premises data gateway.
- If an Azure source sits behind a private endpoint and a dataflow, pipeline or model reads it, use a virtual network data gateway.
- If Spark must reach a private Azure source, use a managed private endpoint.
- If the source is firewalled Azure Storage, use trusted workspace access, and let Spark read it through a shortcut.

Watch out. Trusted workspace access and managed private endpoints both reach firewalled storage, but serve different engines. A shortcut made through trusted access serves Spark, SQL and Power BI, while ADLS Gen2 shortcuts cannot use a managed private endpoint yet.

SCENARIOS **S.24** Every source behind a firewall **S.13** Runtime and gateway, same servers

S.08 Databricks stays, Fabric reads in place

C.15 Workspace roles, item permissions, OneLake security roles and SQL GRANT with row-level security.

All four decide who reads a table, and one user can hold all four at once. They get confused because a rule set in one place looks as if it applies everywhere. Each is enforced by a different layer, and the broader grants pass around the narrower rules.

THE QUESTION THAT SEPARATES THEM

Which engine enforces it, and who gets around it?

Workspace role

Admin, Member, Contributor or Viewer, assigned on a workspace to a user, group or service principal. The role applies to every item in that workspace.

Answer. Fabric enforces it on every item and engine. Admin, Member and Contributor read and write all data through SQL, Spark and OneLake APIs. They pass around OneLake security, and their Spark reads skip SQL rules.

The mix-up. People give engineers Contributor in production and trust the finer rules. Contributor can read the files behind any SQL rule and can delete any item in the workspace. **7.09 · 9.06**

Use it when a person or service principal builds or runs items in the workspace, and keep plain readers on Viewer.

Item permission

Rights on one item, granted by sharing it. Read lets someone open the item and connect, and extra options add SQL reads (ReadData), Spark and OneLake reads (ReadAll), Build, Edit and Execute.

Answer. Fabric checks the grant, and the engine it names does the reading. A ReadAll holder reads the files with Spark and passes around every T-SQL permission and row filter.

The mix-up. People tick the Spark option for convenience. The recipient can then read rows that SQL row-level security hides, and also joins the lakehouse DefaultReader role. **7.08**

Use it when someone needs one item and no workspace role, and grant ReadAll only to people allowed to see every row.

OneLake security role

A role on a lakehouse or mirrored item that grants Read on schemas, tables or folders, with optional row and column filters. Lakehouse roles can grant ReadWrite instead. It became generally available in May 2026.

Answer. OneLake enforces it for Spark, the SQL endpoint in user identity mode and Direct Lake on OneLake. Admin, Member and Contributor pass around it, except endpoint row filters in that mode.

The mix-up. People add careful roles and forget DefaultReader. That default role still grants every lakehouse table to anyone with ReadAll until you edit or remove it. **7.10 · 7.12**

Use it when one set of table, row and column rules must hold for readers in Spark, SQL and Power BI.

SQL GRANT with row-level security

T-SQL permissions and security policies inside a Warehouse or SQL analytics endpoint. GRANT and DENY control objects and columns, and a security policy filters rows through a predicate function.

Answer. The SQL engine enforces it, for T-SQL queries only. It applies to every SQL user, including dbo and workspace admins, but a ReadAll holder reads the files through Spark and skips it.

The mix-up. People add row-level security on a Warehouse under a Direct Lake model. Direct Lake on SQL queries then fall back to DirectQuery, and reports slow down. **5.07 · 7.15**

Use it when readers reach the data only through T-SQL, such as a Warehouse or an endpoint in delegated mode.

WHICH ONE, WHEN

- If someone builds or runs items, give a workspace role; if they only read, share the item.
- If rules must hold in Spark, SQL and Direct Lake alike, put them in OneLake security.
- If data is read only through T-SQL, use GRANT and a security policy in the Warehouse.
- If a user holds Contributor or ReadAll, assume they can read every row in the files.

Watch out. The SQL analytics endpoint has two modes. Delegated mode, the default for new endpoints, applies SQL rules and ignores OneLake roles, and user identity mode ignores table GRANTs and applies OneLake roles.

SCENARIOS **S.22** Branch-level row security for a bank **S.23** External partners read part of Gold

S.05 Three business units, one customer list

C.16 Sensitivity labels, DLP policies, protection policies and OneLake security.

All four sit under the heading of data protection, and three of them live in Microsoft Purview, the compliance portal. People expect a label to stop access, and on its own it does not. They differ in whether they classify, raise a warning, or block a read.

THE QUESTION THAT SEPARATES THEM

Does it classify, warn, or block access?

Sensitivity label

A Microsoft Purview Information Protection tag, such as Confidential, set on a Fabric or Power BI item. It shows on the item and passes down to items built from it.

Answer. It classifies. On its own it blocks nobody inside the service. Any encryption on the label applies only when data leaves through supported exports such as Excel, PDF, PowerPoint or .pbix.

The mix-up. People label a lakehouse Highly Confidential and think access is closed. Everyone with access still reads it, and CSV exports leave with no protection.

Pick it when items need a sensitivity tag that exports and other policies can act on.

DLP policy

A Purview data loss prevention policy that scans Fabric items for sensitive information types, such as card numbers, or for labels. It covers semantic models, lakehouses, warehouses, KQL, mirrored, SQL and Cosmos databases.

Answer. It warns. A match raises a policy tip on the item and an alert for admins, and only the restrict access action, in preview, blocks users.

The mix-up. People expect it to scan every file. It reads only Delta tables, skips complex types, and does not evaluate semantic models owned by a service principal.

Pick it when you want to find sensitive data landing in Fabric and tell the owners about it.

Protection policy

A Purview policy tied to one sensitivity label. Items that carry the label stay open only to the users and groups the policy names.

Answer. It blocks. Named users keep the access they already had, and all other users and groups are blocked from the item.

The mix-up. People expect it to grant access to the named group. It only lets them keep access they already hold, and a new policy can take up to 24 hours to start protecting items.

Pick it when every item with a given label must be closed to everyone outside a named group.

OneLake security

Fabric roles on a lakehouse or mirrored item that grant access to schemas, tables and folders, with row and column filters. Readers see only what their roles allow.

Answer. It blocks at data level, table by table and row by row. It limits Viewers and item-permission readers, and Admin, Member and Contributor largely pass around it.

The mix-up. People expect it to follow the sensitivity label. A role lists tables, filters and members, so relabelling an item leaves its roles as they were. [7.10](#) · [7.11](#)

Pick it when readers may see some tables, rows or columns of an item and not the rest.

WHICH ONE, WHEN

- If you only need to mark data by sensitivity, apply a sensitivity label.
- If you need to know when sensitive values land in a table, use a DLP policy.
- If a label must close items to everyone outside a group, add a protection policy for that label.
- If readers may see part of an item, use OneLake security roles.

Watch out. DLP restrict access and a protection policy both cut access, but they fire at different moments. DLP acts when it evaluates an item and finds a match. A protection policy acts on the label alone.

SCENARIOS **S.21** Patient records from four systems

C.17 External data sharing, Delta Sharing, cross-tenant shortcuts and B2B guest access.

All four let people in one organisation read data held by another, so they get swapped when a partner asks for access. They differ in who signs in to which tenant, which engine does the reading, and whose capacity carries the cost.

THE QUESTION THAT SEPARATES THEM

Who signs in where, and whose capacity pays for the reads?

External data sharing

A Fabric feature that shares tables or files from a lakehouse, warehouse, KQL, SQL or mirrored database with another Fabric tenant. Admins switch it on in both tenants. Accepting the share creates a read-only shortcut in a consumer lakehouse.

Answer. Consumer users sign in to their own tenant only and read through the shortcut. Learn's OneLake rule bills shortcut reads to the shortcut's capacity. It does not mention external shares, so check the Capacity Metrics app.

The mix-up. People expect the provider's labels and access rules to follow the data. They stay in the provider tenant, and the consumer can grant access to anyone.

Pick it when a partner runs its own Fabric tenant and should manage its own users.

Cross-tenant OneLake shortcut

A OneLake shortcut in a lakehouse or KQL database that points at an item in another Fabric tenant. Fabric's What's new page lists it under delegated OneLake shortcuts, in preview.

Answer. It signs in to the producer tenant with a fixed organisational account or service principal that exists there. The general OneLake rule bills shortcut reads to your capacity, but Learn does not confirm this for cross-tenant shortcuts.

The mix-up. People assume a shortcut cannot cross tenants, or that it passes each user through. It always uses the delegated identity. Users see only what both that identity and their own roles allow. **1.06**

Pick it when both tenants are yours, such as test and production, and one identity may read for everyone.

Delta Sharing (now OpenSharing)

An open protocol from Databricks for sharing tables with other organisations and platforms. Databricks now documents it as OpenSharing. Recipients read live data through short-lived pre-signed URLs, and tables are not replicated.

Answer. An open recipient has no sign-in on the provider side. It presents a bearer token or a short-lived token from OIDC federation. The recipient's own client reads shared tables, though shared views can add provider compute. Cloud egress fees can apply across regions or clouds.

The mix-up. People use it to bring Databricks data into Fabric inside one company. A shortcut to the ADLS Gen2 storage avoids managing recipient tokens.

Pick it when Databricks data goes to a partner on Databricks or on another platform.

B2B guest access

Microsoft Entra B2B invites a partner's user into your tenant as a guest. The guest then opens your workspaces and items with the roles and permissions you grant.

Answer. The guest signs in to your tenant, often through its tenant URL, and queries items in your workspaces. Learn does not say which capacity pays, so check the Capacity Metrics app. Their Power BI licence needs depend on your workspace type.

The mix-up. People treat guests as outsiders with limited reach. A guest with a workspace role gets that role's rights, and guest UPN formats can break row-level security mapping tables. **7.07 · 7.08**

Pick it when a few named partner users must work inside your workspaces or read your reports.

WHICH ONE, WHEN

- If the partner runs its own Fabric tenant and manages its own users, use external data sharing.
- If you control both tenants and one identity may read for everyone, use a cross-tenant shortcut, which is in preview.
- If the recipient uses Databricks or a platform outside Fabric, use Delta Sharing.
- If partner users must work inside your workspaces, invite them as B2B guests and plan for their load on your capacity.

Watch out. An external data share gives read access to any user in the invited user's home tenant. Once it is accepted, the provider cannot control who reads the data there.

SCENARIOS **S.23** External partners read part of Gold

C.18 Smoothing, bursting, throttling, surge protection, capacity overage and autoscale billing for Spark.

All six decide what happens when work asks for more compute than the capacity SKU provides. Some spread the cost over time, some stop new work, and some add a charge to the Azure bill. The names sound alike, so admins switch on the wrong one.

THE QUESTION THAT SEPARATES THEM

When demand runs past the SKU, does it spread the cost, stop the work, or add to the bill?

Bursting

Fabric lets an operation use more compute than the SKU provides while it runs. A small capacity can finish a large job quickly this way.

Answer. It borrows compute now, and smoothing spreads that cost over future timepoints. It stops nothing by itself.

The mix-up. People buy a bigger SKU to make one slow query faster. Bursting already gives a single operation extra compute, so the query is often limited by its plan or the data it scans. **8.06**

Use it when sizing, by counting on bursting for short heavy jobs and sizing the SKU for sustained load.

Smoothing

Fabric averages each operation's CU use over future 30-second timepoints. Interactive operations are smoothed over 5 to 64 minutes, and background operations over 24 hours.

Answer. It spreads the cost across time. Nothing stops, but last night's background jobs still take a share of this morning's capacity.

The mix-up. People read a quiet morning as spare room. Background work smoothed over 24 hours can fill much of every timepoint, so a report spike at 09:00 can tip the capacity into throttling. **8.03 · 8.09**

Use it when you plan schedules, since background jobs can run side by side without a spike of their own.

Throttling

The capacity's response once smoothed use runs too far ahead of the SKU. The first 10 minutes of future capacity are free of it, which Learn calls overage protection.

Answer. It stops work in stages. New interactive operations wait 20 seconds, then are rejected past 60 minutes of future use. Past 24 hours, every new request is rejected, while running operations finish.

The mix-up. People treat it as a fault to raise with support. It is how the capacity pays back borrowed compute. It lasts until idle time burns down the carried-forward CUs or the SKU is scaled up. **8.08 · 8.04**

Use it when deciding what to fix, as the signal to tune heavy items, move load or scale the SKU.

Surge protection

Admin limits that reject new background operations before the capacity reaches deep throttling. Capacity-level limits use a rejection and a recovery threshold. Workspace-level limits, in preview, cap one workspace's 24-hour CU use and can block it.

Answer. It stops new background work early, on purpose, to keep room for interactive work. Jobs already running carry on.

The mix-up. People expect it to shield a critical workload. At capacity level it rejects background jobs across the capacity, so Learn advises a separate capacity for critical solutions. **8.05**

Pick it when background loads keep pushing report users into delays or rejections.

Capacity overage

An F SKU setting that pays off excess use at the point throttling would start. A capacity admin sets a rolling 24-hour threshold in CU hours. Learn's setup page still marks it as in preview.

Answer. It adds to the bill. Excess CU hours bill on a separate meter at three times the pay-as-you-go rate, and the threshold is not a hard cap.

The mix-up. People expect it to make work faster. It adds no compute, and a capacity that overloads every day pays the higher rate where a larger SKU would cost less. **8.10**

Pick it when uptime matters and overloads come as rare, short spikes.

Autoscale billing for Spark

A per-capacity opt-in that moves Spark jobs onto serverless compute billed pay-as-you-go. Learn now calls it on-demand billing for Spark. An admin sets a maximum CU limit for Spark.

Answer. It adds to the bill as a separate Spark meter. Spark stops using capacity CUs, with no bursting or smoothing. At the CU limit, interactive jobs are throttled and background jobs queue.

The mix-up. People flip the setting during working hours. Enabling it, disabling it or lowering the CU limit cancels the Spark jobs running under it. **8.01**

Pick it when Spark load is bursty and should stop competing with reports for capacity CUs.

WHICH ONE, WHEN

- If work is slow only at certain hours, read the smoothed background share in the Capacity Metrics app before changing anything.
- If background loads crowd out report users, set capacity-level surge protection.
- If one workspace can starve the rest, use workspace-level surge protection, in preview, or give it its own capacity.
- If rare spikes must never throttle, turn on capacity overage with a threshold you can afford, and check the setting on every capacity.
- If Spark is the bursty workload, move it to autoscale billing for Spark and review the capacity size afterwards.

Watch out. Learn uses overage protection for the free 10 minutes of future capacity before throttling starts. Capacity overage is the separate, billed feature at three times the pay-as-you-go rate.

SCENARIOS **S.26** One capacity, three workloads **S.28** The bill rose 60 percent

C.19 Deployment pipelines, Git integration, fabric-cicd and variable libraries.

All four take part in moving work from development to production, and teams often run two of them together. They differ in what they carry: item definitions, table data or stage settings. Mixing them up leaves production reading development data.

THE QUESTION THAT SEPARATES THEM

What moves between stages: definitions, data or settings?

Deployment pipeline

A Fabric feature that pairs a workspace with each of 2 to 10 stages and copies items from one stage to the next. Deployment rules set data sources, parameters or a notebook's default lakehouse per stage.

Answer. Definitions move. Data, permissions, refresh schedules and data source credentials stay behind, and data already in a target item is kept where possible.

The mix-up. People expect new numbers in production after a deployment. Semantic models need a refresh, and a Direct Lake model keeps reading the source stage's lakehouse until a rule rebinds it. **9.04 · 9.02**

Pick it when a team promotes through fixed workspaces from the Fabric portal and wants compare and deployment history built in.

Git integration

Connects a workspace to a branch in Azure DevOps or GitHub. Commit exports the workspace's supported items to the branch, and update overwrites the workspace from the branch.

Answer. Definitions move, between one workspace and its branch. Lakehouse tables and files are never tracked, so no data travels. It links stages only when each stage workspace connects to its own branch.

The mix-up. People treat it as a release tool. It versions code and keeps durable history, while promotion still needs pull requests, a deployment pipeline or an API tool. **9.09**

Pick it when you need version history, branches and pull request review for workspace items.

fabric-cicd

An open-source Python library from Microsoft that publishes item definitions from a repository folder to a workspace through the Fabric REST APIs. A build pipeline, such as Azure DevOps, runs it, often under a service principal.

Answer. Definitions move, from the repository to the target workspace, as a full deployment each time. A parameter.yml file applies find-and-replace rules for each environment before upload.

The mix-up. People expect it to repoint every reference by itself. Hard-coded GUIDs pass through unless parameter.yml maps them, and unpublish_all_orphan_items deletes workspace items missing from the repository. **9.05**

Pick it when releases run as code from one main branch, with per-environment changes applied before upload.

Variable library

A workspace item holding variables with a value set for each stage, such as dev, test and prod. Pipelines, notebooks, lakehouse shortcuts, Dataflow Gen2, Copy jobs and other items read values from the active set.

Answer. The variables and all their value sets move with the item, through Git or a deployment. The active value set is workspace state kept outside the definition, and deployments do not change it.

The mix-up. People deploy to a new stage and expect production values. A library deployed or committed for the first time has the Default value set active until someone changes it. **9.08**

Pick it when pipelines, notebooks or shortcuts need a different connection, lakehouse or number in each stage.

WHICH ONE, WHEN

- If you need history and review of item definitions, connect the development workspace to Git.
- If you promote from the portal through fixed workspaces, use a deployment pipeline with rules.
- If releases run as code from a main branch, use fabric-cicd with a parameter file.
- If an item needs different values per stage, put them in a variable library and set the active value set once in each stage.
- If a stage needs data, load or refresh it after deployment. Deployment pipelines and Git integration carry no table data.

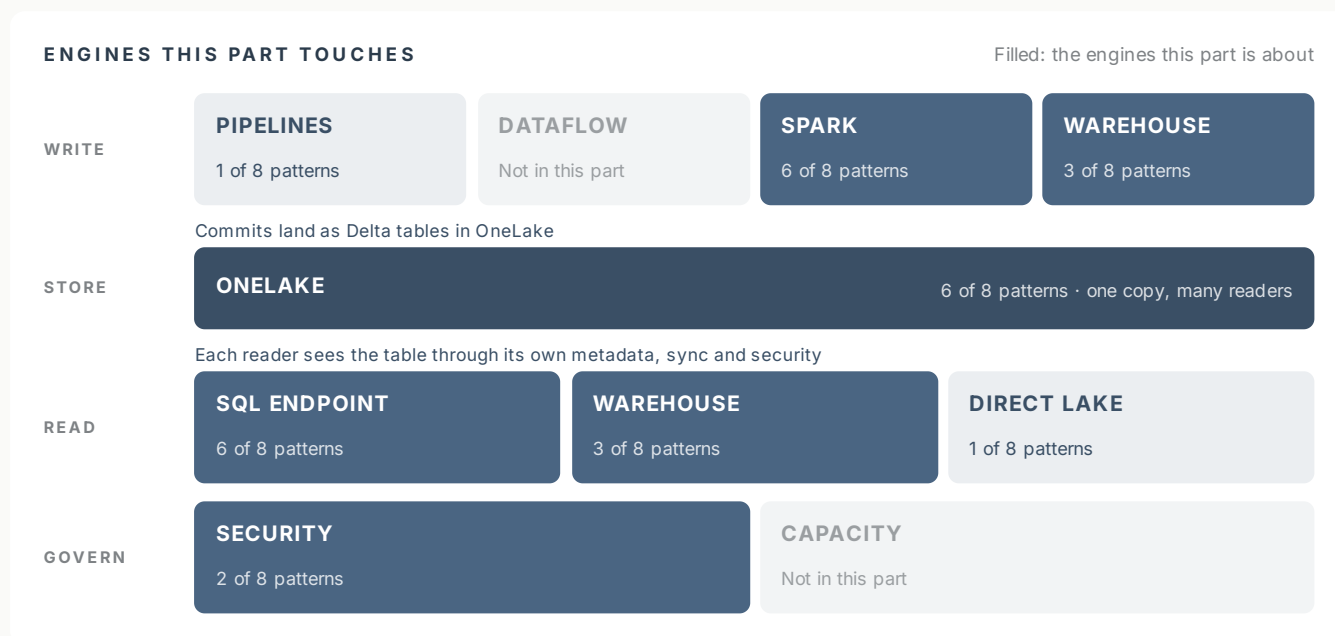
Watch out. External shortcuts, such as ADLS Gen2 or S3, keep the same target in every stage after deployment. Unless a variable library sets the target per stage, production reads the development storage.

SCENARIOS **S.27** Dev, test and prod for two teams **S.29** Recovering from a bad Monday release

01

Storage, OneLake and Data Placement.

Where data lives and how it is laid out across Lakehouse, Warehouse, shortcuts and layers. Most of these failures are decided before the first pipeline runs.



NO.	PATTERN	PIPE	SPARK	ONELAKE	WH	SQL EP	SEC	PAGE
1.01	Your analysts say the Lakehouse SQL endpoint is too slow. They want every table moved into a Warehouse.							34
1.02	Everything now lives in a Warehouse. The data science notebooks can read it but cannot write back.							35
1.03	Two reports show different revenue for the same month. Both read a table called orders.							36
1.04	Shortcuts point at shortcuts across many workspaces. Nobody can say where a table's data lives.							37
1.05	Queries through your ADLS shortcut run far slower than on native tables. Workspace caching is on.							38
1.06	You dropped an unused table during cleanup. Hours later, reports in three other workspaces fail.							39
1.07	You split Bronze, Silver and Gold into separate workspaces. Every debugging join now starts with a shortcut.							40
1.08	Reports refresh without errors but count duplicates and test rows. They read Bronze directly.							41

1.01 Your analysts say the Lakehouse SQL endpoint is too slow. They want every table moved into a Warehouse.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

The SQL analytics endpoint runs on the **same engine** as the Warehouse. Slow queries come from **table layout**: small files, over-partitioning and uncompact deletes.

Cost of failure **Slow queries**Status **GA** · Verified **Sep 2026**

SQL ANALYTICS ENDPOINT

Same engine

Read-only. Views, functions and stored procedures allowed

WAREHOUSE

Same engine

Adds T-SQL writes and transactions

FIG. 1.01 Both paths reach the same engine. Compare the files underneath; a moved copy fragments too.

MEASURE FIRST

- 1 From the endpoint: look for a high `FileCount` skewed to the small size buckets.

```
EXEC sys.sp_get_table_health_metrics 'sales.orders';
-- FileCount, FileSize histogram, DeletedRowCount
```

- 2 From a notebook: partitions well under 1 GB mean the partition column is too fine.

```
DESCRIBE DETAIL sales.orders;
-- numFiles, sizeInBytes, partitionColumns
```

- 3 Rule out metadata sync: a late new table is sync lag, normally under a minute.

CAUSE

The engine parallelises scans per Parquet file. Thousands of small files, or a partition per date or customer, let file and metadata overhead dominate. Updates and deletes add files until compacted. A fresh Warehouse copy starts clean, so the trial migration looked faster.

FIX

Fix the writer: keep the Runtime 2.0 adaptive file size default, drop high-cardinality partitions, and schedule `OPTIMIZE` or auto compaction. The endpoint does not need V-Order.

```
OPTIMIZE sales.orders;
ALTER TABLE sales.orders
SET TBLPROPERTIES ('delta.autoOptimize.autoCompact' = 'true');
```

Move a table to a Warehouse only when it needs T-SQL writes: DML, multi-statement transactions or SQL-owned ELT.

EXPECTED RESULT

Rerun the health check after `OPTIMIZE`: `FileCount` and query duration should both drop.

Go deeper: PySpark Handbook 4.0, table maintenance.

RELATED [1.02](#) Everything now lives in a Warehouse [1.03](#) Two reports show different revenue for the...

1.02 Everything now lives in a Warehouse. The data science notebooks can read it but cannot write back.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Warehouse tables are **Parquet with published Delta logs**, so Spark reads them but cannot write them natively, and there is no Files area. Put each table where its **writer** lives.

Cost of failure **Rework**Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
Everything in a Lakehouse	Spark reads and writes natively. SQL users query through the endpoint on the Warehouse engine. The Files area holds raw JSON, images and model artefacts.	No T-SQL writes. SQL-owned ELT and procedures that change data cannot run on the read-only endpoint.
Everything in a Warehouse	Full T-SQL DML and transactions. Semi-structured data works: OPENROWSET reads JSONL and JSON files, OPENJSON expands nested arrays. Spark reads through the connector or a shortcut.	Other engines get read-only access. Spark writes only through the connector on Runtime 2.0, which stages data and runs COPY INTO . Delta logs publish after commit, so Spark can briefly see an older version. No place for raw files.
Split by writer DEFAULT	Spark-written tables in a Lakehouse, T-SQL-written tables in a Warehouse. Each engine reads the other in place: three-part names in T-SQL, shortcuts or the connector in Spark.	Needs a documented owner per table. Cross-item T-SQL joins work only inside one workspace.

FIG. 1.02 Each option shows which engine writes and which only reads.

DEFAULT

Split by writer. Every table has one engine that writes it, and every other engine reads it where it is.

SIGNS YOU CHOSE WRONG

Notebooks export DataFrames to CSV for a pipeline to load. Raw JSON lands outside Fabric because the Warehouse has no Files area. Large Spark outputs go through connector writes every night. A Warehouse table is reloaded from a Lakehouse copy of itself.

CHECK IT

Spark reads a Warehouse table through the connector, with no copy.

```
import com.microsoft.spark.fabric
df = spark.read.synapsesql("SalesWH.dbo.orders")
df.groupBy("region").count().show()
```

Go deeper: [SQL Handbook](#), [JSON in Fabric Warehouse](#).

RELATED **1.01** Your analysts say the Lakehouse SQL endpoint... **1.03** Two reports show different revenue for the...

1.03 Two reports show different revenue for the same month. Both read a table called orders.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Two pipelines load the table into a Lakehouse and again into a Warehouse, on different schedules with different logic. Both succeed and neither copy is authoritative.

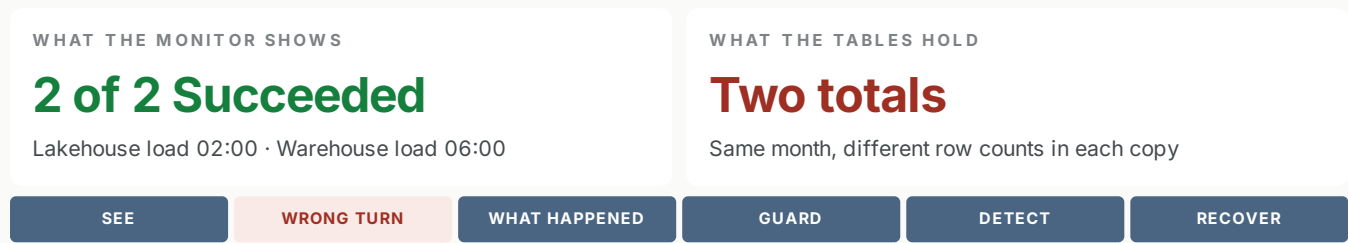
Cost of failure **Wrong data**Status **GA** · Verified **Sep 2026**

FIG. 1.03 The two runs both show Succeeded and the totals differ: each status covers only its own load.

WRONG TURN

"Two copies give us redundancy. If one load breaks, the other still works."

WHAT HAPPENED

Each copy has its own pipeline, filter and schedule. A late file reached the Lakehouse load but missed the Warehouse load. A currency fix went into one transformation only. No check compares the copies, so every run status is Succeeded.

1 GUARD

Give each table one writer. The other engine reads in place: T-SQL with three-part names inside the workspace, Spark through a shortcut or the Warehouse connector.

```
-- In SalesWH: read the Lakehouse table, no copy
CREATE VIEW dbo.orders_v AS
SELECT * FROM SalesLH.dbo.orders;
```

2 DETECT

Compare both copies in one query.

```
SELECT 'LH' AS src, COUNT(*) AS n,
SUM(amount) AS amt
FROM SalesLH.dbo.orders
UNION ALL
SELECT 'WH', COUNT(*), SUM(amount) FROM
SalesWH.dbo.orders;
```

3 RECOVER

Name the authoritative copy per domain and record it. Repoint consumers of the other copy to a view or shortcut over the authoritative one. Retire the second pipeline, then drop the copy once the comparison matches.

RELATED [1.02](#) Everything now lives in a Warehouse [1.04](#) Shortcuts point at shortcuts across many...

1.04 Shortcuts point at shortcuts across many workspaces. Nobody can say where a table's data lives.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

A shortcut stores a **target workspace, item and path**; chains stop at **5 hops**.
Lineage shows shortcuts only inside one workspace, so share raw source tables only through a published contract.

Cost of failure **Rework**
Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
Shortcut straight to source tables	No copy, no pipeline. Consumers see new data as soon as it lands. Reads are billed to the consumer's capacity.	Consumers bind to the source schema, so a renamed or dropped column breaks their views and reports without warning. Chains grow across workspaces and lineage stops at each workspace edge.
Shortcut to a published contract DEFAULT	The owner publishes a Gold schema of materialized lake views or owner-written tables with a fixed column list. Consumers shortcut only to that, one hop. Internal tables can change behind it.	The owner maintains the contract, versions breaking changes and pays for its refresh.
Copy into the consuming workspace	The consumer controls schema, timing and layout. A source change fails a pipeline run, where the monitor shows it, before any report breaks.	Duplicate storage and a second version of the truth. Freshness depends on the copy schedule.

FIG. 1.04 The three ways to share data across workspaces, by who absorbs a schema change.

DEFAULT	SIGNS YOU CHOSE WRONG
Shortcut to a published contract. One hop from consumer to owner, and the owner decides which columns are stable.	A shortcut's target is itself a shortcut. Nobody can list who consumes a table. A column rename broke reports in another workspace. Lineage shows a shortcut but not where it ends.

CHECK IT

List every shortcut in an item with its target. Run it per item on a schedule and store the output as your registry.

```
GET https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}/items/{itemId}/shortcuts
// each row: path, name, target.oneLake.workspaceId, itemId, path
```

RELATED

1.06 You dropped an unused table during cleanup

1.05 Queries through your ADLS shortcut run far...

1.03 Two reports show different revenue for the...

1.05 Queries through your ADLS shortcut run far slower than on native tables. Workspace caching is on.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Shortcut caching covers **S3, GCS, S3-compatible and gateway** sources only.

Cost of failure **Slow queries**

ADLS reads always hit the storage account, so **file layout** and network path set the speed.

Status **GA** · Verified **Sep 2026**

NATIVE ONELAKE TABLE

Fabric-maintained

Your writers, your file sizes, your compaction

ADLS SHORTCUT

No cache

Every read hits the storage account

FIG. 1.05 The two look identical in the explorer; only the native table has files Fabric maintains.

MEASURE FIRST

- 1 Thousands of files of a few MB each mean small-file overhead on every scan.

```
DESCRIBE DETAIL bronze.adls_orders;
-- numFiles, sizeInBytes, partitionColumns
```

- 2 Check the storage account and capacity regions. A cross-region shortcut adds latency to every file request.

- 3 Time one query on the shortcut and on a native one-month copy. The ratio isolates remote reads.

CAUSE

Another engine wrote the table, often uncompact or finely partitioned, and Fabric does not maintain it. Each file open is a storage request with no OneLake cache between. ADLS bills those transactions directly, outside capacity units.

FIX

Compact at the source with the owning engine, or co-locate the account with the capacity. If Fabric queries the data daily, serve it from a native Delta copy.

```
CREATE OR REPLACE TABLE silver.orders AS
SELECT * FROM bronze.adls_orders;
OPTIMIZE silver.orders;
```

If dev is fast and prod slow, compare source type and cache setting. A cached S3 shortcut in dev hides latency an uncached prod pays. The cache skips files over 1 GB and evicts files unread for 1 to 28 days.

EXPECTED RESULT

Hot queries read Fabric-maintained files in the capacity's region. The shortcut is read once per load.

Go deeper: PySpark Handbook 4.0, small files and OPTIMIZE.

RELATED [1.04](#) Shortcuts point at shortcuts across many... [1.01](#) Your analysts say the Lakehouse SQL endpoint...

1.06 You dropped an unused table during cleanup. Hours later, reports in three other workspaces fail.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Shortcuts are **one-way** and the source never sees them; a shortcut breaks once its target path is gone. Recover with **OneLake soft delete** or item recovery.

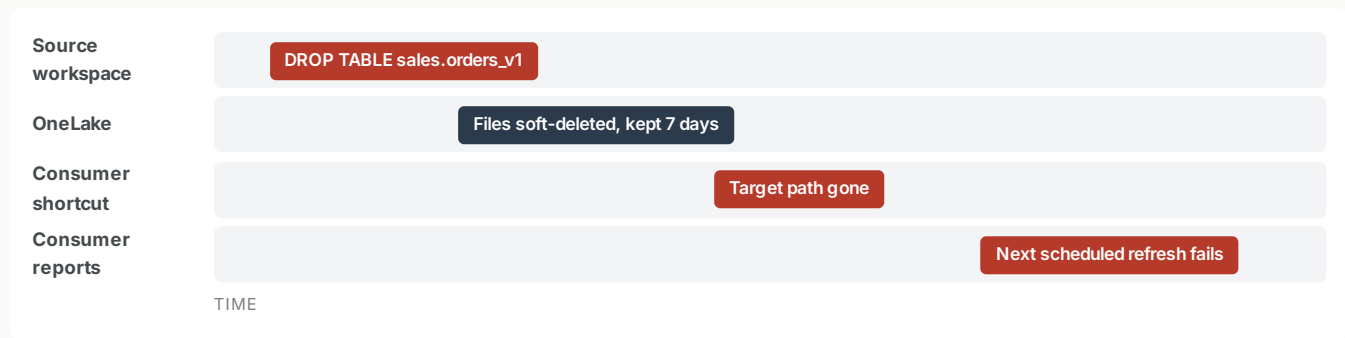
Cost of failure **Outage**Status **GA** · Verified **Sep 2026**

FIG. 1.06 Trace the source drop to the consumer failure hours later in another workspace.

SEEN IN

Consumer workspaces: reports, notebooks, pipelines

CAUSE LIVES IN

Source workspace: a dropped table or deleted item

CAUSE

The shortcut records workspace ID, item ID and path. Dropping the table removes the path. Deleting the Lakehouse removes the item. Lineage shows shortcuts only within one workspace, so the owner's check finds nothing. Recreating the table at the same path restores consumers.

FIX

Dropped table: undelete its folder, including `_delta_log`, from OneLake soft delete within 7 days. Needs write access. Managed-table recovery has documented limits, so rehearse it on a test table first. Deleted item: restore from the workspace recycle bin, kept 3 days by default, up to 90.

```
$ctx = New-AzStorageContext -StorageAccountName onelake -UseConnectedAccount -Endpoint
fabric.microsoft.com
Get-AzDataLakeGen2DeletedItem -Context $ctx -FileSystem SalesWS -Path
'SalesLH.Lakehouse/Tables/sales/orders_v1' |
Restore-AzDataLakeGen2DeletedItem
```

CONFIRM IN BOTH ENGINES

```
-- Before a drop: scan consumers
GET
https://api.fabric.microsoft.com/v1/workspace
s/{ws}/items/{item}/shortcuts
// match target.onelake.itemId and path
```

```
-- After the restore
DESCRIBE HISTORY sales.orders_v1;
```

Deprecate first: announce a sunset date, then drop. The scan needs consumer workspace access or a central registry.

RELATED [1.04](#) Shortcuts point at shortcuts across many... [1.08](#) Reports refresh without errors but count...

1.07 You split Bronze, Silver and Gold into separate workspaces. Every debugging join now starts with a shortcut.

ENGINES PIPELINES DATAFLOW **SPARK** **ONELAKE** WAREHOUSE **SQL ENDPOINT** DIRECT LAKE **SECURITY** CAPACITY

Inside one workspace, T-SQL joins Lakehouses with **three-part names** and Spark reads any Lakehouse by name. Default to **one Lakehouse per layer in one workspace**; separate workspaces give hard access boundaries but need shortcuts.

Cost of failure **Rework**
Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
One Lakehouse, all layers	Schemas <code>bronze</code> , <code>silver</code> , <code>gold</code> in one item. Simplest joins and one set of maintenance jobs.	Item permissions cover every layer. Keeping consumers out of Bronze needs SQL permissions per schema, which apply only through the endpoint, or OneLake security roles.
One Lakehouse per layer, one workspace DEFAULT	Clean item boundaries. Share the Gold item alone. T-SQL joins across layers with three-part names, and Spark reads any layer by name or path.	Workspace Contributors can write every layer. SQL endpoint metadata sync runs as one instance per workspace, so many Lakehouses can add sync lag.
One workspace per layer	Hard boundaries: separate roles, capacity and Git per layer. Fits regulated raw data and separate teams.	T-SQL cross-item queries stop at the workspace edge. Cross-layer joins need shortcuts or Spark four-part names. More deployment pipelines to run.

FIG. 1.07 Three layouts for the same Medallion layers, by where the access boundary sits. Cross-layer joins need shortcuts once that boundary is a workspace.

DEFAULT
One Lakehouse per layer in one workspace. Move to a workspace per layer when a regulator or a separate team needs a hard role boundary.

SIGNS YOU CHOSE WRONG
Debugging starts by creating a shortcut. The same people hold roles in every layer's workspace. Nobody outside engineering reads Bronze or Silver. A regulator requires separate admins for raw data.

CHECK IT
Cross-layer check from the SQL endpoint, with no shortcut.

```
SELECT s.order_id FROM Silver_LH.sales.orders AS s
LEFT JOIN Bronze_LH.sales.orders AS b ON b.order_id = s.order_id
WHERE b.order_id IS NULL;
```

RELATED [1.08](#) Reports refresh without errors but count... [1.04](#) Shortcuts point at shortcuts across many...

1.08 Reports refresh without errors but count duplicates and test rows. They read Bronze directly.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Temporary **Bronze access** became permanent and consumers built on unvalidated tables, so every refresh succeeds with raw data. Expose **Gold only** to anyone outside engineering.

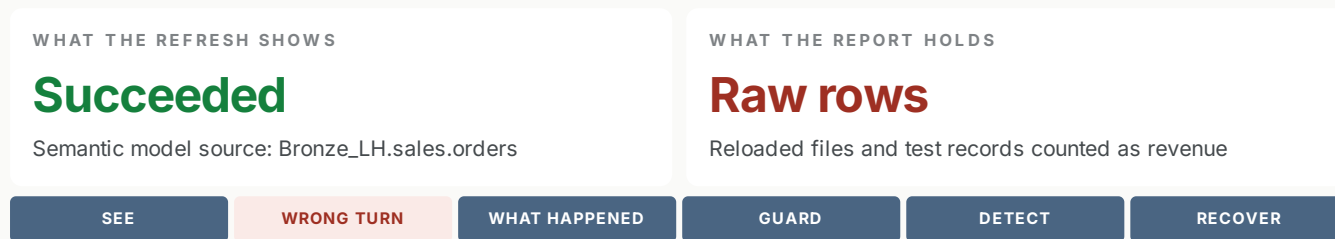
Cost of failure **Wrong data**Status **GA** · Verified **Sep 2026**

FIG. 1.08 The refresh shows Succeeded because its status covers only the source query, whatever the raw rows hold.

WRONG TURN

"They'll move to Gold once it exists. Bronze works for them now."

WHAT HAPPENED

Bronze was shared early to unblock one team. Gold shipped later, but nobody had a reason to repoint working reports. Bronze keeps raw duplicates and bad rows by design. Every Bronze change now needs sign-off from consumers engineering never planned for.

1 GUARD

Give workspace roles, including Viewer, to engineers only. Viewers can query tables through the endpoint when SQL access is granted. Share the Gold item alone, with Read all with SQL analytics endpoint for SQL users.

2 DETECT

Find who queries Bronze. Query insights on the Bronze endpoint keeps 30 days of requests.

```
SELECT login_name, COUNT(*) AS queries,
       MAX(start_time) AS last_seen
FROM queryinsights.exec_requests_history
WHERE command LIKE '%sales.orders%'
GROUP BY login_name;
```

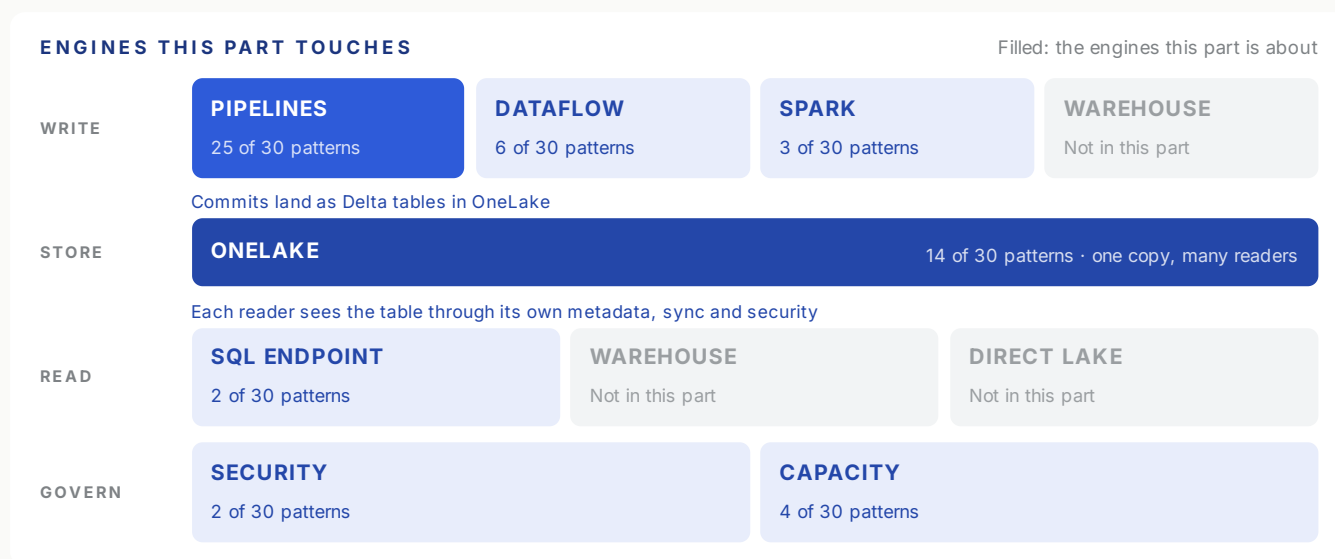
3 RECOVER

Build the Gold table each report needs and repoint the model. Announce a sunset date for Bronze access, then remove the grant. Deliver a minimal Gold table before onboarding the next team.

RELATED [1.07](#) You split Bronze, Silver and Gold into... [1.06](#) You dropped an unused table during cleanup

02 Ingestion and Orchestration.

Pipelines, Copy, Copy job, mirroring, Dataflow Gen2, triggers and schedules. Most of these failures report success while the data is wrong.



NO.	PATTERN	PIPE	DF	SPARK	ONELAKE	SQL EP	CAP	PAGE
2.01	You switched from full load to incremental	●	●	●	●	●	●	44
2.02	You edited a Copy job	●	●	●	●	●	●	45
2.03	Reports on a mirrored database look normal	●	●	●	●	●	●	46
2.04	A copy activity succeeds	●	●	●	●	●	●	47
2.05	Your pipeline copied a CSV into a Lakehouse...	●	●	●	●	●	●	48
2.06	Your copy handled 10 GB in minutes	●	●	●	●	●	●	49
2.07	You parallelised your pipeline	●	●	●	●	●	●	50
2.08	Your ForEach has 1,000 items	●	●	●	●	●	●	51
2.09	Your ForEach sets a variable per item	●	●	●	●	●	●	52
2.10	Lookup returned 5,000 rows	●	●	●	●	●	●	53
2.11	Your scheduled pipeline ran for months	●	●	●	●	●	●	54
2.12	Your pipeline runs once a day	●	●	●	●	●	●	55
2.13	A schedule and a file event start the same...	●	●	●	●	●	●	56
2.14	One file lands in OneLake	●	●	●	●	●	●	57
2.15	Pipeline B must wait for pipeline A	●	●	●	●	●	●	58

Continued on the next page.

NO.	PATTERN	PIPE	DF	SPARK	ONELAKE	SQL EP	CAP	PAGE
2.16	Your on-premises copy fails on some runs...	●	●	●	●	●	●	59
2.17	Your REST API copy fails at about 100 seconds...	●	●	●	●	●	●	60
2.18	You copy CSV files into a Lakehouse table	●	●	●	●	●	●	61
2.19	Dates look right in the source	●	●	●	●	●	●	62
2.20	Your Dataflow Gen2 query looks right in the...	●	●	●	●	●	●	63
2.21	You added one derived column to a Dataflow...	●	●	●	●	●	●	64
2.22	A Dataflow Gen2 that handled 1 million rows...	●	●	●	●	●	●	65
2.23	You set a 2-hour timeout on the Dataflow...	●	●	●	●	●	●	66
2.24	Your Dataflow Gen2 refreshes when you run it	●	●	●	●	●	●	67
2.25	Your incremental Dataflow Gen2 refresh succeeded	●	●	●	●	●	●	68
2.26	An activity failed, retried and succeeded	●	●	●	●	●	●	69
2.27	Your pipeline run succeeded	●	●	●	●	●	●	70
2.28	Your pipeline usually runs for minutes	●	●	●	●	●	●	71
2.29	You pass a load date to your pipeline	●	●	●	●	●	●	72
2.30	Your pipeline ran across midnight	●	●	●	●	●	●	73

2.01 You switched from full load to incremental. Rows that exist in the source never arrive.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

A watermark copies only rows whose column value is **greater than the last one seen**. Rows updated without touching that column, committed late, or holding NULL fall behind it and are never read again.

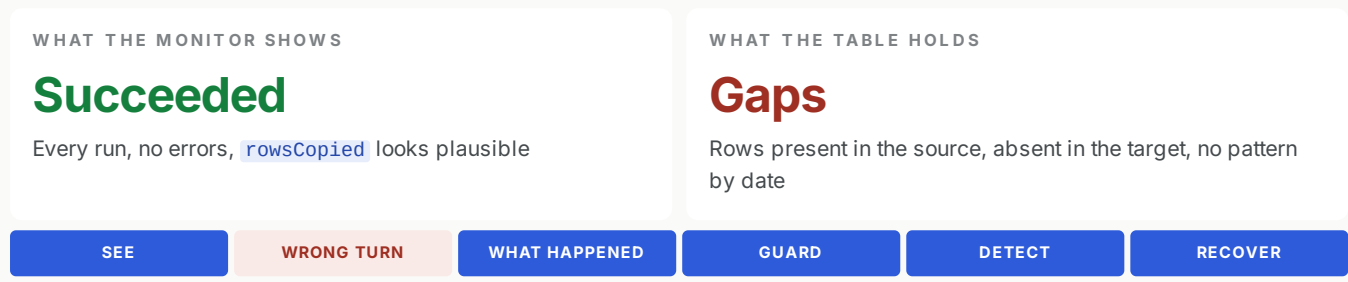
Cost of failure **Data loss**Status **GA** · Verified **Sep 2026**

FIG. 2.01 Every run succeeds while the gap grows.

WRONG TURN

"The watermark logic is right, so the missing rows were never in the source."

WHAT HAPPENED

The full load read every row, so it hid four flaws. The source updates some rows without changing `ModifiedAt`. A long transaction stamps a row at start and commits after the next run has moved the watermark past it. Every incremental load excludes rows with a NULL watermark. Deletes are never seen. Each skipped row stays skipped, because the window only moves forward.

1 GUARD

Re-read an overlap window on every run and write with Upsert on the key, so re-read rows update instead of duplicating.

```
SELECT * FROM dbo.Orders
WHERE ModifiedAt > DATEADD(minute, -30,
  '@{activity('Get watermark').output.firstRow.wm}')
-- Lakehouse sink: Table action Upsert, key OrderID
```

2 DETECT

Run the same daily count on source and target and diff the results.

```
SELECT CAST(ModifiedAt AS date) AS d,
COUNT(*) AS n
FROM dbo.Orders
GROUP BY CAST(ModifiedAt AS date);
```

Go deeper: SQL Handbook, change tracking and CDC.

3 RECOVER

Reload the affected date range with Upsert. For SQL sources, move to a Copy job with CDC: it copies inserts, updates and deletes since the last successful run. On a Copy job watermark, prefer a ROWVERSION column, which changes on every modification.

RELATED [2.02 You edited a Copy job](#) [2.04 A copy activity succeeds](#) [2.27 Your pipeline run succeeded](#)

2.02 You edited a Copy job. The next run copied the whole table again.

[ENGINES](#)
[PIPELINES](#)
[DATAFLOW](#)
[SPARK](#)
[ONELAKE](#)
[WAREHOUSE](#)
[SQL ENDPOINT](#)
[DIRECT LAKE](#)
[SECURITY](#)
[CAPACITY](#)

Some edits, such as changing the incremental column, make a Copy job **reset to a full copy** on the next run. With Append as the write method, every existing row lands twice.

Cost of failure **Wrong data**
Status **GA** · Verified **Sep 2026**

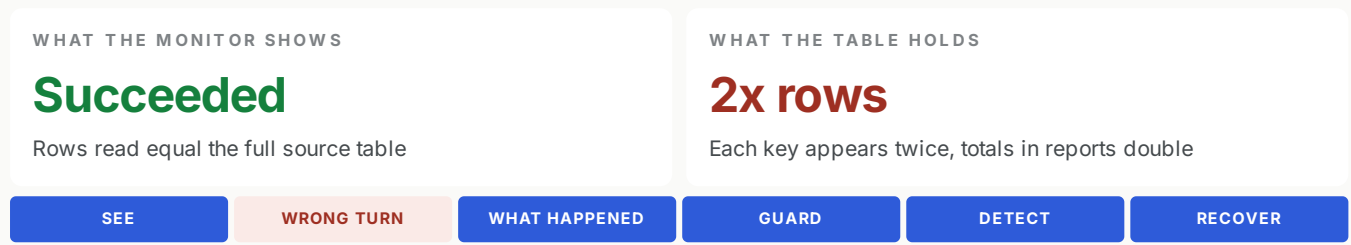


FIG. 2.02 The full read lands on top of the rows already there.

WRONG TURN

"Incremental state is saved, so editing the job only changes what happens next."

WHAT HAPPENED

Copy job tracks the watermark or CDC checkpoint of the last successful run. After certain edits it discards that state to keep source and destination consistent. A manual Reset does the same. Neither touches the destination, so existing rows stay. The next run reads the full range and, with Append, writes it on top.

1 GUARD

Use Merge (upsert) with key columns on incremental Copy jobs, so a full re-read matches existing rows. If Append is required, truncate the destination before the first run after any edit or reset.

2 DETECT

Count duplicate keys after every edit.

```
SELECT order_id, COUNT(*) AS n
FROM sales.orders
GROUP BY order_id
HAVING COUNT(*) > 1;
```

3 RECOVER

Deduplicate on the business key and overwrite. Delta reads a snapshot, so rewriting the same table is safe.

```
df =
spark.table("sales.orders").dropDuplicates([
"order_id"])
df.write.mode("overwrite").saveAsTable("sales.orders")
```

RELATED [2.01](#) You switched from full load to incremental [2.04](#) A copy activity succeeds

[2.27](#) Your pipeline run succeeded

2.03 Reports on a mirrored database look normal. The data stopped changing days ago.

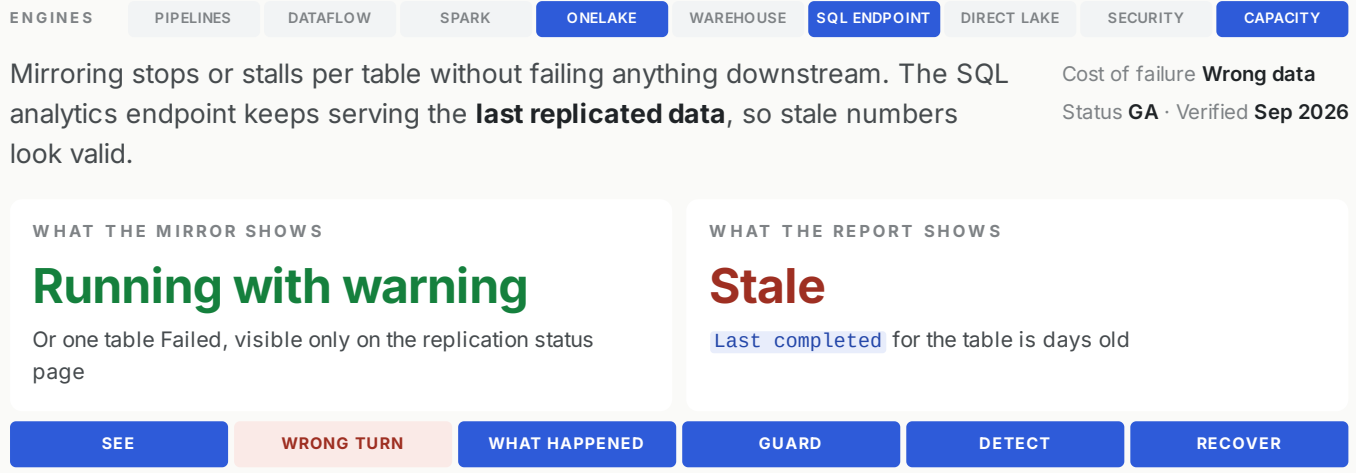


FIG. 2.03 The report stays green while one table's replication has stopped.

WRONG TURN

"Mirroring is continuous, so the endpoint is always current."

WHAT HAPPENED

For Azure SQL Database, any DDL on a mirrored table restarts a full snapshot and reseeds it. Switch partition and altering the primary key are not allowed. Columns of unsupported types, such as xml, json or geography, are skipped. Only the first 1,000 tables are mirrored. Pausing the capacity pauses replication, and it stays Paused after resume until you select Resume replication.

1 GUARD

Poll table status on a schedule and alert when a table is not Replicating or `lastSyncDateTime` is too old.

```
POST
https://api.fabric.microsoft.com/v1/workspaces/{ws}/mirroredDatabases/{id}/getTablesMirroringStatus
# per table: status, metrics.lastSyncDateTime,
# metrics.lastSyncLatencyInSeconds
```

2 DETECT

Open the mirrored database, Monitor replication. Check database status, then each table's status and Last completed. Workspace monitoring logs the same in `MirroredDatabaseTableExecution`.

3 RECOVER

Remove the cause: revert the disallowed DDL, resume the capacity and select Resume replication, or reduce the table list. Stopping and restarting replication resets its state and can reseed tables, so plan it outside reporting hours.

RELATED [2.01](#) You switched from full load to incremental [2.02](#) You edited a Copy job [2.27](#) Your pipeline run succeeded

2.04 A copy activity succeeds. The reconciliation shows a third of the rows missing.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Fault tolerance is on. With **skip incompatible rows** enabled, rows that fail conversion are dropped and counted in `rowsSkipped`, and the activity still succeeds.

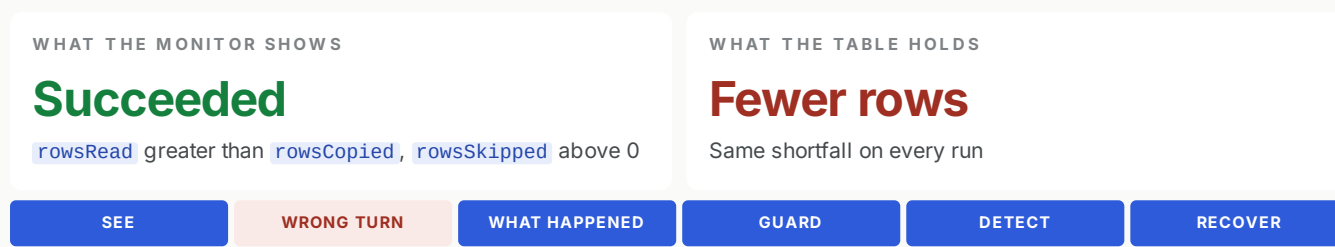
Cost of failure **Data loss**Status **GA** · Verified **Sep 2026**

FIG. 2.04 Compare read, copied and skipped in the output; the monitor status reflects none of them.

WRONG TURN

"The copy succeeded, so the missing rows must be a source problem."

WHAT HAPPENED

By default a copy fails on the first incompatible row. Someone enabled Fault tolerance to get past a failing run. Now a type conversion failure, a column count mismatch or a key violation skips the row instead. The output counts skipped rows in a separate field.

1 GUARD

After each copy, fail the run when read and copied counts differ. Turn fault tolerance off where completeness matters.

```
// If Condition after the copy
@equals(activity('Copy orders').output.rowsRead,
  activity('Copy orders').output.rowsCopied)
// False branch: Fail activity
```

2 DETECT

Enable logging in the copy settings. The log records each skipped row with the reason, so you can see which column failed.

RELATED [2.05](#) Your pipeline copied a CSV into a Lakehouse...

[2.01](#) You switched from full load to incremental

3 RECOVER

Fix the mapping or the source values that fail conversion. Disable fault tolerance and rerun the copy for the affected range with Upsert or Overwrite.

[2.27](#) Your pipeline run succeeded

2.05 Your pipeline copied a CSV into a Lakehouse table. Every column is a string.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Delimited text has **no data types**, so an auto-created table can land every column as string. Set destination types yourself, because numeric conversions can **truncate without error**.

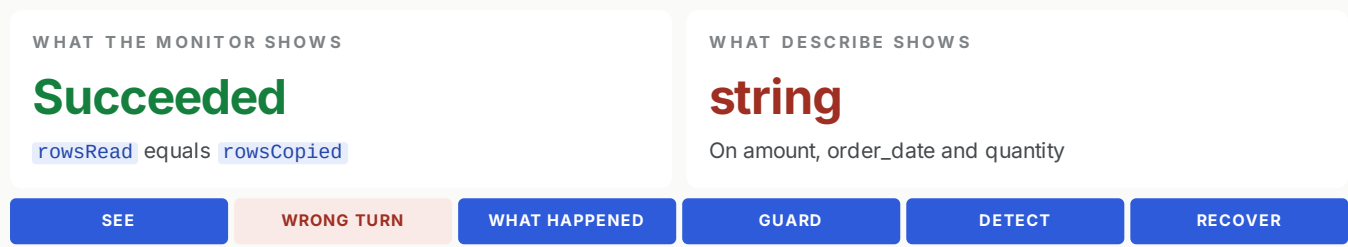
Cost of failure **Wrong data**Status **GA** · Verified **Sep 2026**

FIG. 2.05 Row counts reconcile, but a row check cannot see a column that arrived as text.

WRONG TURN

"Row counts match, so the data landed correctly."

WHAT HAPPENED

Copies from typed sources, such as a Lakehouse table or SQL, map through interim types and keep them. A CSV has only text. In this run every column landed as string. Downstream sums fail or sort as text. Where types are converted, `allowDataTruncation` defaults to true, so decimal to integer drops the fraction without an error.

1 GUARD

In the Mapping tab, select Import schemas and set each destination type. Or create the table first and append into it.

```
CREATE TABLE sales.orders (
  order_id BIGINT, amount DECIMAL(18,2),
  order_date DATE) USING DELTA;
```

2 DETECT

Check types after the first load, before anything consumes the table.

```
DESCRIBE TABLE sales.orders;
```

3 RECOVER

Rewrite with explicit casts. `TRY_CAST` returns NULL on a bad value under either ANSI setting, so count NULLs before and after.

```
CREATE OR REPLACE TABLE sales.orders_t AS
SELECT TRY_CAST(order_id AS BIGINT) AS
order_id,
  TRY_CAST(amount AS DECIMAL(18,2)) AS
amount,
  TRY_CAST(order_date AS DATE) AS order_date
FROM sales.orders;
```

Go deeper: PySpark Handbook 4.0, schemas and casting.

RELATED [2.04](#) A copy activity succeeds [2.27](#) Your pipeline run succeeded [2.02](#) You edited a Copy job

2.06 Your copy handled 10 GB in minutes. At 100 GB it runs for hours on one thread.

ENGINES **PIPELINES** DATAFLOW SPARK ONELAKE WAREHOUSE SQL ENDPOINT DIRECT LAKE SECURITY CAPACITY

Degree of copy parallelism is a **ceiling**. A database source with Partition option None is read by one query, so `usedParallelCopies` stays at 1 whatever you set.

Cost of failure **Slow queries**
Status **GA** · Verified **Sep 2026**



FIG. 2.06 The setting says 32 and the activity output says how many readers ran.

MEASURE FIRST

- 1 Open the copy's output in the Monitoring hub. Compare `usedParallelCopies` with the setting, and note `throughput` and `copyDuration`.
- 2 Check the source's Partition option. None means a single read. For files, check how many files the folder holds.

CAUSE

Parallel copy splits work by partition or by file, so a database source with Partition option None is one unit of work, however high the setting. A long serial read runs until the default 12-hour activity timeout and fails.

FIX

Set Partition option to Dynamic range on an indexed integer or date column. With a query, include the placeholder the service replaces per partition.

```
SELECT * FROM dbo.Sales
WHERE ?DfDynamicRangePartitionCondition
-- Partition column: SaleID, parallelism: 8
```

Each partition is a separate query on the source. High parallelism on an unindexed column can load the source harder than it saves. Bounds set the stride only; rows outside them are still copied.

EXPECTED RESULT

`usedParallelCopies` matches the partitions read, and `copyDuration` drops. Physical partitions of table is the alternative when the source table is already partitioned.

RELATED [2.07](#) You parallelised your pipeline [2.08](#) Your ForEach has 1,000 items

[2.01](#) You switched from full load to incremental

2.07 You parallelised your pipeline. Total runtime stayed the same and new failures appeared.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Parallel activities share the **same limits**: Spark queues notebooks past the capacity's VCores, the source serves every copy at once, and concurrent writes to one Delta table conflict.

Cost of failure **Slow queries**Status **GA** · Verified **Sep 2026**

MEASURE FIRST

- 1 In the Monitoring hub, look for notebook runs in Not started. Pipeline notebooks queue FIFO when Spark VCores are used up (1 CU = 2 VCores, bursting to 3x).
- 2 Compare each copy's **throughput** in parallel with its sequential run. A drop across all of them means the source or network is the bottleneck.
- 3 Check failures for Delta commit conflicts on a shared target table.

CAUSE

Each notebook activity starts its own Spark session unless sessions are shared. Five parallel notebooks on a small capacity queue behind each other and pay startup five times. A pipeline allows at most 100 queued runs.

FIX

Turn on high concurrency for pipelines in workspace Spark settings. Give related notebooks the same session tag; up to 5 share a session. Cap the ForEach batch count at what the source takes. Keep reads parallel and serialise writes to one table.

Sharing a session requires the same user, default lakehouse, Spark configuration and libraries.

EXPECTED RESULT

No notebook sits in Not started, per-copy throughput holds, and runs finish inside the schedule interval.

Go deeper: PySpark Handbook 4.0, sessions and high concurrency.

RELATED [2.08](#) Your ForEach has 1,000 items [2.06](#) Your copy handled 10 GB in minutes

[2.09](#) Your ForEach sets a variable per item

COMPARE

STORAGE

INGESTION

LAKEHOUSE

WAREHOUSE

CROSS-ENGINE

POWER BI

SECURITY

CAPACITY

CI/CD

SCENARIOS

2.08 Your ForEach has 1,000 items. Only 20 ever run at the same time.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

ForEach runs in parallel by default with a **batch count of 20**, maximum 50;
Sequential mode runs one item at a time. Past 50, fan out to child pipelines.

Cost of failure **Slow queries**Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
Sequential	Order matters, or the loop sets variables	Runtime is the sum of every iteration
Parallel, batch count 20 DEFAULT	Most copy and notebook loops	Short iterations wait in waves of 20
Parallel, batch count up to 50	Light activities against a source that scales	Source load and Spark queueing rise with it
Outer ForEach invoking a child pipeline	More than 50 in flight, or nested loops	More runs to monitor; each child counts separately

FIG. 2.08 Batch count sets how many iterations are in flight at once.

DEFAULT

Leave Sequential off and start at 20. Raise toward 50 only while per-item duration holds steady.

SIGNS YOU CHOSE WRONG

Iterations never overlap: Sequential is on. Exactly 20 in progress at any moment. Item durations rise as batch count rises. More than 100,000 items to process.

CHECK IT

Confirm the setting in the pipeline JSON.

```
"type": "ForEach",
"typeProperties": {
  "isSequential": false,
  "batchCount": 20 }
```

RELATED [2.07](#) You parallelised your pipeline [2.09](#) Your ForEach sets a variable per item

[2.10](#) Lookup returned 5,000 rows

2.09 Your ForEach sets a variable per item. In parallel, activities read another item's value.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Pipeline variables are **global to the run**. In a parallel ForEach, every iteration writes the same variable and reads whatever was written last.

Cost of failure **Wrong data**
Status **GA** · Verified **Sep 2026**

	SEQUENTIAL, OR ONE ITEM IN DEBUG	PARALLEL FOREACH IN PRODUCTION
Iterations running	One at a time	Up to 20 at once by default
Writers to the variable	One	Every running iteration
Value the next activity reads	Set by this item	Set by whichever item wrote last
Outcome	Correct file names and targets	Items written to another item's target

FIG. 2.09 With many writers to one variable, the value read depends on timing, so debug with one item passes.

CAUSE

Variables are not thread safe, so parallel iterations overwrite each other between the set and the read. Microsoft answers on the Fabric community forum say a ForEach cannot hold another ForEach.

FIX

Use `@item()` directly in expressions instead of copying it into a variable. When the body needs its own state, move it to a child pipeline and call it with Invoke pipeline, passing `@item()` as a parameter. Each child run has its own variables. The same child pipeline replaces a nested ForEach.

PREVENT

Treat Set Variable and Append Variable inside a parallel ForEach as a review failure. If a variable is unavoidable, set Sequential on and accept the runtime.

RELATED

[2.08 Your ForEach has 1,000 items](#)

[2.07 You parallelised your pipeline](#)

[2.10 Lookup returned 5,000 rows](#)

2.10 Lookup returned 5,000 rows. The source has 12,000.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Lookup returns at most 5,000 rows and **drops the rest**. The ForEach after it processes a partial list and the run succeeds.

Cost of failure **Wrong data**Status **GA** · Verified **Sep 2026**

5,000

Maximum rows a Lookup activity returns. Larger result sets return the first 5,000 rows with no error. Output over 4 MB fails the activity.

Source: Microsoft Learn, Lookup activity, Sep 2026

HOW YOU WILL NOTICE

The ForEach runs exactly 5,000 iterations.

```
@length(activity('Lookup
tables').output.value)
// 5000 means truncated
```

GUARD

Fail the run when the list reaches the cap.

```
@less(length(activity('Lookup
tables').output.value), 5000)
// False branch: Fail activity
```

WORKAROUNDS

Split the query into ranges that each return fewer than 5,000 rows and loop over the ranges. For long lists, write the list to a table and let a notebook or a Copy activity process it.

NEIGHBOURING LIMITS

50

ForEach batch count maximum. Default 20.

100,000

ForEach items maximum.

120

Activities per pipeline, including those inside containers.

RELATED

[2.08](#) Your ForEach has 1,000 items[2.27](#) Your pipeline run succeeded[2.07](#) You parallelised your pipeline

2.11 Your scheduled pipeline ran for months. Then the loads stopped.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

A schedule **expires** when its owner has not signed in to Fabric for 90 consecutive days, and every schedule has a mandatory **end date**. The pipeline's stored token can also expire, and then runs start and fail with a **token error**.

Cost of failure **Outage**Status **GA** · Verified **Sep 2026**

90

A schedule expires when the user who owns it has not signed in to Fabric for 90 consecutive days.

Source: Microsoft Learn, Job scheduler, Sep 2026

HOW YOU WILL NOTICE

The monitor shows a gap in runs, or runs that fail with a token error such as LSROBOTokenFailure or AADSTS700082. Check the owner and end date in the schedule list. Suspect a schedule whose owner left, or a pipeline nobody has saved in 90 days.

GET

```
https://api.fabric.microsoft.com/v1/workspaces/{ws}/items/{pipelineId}/jobs/Pipeline/schedules
# check owner, enabled,
configuration.endDateTime
```

GUARD

Do not wait for a notification when a schedule expires. Alert on missing runs. Run a daily check that compares the last successful run with the expected cadence. It catches expiry, end dates, disabled schedules and token failures. Failure emails arrive only when a run starts and fails.

GET

```
https://api.fabric.microsoft.com/v1/workspaces/{ws}/items/{pipelineId}/jobs/instances
# alert if the newest Completed run
startTimeUtc is older than 1.5 x the
interval
```

WORKAROUNDS

Give production schedules to an owner who signs in, and review ownership when people leave. Set the end date far out, such as 01/01/2099, as Microsoft suggests. When runs fail with a token error, make a small edit to the pipeline, such as its description, and save it to get a new token.

NEIGHBOURING LIMITS

20

Schedules per pipeline

100

Queued runs per pipeline

100

Concurrent jobs per item; more are failed

RELATED [2.12](#) Your pipeline runs once a day [2.15](#) Pipeline B must wait for pipeline A

[2.13](#) A schedule and a file event start the same...

2.12 Your pipeline runs once a day. On some days it runs twice and loads twice.

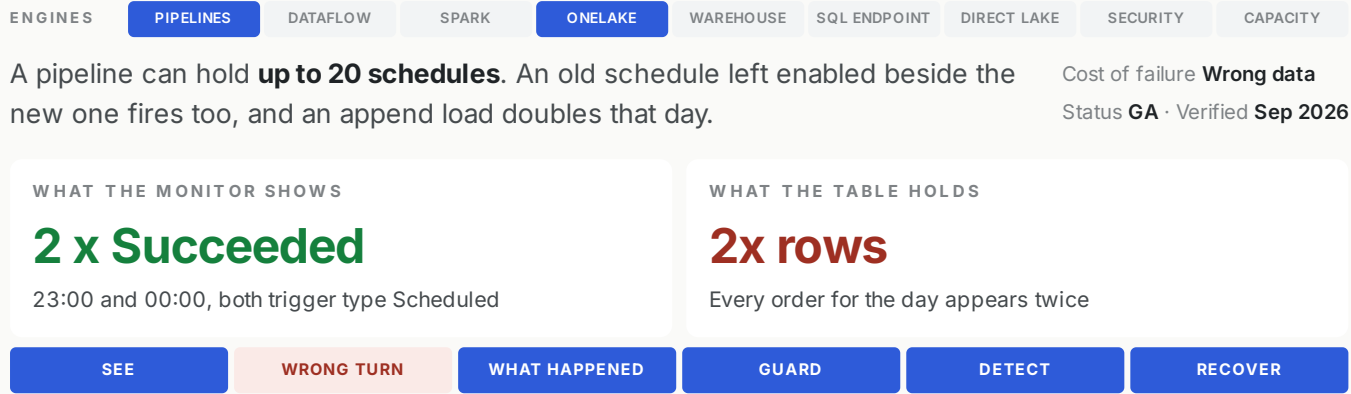


FIG. 2.12 Both schedules fired as set, and each run appended the same rows.

WRONG TURN

"Nobody ran it by hand, so the scheduler fired twice. That is a platform bug."

WHAT HAPPENED

Someone changed the run time by adding a schedule instead of editing the existing one. Both stayed enabled. The 23:00 and 00:00 runs fall on the same local business day. Each appended the same source rows, and both runs show Succeeded.

1 GUARD

Make the load idempotent for its window. Stamp each row with the run id, then replace the window rather than append to it.

```
// Copy additional column load_run_id
@pipeline().RunId
// Load window from the trigger time
@formatDateTime(pipeline().TriggerTime, 'yyyy-MM-dd')
```

2 DETECT

List the schedules through the API or the Schedule pane. More than one enabled entry for a once-a-day pipeline is the cause.

```
GET
https://api.fabric.microsoft.com/v1/workspaces/{ws}/items/{pipelineId}/jobs/Pipeline/schedules
```

3 RECOVER

Delete the rows from the second run, then disable or delete the stale schedule.

```
DELETE FROM sales.orders
WHERE load_date = '2026-09-28'
AND load_run_id = '<second run id>';
```

RELATED [2.11 Your scheduled pipeline ran for months](#) [2.13 A schedule and a file event start the same...](#)

[2.27 Your pipeline run succeeded](#)

2.13 A schedule and a file event start the same pipeline together. One run's output replaces the other's.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Nothing in the two triggers stops both runs executing. Each **overwrites** the same target, and the **last commit wins**.

Cost of failure **Wrong data**Status **GA** · Verified Sep 2026

FIG. 2.13 Run B commits last; Delta keeps its partial load.

SEEN IN

Delta table

CAUSE LIVES IN

Pipeline triggers

CAUSE

The event run reads `@pipeline()?.TriggerEvent?.FileName`. The scheduled run gets null there and reads the whole folder. Delta keeps each commit atomic, so nothing corrupts. The later commit replaces the earlier one.

FIX

Branch on how the run started, so both paths write the same result. Do not count on a concurrency setting to queue the second run.

```
// If Condition: event-triggered run?
@not(empty(pipeline()?.TriggerEvent?.FileName))
// True: load that file. False: load the folder.
```

CONFIRM IN BOTH ENGINES

```
-- Both runs overlap in the monitor
// Monitor > filter the pipeline
// Trigger type: Scheduled and Event, start
times within seconds
```

```
-- Table history shows two overwrites
DESCRIBE HISTORY sales.orders;
-- two WRITE operations, seconds apart
```

Keep one trigger per target. If the event covers arrival, make the schedule a catch-up that exits when the window already loaded.

RELATED [2.14](#) One file lands in OneLake [2.12](#) Your pipeline runs once a day [2.15](#) Pipeline B must wait for pipeline A

2.14 One file lands in OneLake. The event trigger starts the pipeline several times.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

A single write can raise more than one **FileCreated** event. Learn documents one event per `CreateFile` and `FlushWithClose` call for Blob storage, so filter the Activator rule on **api = FlushWithClose** and test it on OneLake.

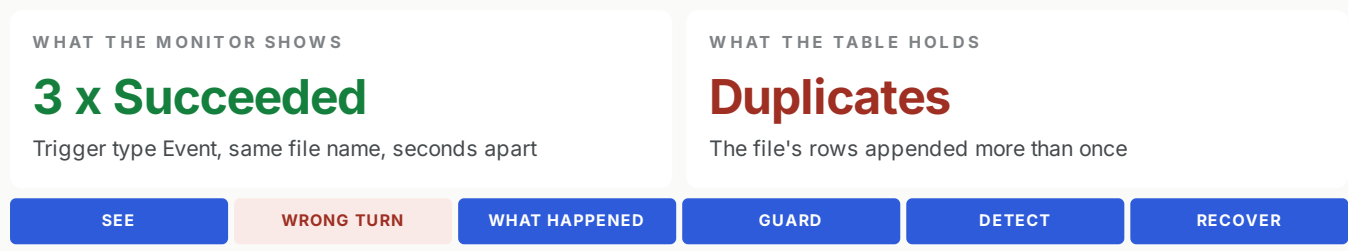
Cost of failure **Wrong data**Status **GA** · Verified **Sep 2026**

FIG. 2.14 Each event from the one file started its own run, and every run appended the same rows.

WRONG TURN

"The source system must be writing the file several times. Raise it with their team."

WHAT HAPPENED

Event triggers are Activator rules on OneLake or Azure Blob events. `FileCreated` fires when a file is created or updated. A writer that creates, then flushes, a file can raise one event per step, such as `CreateFile`. Learn documents this for Blob events; for OneLake it gives only `CreateFile` as an example `api` value. A folder-level rule with no file filter also catches temp files. Each event starts a run.

1 GUARD

Filter the rule on the file suffix and the final commit. Pick the operators that match the suffix and the event type from the list. Record processed files so a repeat run skips them.

```
// Event filters: field, operator, value
subject    ends with  .parquet
data.api    equals     FlushWithClose
```

2 DETECT

Log the trigger's file name in each run, then look for repeats.

```
SELECT file_name, COUNT(*) AS runs
FROM ops.run_log
GROUP BY file_name HAVING COUNT(*) > 1;
```

3 RECOVER

Remove the duplicate loads by run id, or restore the table to the version before the second run.

```
RESTORE TABLE sales.orders TO VERSION AS OF
57;
```

RELATED [2.13](#) A schedule and a file event start the same... [2.27](#) Your pipeline run succeeded

[2.26](#) An activity failed, retried and succeeded

2.15 Pipeline B must wait for pipeline A. On slow days it starts first and reads stale data.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Two schedules at similar times create a race. Call B from A with **Invoke pipeline** and **Wait on completion** on, behind a success path.

Cost of failure **Wrong data**
Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
Invoke pipeline, wait on completion DEFAULT	A calls B on success and waits for B's result. Works across workspaces and shows the child run.	Needs a connection with org account, service principal or workspace identity. The definition schema defaults waitOnCompletion to false, so tick the box and check it in the JSON.
Invoke pipeline, no wait	A finishes early and B runs in parallel.	A shows Succeeded even when B fails.
Activator on A's job event	Decouples teams: B's owner reacts to A's completion without editing A.	Filter on the success status, or B also starts after A fails.
Offset schedule or Wait activity	Nothing to build.	Breaks the first day A runs long, and wastes time when A is quick.

FIG. 2.15 Only the chained option makes B's start depend on A's outcome. Clock-based options start B at a fixed time.

DEFAULT
Remove B's own schedule so only A starts it. Invoke pipeline (Legacy) also chains pipelines, but only within one workspace.

SIGNS YOU CHOSE WRONG
B starts on a clock schedule. A Wait activity sized to A's usual duration. A parent shows success while a child run failed.

CHECK IT
Log the child run id from the activity output. Confirm the property name in a debug run's output first.

@activity('Invoke B').output.pipelineRunId

RELATED

[2.11](#) Your scheduled pipeline ran for months

[2.13](#) A schedule and a file event start the same...

[2.28](#) Your pipeline usually runs for minutes

2.16 Your on-premises copy fails on some runs after a gateway update. The pipeline did not change.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

An admin updates each gateway member separately, so one **cluster member** on a different build makes runs fail depending on which member picks them up. Workspace identity never works through a gateway.

Cost of failure **Outage**
Status **GA** · Verified **Sep 2026**

	RUN THAT SUCCEEDS	RUN THAT FAILS
Cluster member	Updated, current build	Older build, different capabilities
Gateway version	Within the last six releases; 3000.214.2 or later for pipelines	Out of support, or below the pipeline minimum
Connection auth	Org account, service principal, basic	Workspace identity: not supported for gateway connections
Outbound firewall	Gateway reaches *.frontend.clouddatahub.net	Endpoint blocked; pipeline copy cannot move data

FIG. 2.16 The pipeline and connection are unchanged; the failing run landed on a different member.

CAUSE

The cluster load-balances queries across members, so a query can succeed on one member and fail on another.

FIX

Update every member in turn: disable it, wait about 30 minutes for running work, update, re-enable. Use `Update-DataGatewayClusterMember` once members are on the November 2025 baseline. Switch gateway connections from workspace identity to a service principal or account.

PREVENT

Put gateway updates on a monthly calendar so no member falls six releases behind. Allow the outbound pipeline endpoint before the first pipeline uses the gateway.

RELATED

2.01 You switched from full load to incremental

2.07 You parallelised your pipeline

2.17 Your REST API copy fails at about 100 seconds...

2.17 Your REST API copy fails at about 100 seconds every time. The API team sees no timeout.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

The REST source waits **00:01:40** for each HTTP response by default. A page that takes longer to arrive fails the whole copy.

Cost of failure **Outage**Status **GA** · Verified **Sep 2026**

100

Seconds the Copy activity's REST source waits for a response to each request, set by `httpRequestTimeout` (default 00:01:40). It covers getting a response, not reading the data.

Source: Microsoft Learn, Configure REST in a copy activity, Sep 2026

HOW YOU WILL NOTICE

The failed activity's duration sits just past 1 min 40 s. Small pulls from the same API succeed. The clock runs per request, so a paged copy fails on its slowest page.

GUARD

Raise the timeout on the source when the API is slow to assemble a page.

```
"source": {  
  "type": "RestSource",  
  "httpRequestTimeout": "00:05:00"  
}
```

WORKAROUNDS

Ask for smaller pages through the API's page-size parameter and pagination rules, so each response returns sooner. Split a large pull by date range or entity id and run the slices in a `ForEach`.

NEIGHBOURING LIMITS

10 to 60,000 ms

requestInterval between page requests

50

Maximum `ForEach` parallelism (default 20)

24 h

Maximum activity timeout

RELATED [2.16](#) Your on-premises copy fails on some runs... [2.18](#) You copy CSV files into a Lakehouse table

[2.01](#) You switched from full load to incremental

2.18 You copy CSV files into a Lakehouse table. Nulls arrive as empty strings or the text NULL.

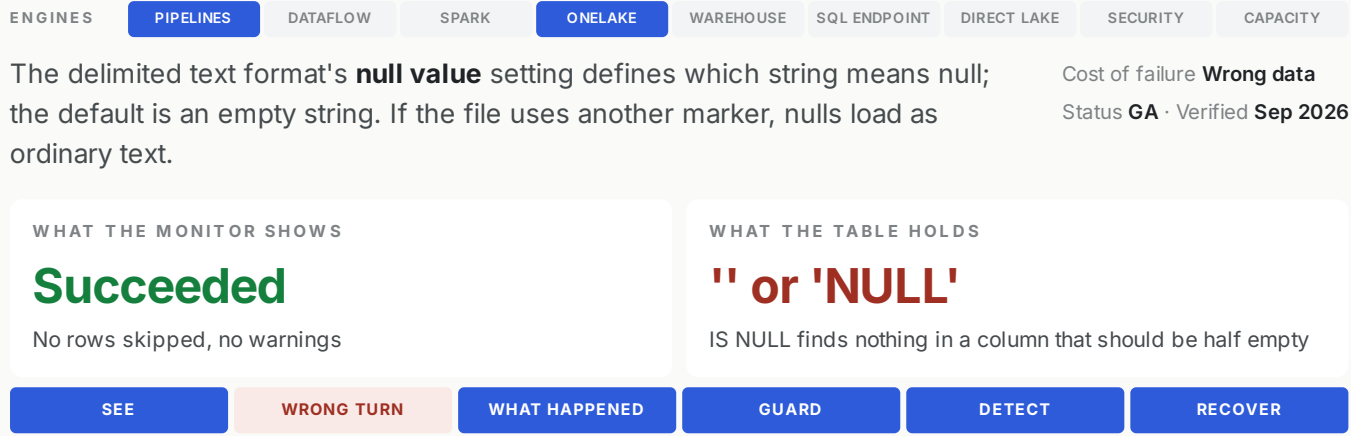


FIG. 2.18 Every row loaded, but the file's null marker differs from the dataset's null value setting.

WRONG TURN

"The source sends empty strings. The copy moves what it gets."

WHAT HAPPENED

A file with empty fields read with null value `NULL` loads empty strings. A file with `NULL` or `\N` markers read with the default loads that text. Relational sources into Delta tables keep nulls, because Delta string columns accept them.

1 GUARD

Set the source dataset's null value to the marker the export uses. Profile a sample of each new feed before the first load.

```
// DelimitedText source dataset
>nullValue": "NULL"
```

2 DETECT

Count each form of empty in the main nullable columns after every load.

```
SELECT COUNT(*) FILTER (WHERE email IS NULL)
AS nulls,
      COUNT(*) FILTER (WHERE email = '') AS
empty,
      COUNT(*) FILTER (WHERE email =
'NULL') AS null_text
FROM bronze.customers;
```

3 RECOVER

Fix the setting and reload. Convert in place only when the source never holds real empty strings.

```
UPDATE bronze.customers
SET email = NULL
WHERE email IN ('', 'NULL');
```

RELATED [2.19](#) Dates look right in the source [2.27](#) Your pipeline run succeeded

[2.17](#) Your REST API copy fails at about 100 seconds...

2.19 Dates look right in the source. In the Lakehouse some are one day earlier.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Copy maps both `DateTime` and **`DateTimeOffset`** to a Delta **timestamp**, which holds an instant with no offset. A local midnight east of UTC becomes the previous evening, and a later cast to date keeps that day.

Cost of failure **Wrong data**Status **GA** · Verified **Sep 2026**

	IN THE SOURCE (SQL SERVER)	IN THE LAKEHOUSE
Column type	<code>datetimeoffset</code>	<code>timestamp</code>
Stored value	2026-03-15 00:30 +02:00	2026-03-14 22:30 UTC
Cast to date	2026-03-15	2026-03-14
date column	<code>date</code>	<code>date</code> , unchanged

FIG. 2.19 Both stored values are the same instant; only the Cast to date row differs.

CAUSE

A timestamp stores UTC. The offset is applied on the way in, then dropped. Spark renders it in the session time zone. Check the default on your runtime with `spark.conf.get("spark.sql.session.timeZone")`. With a UTC session, casting it to a date gives the UTC calendar day.

FIX

Decide which day is the business day. If it is the source's local day, derive it at the source: `CAST(order_dto AS date)` keeps the local date. If you keep the timestamp, convert with the zone explicitly: `to_date(from_utc_timestamp(order_ts, 'Europe/Berlin'))`.

PREVENT

Land a date column beside every timestamp that reports use by day. Check the Spark session zone with `SET spark.sql.session.timeZone`. Compare a sample of source and target dates after each first load.

RELATED [2.18](#) You copy CSV files into a Lakehouse table [2.27](#) Your pipeline run succeeded

2.20 Your Dataflow Gen2 query looks right in the editor preview. The refreshed table holds different rows.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

The preview shows the first rows and runs **preview-only steps**. The refresh reads the **full source** without them, so duplicate keys, nulls and edge cases appear only there.

Cost of failure **Wrong data**
Status **GA** · Verified **Sep 2026**

	IN THE EDITOR PREVIEW	IN THE REFRESH
Rows evaluated	The top of the result; column profiling defaults to the first 1,000 rows	Every row the source returns
Steps set to Enable only in previews	Run, step name in italics	Skipped
Merge keys	Unique in the rows shown	Repeated in the full table, so rows multiply
Filters and type changes	Match the values shown	Meet nulls and formats the preview never held

FIG. 2.20 The editor runs preview-only steps on a slice; the refresh runs every row without them.

CAUSE

Every run skips a step set to Enable only in previews, often a Keep top rows added to speed up authoring. A merge that looked one-to-one on the preview rows fans out when the full right-hand table repeats a key. Version history shows no edit to the query.

FIX

Look for italic step names in Applied steps and remove the preview-only setting from any step that carries logic. Before a merge, group the right-hand query by the key and keep groups with a count above 1. Switch the status bar to Column profiling based on entire data set to see full-data nulls and distinct counts.

PREVENT

Validate the destination table after each refresh, never the preview. Compare its row count with Rows written in the refresh history and fail the calling pipeline when a key repeats.

RELATED

2.28 Your pipeline usually runs for minutes

2.21 You added one derived column to a Dataflow...

2.27 Your pipeline run succeeded

2.21 You added one derived column to a Dataflow Gen2 query. The refresh now takes many times longer.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

The new step **broke query folding**. Every step after it now runs in the dataflow engine on all rows, and **fast copy** no longer applies to that query.

Cost of failure **Cost**Status **GA** · Verified **Sep 2026**

BEFORE

Folding

Last step indicator; the source runs one query

AFTER

Not folding

Rows leave the source, then get transformed

FIG. 2.21 Find the first Not folding step; every step after it runs in the dataflow engine.

MEASURE FIRST

- 1 Read the step folding indicators in Applied steps. The first step marked Not folding is where work leaves the source.
- 2 Compare this run with an earlier one in refresh history: rows read, duration and Engine. Engine shows CopyActivity only when fast copy ran.

CAUSE

Folding turns the steps into one source query. A step the connector cannot translate, such as a custom M function or some text functions, ends folding there. Every later step then runs outside the source, including steps that folded before. Fast copy is disabled when a query adds transformations that do not fold, so data moves by the slower default path.

FIX

Rewrite the column with operations the source can translate, or compute it in a source view. If it cannot fold, split the query: the first lands raw data with fast copy, the second references it and adds the column.

```
// Query Orders references Orders_raw (folds, fast copy)
= Table.AddColumn(Orders_raw, "FullName",
    each [First] & " " & [Last], type text)
```

Folding depends on the connector. The same step can fold against Azure SQL Database and not fold against CSV files in ADLS Gen2.

EXPECTED RESULT

The source query shows Folding to its last step and the refresh history shows CopyActivity again.

RELATED [2.22](#) A Dataflow Gen2 that handled 1 million rows... [2.20](#) Your Dataflow Gen2 query looks right in the...

[2.23](#) You set a 2-hour timeout on the Dataflow...

2.22 A Dataflow Gen2 that handled 1 million rows fails or stalls at 10 million, with a vague error.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

There is no compute size to raise. Find the **non-folding step** that holds every row, then land data with **fast copy**, transform from staging, or move the step to Spark.

Cost of failure **Outage**Status **GA** · Verified **Sep 2026**

MEASURE FIRST

- 1 Open the failed run in refresh history. Under Activities, compare duration, rows read and rows written to find the query that failed or ran longest.
- 2 Download detailed logs for the Mashup engine diagnostics. They stay available for up to 28 days.
- 3 In that query, find merges, group-bys, sorts and pivots placed after the first Not folding indicator.

CAUSE

After folding ends, merges, group-bys, sorts and pivots run in the dataflow engine and need all their input before producing output. A volume that fits at 1 million rows can fail or stall at 10 million. A single query evaluation is also capped at 8 hours, and a whole refresh at 24 hours. The top-level error may not name the step.

FIX

One query ingests with fast copy into staging; a second references it, so merges run on the staging warehouse compute. Filter both sides before a merge. In Dataflow Gen2 (CI/CD), keep Modern query evaluation engine on (Options, Scale). If the join still does not fit, do it in a notebook.

```
o = spark.read.table("stg.orders")
c = spark.read.table("stg.customers")
(o.join(c, "customer_id")
 .write.mode("overwrite").saveAsTable("sales.orders_enriched"))
```

The Modern query evaluation engine covers supported connectors only; other queries use the standard engine. Partitioned compute is in preview. Test at several times today's volume before go-live.

EXPECTED RESULT

The heavy step becomes its own query with rows and duration you can trend, and the ingest query shows CopyActivity.

RELATED [2.21](#) You added one derived column to a Dataflow... [2.23](#) You set a 2-hour timeout on the Dataflow...

[2.20](#) Your Dataflow Gen2 query looks right in the...

2.23 You set a 2-hour timeout on the Dataflow activity. The dataflow kept running for hours.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Activity **timeout and cancellation** reach only Dataflow Gen2 (**CI/CD**). Any other Dataflow Gen2 runs until it finishes or hits the service limits.

Cost of failure **Cost**Status **GA** · Verified **Sep 2026**

8 hours

Maximum evaluation time for one query in a Dataflow Gen2 refresh; the whole refresh stops at 24 hours.

Source: Microsoft Learn, Dataflow Gen2 refresh and Dataflow activity, Sep 2026

HOW YOU WILL NOTICE

The pipeline moves on or reports cancelled while the dataflow's refresh history still shows the run in progress.

GUARD

Upgrade with Save As to Dataflow Gen2 (CI/CD), point the activity at the new item and set its timeout (D.HH:MM:SS). Save As does not copy the refresh schedule.

```
"policy": { "timeout": "0.02:00:00",  
  "retry": 0 }
```

WORKAROUNDS

Until you upgrade, stop a runaway run with Cancel in the workspace or lineage view. Split long dataflows so each query stays well under 8 hours. Passing parameters from the activity also needs CI/CD, and public parameters are in preview.

NEIGHBOURING LIMITS

300

Refreshes per CI/CD dataflow per 24 hours; 150 without CI/CD

50

Staged queries or queries with a destination per dataflow

72 h

All scheduled refreshes failing (6 or more) pauses the schedule

RELATED [2.24](#) Your Dataflow Gen2 refreshes when you run it [2.22](#) A Dataflow Gen2 that handled 1 million rows...

[2.26](#) An activity failed, retried and succeeded

2.24 Your Dataflow Gen2 refreshes when you run it. Triggered from a pipeline, it fails.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

A run needs **Member** or higher on the workspace and access to **every connection** the dataflow uses. You have both; the identity behind the pipeline run may not.

Cost of failure **Outage**

Status **GA** · Verified **Sep 2026**

	MANUAL REFRESH	FROM A PIPELINE
Who runs it	You, signed in	The pipeline's run identity
Workspace role	Member or Admin	Can be Contributor or lower
Connections	Yours, so usable	Usable only if shared with that identity

FIG. 2.24 Read the role and connection rows: the manual run carries the author's access, the pipeline its own.

CAUSE

The manual refresh passes because the author created the connections. A Wait activity or a rerun changes nothing while the pipeline identity lacks that access.

FIX

In Manage connections and gateways, share every connection in the dataflow with the pipeline's run identity, or bind them to a service principal credential. Grant that identity Member on the dataflow's workspace. The activity error can be incomplete, so rerun and read the error in the dataflow's own refresh history.

PREVENT

Own production pipelines and connections with a service account or service principal. Retest the pipeline when ownership changes.

RELATED

[2.23](#) You set a 2-hour timeout on the Dataflow...

[2.29](#) You pass a load date to your pipeline

[2.20](#) Your Dataflow Gen2 query looks right in the...

2.25 Your incremental Dataflow Gen2 refresh succeeded. Rows posted late for last month never loaded.

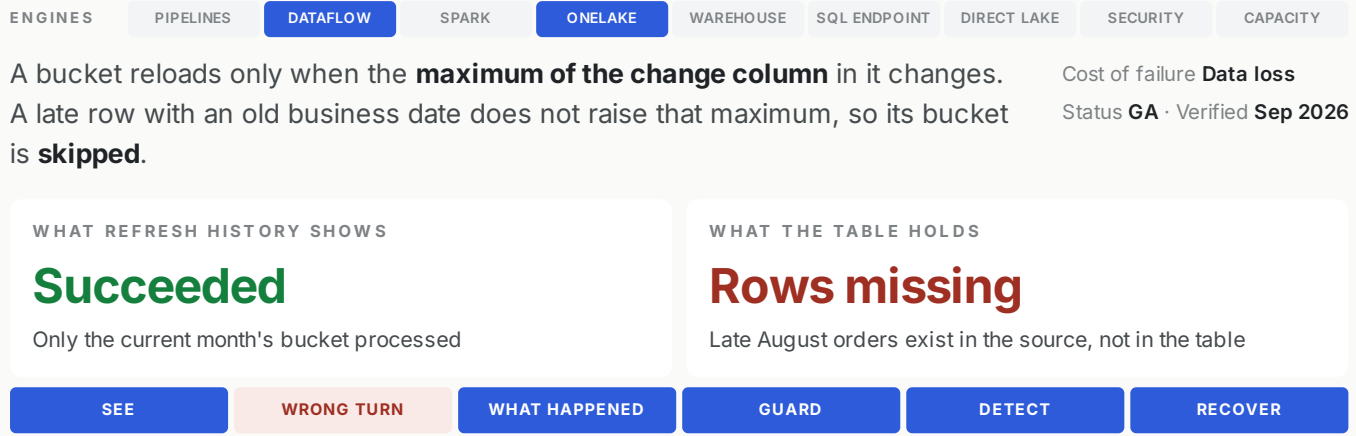


FIG. 2.25 Look at August's maximum: the late row leaves it unchanged, so the refresh never reads the bucket.

WRONG TURN

"Incremental refresh re-reads the recent window on every run, so late rows get picked up."

WHAT HAPPENED

The query filters on OrderDate and also uses OrderDate for Only extract new data when the maximum value in this column changes. Each refresh compares every bucket's maximum with the previous refresh. A row posted today for 28 August sits below August's existing maximum, so August looks unchanged and is not fetched. Rows older than Extract data from the past are never read.

1 GUARD

Detect change on a column the source sets on every insert and update, such as ModifiedAt, not on the business date. Set Extract data from the past to cover your longest posting delay.

2 DETECT

Compare monthly counts in the source and in the destination.

```
SELECT YEAR(OrderDate) AS y,
       MONTH(OrderDate) AS m, COUNT(*) AS n
FROM   dbo.orders
GROUP BY YEAR(OrderDate), MONTH(OrderDate);
```

3 RECOVER

Reload the affected months with a one-off load, then fix change detection. On a Lakehouse destination, do not patch the table from Spark or run OPTIMIZE; incremental refresh tables support neither.

RELATED [2.28](#) Your pipeline usually runs for minutes [2.27](#) Your pipeline run succeeded

[2.20](#) Your Dataflow Gen2 query looks right in the...

2.26 An activity failed, retried and succeeded. The table now holds some rows twice.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

A retry reruns the **whole activity**; if the failed attempt had already committed, an Append writes the same rows again. Make the write **idempotent** before adding retries.

Cost of failure **Wrong data**
Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
Append	Fastest; fine for immutable batches loaded exactly once.	A retry or rerun after a committed write duplicates rows.
Overwrite table or partition	Rerun-safe for full or per-partition reloads.	Replaces the target with whatever the rerun read, including nothing (2.27).
Upsert on key columns DEFAULT	Copy activity matches Lakehouse rows on key columns, updates or inserts; reruns converge.	Needs a unique business key. Partition columns cannot overlap key columns.
Notebook MERGE	Deduplicate the batch, then MERGE; can handle deletes.	More code and a Spark session per run.

FIG. 2.26 Compare what a retry does to the target in each write mode; only idempotent writes land the rows once.

DEFAULT

Upsert on a business key for incremental loads, Overwrite for full reloads. Then tune retries: the default is 1 retry after 30 seconds, with no exponential backoff. For throttling, lengthen the interval rather than the count. Retry conditions, which retry only on chosen errors, are in preview.

SIGNS YOU CHOSE WRONG

Duplicates cluster on runs whose activity shows more than one attempt. Duplicate rows share one load timestamp. Every attempt fails with the same error, so it is not transient.

CHECK IT

Find duplicate keys after a retried run.

```
SELECT order_id, COUNT(*) AS n
FROM sales.orders
GROUP BY order_id
HAVING COUNT(*) > 1;
```

RELATED [2.27](#) Your pipeline run succeeded [2.28](#) Your pipeline usually runs for minutes

[2.25](#) Your incremental Dataflow Gen2 refresh succeeded

2.27 Your pipeline run succeeded. The target table is empty.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

The source returned no rows and the copy **overwrote** the table with them.
Writing zero rows is valid, so the run succeeds.

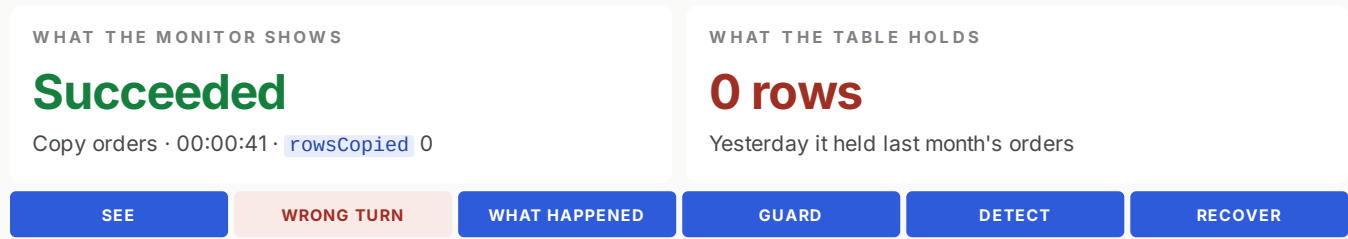
Cost of failure **Data loss**Status **GA** · Verified **Sep 2026**

FIG. 2.27 The run shows Succeeded and the table holds 0 rows: the copy overwrote it with nothing.

WRONG TURN

"Someone dropped the table, or the pipeline wrote somewhere else."

WHAT HAPPENED

The source query returned nothing: a filter that matched no rows, an empty staging folder, or a watermark past the latest data. The copy wrote zero rows in Overwrite mode. Delta replaced the table with an empty version; the run succeeded.

1 GUARD

Fail the run when a write produces no rows.

```
// If Condition after the copy
@greater(activity('Copy orders').output.rowsCopied, 0)
// False branch: Fail activity
```

2 DETECT

The table history shows the empty overwrite.

```
DESCRIBE HISTORY sales.orders;
-- Overwrite, numOutputRows = 0
```

3 RECOVER

Restore the previous version. It stays until VACUUM removes its files, 7 days by default.

```
RESTORE TABLE sales.orders TO VERSION AS OF
41;
```

RELATED [2.28](#) Your pipeline usually runs for minutes [2.26](#) An activity failed, retried and succeeded

[2.01](#) You switched from full load to incremental

2.28 Your pipeline usually runs for minutes. Today it succeeded in seconds and loaded almost nothing.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

An empty Lookup or a null value sent the run down a **do-nothing path**, and every activity succeeded. Check **preconditions** first and fail the run when they are unmet.

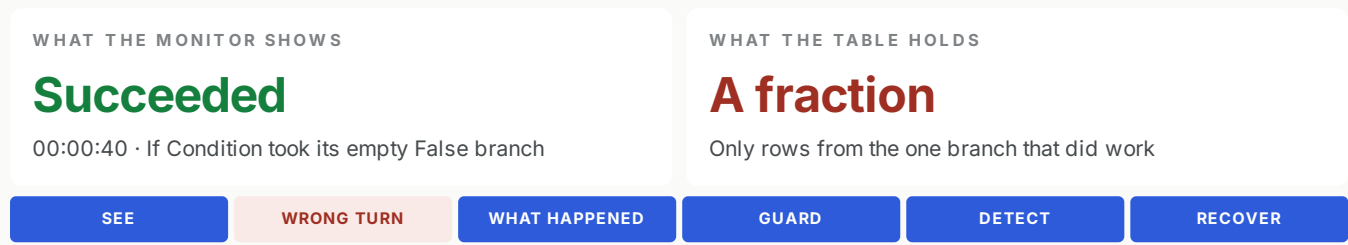
Cost of failure **Wrong data**Status **GA** · Verified **Sep 2026**

FIG. 2.28 A run far shorter than its recent history means a branch did no work.

WRONG TURN

"It ran faster today. The source must have been small."

WHAT HAPPENED

The first Lookup returned no row, or a Set Variable received null. An If Condition evaluated false and took an empty branch. Copies built their query from an empty string and matched nothing. A ForEach over an empty array ran zero iterations. Every activity shows Succeeded; the short duration is the visible difference.

1 GUARD

Make preconditions the first step: look up the source count and required parameters, and fail with a clear message if either is off.

```
// If Condition after Lookup 'Check source'
@and(greater(activity('Check source').output.firstRow.n, 0),
    not(empty(pipeline().parameters.load_date)))
// False branch: Fail activity, code SRC_EMPTY
```

2 DETECT

Pull run durations and flag runs far below their recent baseline.

```
GET
https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}/items/{pipelineId}/jobs/instances
# compare endTimeUtc - startTimeUtc per run
```

3 RECOVER

Fix the precondition, then rerun for the missed window with explicit parameters. If the empty path overwrote data, restore the Delta version as in 2.27.

RELATED [2.27](#) Your pipeline run succeeded [2.29](#) You pass a load date to your pipeline

[2.26](#) An activity failed, retried and succeeded

2.29 You pass a load date to your pipeline. The run uses the parameter's default instead.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Schedules ignore **mismatched parameter names**, so the **default** runs. A variable set from a literal also hides the value you passed.

Cost of failure **Wrong data**
Status **GA** · Verified **Sep 2026**

	MANUAL RUN	SCHEDULED OR INVOKED RUN
Where the value is set	Run dialog, listing current parameter names	Schedule or Invoke pipeline settings, saved earlier
Parameter name	Always current	Must match exactly; a mismatch is ignored
What activities read	@pipeline().parameters.load_date	A variable initialised from a literal

FIG. 2.29 A value binds only to the exact current parameter name; any other name leaves the default in place.

CAUSE

Renaming loadDate to load_date breaks every schedule still passing loadDate. A Set Variable at the start that assigns a hardcoded date has the same effect, because activities read the variable and never the parameter.

FIX

Open each schedule and each Invoke pipeline activity and re-enter values against the current names. Reference the parameter directly in activity settings. Set the default to an empty string and fail the run in a precondition check when it is empty.

PREVENT

Test with a value far from the default, such as 1999-01-01, and read the activity input in the run to see what arrived. Search schedules and callers before renaming a parameter.

RELATED

2.30 Your pipeline ran across midnight

2.28 Your pipeline usually runs for minutes

2.24 Your Dataflow Gen2 refreshes when you run it

2.30 Your pipeline ran across midnight. Half of its files landed in the next day's folder.

Each `utcNow()` is evaluated when **its own activity** runs. Use `pipeline().TriggerTime`, which is fixed for the run, or capture the time once in a variable.

Cost of failure **Wrong data**
Status **GA** · Verified **Sep 2026**

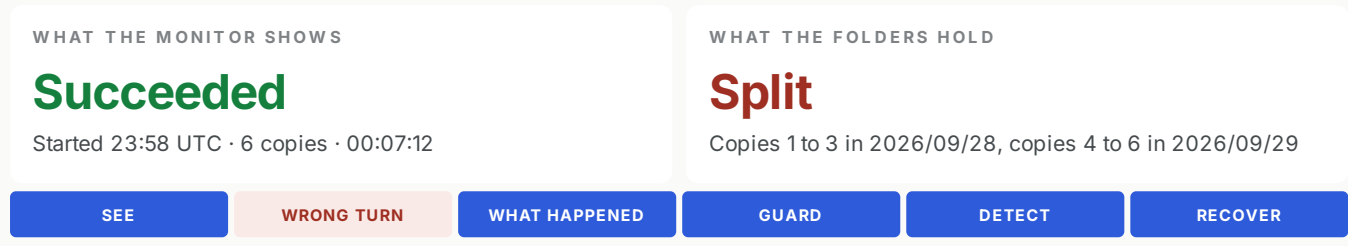


FIG. 2.30 Follow the copies across midnight: each read the clock when it started.

WRONG TURN

"The expression shows the right date when I check it, so something must be caching an old value."

WHAT HAPPENED

Every copy built its path with `formatDateTime(utcNow(), 'yyyy/MM/dd')`. The first copies ran before midnight and the rest after. Checking later shows today's date, which hides the split. Times are UTC, so the split falls at UTC midnight, not local midnight.

1 GUARD

Derive every run date from the trigger time. Interval-based schedules, in preview, also offer Window start time and Window end time.

```
@formatDateTime(pipeline().TriggerTime, 'yyyy/MM/dd')
```

2 DETECT

Look for one run's files spread across two date folders. Each copy's input in the monitor shows the path it resolved.

3 RECOVER

Move the stray files, or rerun with the date passed in. A rerun is safe only if the write is idempotent (2.26).

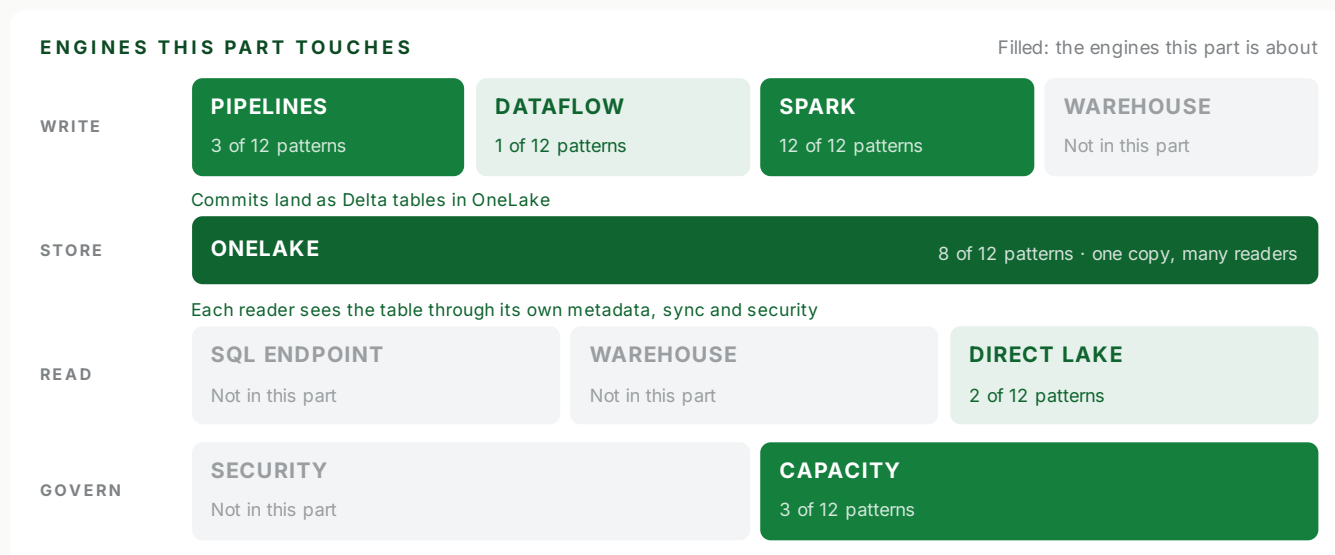
```
@if(empty(pipeline().parameters.run_date),
    formatDateTime(pipeline().TriggerTime,
        'yyyy/MM/dd'),
    pipeline().parameters.run_date)
```

RELATED [2.29](#) You pass a load date to your pipeline [2.26](#) An activity failed, retried and succeeded

[2.28](#) Your pipeline usually runs for minutes

03 Lakehouse, Spark and Delta.

Fabric-specific Spark and Delta behaviour: runtimes, resource profiles, maintenance, deletion vectors, pools. Generic Spark mechanics are in the PySpark Practitioner's Handbook 4.0.



NO.	PATTERN	PIPE	DF	SPARK	ONELAKE	DL	CAP	PAGE
3.01	Your Delta table was fast last month	●	●	●	●	●	●	75
3.02	You deleted half the rows in a Delta table	●	●	●	●	●	●	76
3.03	A pipeline or external engine cannot read...	●	●	●	●	●	●	76
3.04	You merged a few thousand rows into a large...	●	●	●	●	●	●	78
3.05	Your code and data barely changed	●	●	●	●	●	●	79
3.06	Your Spark notebook runs fine in dev	●	●	●	●	●	●	80
3.07	You moved to Runtime 2.0	●	●	●	●	●	●	81
3.08	Two teams write to the same Lakehouse table	●	●	●	●	●	●	82
3.09	Two Lakehouses hold the same data	●	●	●	●	●	●	83
3.10	You restored a Delta table to a good version	●	●	●	●	●	●	84
3.11	The first notebook run of the day takes...	●	●	●	●	●	●	85
3.12	Bronze to Silver reprocesses everything daily	●	●	●	●	●	●	86

3.01 Your Delta table was fast last month. Now it is slow, and nobody changed the notebook.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Every append and merge adds Parquet files, and in Fabric **nothing compacts them unless you enable it.**

Cost of failure **Slow queries**Status **GA** · Verified **Sep 2026**

MEASURE FIRST

- 1 Check file count against table size. Thousands of files averaging a few MB means fragmentation.

```
DESCRIBE DETAIL sales.orders;
-- numFiles, sizeInBytes: average file size = sizeInBytes / numFiles
```

- 2 Count writes and maintenance runs in the history.

```
from delta.tables import DeltaTable
h = DeltaTable.forName(spark, "sales.orders").history()
h.groupBy("operation").count().show() # WRITE, MERGE vs OPTIMIZE
```

CAUSE

Each write adds files sized to that batch. New workspaces default to the `writeHeavy` resource profile: V-Order off and optimize write unset. Auto compaction and fast optimize are opt-in session or table settings. The pipeline Lakehouse Maintenance activity is still in preview. The Delta log checkpoints about every 10 commits, which keeps log replay fast; the bottleneck is file count.

FIX

Turn on auto compaction for tables written in small batches, and schedule `OPTIMIZE` and `VACUUM` for the rest. Set `spark.microsoft.delta.optimize.fast.enabled` to skip compactions that would not help.

```
ALTER TABLE sales.orders
  SET TBLPROPERTIES ('delta.autoOptimize.autoCompact' = 'true');
-- nightly notebook
OPTIMIZE sales.orders;
VACUUM sales.orders; -- 7-day default retention
```

On Runtime 1.3, avoid auto compaction on liquid clustered tables: it triggers full rewrites. Runtime 2.0 reclusters incrementally.

EXPECTED RESULT

File count falls to roughly table size divided by 1 GB, the default `OPTIMIZE` target, and read times return to their earlier level.

Go deeper: PySpark Practitioner's Handbook 4.0, ch. VII, small files and compaction.

RELATED **3.02** You deleted half the rows in a Delta table **3.04** You merged a few thousand rows into a large... **3.05** Your code and data barely changed

3.02 You deleted half the rows in a Delta table. OneLake storage did not go down.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

A Delta **DELETE** only records which rows are gone. Storage falls after **OPTIMIZE**, **VACUUM** and OneLake soft delete have all run their course.

Cost of failure **Cost**Status **GA** · Verified **Sep 2026**

YOU'D EXPECT (SQL SERVER)

"The rows are deleted, so the space they used is free."

IN A FABRIC DELTA TABLE

On tables with deletion vectors, the Runtime 2.0 default, **DELETE** writes a deletion vector and leaves the Parquet files in place. Without deletion vectors it writes new files and keeps the old ones for time travel. Either way storage grows first. **OPTIMIZE** purges files where over 5% of rows are deleted. **VACUUM** then removes unreferenced files older than 7 days. OneLake soft delete keeps them 7 more days, billed as active data.

Do this. Run **OPTIMIZE** then **VACUUM** on a schedule. For compliance deletes, force the rewrite with **REORG TABLE t APPLY (PURGE)**.

RELATED **3.01** Your Delta table was fast last month **3.03** A pipeline or external engine cannot read...

2.27 Your pipeline run succeeded

3.03 A pipeline or external engine cannot read your Fabric Delta table after a delete.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Runtime 2.0 creates tables with **deletion vectors on**, which needs Delta reader version 3. Any reader without deletion vector support rejects the table or reads it incorrectly.

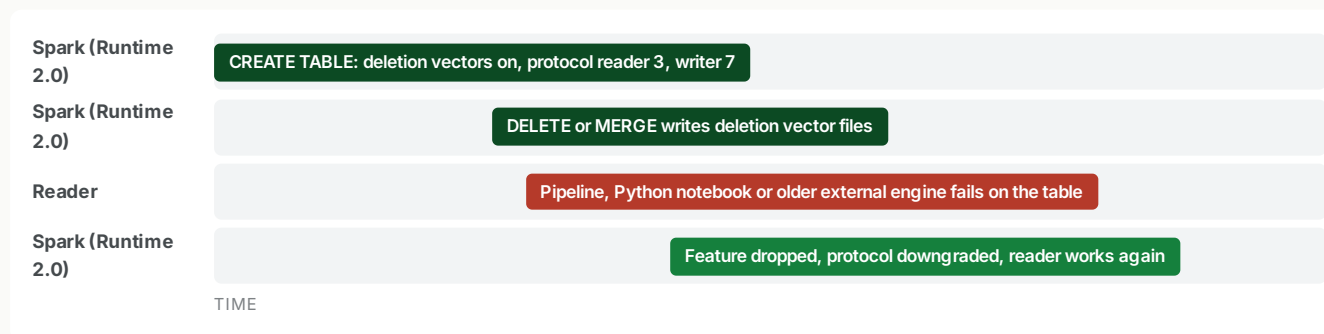
Cost of failure **Outage**Status **GA** · Verified **Sep 2026**

FIG. 3.03 The write succeeds in Spark and the failure surfaces later in another engine.

SEEN IN

Pipelines, Python notebooks or an engine outside
Fabric

CAUSE LIVES IN

Spark Runtime 2.0 table defaults

CAUSE

Runtime 1.3 wrote tables at reader 1, writer 2. Runtime 2.0 enables deletion vectors by default, so new tables need reader 3 and writer 7. The Fabric interoperability matrix lists Pipelines, Eventstreams and Python notebooks as not supporting deletion vectors. External engines on older Delta readers have the same gap. The break appears at the first reader after the switch, often days later. Turning the property off stops new vectors but does not lower the protocol.

FIX

For tables shared with such readers, set the default off in the Environment before creating them. For an existing table, drop the feature. Run it twice, 24 hours apart; it truncates older history. Dropping deletion vectors needs open-source Delta 4.0 or later, and Runtime 2.0 ships 4.2. Test it on a copy first.

```
-- Environment Spark property, before CREATE
SET spark.databricks.delta.properties.defaults.enableDeletionVectors = false;
-- existing table
ALTER TABLE sales.orders DROP FEATURE deletionVectors TRUNCATE HISTORY;
```

CONFIRM IN BOTH ENGINES

```
-- Protocol and features
DESCRIBE DETAIL sales.orders;
-- minReaderVersion 3, tableFeatures contains
deletionVectors
```

```
-- Vectors written
DESCRIBE HISTORY sales.orders;
-- operationMetrics: numDeletionVectorsAdded
> 0
```

Fabric SQL analytics endpoint, Direct Lake, Dataflows Gen2 and KQL databases read deletion vectors.

RELATED **3.02** You deleted half the rows in a Delta table **3.04** You merged a few thousand rows into a large...

3.07 You moved to Runtime 2.0

3.04 You merged a few thousand rows into a large Delta table. It rewrote most of the table.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Without deletion vectors, **MERGE** rewrites **every file holding a matched row**.

Cost of failure **Slow queries**

Tables created on Runtime 1.3 have none, and matches spread across all files when the table is not clustered on the key.

Status **GA** · Verified **Sep 2026**

MEASURE FIRST

- 1 Read the MERGE metrics. Copied rows are unchanged rows rewritten.

```
from delta.tables import DeltaTable
h = DeltaTable.forName(spark, "sales.orders").history(5)
h.selectExpr("version", "operationMetrics['numTargetRowsCopied'] AS copied",
"operationMetrics['numTargetFilesRemoved'] AS files_removed").show()
```

- 2 Check the layout. **DESCRIBE DETAIL** shows **clusteringColumns** and whether **tableFeatures** includes **deletionVectors**.

CAUSE

Copy-on-write works at file level. One matched row makes Spark rewrite the whole file, all columns. With deletion vectors, **MERGE** marks old rows and writes only the changed ones. If keys arrive in random order, matches touch nearly every file.

FIX

Enable deletion vectors once every reader supports them. Cluster on a column the merge can filter on, then add a target predicate on it to the **ON** clause so Spark skips files.

```
ALTER TABLE sales.orders SET TBLPROPERTIES ('delta.enableDeletionVectors' = true);
ALTER TABLE sales.orders CLUSTER BY (order_date);
OPTIMIZE sales.orders;
-- MERGE ... ON t.order_id = s.order_id AND t.order_date >= '2026-09-01'
```

Liquid clustering cannot combine with partitioning, and the Warehouse does not support it. On Runtime 1.3, each **OPTIMIZE** on a clustered table does a full rewrite, so adopt it on Runtime 2.0.

EXPECTED RESULT

numTargetRowsCopied falls towards zero and write volume tracks the changed rows.

Go deeper: PySpark Practitioner's Handbook 4.0, MERGE and copy-on-write.

RELATED **3.03** A pipeline or external engine cannot read... **3.01** Your Delta table was fast last month

3.02 You deleted half the rows in a Delta table

3.05 Your code and data barely changed. The join plan did, and the query now fails or crawls.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Fabric injects **automated table statistics** collected at write time into the optimiser. Deletes, updates and schema changes make them stale, and **ANALYZE TABLE** does not refresh them.

Cost of failure **Slow queries**Status **GA** · Verified **Sep 2026**

MEASURE FIRST

- 1 Check which join Spark chose.

```
q = spark.table("sales.orders").join(spark.table("sales.customers"), "customer_id")
q.explain() # BroadcastHashJoin or SortMergeJoin
```

- 2 Compare the optimiser's size estimate with the real size from **DESCRIBE DETAIL**.

```
df = spark.table("sales.customers")
print(df._jdf.queryExecution().optimizedPlan().stats().toString())
```

CAUSE

Automated table statistics are on by default for Delta tables. They cover the first 32 columns: row counts, nulls, min, max, distinct counts. Spark collects them only when it writes files. Deletes, updates and dropped or added columns leave them stale. On deletion vector tables they can underestimate size, so Spark broadcasts a large table and the query can fail. **ANALYZE TABLE** refreshes catalog statistics only. **COMPUTE DELTA STATISTICS** is Databricks syntax.

FIX

Recompute the extended statistics. On deletion vector tables with frequent updates, stop injecting them into the optimiser and keep collecting.

```
from pyspark.sql.delta import StatisticsStore
StatisticsStore.recomputeStatisticsWithCompaction(spark, "sales.customers")
spark.sql("ALTER TABLE sales.customers SET TBLPROPERTIES ('delta.stats.extended.inject' = 'false')")
```

After a schema change, call **StatisticsStore.removeStatisticsData** before recomputing. **OPTIMIZE** and **REORG TABLE ... APPLY (PURGE)** also refresh statistics for the files they rewrite.

EXPECTED RESULT

The plan shows the expected join strategy and the query completes at its earlier speed.

Go deeper: PySpark Practitioner's Handbook 4.0, join strategies and AQE.

RELATED **3.01** Your Delta table was fast last month **3.04** You merged a few thousand rows into a large...

3.03 A pipeline or external engine cannot read...

3.06 Your Spark notebook runs fine in dev. In production it is many times slower.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Spark settings come from the **workspace and the attached Environment**; the notebook carries only code. Production often runs a different runtime, profile, pool or queue, against tables nobody has maintained.

Cost of failure **Slow queries**Status **GA** · Verified **Sep 2026**

	DEV WORKSPACE	PROD WORKSPACE
Spark properties	Environment with tuned properties attached	Workspace defaults: writeHeavy, V-Order off
Run type	Interactive session starts at once	Scheduled or pipeline run queues FIFO behind other jobs
Runtime	Environment pinned to 2.0	Workspace default, still 1.3 in older workspaces
Native execution engine	Enabled in the Environment	Off unless the Environment enables it
Tables	Small, recently optimised copies	Months of unmaintained appends

FIG. 3.06 Read the two highlighted rows: they usually explain the gap between the workspaces.

CAUSE

The attached Environment sets runtime, pool, resource profile and native engine. Without it, the workspace defaults apply. Only pipeline, scheduler and job definition runs queue, so interactive dev sessions never see it. Queued jobs expire after 24 hours. Larger volumes also expose shuffle and skew problems that small dev samples hide.

FIX

Log the effective settings in both workspaces and diff them. Print `spark.version` and `spark.conf.get(k, None)` for `spark.fabric.resourceProfile`, `spark.native.enabled` and `spark.sql.shuffle.partitions`. In the Monitoring hub, separate queued time from run time. Then check prod tables for small files (3.01).

PREVENT

Deploy the Environment item with the notebook and attach it explicitly. Keep a first cell that logs runtime and key properties on every run.

Go deeper: PySpark Practitioner's Handbook 4.0, shuffle, skew and the Spark UI.

RELATED **3.01** Your Delta table was fast last month **3.07** You moved to Runtime 2.0

3.05 Your code and data barely changed

3.07 You moved to Runtime 2.0. Casts that returned null now fail the job.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Runtime 2.0 runs Spark 4.1, and Apache Spark 4 turns **ANSI mode on by default**. With it on, invalid casts and overflows raise errors instead of returning null or wrapping.

Cost of failure **Outage**
Status **GA** · Verified **Sep 2026**

	RUNTIME 1.3 (SPARK 3.5)	RUNTIME 2.0 (SPARK 4.1)
<code>CAST('a' AS INT)</code>	Returns null	Fails: <code>CAST_INVALID_INPUT</code>
<code>to_date('2026-13-45')</code>	Returns null	Fails with a parse error
INT overflow	Wraps with no error	Fails: <code>ARITHMETIC_OVERFLOW</code>
Environment libraries	Built for Python 3.11	Python 3.13; republish the Environment

FIG. 3.07 Spark 4 fails on invalid values that Runtime 1.3 turned into nulls, and the Python upgrade breaks libraries.

CAUSE

Apache Spark 4 sets `spark.sql.ansi.enabled` to true. Check the Fabric value on 2.0 with `spark.conf.get`. Code that relied on null results to discard bad values now fails at the first dirty row. Runtime 1.3 leaves standard support on 30 September 2026 and gets six months of long-term support to March 2027. Microsoft planned to make Runtime 2.0 the default for new workspaces and Environments in late September 2026.

FIX

Replace casts on untrusted input with `try_cast`, `try_to_date`, `try_to_timestamp` or `try_add`. These return null as before. As a short bridge, set `spark.sql.ansi.enabled` to false in the Environment. Remove and re-add libraries, then publish the Environment.

PREVENT

Clone the Environment on Runtime 2.0 and run jobs against production-shaped data before switching. Count rows where `try_cast` returns null; 1.3 nulled them with no error.

Go deeper: PySpark Practitioner's Handbook 4.0, ANSI mode and Spark 4 migration.

RELATED **3.06** Your Spark notebook runs fine in dev **3.03** A pipeline or external engine cannot read...
3.05 Your code and data barely changed

3.08 Two teams write to the same Lakehouse table. Rows one team loaded are missing.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

One writer runs a **full overwrite** and replaces every file, including the other team's rows, and both runs succeed. When the writes overlap, one can fail with a **concurrency exception** instead.

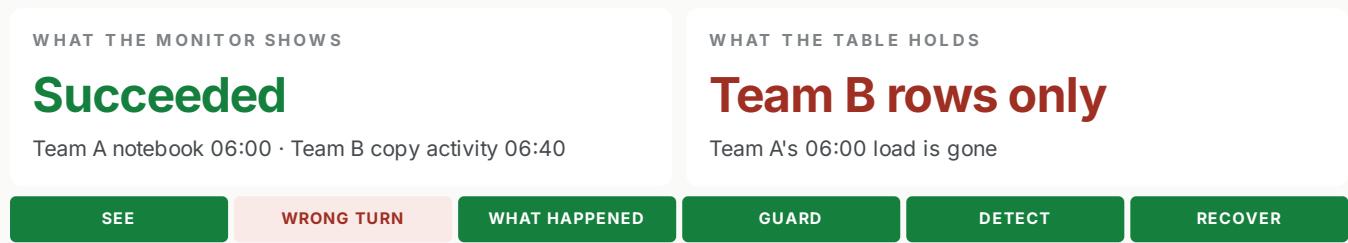
Cost of failure **Data loss**Status **GA** · Verified **Sep 2026**

FIG. 3.08 Both runs show Succeeded, yet the table holds only the last overwrite.

WRONG TURN

"Both jobs succeeded, so OneLake must be losing files."

WHAT HAPPENED

Team B's writer overwrites the whole table: a notebook with `mode("overwrite")`, a copy activity with table action Overwrite, or a Dataflow Gen2 destination set to Replace. Delta commits a new version without Team A's files. When the two writes overlap, optimistic concurrency can detect a conflict and fail one with `ConcurrentAppendException` or `ConcurrentDeleteDeleteException`. Two blind appends never conflict.

1 GUARD

Give each writer its own table or its own partition. `replaceWhere` limits the overwrite and fails if the data falls outside the predicate. Merges must carry the partition column in the ON clause.

```
(df.write.format("delta").mode("overwrite")
  .option("replaceWhere", "team = 'finance'")
  .saveAsTable("sales.orders"))
```

2 DETECT

History shows when the table was overwritten and in which mode.

```
DESCRIBE HISTORY sales.orders;
-- operation WRITE, operationParameters mode
Overwrite, timestamp
```

3 RECOVER

Restore the version before the overwrite, then rerun Team B's load with `replaceWhere`.

```
RESTORE TABLE sales.orders TO VERSION AS OF
57;
```

Go deeper: PySpark Handbook 4.0, Delta concurrency and `replaceWhere`.

RELATED [3.10](#) You restored a Delta table to a good version [2.27](#) Your pipeline run succeeded

[3.09](#) Two Lakehouses hold the same data

3.09 Two Lakehouses hold the same data. Reports on one are much slower.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

New workspaces default to the **writeHeavy** profile with **V-Order off**. Tables written there lack the layout Direct Lake reads fastest.

Cost of failure **Slow reports**Status **GA** · Verified **Sep 2026**

FAST LAKEHOUSE

V-Order on

Older workspace or **readHeavyForPBI** profile

SLOW LAKEHOUSE

V-Order off

New workspace, **writeHeavy** default

FIG. 3.09 The schemas and row counts match; the resource profile at write time decided the Parquet layout.

MEASURE FIRST

- 1 Check the resource profile in a notebook attached to each workspace.

```
spark.conf.get("spark.fabric.resourceProfile")
# 'writeHeavy' on new workspaces
```

- 2 Check the table property on both tables.

```
SHOW TBLPROPERTIES sales.orders;
-- look for delta.parquet.vorder.enabled
```

CAUSE

In **writeHeavy**, **spark.sql.parquet.vorder.default** is false. Plain **OPTIMIZE** inherits the table property or session setting, so it does not add V-Order either. Converting CSV to Parquet under this profile also writes columnar files without V-Order.

FIX

Set the property so future writes use V-Order, then rewrite existing files. Or switch the workspace or session to **readHeavyForPBI**.

```
ALTER TABLE sales.orders SET TBLPROPERTIES ('delta.parquet.vorder.enabled' = 'true');
OPTIMIZE sales.orders VORDER;
```

V-Order slows writes, by about 15% on average. Enable it on read-heavy tables, such as those feeding Direct Lake reports, not on ingestion tables.

EXPECTED RESULT

Both tables report **delta.parquet.vorder.enabled = true** and report visuals on the two Lakehouses respond alike.

Go deeper: PySpark Handbook 4.0, V-Order and file layout.

RELATED **3.08** Two teams write to the same Lakehouse table **4.01** Your query ran in 2 seconds last week

3.12 Bronze to Silver reprocesses everything daily

3.10 You restored a Delta table to a good version. Downstream Gold tables still show bad data.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

RESTORE changes one table. Gold tables built from the bad version keep it, and **watermark** loads never revisit rows already processed.

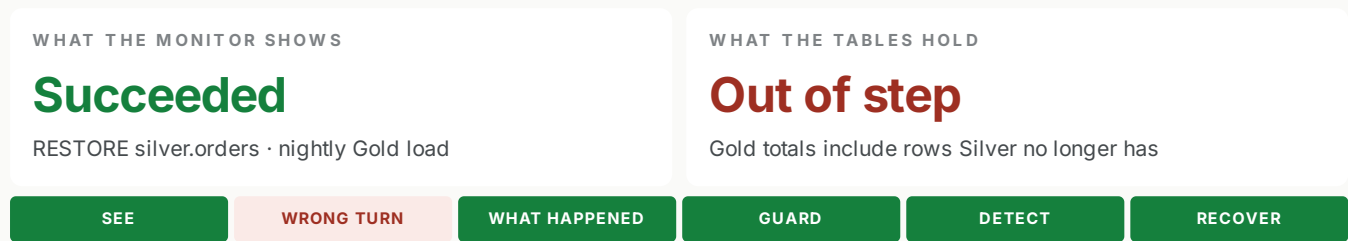
Cost of failure **Wrong data**Status **GA** · Verified **Sep 2026**

FIG. 3.10 Follow the lineage from the restored table: every table downstream loaded in the bad window needs its own rebuild.

WRONG TURN

"Silver is clean now, so tonight's run will fix Gold."

WHAT HAPPENED

RESTORE commits a new Silver version that removes the bad files. Gold tables loaded during the bad window still hold derived rows. Incremental loads read only data after their watermark, so they skip the window. Two Gold tables refreshed at different times now disagree with each other and with Silver. Optimal refresh on a materialized lake view falls back to a full refresh when a source records deletes or updates. A RESTORE may not trigger that fallback, so force a full refresh yourself.

1 GUARD

Make every Gold load replayable for a date range, so a window can be rebuilt on demand.

2 DETECT

Compare each Gold table's write times with the bad window in Silver's history.

```
DESCRIBE HISTORY silver.orders; -- bad
version, RESTORE version
DESCRIBE HISTORY gold.daily_sales; -- loads
inside that window
```

3 RECOVER

Rebuild the affected window in each Gold table. Force a full refresh of any materialized lake view.

```
(df.write.format("delta").mode("overwrite")
  .option("replaceWhere", "order_date >=
'2026-09-20'")
  .saveAsTable("gold.daily_sales"))
-- REFRESH MATERIALIZED LAKE VIEW
gold.sales_summary FULL
```

Go deeper: PySpark Handbook 4.0, time travel and RESTORE.

RELATED [3.08](#) Two teams write to the same Lakehouse table

[3.12](#) Bronze to Silver reprocesses everything daily

[2.27](#) Your pipeline run succeeded

3.11 The first notebook run of the day takes minutes to start. Later runs start fast.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Starter pools start in **seconds**; minutes come from **private link or managed VNet**, custom pools or **Environment libraries**. Later runs reuse a live session.

Cost of failure **Slow queries**Status **GA** · Verified **Sep 2026**

STARTER POOL, NO LIBRARIES

5 to 10 s

Pre-warmed capacity, best effort

PRIVATE LINK OR MANAGED VNET

+2 to 5 min

Starter pools unavailable, cluster built on demand

FIG. 3.11 Networking and libraries stretch the start-time bar; the notebook code sets the run-time bar.

MEASURE FIRST

- 1 Compare session start time with code run time for the first and second run in the Monitoring hub. If start time dominates, the notebook code is not the problem.
- 2 Check workspace network security for tenant Private Link or a managed VNet. Either disables starter pools.
- 3 Check the attached Environment. Quick mode installs libraries at start, 30 s to 5 min. Full mode deploys a snapshot, 1 to 3 min.

CAUSE

The first run pays for cluster creation and library setup. Later interactive runs reuse the session until it expires, 20 minutes idle by default. Pipeline notebooks share a session only under high concurrency mode. A warm-up notebook does not help scheduled runs, which start their own session.

FIX

Remove unused Environment libraries or move them to full mode. For pipelines running multiple notebooks, enable high concurrency and give activities the same session tag. Up to 5 notebooks share one session. Custom live pools with a Full mode Environment keep clusters hydrated and start sessions in about 5 s.

Sharing needs the same user, default Lakehouse, compute settings and libraries. Any difference starts a new session.

EXPECTED RESULT

Monitoring hub shows start times in seconds, or one start per pipeline run instead of one per notebook.

Go deeper: PySpark Handbook 4.0, session lifecycle.

RELATED [6.06](#) Someone renamed a Lakehouse table [3.09](#) Two Lakehouses hold the same data

[3.12](#) Bronze to Silver reprocesses everything daily

3.12 Bronze to Silver reprocesses everything daily. It takes hours when 1 percent changed.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Full reprocess cost grows with table size. Process **only changes**: a watermark, Delta **change data feed**, or a **materialized lake view** that refreshes incrementally.

Cost of failure **Cost**Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
Full reprocess	Simple. Always correct, handles updates and deletes.	Runtime and CU scale with total volume.
Watermark column	Filter Bronze on a modified timestamp stored in a control table.	Misses hard deletes and late rows stamped behind the watermark.
Change data feed DEFAULT	Reads inserts, updates and deletes since the last processed version.	Only covers versions after CDF was enabled and before VACUUM removes change files.
Materialized lake view	Declare Silver in SQL. Fabric picks skip, incremental or full refresh.	Incremental only for append-only sources with CDF on. Any update or delete forces full refresh.

FIG. 3.12 The four options are ordered by how much unchanged data each rereads; pick by whether Bronze ever updates or deletes.

DEFAULT

Enable CDF on Bronze and read changes from the last committed version. Use a materialized lake view where Bronze is append-only.

SIGNS YOU CHOSE WRONG

Runtime grows month on month while daily input stays flat. Most Silver rows rewritten are unchanged. Gold refresh waits on Silver. Bronze is append-only.

CHECK IT

Enable CDF, then read changes since the version stored in your control table.

```
ALTER TABLE bronze.orders SET TBLPROPERTIES (delta.enableChangeDataFeed = true);
SELECT * FROM table_changes('bronze.orders', 42);
```

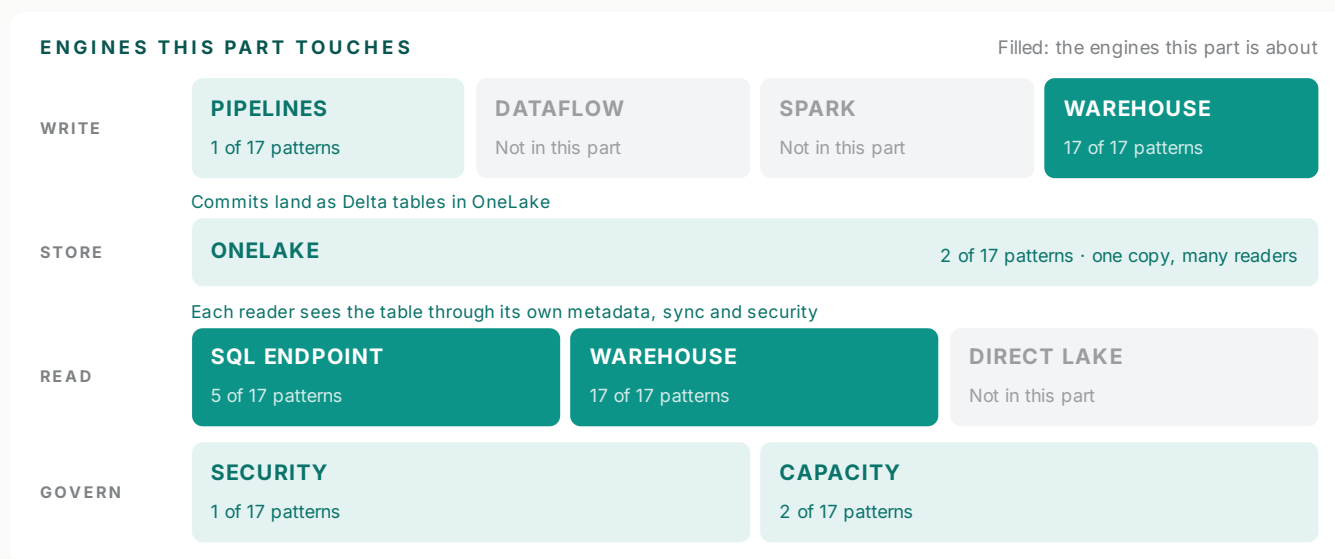
Go deeper: PySpark Handbook 4.0, change data feed and incremental loads.

RELATED [3.10](#) You restored a Delta table to a good version [3.09](#) Two Lakehouses hold the same data

[3.08](#) Two teams write to the same Lakehouse table

04 Warehouse and SQL Endpoint.

Fabric Warehouse and the SQL analytics endpoint as they actually behave, including SQL Server and Synapse habits that do not carry over.



NO.	PATTERN	PIPE	ONELAKE	WH	SQL EP	SEC	CAP	PAGE
4.01	Your query ran in 2 seconds last week							88
4.02	You ported index and partition DDL from SQL...							89
4.03	You declared every string column varchar(MAX)...							89
4.04	You designed hash distribution for your star...							90
4.05	Automatic statistics are on							90
4.06	You added OPTION (RECOMPILE) to a procedure...							92
4.07	ETL loads, Power BI reports and ad hoc users...							92
4.08	One long session holds up every query on a table							93
4.09	Loading row by row into the warehouse takes...							94
4.10	COPY INTO succeeded and row counts match							95
4.11	Your daily dimension MERGE changes a few...							96
4.12	Two procedures update the same table							97
4.13	Reports during the nightly load show new...							98
4.14	You truncated and reloaded a dimension							99
4.15	The dimension has a primary key							100
4.16	A script that runs on SQL Server fails in...							102
4.17	Your cross-warehouse query works in dev							103

4.01 Your query ran in 2 seconds last week. This week it takes 3 minutes. Nothing changed.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Check what changed: the **cache**, the **statistics**, the file layout or the capacity.
Compare the fast run and the slow run in query insights before filing a ticket.

Cost of failure **Slow queries**Status **GA** · Verified **Sep 2026**

LAST WEEK (EXAMPLE)

2 s

result_cache_hit 2 · remote storage 0 MB

THIS WEEK (EXAMPLE)

3 min

result_cache_hit 0 · remote storage 4,800 MB

FIG. 4.01 Compare the cache and scan columns of the two runs; the SQL text is identical.

MEASURE FIRST

- 1 Pull every run of the query from query insights. Rows appear within 15 minutes and stay 30 days.

```
SELECT start_time, total_elapsed_time_ms, allocated_cpu_time_ms,
       data_scanned_remote_storage_mb, result_cache_hit
FROM queryinsights.exec_requests_history
WHERE command LIKE '%FactSales%'
ORDER BY start_time DESC;
```

- 2 Nonzero `data_scanned_remote_storage_mb` means a cold start that read data from OneLake.
`result_cache_hit 2` is a cache hit; 0 means not eligible.

- 3 Check when statistics on the queried columns last changed.

```
SELECT s.name, STATS_DATE(s.object_id, s.stats_id) AS updated
FROM sys.stats AS s
WHERE s.object_id = OBJECT_ID('dbo.FactSales');
```

CAUSE

The result set cache drops an entry unused for 24 hours or when a referenced table changes. After a load or scale event the first run reads from OneLake. Automatic statistics refresh synchronously, inside the first query after a large change. Small writes fragment files until background compaction finishes. Capacity overuse beyond smoothing leads to throttling.

FIX

Fix the column that moved. Refresh statistics as the last load step. Warm the cache with scheduled runs of the main report queries. For throttling, check the Capacity Metrics app and reschedule load.

Never judge a query on its first execution. Compare warm runs.

EXPECTED RESULT

Each regression maps to one measured cause.

Go deeper: SQL Handbook, reading execution plans.

RELATED [4.05](#) Automatic statistics are on [4.07](#) ETL loads, Power BI reports and ad hoc users...

[4.02](#) You ported index and partition DDL from SQL...

4.02 You ported index and partition DDL from SQL Server. The Warehouse rejects it.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Fabric Warehouse accepts **no index, partition or distribution DDL**, apart from **PRIMARY KEY** and **UNIQUE** declared **NONCLUSTERED NOT ENFORCED**. The engine manages file layout, statistics and compaction itself.

Cost of failure **Rework**Status **GA** · Verified **Sep 2026**

YOU'D EXPECT (SQL SERVER / SYNAPSE)

Nonclustered indexes on filter columns, partitions on the date key, and a hash distribution on the join key.

IN FABRIC WAREHOUSE

Partitioned tables and unique indexes are unsupported. **PRIMARY KEY** and **UNIQUE** work only as **NONCLUSTERED NOT ENFORCED**. Data lives in V-Order Parquet, and background compaction merges small files.

Do this. Strip index, partition and distribution clauses from ported scripts. Keep types narrow and create single-column statistics on the main filter and join columns. Data clustering (**CLUSTER BY**) is in preview: see the Watch list.

RELATED [4.04](#) You designed hash distribution for your star... [4.05](#) Automatic statistics are on

[4.03](#) You declared every string column varchar(MAX)...

4.03 You declared every string column varchar(MAX) for flexibility. Queries and caching both suffer.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

In Fabric Warehouse, declared length feeds **cost estimates**, and **varchar(max)** columns block **result set caching**.

Cost of failure **Slow queries**Status **GA** · Verified **Sep 2026**

YOU'D EXPECT (SQL SERVER)

A short value in a MAX column costs the same as in **varchar(100)**. Length is only a ceiling.

IN FABRIC WAREHOUSE

Statistics and cost estimates are more accurate when declared length is close to the data. Queries returning **varchar(max)** are not eligible for the result set cache. **CLUSTER BY** (in preview) rejects MAX columns. Lakehouse **STRING** columns surface as **varchar(8000)** in the SQL analytics endpoint.

Do this. Declare the smallest length that fits, such as **varchar(100)** for names. Keep MAX for genuine large text, up to 16 MB. In Spark, define **varchar(n)** in **CREATE TABLE**; below 2000 it surfaces as **varchar(4*n)**.

RELATED [4.02](#) You ported index and partition DDL from SQL... [4.01](#) Your query ran in 2 seconds last week
[4.05](#) Automatic statistics are on

4.04 You designed hash distribution for your star schema. The Warehouse has no distribution option.

ENGINES PIPELINES DATAFLOW SPARK ONELAKE **WAREHOUSE** SQL ENDPOINT DIRECT LAKE SECURITY CAPACITY

The engine decides data placement and data movement. Your levers are **statistics**, matching join types and **join hints**.

Cost of failure **Rework**

Status **GA** · Verified **Sep 2026**

YOU'D EXPECT (SYNAPSE DEDICATED POOL)

Hash-distribute the fact and large dimensions on the join key, replicate small dimensions, and avoid shuffle moves.

IN FABRIC WAREHOUSE

The optimizer picks broadcast or shuffle per query. Distributed bitmap filters have been generally available since September 2026. Join hints **LOOP**, **HASH**, **MERGE**, **REPLICATE** and **REDISTRIBUTE** are supported.

Do this. Give join columns the same data type on both sides and fresh statistics. When a plan still moves the wrong side, test a hint on that join. **REPLICATE** on an inner join broadcasts the right side: `FROM FactSales AS f INNER REPLICATE JOIN DimProduct AS dp ON f.ProductKey = dp.ProductKey`.

RELATED [4.02](#) You ported index and partition DDL from SQL... [4.05](#) Automatic statistics are on
[4.06](#) You added `OPTION (RECOMPILE)` to a procedure...

4.05 Automatic statistics are on. The first report query after the nightly load is still slow.

ENGINES **PIPELINES** DATAFLOW SPARK ONELAKE **WAREHOUSE** SQL ENDPOINT DIRECT LAKE SECURITY CAPACITY

Fabric creates and refreshes statistics **synchronously**, inside the query that needs them. Refresh them as the **last load step** so no user pays for it.

Cost of failure **Slow reports**

Status **GA** · Verified **Sep 2026**

FIRST QUERY AFTER LOAD (EXAMPLE)

95 s

Includes the statistics refresh

SECOND RUN (EXAMPLE)

6 s

Statistics already current

FIG. 4.05 The first run after the load includes the statistics refresh; the second run does not.

MEASURE FIRST

- 1 Check statistics dates on the filter and join columns against the load end time.

```
SELECT s.name, s.auto_created,
       STATS_DATE(s.object_id, s.stats_id) AS updated
FROM sys.stats AS s
WHERE s.object_id = OBJECT_ID('dbo.FactSales');
```

- 2 Compare the histogram row count with `COUNT(*)`. A large gap means estimates predate the load.

```
DBCC SHOW_STATISTICS ("dbo.FactSales", "stat_FactSales_DateKey");
```

CAUSE

Refresh uses the SQL Server 2016 threshold, so auto stats do fire. Proactive refresh, on by default, front-loads only auto-created `_WA_Sys_` statistics and skips the ones you create. Only single-column statistics exist, so correlated filters such as region and city still misestimate.

FIX

Create statistics on the filter and join columns once, then update them after each commit in the pipeline.

```
CREATE STATISTICS stat_FactSales_DateKey
  ON dbo.FactSales (DateKey) WITH FULLSCAN;
-- final pipeline step, after the load commits
UPDATE STATISTICS dbo.FactSales (stat_FactSales_DateKey) WITH FULLSCAN;
```

For correlated filters, test `OPTION (USE HINT('ASSUME_PARTIAL_CORRELATION_FOR_FILTER_ESTIMATES'))`. After a reload, nonzero `data_scanned_remote_storage_mb` on the first run points to a cold cache.

EXPECTED RESULT

The first user query after the load runs at warm-run speed.

Go deeper: SQL Handbook, statistics and cardinality estimation.

RELATED **4.01** Your query ran in 2 seconds last week **4.06** You added `OPTION (RECOMPILE)` to a procedure...

4.02 You ported index and partition DDL from SQL...

4.06 You added **OPTION (RECOMPILE)** to a procedure with erratic timings. Fabric Warehouse does not support it.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

RECOMPILE, **OPTIMIZE FOR** and **MAXRECURSION** are not among the supported hints, so the SQL Server parameter sniffing toolkit is unavailable.

Cost of failure **Rework**Status **GA** · Verified **Sep 2026**

YOU'D EXPECT (SQL SERVER)

Add **OPTION (RECOMPILE)** or **OPTIMIZE FOR UNKNOWN**, or clear the plan cache with **DBCC FREEPROCCACHE**.

IN FABRIC WAREHOUSE

Supported query hints are **FORCE ORDER**, group and union hints, four **USE HINT** estimation options, **FORCE SINGLE NODE PLAN**, **FORCE DISTRIBUTED PLAN** and **DISABLE_RESULT_SET_CACHE**. **DBCC FREEPROCCACHE** is not documented for Fabric.

Do this. Find which runs are slow in query insights by **query_hash**. Fix the estimate with fresh statistics, then an **ASSUME_*** hint or **FORCE ORDER**. Split wide and narrow ranges into separate procedures if their best plans differ.

RELATED **4.05** Automatic statistics are on **4.04** You designed hash distribution for your star...

4.01 Your query ran in 2 seconds last week

4.07 ETL loads, Power BI reports and ad hoc users share one warehouse. Each slows the others.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Loads and **SELECT**s already run in **separate pools**, so the remaining contention is reads against reads and **table locks** from load DDL. A **warehouse snapshot** shields reports from loads.

Cost of failure **Slow reports**Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
Default isolation DEFAULT	Every workspace splits compute 50/50 into a SELECT pool and a non- SELECT pool. Loads never take read compute.	All readers share one pool, so a heavy ad hoc query slows Power BI.
Custom SQL pools	Up to eight pools per workspace, each with a maximum resource percentage. Queries route to a pool by the application name in the connection string, so reports and ad hoc tools can get their own compute.	Removing a pool fails any query running on it. Each application name maps to one pool only.
Warehouse snapshot for reports	Reports read a fixed point in time while the load runs. Roll it forward when the load commits.	Reports see new data only after the roll-forward step.

FIG. 4.07 Three ways to separate workloads on one warehouse.

DEFAULT

Keep default isolation. Add a snapshot when reports must not see a half-finished load. Consider custom SQL pools for splitting ad hoc readers from reports. Learn still labels them preview, and the September 2026 feature summary lists them GA.

CHECK IT

Roll the snapshot forward as the last pipeline step.

```
ALTER DATABASE [Sales_Snapshot]
SET TIMESTAMP = CURRENT_TIMESTAMP;
```

Go deeper: SQL Handbook, concurrency and isolation.

RELATED [4.08](#) One long session holds up every query on a table [4.01](#) Your query ran in 2 seconds last week

4.08 One long session holds up every query on a table. New queries queue until it ends.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

The usual cause is an **open transaction holding a Sch-M lock** from DDL. Warehouse locks **whole tables**, so one uncommitted **ALTER** or **TRUNCATE** blocks every reader.

Cost of failure **Outage**

Status **GA** · Verified **Sep 2026**

MEASURE FIRST

- 1 List blocked requests and who blocks them.

```
SELECT session_id, blocking_session_id, wait_type,
       total_elapsed_time, open_transaction_count
FROM sys.dm_exec_requests
WHERE blocking_session_id <> 0;
```

- 2 Find sessions holding open transactions. **sleeping** with a count above zero is an idle session keeping its locks.

```
SELECT session_id, login_name, program_name, status,
       open_transaction_count, last_request_start_time
FROM sys.dm_exec_sessions WHERE open_transaction_count > 0;
```

- 3 Confirm the lock mode in **sys.dm_tran_locks**. **Sch-M** on the table is the blocker.

CAUSE

Every transaction runs under snapshot isolation with table-level locks. **SELECT** takes **Sch-S** and DML takes **IX**. DDL such as **CREATE**, **ALTER**, **DROP**, **TRUNCATE** and CTAS takes **Sch-M**, which blocks all reads and writes on the table. Inside **BEGIN TRAN** the lock holds until commit, so a paused script blocks every report.

FIX

End the blocking session. It rolls back uncommitted work and needs the workspace Admin role. To prevent a repeat, keep DDL out of long transactions, commit promptly, and schedule DDL outside reporting hours.

```
KILL 87; -- blocking_session_id from step 1
```

Two concurrent **UPDATE**, **DELETE**, **MERGE** or **TRUNCATE** on one table do not block; they fail with error 24556 or 24706. Retry with backoff.

EXPECTED RESULT

Blocked requests clear when the session ends, and during reporting hours `sys.dm_exec_sessions` shows no sleeping session with open transactions.

Go deeper: SQL Handbook, transactions and locking.

RELATED **4.07** ETL loads, Power BI reports and ad hoc users... **4.01** Your query ran in 2 seconds last week

4.05 Automatic statistics are on

4.09 Loading row by row into the warehouse takes hours. Queries on the table stay slow afterwards.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE**WAREHOUSE**

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Every singleton **INSERT** is a full distributed transaction and writes its own small Parquet file. Load in sets from staged files instead.

Cost of failure **Slow queries**

Status **GA** · Verified **Sep 2026**

MEASURE FIRST

- 1 Count yesterday's singleton inserts and their total time in Query Insights.

```
SELECT COUNT(*) AS inserts, SUM(total_elapsed_time_ms) / 1000 AS secs
FROM queryinsights.exec_requests_history
WHERE command LIKE 'INSERT%VALUES%'
AND start_time > DATEADD(day, -1, GETUTCDATE());
```

- 2 Run a typical report query twice. If the second, warm run is still slow, the table is fragmented.

CAUSE

Each statement pays parsing, planning, distributed execution and commit, which dwarf one row of data. Every committed **INSERT**, **UPDATE** or **DELETE** also writes a new Parquet file. Thousands of tiny files mean fragmented row groups and slow scans. Background data compaction merges small files automatically, and queries stay slow until it catches up.

FIX

Stage rows as files of 100 MB to 1 GB, then load them in one statement with `COPY INTO`. From a Lakehouse table in the same workspace, use `INSERT ... SELECT` or CTAS. Rows that arrive one at a time go to a landing table or file first and move in batches.

```
COPY INTO dbo.sales
FROM 'https://myaccount.dfs.core.windows.net/landing/sales/*.parquet'
WITH (FILE_TYPE = 'PARQUET');
```

`ALTER INDEX ... REORGANIZE` is not available. Compaction waits for user queries holding locks.

EXPECTED RESULT

One load statement per batch, a few large files per load, and report queries that no longer depend on compaction to recover.

Go deeper: SQL Handbook, set-based loading.

RELATED [4.10 COPY INTO succeeded and row counts match](#) [4.11 Your daily dimension MERGE changes a few...](#)

[2.26](#) An activity failed, retried and succeeded

4.10 COPY INTO succeeded and row counts match. The totals are lower than the source.

ENGINES PIPELINES DATAFLOW SPARK **ONELAKE** **WAREHOUSE** SQL ENDPOINT DIRECT LAKE SECURITY CAPACITY

The target column has a smaller scale than the source. `COPY INTO` **truncates** the extra digits without rounding or error, so row counts match and sums drift.

Cost of failure **Wrong data**

Status **GA** · Verified **Sep 2026**

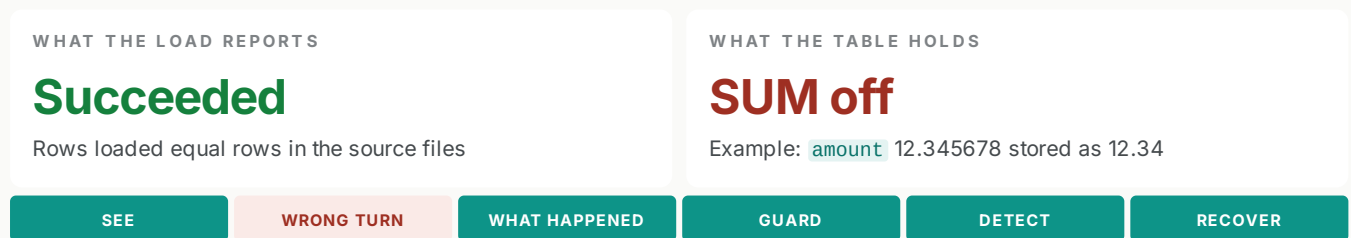


FIG. 4.10 The load status and row count match the source; compare the SUM to see the lost digits.

WRONG TURN

"Row counts match, so the load is fine. The source file must have a quality problem."

WHAT HAPPENED

The source held `DECIMAL(18, 6)` values and the table was created as `DECIMAL(10, 2)`. When the source has greater precision than the target, `COPY INTO` truncates numeric, date and time values. No row fails and nothing is reported. Fabric does not allow `MAXERRORS` with Parquet, so a real conversion error fails the whole load.

1 GUARD

Read the source schema before creating the target. Match decimal scale and time precision exactly.

```
SELECT TOP 10 *
FROM OPENROWSET(BULK 'https://myaccount.dfs.core.windows.net/landing/sales/*.parquet',
  FORMAT = 'PARQUET') AS src;
```

2 DETECT

Compare an aggregate on the files with the same aggregate on the table.

```
SELECT SUM(amount) FROM OPENROWSET(BULK
'https://myaccount.dfs.core.windows.net/land
ing/sales/*.parquet', FORMAT = 'PARQUET') AS
src;
SELECT SUM(amount) FROM dbo.sales;
```

3 RECOVER

The truncated digits are gone from the table. Recreate it with the right type and reload from the source files.

```
DROP TABLE dbo.sales;
CREATE TABLE dbo.sales (order_id BIGINT,
amount DECIMAL(18,6));
COPY INTO dbo.sales FROM
'https://myaccount.dfs.core.windows.net/land
ing/sales/*.parquet'
WITH (FILE_TYPE = 'PARQUET');
```

RELATED [4.09](#) Loading row by row into the warehouse takes... [2.27](#) Your pipeline run succeeded

[4.11](#) Your daily dimension MERGE changes a few...

4.11 Your daily dimension MERGE changes a few hundred rows. It writes the whole table.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

WHEN MATCHED THEN UPDATE with no change test updates **every matched row**, changed or not. Add a change predicate and feed MERGE only changed rows.

Cost of failure **Slow queries**

Status **GA** · Verified **Sep 2026**

MEASURE FIRST**1** See how many rows each MERGE reported.

```
SELECT TOP 5 start_time, row_count, total_elapsed_time_ms
FROM queryinsights.exec_requests_history
WHERE command LIKE 'MERGE%dim_customer%'
ORDER BY start_time DESC;
```

2 Count the rows that differ.

```
SELECT COUNT(*) FROM (
  SELECT customer_id, name, segment FROM stg.customer
  EXCEPT SELECT customer_id, name, segment FROM dbo.dim_customer) AS d;
```

CAUSE

The ON clause decides only which rows match. A plain **WHEN MATCHED** rewrites every matched row, so a full-extract source rewrites the dimension daily.

FIX

Test for change in the MERGE clause. Load only changed rows into staging, from a watermark or hash.

```

MERGE dbo.dim_customer AS t
USING stg.customer AS s ON t.customer_id = s.customer_id
WHEN MATCHED AND (t.name <> s.name OR t.segment <> s.segment)
THEN UPDATE SET name = s.name, segment = s.segment;

```

<> skips NULL changes; handle nullable columns explicitly. MERGE fails if two source rows match one target row, so deduplicate staging. `WHEN NOT MATCHED ... INSERT` is unsupported on tables with IDENTITY; insert new members separately. MERGE takes a table-level lock and conflicts with any concurrent writer (4.12).

EXPECTED RESULT

`row_count` matches the real change count and MERGE time tracks the change rate.

Go deeper: SQL Handbook, MERGE and NULL-safe change detection.

RELATED [4.12](#) Two procedures update the same table [4.14](#) You truncated and reloaded a dimension

[4.09](#) Loading row by row into the warehouse takes...

4.12 Two procedures update the same table. One fails at commit.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Fabric Warehouse runs every transaction under **snapshot isolation** and checks write conflicts **per table**.

Cost of failure **Outage**

Status **GA** · Verified **Sep 2026**

YOU'D EXPECT (SQL SERVER)

Row locks keep writers apart. Two procedures touching different rows of one table both commit.

IN FABRIC WAREHOUSE

UPDATE, DELETE, MERGE and TRUNCATE take a table-level lock. When two transactions modify one table, the later commit fails with error 24556 or 24706, even if the rows differ. `SET TRANSACTION ISOLATION LEVEL` is ignored. Savepoints and named transactions are not supported.

Do this. Run writers to the same table one after another. Wrap each procedure in retry logic with exponential backoff on 24556 and 24706.

RELATED [4.11](#) Your daily dimension MERGE changes a few... [4.13](#) Reports during the nightly load show new...

[4.16](#) A script that runs on SQL Server fails in...

4.13 Reports during the nightly load show new facts with missing customers.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Each ETL step commits on its own, so readers see the warehouse between steps. Point reports at a **warehouse snapshot** and roll it forward once the load finishes.

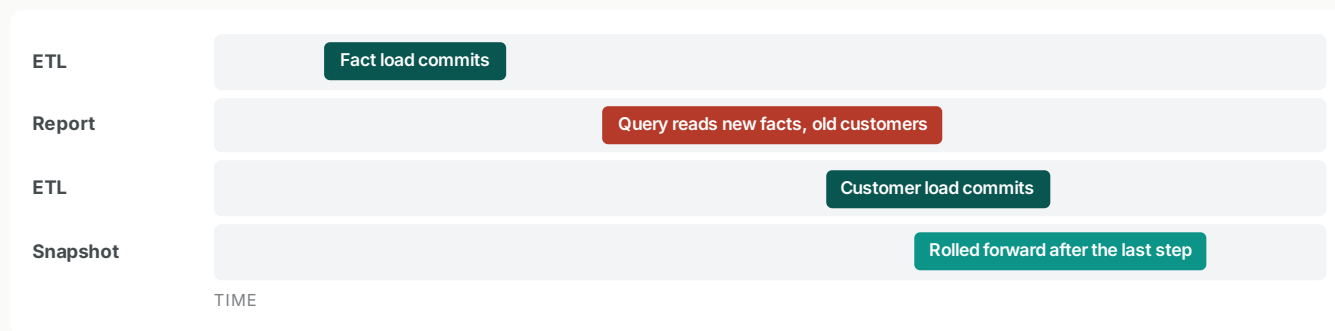
Cost of failure **Wrong data**Status **GA** · Verified **Sep 2026**

FIG. 4.13 The report reads between two ETL commits. The snapshot only moves once every step has committed.

SEEN IN

Report (DirectQuery or Import refresh)

CAUSE LIVES IN

Warehouse ETL

CAUSE

Snapshot isolation protects each transaction separately. Wrapping every step in one transaction holds table locks for the full run and blocks other writers. A warehouse snapshot is a read-only child item frozen at a timestamp. Rolling it forward applies all changes atomically and completes instantly.

FIX

Create a snapshot in the portal (Management, New warehouse snapshot). Connect reports to it. As the pipeline's last step, run this in the parent warehouse.

```
ALTER DATABASE [sales_wh_reporting]
SET TIMESTAMP = CURRENT_TIMESTAMP;
```

CONFIRM IN BOTH ENGINES

```
-- Snapshot lags the warehouse during a load
-- connected to the snapshot
SELECT COUNT(*) FROM dbo.fact_sales;
-- connected to sales_wh
SELECT COUNT(*) FROM dbo.fact_sales;
```

Snapshots do not support Direct Lake; use DirectQuery or Import. The timestamp must fall within the parent's retention period, up to 120 days. Tables changed by DDL after the timestamp become invalid in the snapshot.

RELATED [4.12](#) Two procedures update the same table [4.09](#) Loading row by row into the warehouse takes...

[1.07](#) You split Bronze, Silver and Gold into...

4.14 You truncated and reloaded a dimension. Facts now join to nothing.

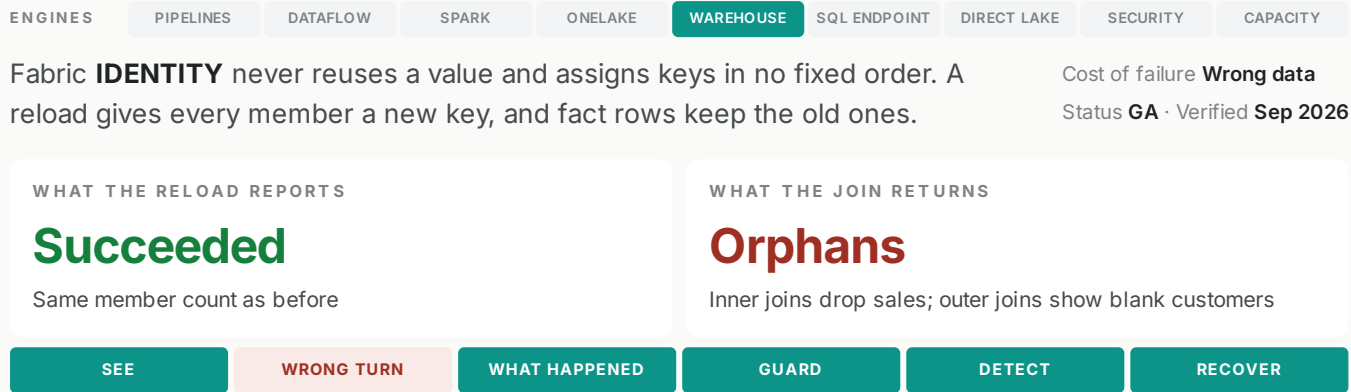


FIG. 4.14 The reload succeeds and the member count is unchanged; run the join to facts to see the keys moved.

WRONG TURN

"TRUNCATE resets IDENTITY to 1, so the keys come back the same."

WHAT HAPPENED

IDENTITY in Fabric is **BIGINT** only, with no seed or increment. A value once used is not handed out again. Values are unique for the table's lifetime but not sequential, because nodes get separate ranges and gaps are normal. Even if TRUNCATE reset the counter, the reload would not hand members their old keys. Duplicate keys appear only when someone uses **IDENTITY_INSERT**. Fact keys now point at values that no longer exist.

1 GUARD

Never reload a keyed dimension. Update changed members, insert new ones. MERGE cannot insert into an IDENTITY table, so insert separately.

```
INSERT INTO dbo.dim_customer (customer_id, name)
SELECT s.customer_id, s.name FROM stg.customer AS s
WHERE NOT EXISTS (SELECT 1 FROM dbo.dim_customer AS d
  WHERE d.customer_id = s.customer_id);
```

2 DETECT

Count fact rows without a dimension member after every dimension load.

```
SELECT COUNT(*) FROM dbo.fact_sales AS f
LEFT JOIN dbo.dim_customer AS d ON
d.customer_sk = f.customer_sk
WHERE d.customer_sk IS NULL;
```

Go deeper: SQL Handbook, surrogate keys.

RELATED **4.15** The dimension has a primary key **4.11** Your daily dimension MERGE changes a few...

4.12 Two procedures update the same table

3 RECOVER

Read the pre-reload keys with time travel, which is in preview. Write them back with `IDENTITY_INSERT`. It needs a column list and works on one table per session. Then run `DBCC CHECKIDENT` with `RESEED`. The warehouse picks the next value itself, and a custom seed value returns an error.

```
SELECT customer_sk, customer_id FROM
dbo.dim_customer
OPTION (FOR TIMESTAMP AS OF '2026-09-
28T01:00:00');
-- after re-inserting old keys:
DBCC CHECKIDENT ('dbo.dim_customer',
RESEED);
```

4.15 The dimension has a primary key. Joins still double the sales.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Warehouse keys are **NOT ENFORCED** metadata. Duplicates load without error, and each duplicate member multiplies its fact rows in a join.

Cost of failure **Wrong data**

Status **GA** · Verified **Sep 2026**

WHAT THE LOAD REPORTS

Succeeded

No constraint violation raised

WHAT THE JOIN RETURNS

2x sales

Example: one customer loaded twice

SEE

WRONG TURN

WHAT HAPPENED

GUARD

DETECT

RECOVER

FIG. 4.15 The declared key says unique; the stored rows hold duplicates and the load shows Succeeded.

WRONG TURN

"The table has a primary key, so `customer_id` must be unique."

WHAT HAPPENED

Fabric accepts `PRIMARY KEY` and `UNIQUE` only as `NONCLUSTERED NOT ENFORCED`, and `FOREIGN KEY` only as `NOT ENFORCED`. Nothing checks the data. A rerun load inserted the same members again. In Synapse dedicated pools the optimizer trusts such keys and returns wrong results when they are violated. Fabric documentation does not say whether its optimizer does the same.

1 GUARD

Keep the key as documentation, and test uniqueness as the last step of every load. Fail the pipeline if the query returns rows.

```
ALTER TABLE dbo.dim_customer ADD CONSTRAINT pk_dim_customer
PRIMARY KEY NONCLUSTERED (customer_sk) NOT ENFORCED;
```

2 DETECT

List key values that occur more than once.

```
SELECT customer_sk, COUNT(*) AS n FROM
dbo.dim_customer
GROUP BY customer_sk HAVING COUNT(*) > 1;
```

3 RECOVER

The rerun inserted identical rows, so rebuild the table from a DISTINCT select. If duplicate rows differ, keep one per key with `ROW_NUMBER()` ordered by a load timestamp.

```
CREATE TABLE dbo.dim_customer_new AS
SELECT DISTINCT customer_sk, customer_id,
name
FROM dbo.dim_customer;
```

Go deeper: SQL Handbook, deduplication with ROW_NUMBER.

RELATED [4.14](#) You truncated and reloaded a dimension [4.11](#) Your daily dimension MERGE changes a few...

[4.10](#) COPY INTO succeeded and row counts match

4.16 A script that runs on SQL Server fails in Fabric Warehouse, or runs and misbehaves.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Fabric Warehouse implements a **subset of T-SQL**; some unsupported commands error, and some appear to succeed. Check the surface-area page before porting.

Cost of failure **Rework**

Status **GA** · Verified **Sep 2026**

	IN SQL SERVER	IN FABRIC WAREHOUSE
Recursive CTE with MAXRECURSION	Walks the hierarchy	Recursive queries not supported
SET TRANSACTION ISOLATION LEVEL	Changes isolation	Ignored; snapshot isolation always applies
##global temp table	Shared across sessions	Not supported; #temp is session-scoped
Scalar UDF with GETDATE()	Runs per row	In preview; non-inlineable UDFs fail over user tables
Triggers, synonyms, SET ROWCOUNT	Supported	Not supported

FIG. 4.16 The same statements on both engines; the highlighted row runs without error in Fabric but does not do what the script intends.

CAUSE

Some unsupported commands fail with generic parser errors that do not name the feature. Scalar UDFs that are not inlineable raise **Scalar UDF execution is currently unavailable in this context**.

FIX

Run the script statement by statement to find the failing one, and check it against the surface-area list. Replace recursion with a loop that inserts one level per pass, or flatten the hierarchy in Spark. Create temp tables as **#name ... WITH (DISTRIBUTION = ROUND_ROBIN)**; the distributed form is recommended and supports **ALTER TABLE**. Check UDFs with **sys.sql_modules.is_inlineable**.

PREVENT

Scan ported code against the unsupported list before the first run, including commands that return no error.

Go deeper: SQL Handbook, hierarchies without recursion.

RELATED **4.12** Two procedures update the same table **4.17** Your cross-warehouse query works in dev
4.14 You truncated and reloaded a dimension

4.17 Your cross-warehouse query works in dev. In production it cannot see the other warehouse.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Three-part names resolve only to items in the **current workspace**. Production split the items across workspaces, or the running identity cannot read the second item.

Cost of failure **Outage**
Status **GA** · Verified **Sep 2026**

	DEV	PRODUCTION
Workspace layout	Warehouse and Lakehouse together	Lakehouse in a separate workspace
Identity running the query	Developer, workspace Contributor	Service principal with access to one item
Query text	lake.dbo.orders	Unchanged

FIG. 4.17 The workspace layout and identity differ between environments; the query text is unchanged.

CAUSE

Cross-database queries reach warehouses, SQL analytics endpoints and mirrored databases in the current active workspace only. The identity also needs access to every item it names. A share grants one item only; the developer had all of them through a workspace role.

FIX

Keep items that query each other in one workspace per environment. For data owned elsewhere, create a OneLake shortcut in a Lakehouse in this workspace and query it through that Lakehouse's SQL analytics endpoint. Give the production identity a workspace role, or Read plus ReadData or T-SQL GRANT on each referenced item. Sharing a warehouse directly with a service principal is not supported.

PREVENT

Test deployed queries with the production identity, not a developer account, and record cross-item dependencies in the deployment checklist.

RELATED

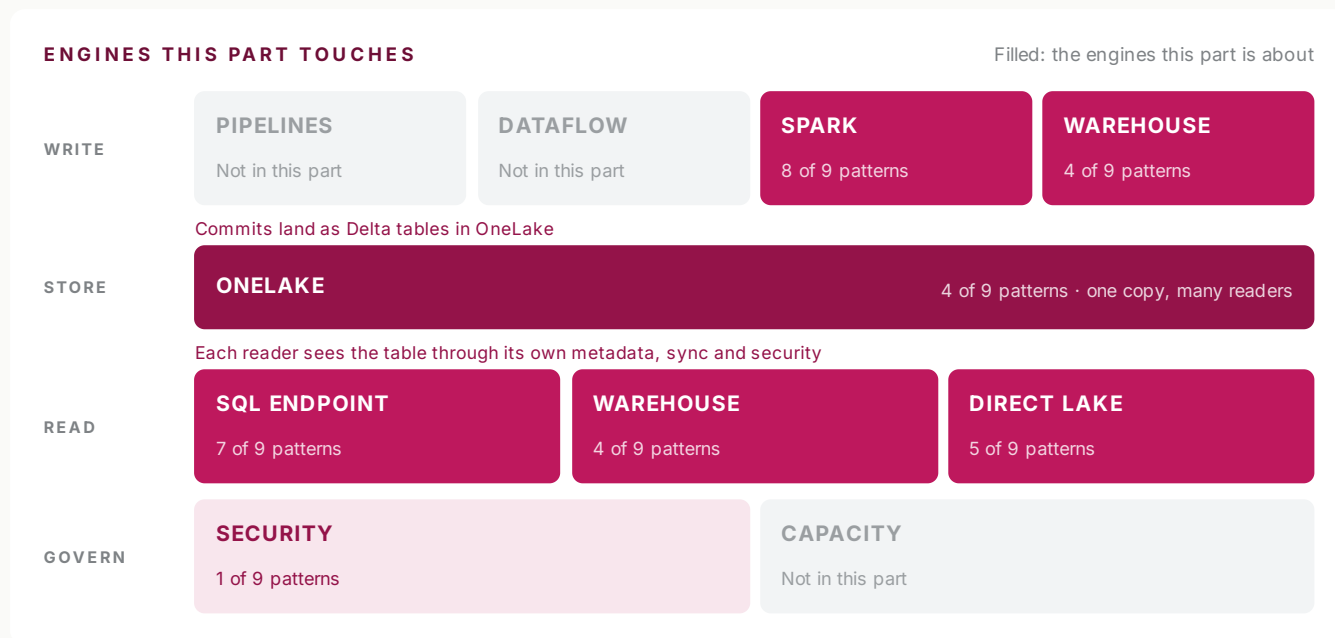
4.16 A script that runs on SQL Server fails in...

4.12 Two procedures update the same table

2.01 You switched from full load to incremental

05 Cross-engine Consistency.

The symptom appears in one engine and the cause lives in another. Spark writes, and the SQL endpoint, the Warehouse or a report sees something different.



NO.	PATTERN	SPARK	ONELAKE	WH	SQL EP	DL	SEC	PAGE
5.01	Spark wrote the rows	●	●	●	●	●	●	105
5.02	The Warehouse shows today's load	●	●	●	●	●	●	106
5.03	The same T-SQL query returns different rows...	●	●	●	●	●	●	107
5.04	A Spark change to a shared table succeeded	●	●	●	●	●	●	108
5.05	Your JSON column parses fine in Spark	●	●	●	●	●	●	109
5.06	Your OneLake shortcut works in a notebook	●	●	●	●	●	●	110
5.07	You added row-level security on the SQL endpoint	●	●	●	●	●	●	111
5.08	Your Gold table is a fraction of Silver's size	●	●	●	●	●	●	112
5.09	A Warehouse query that joins a Lakehouse...	●	●	●	●	●	●	113

5.01 Spark wrote the rows. The SQL endpoint does not show them yet.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

The SQL analytics endpoint picks up Delta commits through a **background metadata sync**, so queries before the sync see the old table. Force the sync with the **refresh metadata API**.

Cost of failure **Wrong data**Status **GA** · Verified **Sep 2026**

FIG. 5.01 The T-SQL check runs before the sync, so it reads the old table.

SEEN IN

Endpoint queries and reports right after a load.

CAUSE LIVES IN

The sync gap after the Spark commit.

CAUSE

The endpoint syncs metadata from the Delta logs in the background, usually within a minute. Many lakehouses per workspace, small files and large change volumes slow it. Sync stops after 15 minutes without queries, so the first query after a quiet spell can be stale.

FIX

Call the **Refresh SQL analytics endpoint metadata** API before any step that reads through the endpoint. It needs Contributor and returns a status per table. Endpoints under the new metadata sync, in preview, also offer per-table `sys.sp_dw_refresh_ext_table`.

```
import sempy.fabric as fabric
ws, ep = fabric.get_workspace_id(), "<sql-endpoint-id>"
r = fabric.FabricRestClient().post(
    f"/v1/workspaces/{ws}/sqlEndpoints/{ep}/refreshMetadata", json={})
print(r.status_code) # 200: per-table status; 202: in progress
```

CONFIRM IN BOTH ENGINES

```
-- Spark
spark.table("silver.orders").count()
```

```
-- SQL endpoint
SELECT COUNT(*) FROM silver.orders;
```

If counts differ, look for status Failure in the response.

RELATED **6.04** The semantic model refresh succeeded **5.06** Your OneLake shortcut works in a notebook
5.04 A Spark change to a shared table succeeded

5.02 The Warehouse shows today's load. Spark, reading the same table through a shortcut, shows older data.

ENGINES PIPELINES DATAFLOW **SPARK** ONELAKE **WAREHOUSE** SQL ENDPOINT DIRECT LAKE SECURITY CAPACITY

Spark reads the Delta log the Warehouse publishes **asynchronously** after each commit. If publishing is **PAUSED**, engines outside the Warehouse stay frozen at the pause point.

Cost of failure **Wrong data**
Status **GA** · Verified **Sep 2026**

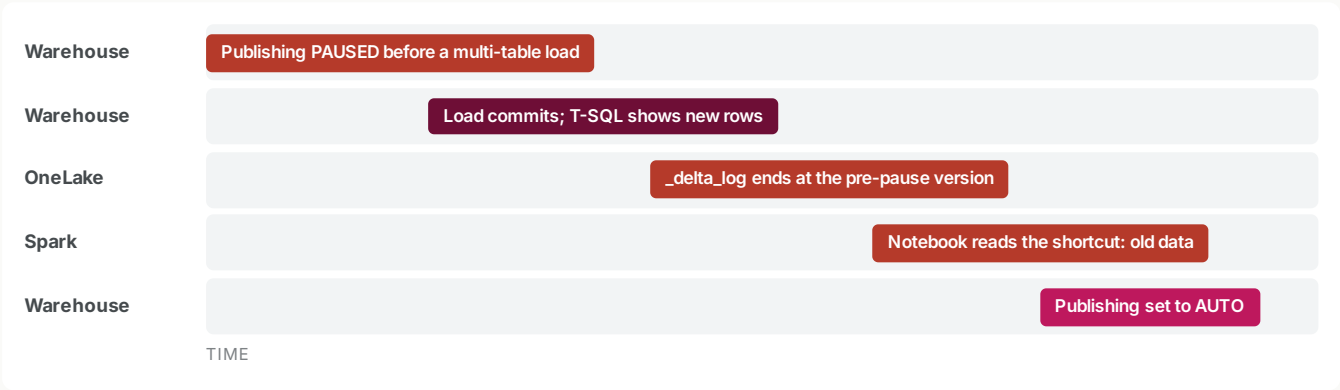


FIG. 5.02 Spark reads the pre-pause data with no error until publishing resumes.

SEEN IN

Spark notebooks and other engines reading Warehouse tables through a shortcut.

CAUSE LIVES IN

The Warehouse: Delta log publishing, delayed or paused.

CAUSE

The publishing delay has no documented upper bound. The longer gap is a pause set for a multi-table load and never resumed.

FIX

Resume publishing in the same job that paused it. AUTO publishes all pending changes. If Spark must see a commit at once, read through the Warehouse Spark connector, which queries the SQL engine.

```
SELECT [name], [DATA_LAKE_LOG_PUBLISHING_DESC] FROM sys.databases;  
ALTER DATABASE CURRENT SET DATA_LAKE_LOG_PUBLISHING = AUTO;
```

CONFIRM IN BOTH ENGINES

```
-- Spark through the shortcut  
spark.table("lh_sales.dim_customer").count()
```

```
-- Spark through the Warehouse connector  
import com.microsoft.spark.fabric  
spark.read.synapsesql("wh_sales.dbo.dim_customer").count()
```

The connector matches T-SQL at once; the shortcut matches after publishing.

RELATED **5.01** Spark wrote the rows **5.03** The same T-SQL query returns different rows...
5.09 A Warehouse query that joins a Lakehouse...

5.03 The same T-SQL query returns different rows in the Warehouse and in the Lakehouse SQL endpoint.

ENGINES PIPELINES DATAFLOW **SPARK** ONELAKE **WAREHOUSE** **SQL ENDPOINT** DIRECT LAKE SECURITY CAPACITY

A Warehouse and a Lakehouse SQL endpoint can have **different collations**. Cost of failure **Wrong data**
When one is **case-insensitive**, 'UK' and 'uk' match only there, so filters, Status **GA** · Verified **Sep 2026**
DISTINCT and GROUP BY disagree.

	CASE-INSENSITIVE ITEM	CASE-SENSITIVE ITEM (BIN2)
WHERE country = 'uk'	Matches UK, uk and Uk	Matches only uk
COUNT(DISTINCT country)	UK and uk count once	UK and uk count twice
ORDER BY name	Case ignored	All uppercase before lowercase
Spark on the same data	Disagrees with this item	Agrees: Spark compares strings case-sensitively by default

FIG. 5.03 The hot rows are the ones that change counts in a reconciliation.

CAUSE

Each Warehouse and SQL analytics endpoint takes the workspace default collation when it is created. That default is `Latin1_General_100_BIN2_UTF8`, which is case-sensitive. A workspace admin can switch it to `Latin1_General_100_CI_AS_KS_WS_SC_UTF8`, which is case-insensitive. The switch affects only items created afterwards. A Warehouse can also get the case-insensitive collation through the REST API. An item's collation cannot be changed after creation. The data types page lists Lakehouse string columns as BIN2, and the collation page says endpoints follow the workspace default. Check yours with `sys.columns`. Queries across items with different collations can fail or return unexpected results.

FIX

Normalise case where the data is written, for example `upper(trim(country))` in Spark, so both collations return the same rows. For a one-off reconciliation, force the collation in the query: `WHERE country COLLATE Latin1_General_100_CI_AS_KS_WS_SC_UTF8 = 'uk'`. The COLLATE clause in a SELECT is unrestricted.

PREVENT

Pick one collation per workspace before creating any Warehouse or Lakehouse. Before a migration or reconciliation, check each item with `SELECT name, collation_name FROM sys.databases;` and the string columns with `SELECT name, collation_name FROM sys.columns;`

RELATED **5.01** Spark wrote the rows **5.02** The Warehouse shows today's load
5.09 A Warehouse query that joins a Lakehouse...

5.04 A Spark change to a shared table succeeded. The SQL endpoint lost columns, or the whole table.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Spark accepts Delta features other engines cannot read. **Column mapping by ID, V2 checkpoints, TIMESTAMP_NTZ** and Delta 4.x features hide columns or tables from the endpoint.

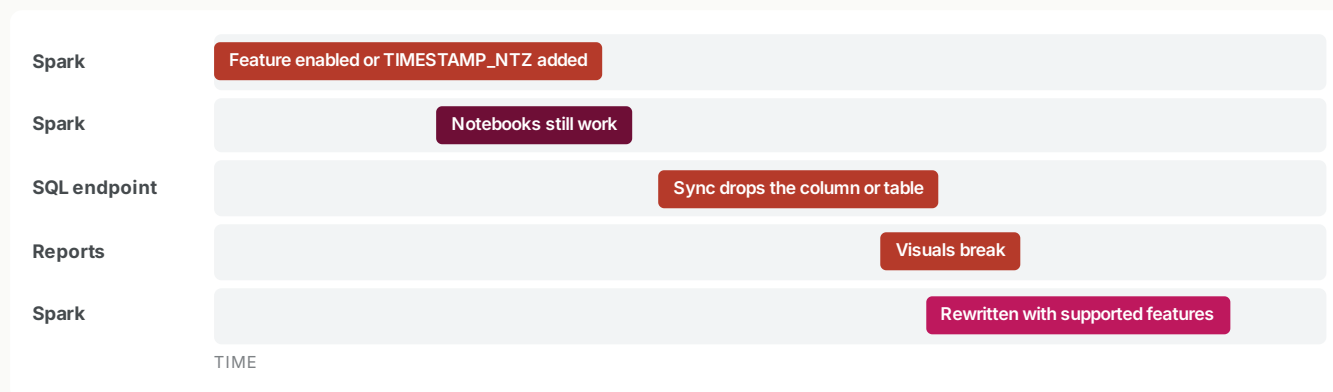
Cost of failure **Outage**Status **GA** · Verified **Sep 2026**

FIG. 5.04 Spark succeeds; the endpoint loses the column or table at the next sync.

SEEN IN

SQL endpoint, cross-database queries, Direct Lake on SQL models.

CAUSE LIVES IN

A Spark change to the protocol or a column type.

CAUSE

The endpoint maps columns by name only and misses tables with V2 checkpoints. Neither it nor Direct Lake supports `TIMESTAMP_NTZ`, and the endpoint omits unmapped columns. Variant and other Delta 4.x features are Spark-only. Type widening is safe. A foreign key added on the endpoint blocks later schema changes.

FIX

Keep shared tables to features every reader supports. Protocol upgrades are permanent: rewrite into a new table and repoint readers.

```
from pyspark.sql.functions import col
spark.table("sales.orders").withColumn("order_ts", col("order_ts").cast("timestamp")) \
    .write.saveAsTable("sales.orders_clean")
```

CONFIRM IN BOTH ENGINES

```
-- Spark: features and properties
DESCRIBE DETAIL sales.orders; --
tableFeatures
SHOW TBLPROPERTIES sales.orders;
```

```
-- SQL endpoint: exposed columns
SELECT name, TYPE_NAME(user_type_id) FROM
sys.columns
WHERE object_id = OBJECT_ID('sales.orders');
```

Compare with the Spark schema. The refresh API reports sync errors per table.

RELATED **5.05** Your JSON column parses fine in Spark **5.01** Spark wrote the rows
5.06 Your OneLake shortcut works in a notebook

5.05 Your JSON column parses fine in Spark. Through the SQL endpoint, long documents break and nested fields vanish.

ENGINES PIPELINES DATAFLOW **SPARK** ONELAKE WAREHOUSE **SQL ENDPOINT** DIRECT LAKE SECURITY CAPACITY

A Spark STRING surfaces as **varchar(8000)** on the Lakehouse SQL endpoint, with values **truncated at 8 KB**; struct, array and variant columns are not exposed. Write needed fields as typed columns.

Cost of failure **Wrong data**
Status **GA** · Verified **Sep 2026**

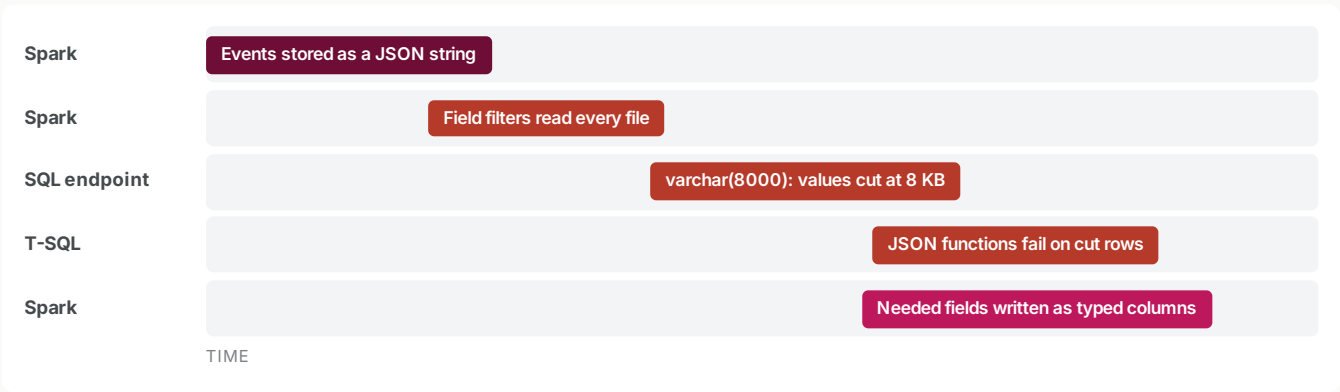


FIG. 5.05 The loss happens at the type mapping between Spark and the endpoint.

SEEN IN

T-SQL on the endpoint and reports built on it.

CAUSE LIVES IN

The Spark write: JSON as one STRING, or a nested type.

CAUSE

STRUCT and ARRAY are missing from the type mapping, so they are not exposed. VARIANT needs runtime 2.0, upgrades the protocol permanently and is Spark-only. A JSON string has no per-field statistics, so Spark cannot skip files on field filters.

FIX

Parse once with an explicit schema. Write filtered or displayed fields as top-level columns and keep the raw payload.

```
from pyspark.sql.functions import from_json
schema = "order_id BIGINT, status STRING, amount DECIMAL(18,2)"
raw.withColumn("j", from_json("payload", schema)) \
    .select("event_id", "j.*", "payload") \
    .write.mode("append").saveAsTable("silver.events")
```

CONFIRM IN BOTH ENGINES

```
-- Spark: rows over 8 KB
spark.sql("SELECT COUNT(*) FROM bronze.events
WHERE octet_length(payload) > 8000").show()
```

```
-- SQL endpoint: what it holds
SELECT MAX(DATALENGTH(payload)) FROM
dbo.events;
-- 8000 means values were cut
```

RELATED 5.04 A Spark change to a shared table succeeded 5.09 A Warehouse query that joins a Lakehouse...
5.08 Your Gold table is a fraction of Silver's size

5.06 Your OneLake shortcut works in a notebook.
The Direct Lake model cannot see it as a table.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Only **Delta data at the top level of Tables** becomes a table; shortcuts in Files, or in Tables over CSV, Parquet or JSON, never register. Use a **shortcut transformation** or write Delta.

Cost of failure **Rework**
Status **GA** · Verified **Sep 2026**

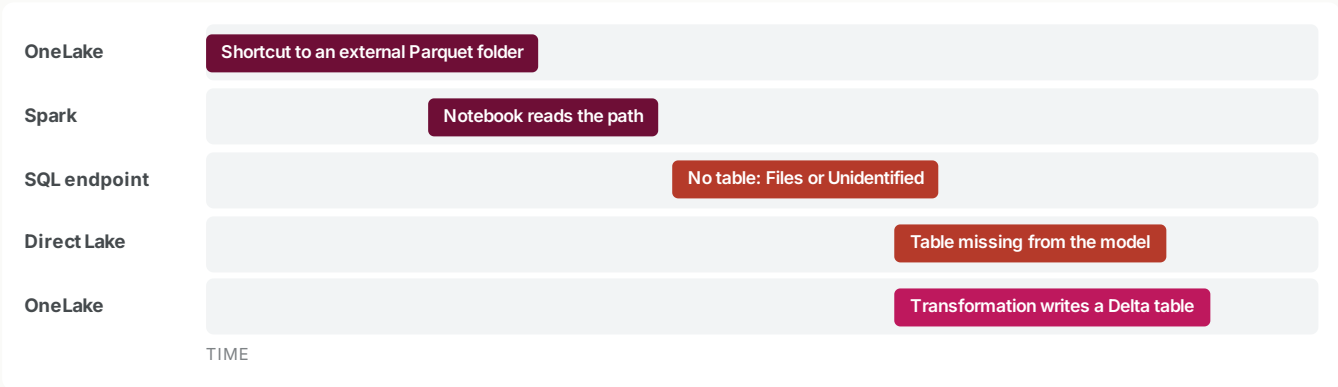


FIG. 5.06 Spark reads by path, so the notebook works; the endpoint and Direct Lake see only registered tables.

SEEN IN

CAUSE LIVES IN

The Direct Lake model and the SQL endpoint.

OneLake: where the shortcut sits and what format its target holds.

CAUSE

Table discovery runs only at the top level of Tables. A shortcut name with a space is not recognised as a Delta table. Direct Lake on SQL discovers tables through the SQL endpoint, Direct Lake on OneLake through OneLake APIs. A broken target, for example after a Git sync, moves the shortcut to Unidentified.

FIX

For CSV, Parquet or JSON, a shortcut transformation writes a Delta table in Tables and polls the source every two minutes. Or convert with Spark `saveAsTable`. A `save` to Files registers nothing.

```
df = spark.read.parquet("Files/ext_orders/")
df.write.format("delta").mode("overwrite").saveAsTable("ext_orders")
```

CONFIRM IN BOTH ENGINES

```
-- Spark
spark.sql("SHOW TABLES").show()
```

```
-- SQL endpoint
SELECT name FROM sys.tables WHERE name =
'ext_orders';
```

Both must list the table before the model can use it.

RELATED **5.01** Spark wrote the rows **5.04** A Spark change to a shared table succeeded
5.07 You added row-level security on the SQL endpoint

5.07 You added row-level security on the SQL endpoint. Direct Lake reports now run as slowly as DirectQuery.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Direct Lake on SQL **falls back to DirectQuery** for any query touching a table with **SQL endpoint RLS**, but not for semantic model RLS. Move the rules to the model or to OneLake security.

Cost of failure **Slow reports**
Status **GA** · Verified **Sep 2026**

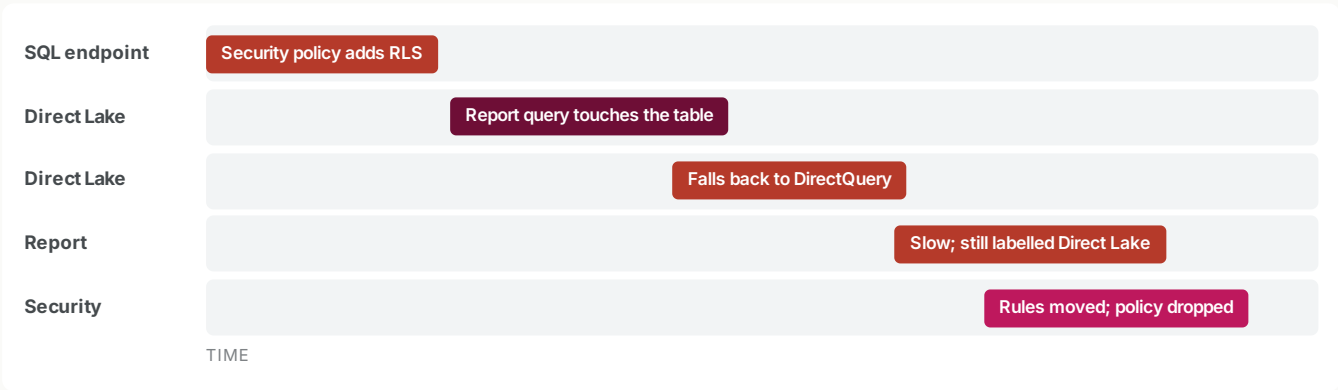


FIG. 5.07 Reports show no error, only slower queries.

SEEN IN
Reports on a Direct Lake on SQL model.

CAUSE LIVES IN
SQL security on the source's SQL endpoint.

CAUSE
Direct Lake on SQL checks the endpoint's security before reading Delta. A query touching a table with endpoint RLS, or a view, goes to DirectQuery for every user. Endpoint CLS and denied tables return errors instead. Under the default `DirectLakeBehavior`, Automatic, the fallback raises no error.

FIX
Option one: OneLake security RLS with Direct Lake on OneLake, which never falls back. It restricts Viewers only. Option two: semantic model roles over a fixed-identity connection. Then drop the endpoint policy: Direct Lake on OneLake errors on tables that keep it.

```
-- On the SQL endpoint, once the new rules are live
DROP SECURITY POLICY rls.SalesFilter;
```

CONFIRM IN BOTH ENGINES

```
-- DAX: fallback reason per table
EVALUATE TABLETRAITS()
// DirectLakeFallbackInfo = None means Direct Lake
```

In a development copy, set Direct Lake behavior to DirectLakeOnly so would-be fallbacks fail with an error.

RELATED **5.06** Your OneLake shortcut works in a notebook **5.08** Your Gold table is a fraction of Silver's size

5.01 Spark wrote the rows

5.08 Your Gold table is a fraction of Silver's size. Direct Lake reports on it are slower.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

New workspaces default to **writeHeavy**, with **V-Order off**, so small Gold writes leave many small row groups that Direct Lake transcodes slowly. Use **readHeavyForPBI** for Gold.

Cost of failure **Slow reports**

Status **GA** · Verified **Sep 2026**

SILVER

Few large row groups

V-Ordered, compacted

GOLD

Many small row groups

writeHeavy, no V-Order

FIG. 5.08 Row group size and V-Order decide how fast Direct Lake reads a table.

MEASURE FIRST

- 1 Rule out fallback first.

```
EVALUATE TABLETRAITS()
```

- 2 Check how Spark writes.

```
spark.conf.get("spark.sql.parquet.vorder.default") # 'false' under writeHeavy
```

- 3 Count row groups. Direct Lake prefers 1 to 16 million rows each.

```
import sempy_labs as labs
display(labs.delta_analyzer("sales_summary")["Summary"])
```

CAUSE

Each row group becomes a column segment with its own dictionary to merge. V-Order improves RLE compression and lets VertiScan scan compressed data. A model refresh only reframes metadata, so it does not fix layout.

FIX

Set the profile, pin V-Order on the table, compact once.

```
spark.conf.set("spark.fabric.resourceProfile", "readHeavyForPBI")
spark.sql("ALTER TABLE gold.sales_summary SET TBLPROPERTIES ('delta.parquet.vorder.enabled' = 'true')")
spark.sql("OPTIMIZE gold.sales_summary VORDER")
```

OPTIMIZE forces retranscoding, so run it off-hours. V-Order adds about 15% to write time. Use it for Direct Lake tables; it does little for SQL endpoint speed.

EXPECTED RESULT

Fewer, larger V-Ordered row groups and less transcoding.

Go deeper: PySpark Handbook 4.0, V-Order.

RELATED **5.05** Your JSON column parses fine in Spark **5.07** You added row-level security on the SQL endpoint

5.09 A Warehouse query that joins a Lakehouse...

5.09 A Warehouse query that joins a Lakehouse table is far slower than either table queried alone.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

The Warehouse reads the table through its SQL endpoint, in the same engine.

The cost comes from Spark's layout: **STRING keys become varchar(8000)** and **small files** slow every scan.

Cost of failure **Slow queries**

Status **GA** · Verified Sep 2026

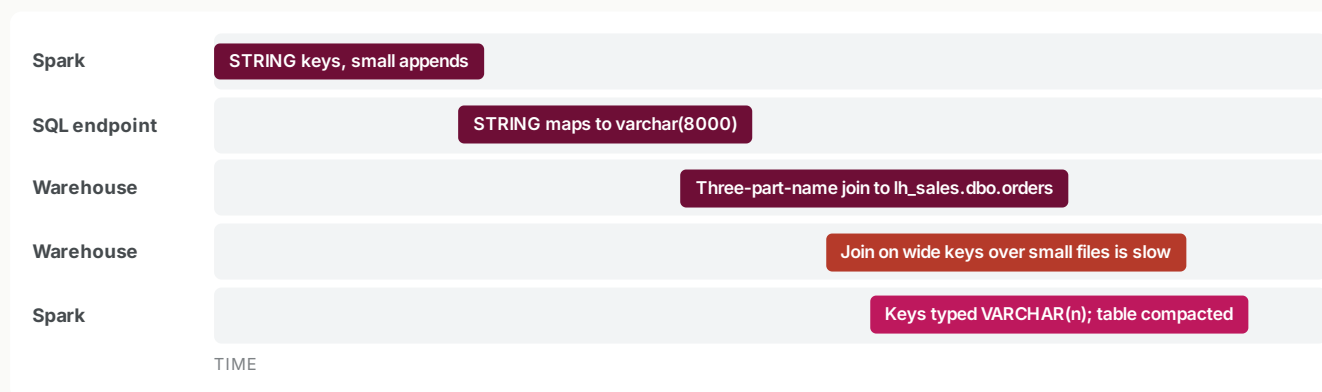


FIG. 5.09 The Warehouse join is slow because of the types and file layout Spark wrote.

SEEN IN

Warehouse T-SQL joining a Lakehouse table by three-part name.

CAUSE LIVES IN

The Spark job that creates and feeds the table.

CAUSE

A Spark STRING surfaces as varchar(8000), and Microsoft advises the smallest string type that fits. Spark VARCHAR(n) under 2000 surfaces as varchar(4n). Frequent small appends leave many small files, adding overhead to every read. Warehouse tables are V-Ordered by default. Spark tables in new workspaces are not.

FIX

Declare string lengths in Spark and compact the table. If the Spark job cannot change, copy the table into the Warehouse with CTAS, casting the join keys to narrow types in the SELECT. Rebuild the copy after each load.

```
-- Spark SQL: VARCHAR(20) surfaces as varchar(80)
CREATE TABLE lh_sales.orders_typed (order_id BIGINT,
  customer_code VARCHAR(20), amount DECIMAL(18,2)) USING DELTA;
INSERT INTO lh_sales.orders_typed SELECT order_id, customer_code, amount FROM lh_sales.orders;
OPTIMIZE lh_sales.orders_typed;
-- T-SQL in a Warehouse in the same workspace
CREATE TABLE dbo.orders AS SELECT order_id, amount,
  CAST(customer_code AS varchar(20)) AS customer_code FROM lh_sales.dbo.orders;
```

CONFIRM IN BOTH ENGINES

```
-- SQL endpoint: key widths
SELECT name, max_length FROM sys.columns
WHERE object_id = OBJECT_ID('dbo.orders');
-- 8000 on a join key is the problem
```

```
-- Spark: file count
spark.sql("DESCRIBE DETAIL
lh_sales.orders").select("numFiles",
"sizeInBytes").show()
```

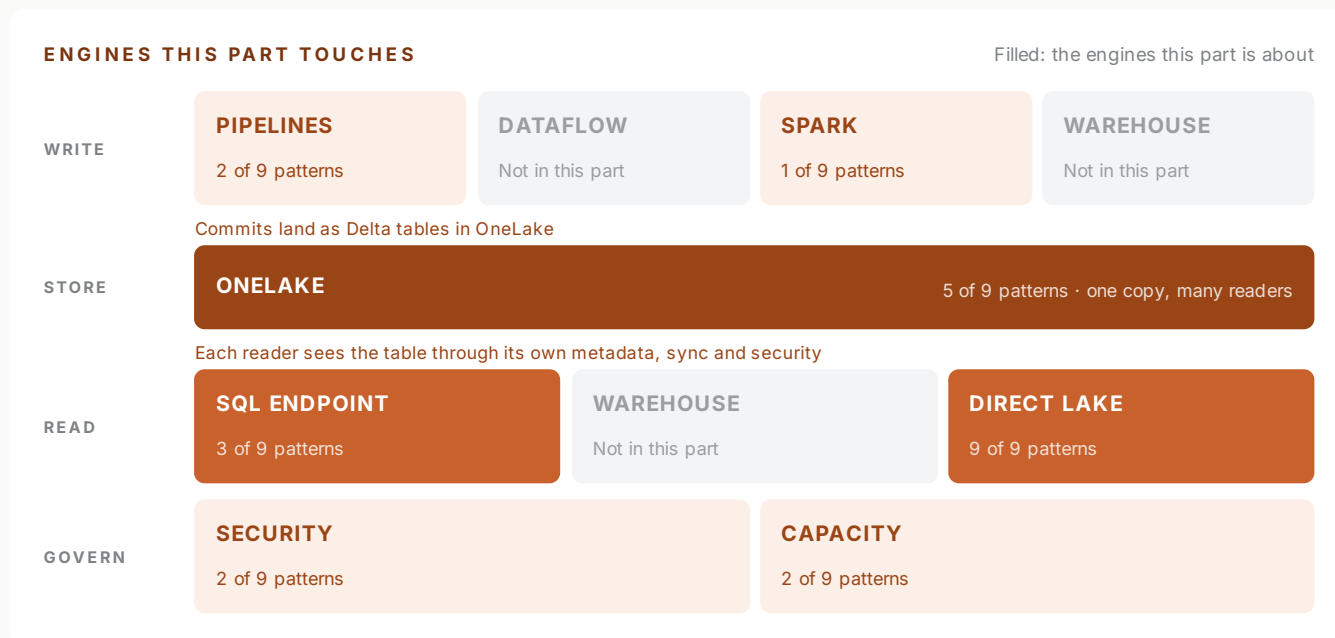
A Warehouse and a Lakehouse endpoint with different collations can also mismatch rows (5.03).

RELATED **5.03** The same T-SQL query returns different rows... **5.08** Your Gold table is a fraction of Silver's size

5.01 Spark wrote the rows

06 Direct Lake and Power BI.

Direct Lake flavours, framing, automatic updates, fallback and refresh identity.
Generic DAX and modelling live elsewhere; this part covers what Fabric changes.



NO.	PATTERN	PIPE	ONELAKE	SQL EP	DL	SEC	CAP	PAGE
6.01	You moved a model from Direct Lake on SQL to...	●	●	●	●	●	●	116
6.02	A view that worked fine in Import mode makes...	●	●	●	●	●	●	117
6.03	You add Import lookup tables to a Direct Lake...	●	●	●	●	●	●	117
6.04	The semantic model refresh succeeded	●	●	●	●	●	●	118
6.05	During the nightly load, reports show a...	●	●	●	●	●	●	119
6.06	Someone renamed a Lakehouse table	●	●	●	●	●	●	120
6.07	Direct Lake on SQL report pages got slower	●	●	●	●	●	●	121
6.08	Reports crawl on the first open after every load	●	●	●	●	●	●	122
6.09	Every Direct Lake refresh fails with access...	●	●	●	●	●	●	123

6.01 You moved a model from Direct Lake on SQL to OneLake. Refresh now fails instead of slowing down.

ENGINES PIPELINES DATAFLOW SPARK ONELAKE WAREHOUSE SQL ENDPOINT DIRECT LAKE SECURITY CAPACITY

Direct Lake on OneLake has **no DirectQuery fallback**. Conditions that made the SQL model slow now fail refresh or queries, and SQL endpoint RLS **no longer applies**.
Cost of failure **Outage**
Status **GA** · Verified **Sep 2026**

	DIRECT LAKE ON SQL	DIRECT LAKE ON ONELAKE
Table over a guardrail (5,000 files at F64)	Falls back to DirectQuery, refresh succeeds with a warning	Refresh fails, model cannot be queried until the table is optimized
Table based on a SQL view	Falls back to DirectQuery	Cannot be a Direct Lake table
RLS defined on the SQL analytics endpoint	Falls back, RLS enforced	Queries succeed, SQL RLS not applied
Permission checked	SELECT through the SQL analytics endpoint	Read and ReadAll, or a OneLake security role

FIG. 6.01 Read each row across: SQL falls back and slows, OneLake fails or skips SQL RLS.

CAUSE

Direct Lake on SQL ships with Direct Lake behavior set to Automatic, which turns every unmet condition into a DirectQuery query. Direct Lake on OneLake runs only in DirectLakeOnly mode. It reads Delta files through OneLake APIs, so rules on the SQL analytics endpoint never run. Both show as Direct Lake in the service, so the difference surfaces only after the switch.

FIX

Before switching, set Direct Lake behavior to DirectLakeOnly on a test copy of the SQL model and run `EVALUATE TABLETRAITS()`. Fix every table whose `[DirectLakeFallbackInfo]` is not None: OPTIMIZE oversized tables, materialize views as Delta tables, move SQL RLS into OneLake security or model RLS. Then replace the connection expression in TMDL view and test with DAX queries.

PREVENT

Build new models on Direct Lake on OneLake, the option Microsoft recommends, GA since March 2026. Tell the flavours apart in TMDL view: an `AzureStorage.DataLake` expression is OneLake, a `Sql.Database` expression is SQL.

RELATED **6.02** A view that worked fine in Import mode makes... **6.07** Direct Lake on SQL report pages got slower
6.03 You add Import lookup tables to a Direct Lake...

6.02 A view that worked fine in Import mode makes the Direct Lake model slow or unusable.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Direct Lake reads Delta tables. On SQL a view table **falls back to DirectQuery**; on OneLake a view **cannot be a Direct Lake table** at all.

Cost of failure **Slow reports**Status **GA** · Verified **Sep 2026**

YOU'D EXPECT (IMPORT MODE)

A SQL view is a table like any other. Refresh reads it once and reports query the imported copy.

IN DIRECT LAKE

Direct Lake on SQL sends every query on a view table to the SQL analytics endpoint. Those queries return live data while the other tables stay at the last frame, so totals can disagree. Direct Lake on OneLake cannot add a non-materialized view as a Direct Lake table.

Do this. Materialize the view as a Delta table in the pipeline, or as a materialized lake view. On OneLake, a view that must stay a view can join the model as an Import table.

RELATED **6.01** You moved a model from Direct Lake on SQL to... **6.07** Direct Lake on SQL report pages got slower

6.03 You add Import lookup tables to a Direct Lake...

6.03 You add Import lookup tables to a Direct Lake model and expect composite model penalties.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

With Direct Lake on OneLake, Import and Direct Lake tables form **one source group** and use **regular relationships**. Direct Lake on SQL cannot hold Import tables.

Cost of failure **Rework**Status **GA** · Verified **Sep 2026**

YOU'D EXPECT (IMPORT PLUS DIRECTQUERY COMPOSITE)

Two storage modes mean two source groups, limited relationships, slow cross-source joins and totals that can disagree between visuals.

IN DIRECT LAKE ON ONELAKE

Direct Lake and Import tables are one source group. Relationships between them are regular, except many-to-many, which stay limited.

Do this. Check the flavour in TMDL view first. Combine the tables in Power BI web modeling. Set credentials for the Import sources in the model's refresh settings. The web modeling Refresh reloads Import tables, reframes Direct Lake tables and syncs the schema.

RELATED **6.01** You moved a model from Direct Lake on SQL to... **6.02** A view that worked fine in Import mode makes...

6.08 Reports crawl on the first open after every load

6.04 The semantic model refresh succeeded. The report still shows yesterday's data.

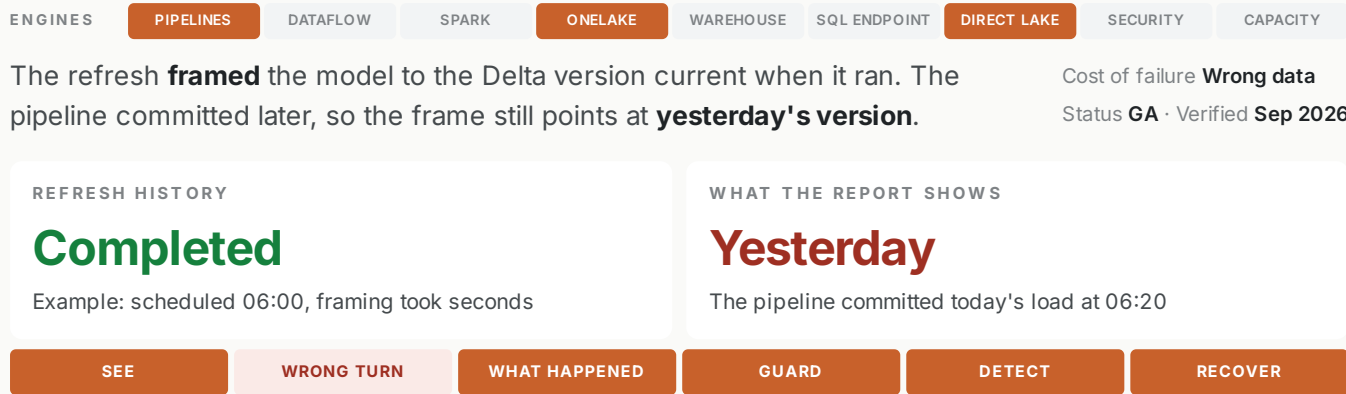


FIG. 6.04 Refresh history shows Completed at 06:00; the load committed at 06:20, after the frame.

WRONG TURN

"Power BI is caching old results. Refresh again and clear the browser cache."

WHAT HAPPENED

Framing records which Delta version each table reads, and queries see only that version. With automatic updates off, the model framed on its schedule, before the load committed. Automatic updates can also be on but suspended after a non-recoverable refresh error. Only a successful on-demand refresh resumes them.

1 GUARD

Frame after the data lands. End the pipeline with a Semantic model refresh activity, Wait on completion on, and delete the fixed schedule.

2 DETECT

Compare the latest Delta commit with the last refresh time. Check refresh history for failed automatic updates.

```
DESCRIBE HISTORY gold.fact_sales LIMIT 1;
-- commit later than the last refresh =
stale frame
```

3 RECOVER

Refresh on demand. Framing reads metadata only and takes seconds.

```
POST
https://api.powerbi.com/v1.0/myorg/groups/{workspaceId}/datasets/{modelId}/refreshes
{"type": "full"}
```

RELATED **6.05** During the nightly load, reports show a... **6.06** Someone renamed a Lakehouse table

6.09 Every Direct Lake refresh fails with access...

6.05 During the nightly load, reports show a half-loaded table. An hour later the numbers are right.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Automatic updates reframe a table as soon as a commit changes it, so reports see every intermediate state. Turn them off and **frame once at pipeline end**.

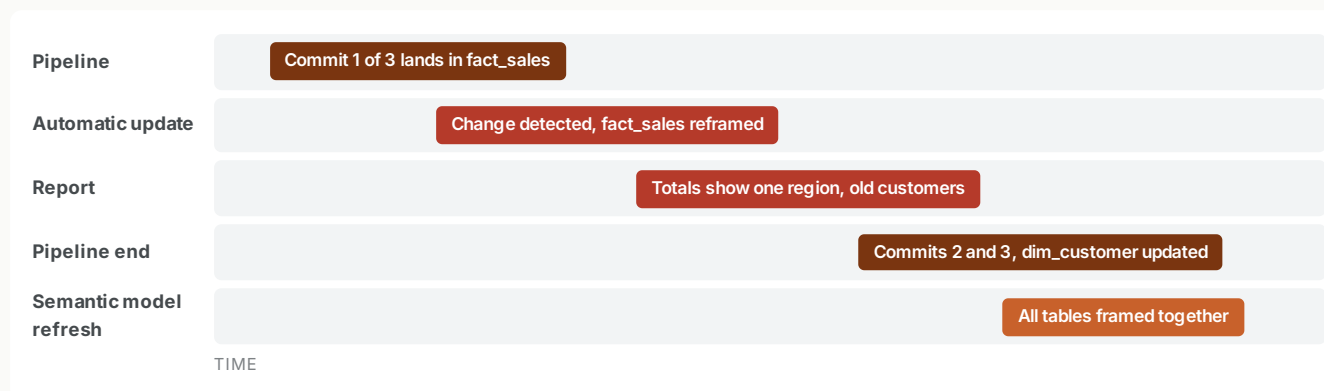
Cost of failure **Wrong data**Status **GA** · Verified Sep 2026

FIG. 6.05 Automatic updates reframe fact_sales after commit 1, so the report shows one region until the pipeline-end refresh.

SEEN IN

Direct Lake

CAUSE LIVES IN

Pipelines

CAUSE

Keep your Direct Lake data up to date is on by default. It reframes the changed tables whenever it detects a data change. A load that commits in stages, or updates facts before dimensions, exposes each stage to users. Framing takes seconds, so the half-loaded state appears almost at once.

FIX

In the semantic model settings, Refresh section, turn off Keep your Direct Lake data up to date. Add a Semantic model refresh activity as the last pipeline step with Wait on completion on.

```
POST https://api.powerbi.com/v1.0/myorg/groups/{workspaceId}/datasets/{modelId}/refreshes
{"type": "full"}
```

CONFIRM IN BOTH ENGINES

```
-- Commits inside the load window
DESCRIBE HISTORY gold.fact_sales LIMIT 10;
-- several commits minutes apart: each was reframed
```

RELATED **6.04** The semantic model refresh succeeded **6.08** Reports crawl on the first open after every load

6.06 Someone renamed a Lakehouse table. Scheduled refresh now fails and new columns never reach the model.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Scheduled and automatic refreshes **only reframe**; they never sync the schema. Cost of failure **Outage**

Added columns stay invisible, and a renamed or dropped source table breaks framing, so the **refresh fails**. Status **GA** · Verified **Sep 2026**

	REFRESH IN WEB MODELING OR DESKTOP LIVE EDIT	SCHEDULED OR AUTOMATIC REFRESH
Schema sync	Yes, then reframe	No, reframe only
New column in the Delta table	Added to the model	Not visible
Table renamed or dropped at source	Removed from the model, visuals using it break	Framing fails, refresh errors
Automatic updates after the failure	Not relevant	Suspended until an on-demand refresh succeeds

FIG. 6.06 Only interactive refresh syncs the schema, so a renamed source table works in development and fails the scheduled refresh.

CAUSE

The model binds to source table and column names. Framing reads Delta metadata for the objects the model lists. A referenced table that no longer exists stops the frame. After a non-recoverable refresh error Power BI suspends automatic updates, so no new data arrives either.

FIX

Open the model in web modeling or Desktop live edit and select Refresh, or use Edit tables, to sync the schema. For a rename, set `SourceLineageTag` to the new name in TMDL view before syncing, so the column keeps its measures and visuals. Then run an on-demand refresh to resume automatic updates.

PREVENT

Treat source table and column names as a contract with the model. Add columns freely. Rename or drop only in the same release as the model change.

RELATED

6.04 The semantic model refresh succeeded

6.07 Direct Lake on SQL report pages got slower

6.07 Direct Lake on SQL report pages got slower. No error appears and the model still says Direct Lake.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Queries are **falling back to DirectQuery** through the SQL analytics endpoint. Performance Analyzer and `TABLETRAITS()` show which tables fall back and why.

Cost of failure **Slow reports**Status **GA** · Verified **Sep 2026**

PARQUET FILES OR ROW GROUPS PER TABLE, F64

5,000

Rows: 1.5 billion. One table over any limit sends the whole model to DirectQuery

SAME LIMITS, F2 TO F32

1,000

Rows: 300 million

FIG. 6.07 The storage mode label says Direct Lake while queries run as DirectQuery; Performance Analyzer or `TABLETRAITS()` shows the fallback.

MEASURE FIRST

- 1 In Power BI Desktop, record the page in Performance Analyzer. A Direct query line under a visual means that query fell back.
- 2 Ask the model which tables fall back and why. None means Direct Lake.

```
EVALUATE TABLETRAITS()
-- read [DirectLakeFallbackInfo] per table
```

- 3 For a trace, capture `DirectQuery_Begin` and `DirectQuery_End` in SQL Server Profiler. EngineEdition and OLS checks always use DirectQuery and do not count.

CAUSE

Direct Lake on SQL falls back when any condition fails: a table on a non-materialized view, RLS, OLS or dynamic data masking on the SQL analytics endpoint, a table over a guardrail, or a table not framed since it changed. Switching the endpoint to user identity mode under OneLake security makes every query fall back. Under the default Automatic behavior, none of this shows as an error.

FIX

Fix by cause: `OPTIMIZE` and `VACUUM` oversized tables, materialize views as Delta tables, move endpoint RLS or OLS into the model, refresh unframed tables. If a table still exceeds the guardrail, move to a larger SKU.

```
OPTIMIZE gold.fact_sales;
VACUUM gold.fact_sales;
```

Direct Lake behavior set to `DirectLakeOnly` turns fallback into query errors. Use it in development to catch fallback before users see slow pages.

EXPECTED RESULT

Performance Analyzer shows no Direct query lines and every table reports None.

RELATED [6.01](#) You moved a model from Direct Lake on SQL to... [6.02](#) A view that worked fine in Import mode makes...

[6.08](#) Reports crawl on the first open after every load

6.08 Reports crawl on the first open after every load. Reopening the same page is fast.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Framing **evicted** the columns the load changed, and first queries **transcode** them back from Delta. Overwrite loads and small row groups make it worse.

Cost of failure **Slow reports**Status **GA** · Verified **Sep 2026**

FIRST QUERY AFTER REFRAME

Cold

Segments, dictionaries and join indexes rebuilt from Parquet

SAME QUERY AGAIN

Warm

Resident columns, on par with Import mode

FIG. 6.08 The gap between the cold and warm runs is transcoding cost from the load pattern.

MEASURE FIRST

- 1 Record the page twice in Performance Analyzer after a refresh. The second run's drop in DAX query time is transcoding cost.
- 2 Non-resident segments right after framing, in a model that was warm, mean the load was destructive.

```
EVALUATE INFO.STORAGETABLECOLUMNSEGMENTS()
-- inspect segment residency per column
```

- 3 Check how the load writes.

```
DESCRIBE HISTORY gold.fact_sales LIMIT 5;
-- operationParameters mode Overwrite = full cold reload
```

CAUSE

Direct Lake loads a column only when a query first needs it. Incremental framing drops only segments of removed row groups. An Overwrite commit removes all Parquet files, so every column, dictionary and join index reloads. Row groups well under 1 million rows multiply segments and slow transcoding. Above its memory limit, 25 GB at F64, a model pages data in and out.

FIX

Load large tables with append or partition-scoped writes so incremental framing keeps resident data. Run OPTIMIZE with V-Order off-hours. After the pipeline refresh, query the main report measures once to load their columns.

```
import sempy.fabric as fabric
# after the refresh activity: load the columns users hit first
fabric.evaluate_dax("Sales", "EVALUATE ROW(\"Total\", [Total Sales])", workspace="Finance")
```

OPTIMIZE rewrites Parquet files, so it causes one cold reload itself. A model can also reload cold on another node.

EXPECTED RESULT

First opens run close to repeat opens, and segments survive daily loads.

RELATED

[6.07 Direct Lake on SQL report pages got slower](#)

[6.05 During the nightly load, reports show a...](#)

[6.03 You add Import lookup tables to a Direct Lake...](#)

6.09 Every Direct Lake refresh fails with access denied, even when a workspace admin runs it.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Direct Lake checks the **semantic model owner's** read access on every framing, whoever starts the refresh. When the owner loses access, every refresh fails until someone **takes over** the model.

Cost of failure **Outage**

Status **GA** · Verified **Sep 2026**

	REPORT QUERIES	REFRESH (FRAMING)
Identity checked	Viewer under SSO, or the fixed identity	Model owner, plus the fixed identity if bound
Owner loses read access	Effective identity checked as before	"Source tables either do not exist or access was denied"
Where to fix	Viewer or fixed identity permissions	Model ownership

FIG. 6.09 Framing checks the model owner, so fixing the viewer's access does not fix the refresh.

CAUSE

Framing needs the owner to read the source tables, in addition to the effective identity. By default the effective identity is the viewer through SSO. A cloud connection with a fixed identity replaces it. With SSO and a fixed identity both configured, the fixed identity frames and the viewer is checked at query time.

FIX

Take over the model in its settings as an account with read access to the source, or call the Take Over In Group REST API. Then refresh on demand to confirm framing succeeds and to resume automatic updates.

PREVENT

Own production models with a dedicated service account. Bind them to a fixed-identity cloud connection with SSO off when viewers should not read the lakehouse. Review model owners when people change roles.

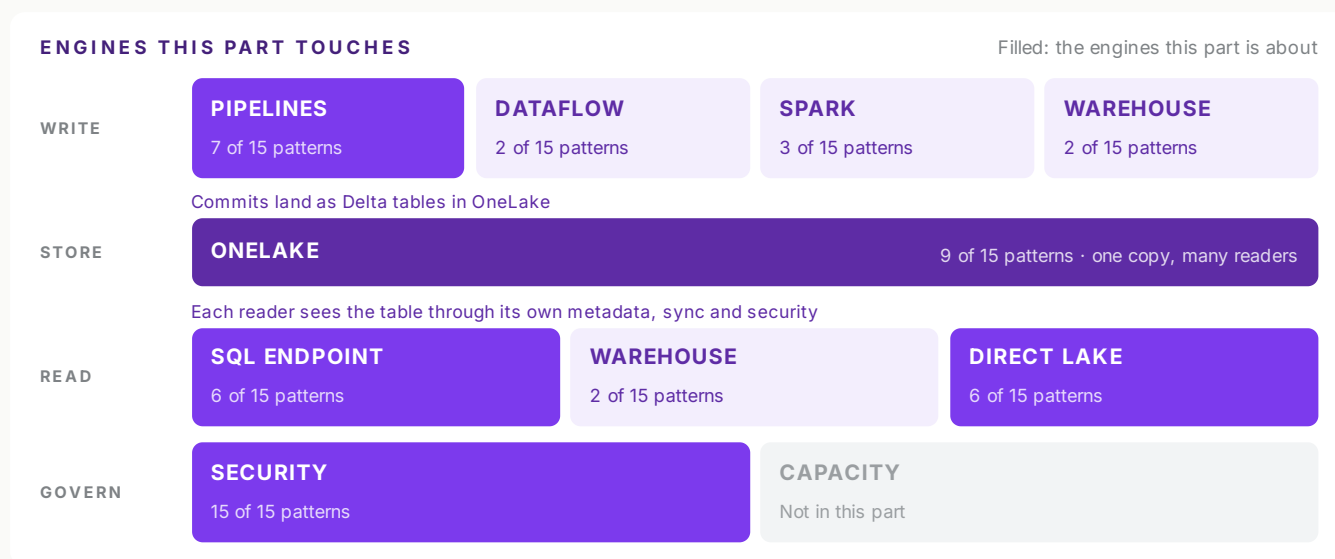
RELATED

[6.04 The semantic model refresh succeeded](#)

[6.06 Someone renamed a Lakehouse table](#)

07 Security and Identity.

Run identity, connections, credentials, workspace roles and OneLake security. Who a job runs as decides whether it runs at all.



NO.	PATTERN	PIPE	SPARK	ONELAKE	SQL EP	DL	SEC	PAGE
7.01	A pipeline schedule ran for months, then stopped	●	●	●	●	●	●	125
7.02	The connection works when you test it	●	●	●	●	●	●	126
7.03	The engineer who built the connections left	●	●	●	●	●	●	127
7.04	Every workload runs as one service principal	●	●	●	●	●	●	128
7.05	The shortcut to firewalled storage worked for...	●	●	●	●	●	●	129
7.06	Pipelines ran fine yesterday	●	●	●	●	●	●	130
7.07	RLS with USERPRINCIPALNAME() works in Desktop	●	●	●	●	●	●	131
7.08	The report works for workspace members	●	●	●	●	●	●	132
7.09	Everyone holds Contributor on the production...	●	●	●	●	●	●	133
7.10	OneLake security hides the PII tables from...	●	●	●	●	●	●	134
7.11	Bronze, Silver and Gold share one lakehouse	●	●	●	●	●	●	135
7.12	RLS sits in OneLake security, the SQL...	●	●	●	●	●	●	136
7.13	Secured users wait seconds for Direct Lake...	●	●	●	●	●	●	137
7.14	You removed a group from a OneLake security role	●	●	●	●	●	●	138
7.15	You added column-level security in the Warehouse	●	●	●	●	●	●	139

7.01 A pipeline schedule ran for months, then stopped. Nobody changed the pipeline.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Pipelines run as the **Last Modified By** user. When that user leaves, resets a password or skips Fabric for **90 days**, runs stop.

Cost of failure **Outage**

Status **GA** · Verified **Sep 2026**

	WHILE THE LAST EDITOR IS ACTIVE	AFTER THEY LEAVE OR GO QUIET
Run identity	Last Modified By user with a valid token	Same user, account disabled or password reset
Schedule	Fires on time	Expired: that user has not signed in to Fabric for 90 days
Ownership takeover	Not needed	Owner changes, but runs keep the old identity until a re-save
Notebook activity	Runs through the connection it was given	A user connection fails; a service principal or workspace identity connection keeps running

FIG. 7.01 The pipeline and schedule are unchanged; compare the state of the account it runs as.

CAUSE

A disabled account or password change revokes the cached token. A scheduled notebook runs as the user who created or last updated its schedule. Schedules expire after the Last Modified By user goes 90 consecutive days without signing in to Fabric. Fabric auto-disables a schedule that fails about 10 times in a row, and you must restart it.

FIX

Take over the pipeline, make a small edit such as the description, and save. That updates Last Modified By and refreshes the stored auth context. Takeover does not cascade, so take over child notebooks separately. For production, save the pipeline as a service principal: `PATCH /dataPipelines/{id}` with a new description. The service principal needs Contributor on the workspace. In the Notebook activity, choose a service principal or workspace identity connection. Workspace identity there needs the tenant setting *Service principals can call Fabric public APIs*.

PREVENT

Read each schedule's `owner` and `enabled` fields with `GET /v1/workspaces/{ws}/items/{id}/jobs/Pipeline/schedules`. Alert when an expected run is missing from job history. Re-home pipelines before a leaver's account is disabled.

RELATED

7.02 The connection works when you test it

7.03 The engineer who built the connections left

7.04 Every workload runs as one service principal

7.02 The connection works when you test it. The scheduled run fails.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

OAuth connections keep a **refresh token**, which stops working on expiry, a password reset or credential revocation, or when a policy blocks non-interactive use. Switch to a **service principal** or **workspace identity**.

Cost of failure **Outage**
Status **GA** · Verified **Sep 2026**

	TEST OR MANUAL RUN	SCHEDULED RUN
Token source	Fresh token from your interactive sign-in	Stored refresh token, nobody signed in
Credential change	None	Password reset or credential revocation invalidated the token
Conditional Access	Your sign-in meets the policy	Policy blocks token issuance for non-interactive use
Retry	Not needed	Fails again: the token cannot renew itself

FIG. 7.02 The test uses your live sign-in; the schedule uses a stored refresh token.

CAUSE

The pipeline stores the connection's refresh token and exchanges it for an access token at run time. Test Connection and manual runs use your live session instead. Microsoft Entra refuses the stored token after expiry or revocation, or when Conditional Access requires a device that no longer qualifies. The run fails with `LSROBOTokenFailure`.

FIX

Short term: edit and save each affected pipeline to cache a new token. Lasting fix: recreate the connection with a service principal or workspace identity and grant that identity access on the source. Workspace identity covers the Copy, Lookup and Get Metadata activities. It works on cloud connections only, not through an on-premises or VNet gateway, so gateway sources need a service principal.

PREVENT

List connections with `GET /v1/connections` and flag `credentialType OAuth2` on anything a schedule uses. Treat a personal OAuth connection in production as a defect in code review.

RELATED [7.01](#) A pipeline schedule ran for months, then stopped [7.03](#) The engineer who built the connections left

7.03 The engineer who built the connections left. Every pipeline failed the next morning.

ENGINESPIPELINESDATAFLOWSPARKONELAKEWAREHOUSESQL ENDPOINTDIRECT LAKESECURITYCAPACITY

Every shared connection held that engineer's **OAuth token**, revoked when the account was disabled. Move production connections to a **workspace identity** or service principal before anyone leaves.

Cost of failure **Outage**
Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
Personal OAuth	Quickest to create. Fine for development.	Revoked when the user leaves or changes password. One account can fail every pipeline sharing the connection.
Workspace identity DEFAULT	Fabric manages the credential. No secret to leak or rotate, scoped to one workspace.	Cloud connections and supported items only. Deleting it breaks every item that uses it, and it cannot be restored.
Service principal with secret	Works through gateways and for connectors without workspace identity support.	The client secret expires on its Entra end date. Every connection using it fails at once.

FIG. 7.03 Read each credential's breaks row; personal OAuth fails when one person leaves.

DEFAULT

Workspace identity wherever the connector supports it, a service principal elsewhere. Record every secret's end date and rotate before it. Azure Key Vault references for connection secrets were announced in preview; confirm their status before relying on them.

SIGNS YOU CHOSE WRONG

Many pipelines fail within the same hour with token or credential errors. `LSROBOTokenFailure` citing a revoked grant or password change. Shared cloud connections show `credentialType OAuth2`. Failures start the day an account was disabled or a secret expired.

CHECK IT

Find shared connections that still carry a user token.

```
GET https://api.fabric.microsoft.com/v1/connections
# scope: Connection.Read.All
# flag connectivityType = ShareableCloud
# and credentialDetails.credentialType = OAuth2
```

RELATED7.02 The connection works when you test it7.01 A pipeline schedule ran for months, then stopped7.04 Every workload runs as one service principal

7.04 Every workload runs as one service principal. One leaked secret opens every workspace.

ENGINES PIPELINES DATAFLOW SPARK ONELAKE WAREHOUSE SQL ENDPOINT DIRECT LAKE SECURITY CAPACITY

Split identities by **workspace** or **workload domain** so one leaked secret exposes one domain.

Cost of failure **Security**
Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
One service principal for everything	One secret to rotate, one app to grant.	A secret in a repo or a log exposes every workspace. The audit log shows one identity for every workload.
Service principal per workload domain	The ingestion identity writes Bronze only; the reporting identity reads Gold only.	More secrets and end dates to track.
Workspace identity per workspace DEFAULT	One identity per workspace with Fabric-managed credentials. Nothing to leak.	Limited to supported items and cloud connections. Not usable in B2B or cross-tenant scenarios.

FIG. 7.04 Each option draws a different boundary around what one stolen credential can reach.

DEFAULT

Workspace identity where the item supports it. Elsewhere, one service principal per domain, with workspace roles only on that domain's workspaces. Grant Contributor, not Admin, unless the identity manages access.

SIGNS YOU CHOSE WRONG

The same app ID holds a role in every workspace. One client secret appears in dozens of connections. Rotating that secret needs a platform-wide change window.

RELATED 7.03 The engineer who built the connections left

7.02 The connection works when you test it

7.05 The shortcut to firewalled storage worked for months. After the workspace identity was recreated, access is denied.

ENGINES PIPELINES DATAFLOW SPARK ONELAKE WAREHOUSE SQL ENDPOINT DIRECT LAKE SECURITY CAPACITY

Trusted workspace access depends on the **workspace identity**. A deleted identity cannot be restored, and its replacement has a **new ID** with no storage roles.

Cost of failure **Outage**
Status **GA** · Verified **Sep 2026**

	ORIGINAL IDENTITY	DELETED OR RECREATED IDENTITY
Trusted access	Identity exists; the firewall admits the workspace	Identity deleted: every item relying on it breaks
Storage role	Old identity holds Storage Blob Data Reader	New identity has a new ID and no Azure roles
Resource instance rule	Names the workspace ID	Unchanged, still matches
Capacity	Purchased F SKU	Trial or non-F SKU: trusted access stops after one hour

FIG. 7.05 The resource instance rule still matches; the storage role sits on the deleted identity.

CAUSE

Trusted workspace access lets shortcuts, pipelines, COPY and import semantic models read a storage account restricted to selected networks. It needs a workspace identity on the workspace. Any workspace admin can delete that identity, and a deleted identity cannot be restored. Restoring a deleted workspace does not restore its identity either.

FIX

Create the workspace identity again in workspace settings. Grant it Storage Blob Data Reader or Contributor on the storage account. The resource instance rule references the workspace ID, so it needs no change. Confirm the workspace is on a purchased F SKU. If a Conditional Access policy targets all service principals, exclude Fabric workspace identities.

PREVENT

Keep workspace admin to a small group in workspaces that use trusted access. Record the identity's ID and its storage role assignments. Alert on shortcut and copy failures against firewalled accounts.

RELATED 7.04 Every workload runs as one service principal 7.03 The engineer who built the connections left 7.02 The connection works when you test it

7.06 Pipelines ran fine yesterday. Since outbound access protection was switched on, every copy fails.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Outbound access protection **blocks all outbound connections** by default.

Pipelines and dataflows need **data connection rules**, Spark needs managed private endpoints, and Warehouse gets no exceptions.

Cost of failure **Outage**

Status **GA** · Verified **Sep 2026**

	BEFORE PROTECTION	AFTER PROTECTION
Copy from an external source	Connects to any reachable endpoint	Blocked until a data connection rule allows the connector
Gateway sources	Reached through the gateway	Blocked until the gateway is allowed
Notebooks	Call any endpoint	Need a managed private endpoint per target
Warehouse COPY INTO, OPENROWSET	Read external storage	Blocked; only OneLake in the same workspace, no exceptions

FIG. 7.06 One setting blocks four workloads; they use three different ways to allow traffic.

CAUSE

Outbound access protection is GA for Data Factory, Data Engineering, OneLake shortcuts and semantic models. Data Factory items accept data connection rules only, not managed private endpoints. Warehouses and SQL analytics endpoints allow no exceptions. Workspace staging in pipelines is unsupported.

FIX

Add a data connection rule for each connector the pipelines use, such as SQL Server or ADLS Gen2. Allow gateways explicitly; an allowed gateway admits any source behind it. Create managed private endpoints for notebook targets. Land external files in OneLake with a pipeline, then run **COPY INTO** from OneLake. Point Copy staging at external storage.

PREVENT

Inventory each workspace's outbound connections before enabling protection, and trial it in a test workspace. It needs a purchased F SKU; trials are not supported. Teams and Office 365 Outlook activities do not support it.

RELATED

7.05 The shortcut to firewalled storage worked for...

7.02 The connection works when you test it

7.07 RLS with USERPRINCIPALNAME() works in Desktop. In the service, users see blank visuals.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

The service compares the signed-in user's Entra **UPN**. Guest UPN formats, stale mapping tables and case-sensitive source values make the filter match nothing.

Cost of failure **Security**

Status **GA** · Verified **Sep 2026**

	DESKTOP VIEW AS	SERVICE, REAL USER
Identity	The user name you type in	Entra UPN of the signed-in user
Guest users	Mapping table holds user@partner.com	Some configurations return user_partner.com#EXT#@yourtenant.onmicrosoft.com
Mapping table	Built from HR email	UPN changed after a rename or domain move
Tester	Sees filtered rows	Admin, Member and Contributor bypass RLS, so the author sees everything

FIG. 7.07 Desktop tests a name you typed; the service tests the Entra-issued UPN, which your table may lack.

CAUSE

For B2B guests `USERPRINCIPALNAME()` usually returns an email-style value, but some configurations return the `#EXT#` form. HR email drifts from UPN after renames. Test as role evaluates dynamic RLS with your own identity. The model engine ignores case. Learn warns that a case-sensitive DirectQuery source can compare differently, and does not document how RLS behaves after a Direct Lake on SQL fallback.

FIX

Publish a measure `UPN Diag = USERPRINCIPALNAME()` and have the affected user open it. Store exactly that value in the mapping table. Build the table from Entra UPNs, and store guests in the form they return. Store UPNs in lowercase and filter with `[UPN] = LOWER(USERPRINCIPALNAME())`.

PREVENT

After every mapping change, validate with a real Viewer account and a real guest account.

RELATED 7.08 The report works for workspace members 7.13 Secured users wait seconds for Direct Lake...

7.08 The report works for workspace members. Viewers and guests get errors on every visual.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

By default Direct Lake queries OneLake as the **signed-in user**; Contributors and above can read it, Viewers and share recipients cannot. Grant read, or bind a **fixed-identity cloud connection**.

Cost of failure **Outage**
Status **GA** · Verified **Sep 2026**

	WORKSPACE MEMBER	VIEWER, SHARE OR GUEST USER
Report and model	Read through workspace role	Read through share or app
OneLake read	Contributor role implies Read and ReadAll	None: Viewers get no OneLake read
Direct Lake query	SSO as the member, succeeds	SSO as the viewer, access denied
Refresh (framing)	Uses the model owner's permissions	Same: refresh succeeds while queries fail

FIG. 7.08 Report access and data access are separate grants; members hold both and viewers hold only report access.

CAUSE

Direct Lake uses single sign-on by default. Read on the report and model does not grant data access. On Direct Lake on SQL, users also need SELECT through the SQL analytics endpoint.

FIX

Option one: give viewers Read and ReadAll on the lakehouse, or add them to a OneLake security role. Option two: bind the model to a cloud connection with a fixed identity, such as a service principal or workspace identity, and leave SSO disabled. Grant that identity read on the item and give users nothing on it. Define RLS in the semantic model, because the source now sees only the fixed identity.

PREVENT

Choose SSO or fixed identity per model before sharing outside the workspace. Test with a Viewer account and a guest account before publishing the app.

RELATED [7.07](#) RLS with USERPRINCIPALNAME() works in Desktop [7.13](#) Secured users wait seconds for Direct Lake...

7.09 Everyone holds Contributor on the production workspace. One wrong script deleted the Gold lakehouse.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Contributor can **create, change and delete** every item in the workspace, and Member ranks above it. Keep people on **Viewer** in production and deploy with a service principal.

Cost of failure **Data loss**

Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
Viewer DEFAULT	Reads item content and queries data through the SQL analytics endpoint. OneLake security roles apply to this role.	Cannot run, schedule, edit or delete pipelines, notebooks or data. Reruns go through someone with write access.
Contributor	Runs pipelines, notebooks and Spark jobs. Writes to lakehouses and warehouses. Restores deleted items.	Can delete any item and its data. Bypasses OneLake security, so every table is readable.
Member	Everything Contributor has, plus adding people with lower roles and allowing reshare.	Granting it as the "safer" role widens access.
Admin	Workspace settings, Git connection, workspace identity, deleting the workspace.	Can remove other admins and delete everything. Keep it to two named people.

FIG. 7.09 Each step up the role ladder adds destructive rights; only Viewer stays inside OneLake security.

DEFAULT

In production, people get Viewer. A service principal with Contributor deploys through Git or deployment pipelines. Grant Contributor to a person for an incident, then remove it.

SIGNS YOU CHOSE WRONG

More than a handful of named users hold Contributor or Member in production. Engineers run ad hoc notebooks against production items. Role assignments were never reviewed after go-live. Nested groups hide who holds write access.

CHECK IT

List assignments and count everyone above Viewer. A deleted lakehouse, warehouse, pipeline or notebook stays recoverable for 3 days by default; a tenant admin can set 3 to 90.

```
GET https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}/roleAssignments
# value[].principal.displayName, value[].role
```

RELATED

7.10 OneLake security hides the PII tables from...

7.11 Bronze, Silver and Gold share one lakehouse

7.10 OneLake security hides the PII tables from analysts. Engineers and some analysts still read every row.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

OneLake security restricts only the **Viewer** role. Admin, Member and Contributor bypass it, and the lakehouse **DefaultReader** role still grants every table to users with ReadAll.

Cost of failure **Security**

Status **GA** · Verified **Sep 2026**

	USER IS RESTRICTED	USER STILL SEES EVERYTHING
Workspace role	Viewer	Contributor, Member or Admin
OneLake roles held	One custom role scoped to allowed tables	Custom role plus DefaultReader through ReadAll
How roles combine	Union covers only allowed tables	Any role that grants the table wins
SQL endpoint access mode	User's identity: OneLake roles enforced	Delegated: SQL permissions govern tables

FIG. 7.10 The workspace role and DefaultReader decide whether the same OneLake roles restrict a user.

CAUSE

Workspace roles above Viewer carry Write on every item, so OneLake security never evaluates for them. Every lakehouse gets a DefaultReader role whose members are all users with ReadAll. Roles combine as a union and OneLake has no deny, so DefaultReader outweighs a narrow custom role. An endpoint in delegated mode ignores OneLake roles for table access.

FIX

Move data consumers to Viewer. Remove ReadAll from their item shares, or edit DefaultReader so it excludes sensitive tables. Give each group one custom role scoped to schemas or tables, with RLS or CLS where needed. Switch the SQL analytics endpoint to User's identity access mode. The switch cancels running and queued queries on every endpoint in the workspace, so schedule it.

PREVENT

Test with a Viewer account through Spark, the SQL endpoint and a report before sign-off. Role definition changes apply in about 5 minutes, group membership changes in about an hour.

RELATED

7.11 Bronze, Silver and Gold share one lakehouse

7.09 Everyone holds Contributor on the production...

7.12 RLS sits in OneLake security, the SQL...

7.11 Bronze, Silver and Gold share one lakehouse. Gold readers can also query raw Bronze PII.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

OneLake security is GA with **schema, folder and table** scope plus RLS and CLS, so one lakehouse can separate readers. Every Contributor still sees every layer, so it cannot separate **writers**.

Cost of failure **Security**

Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
One lakehouse, schema per layer	One OneLake role per schema for Viewers. One item, one endpoint, one set of shortcuts.	Contributors read and write Bronze. DefaultReader exposes all schemas to ReadAll holders until edited.
Lakehouse per layer, one workspace	Separate item shares and endpoints per layer. Shortcuts let Silver jobs read Bronze.	Workspace roles span all items, so Contributors still reach Bronze.
Workspace per layer DEFAULT	Workspace roles become the layer boundary. Ingestion engineers hold Contributor on Bronze only. Gold consumers are Viewers with OneLake roles.	More workspaces, connections and deployment pipelines. Shortcut readers need a OneLake role or ReadAll at the target.

FIG. 7.11 For each layout, OneLake roles fence readers and only a separate workspace fences writers.

DEFAULT

Give Bronze its own workspace when it holds PII or raw extracts that non-ingestion engineers must not touch. Silver and Gold can share a workspace, with one OneLake role per schema for Viewers.

CHECK IT

Turn on OneLake diagnostics (GA). It writes data access events for the workspace to a lakehouse you choose, which answers who read Bronze.

RELATED

7.10 OneLake security hides the PII tables from...

7.09 Everyone holds Contributor on the production...

7.12 RLS sits in OneLake security, the SQL...

7.12 RLS sits in OneLake security, the SQL endpoint and the model. Nobody knows which rule filtered a row.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Define row and column rules once in **OneLake security**, which Spark, the SQL endpoint in user identity mode and Direct Lake on OneLake enforce. Add model RLS only behind a **fixed identity**.

Cost of failure **Security**

Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
OneLake security only DEFAULT	One role set for Spark, the SQL analytics endpoint and Direct Lake on OneLake. Direct Lake stays in memory.	RLS rules are static single-table predicates. A user cannot hold two roles with different column sets on a table if either applies RLS.
Semantic model RLS, fixed identity	Dynamic DAX rules keyed on USERPRINCIPALNAME(). Consumers need no lakehouse access.	Protects that model only. Anyone with lakehouse access, or Write on the model, bypasses it.
SQL endpoint RLS, delegated mode	CREATE SECURITY POLICY with predicate functions, plus dynamic data masking.	OneLake roles are ignored for tables. Direct Lake on SQL falls back to DirectQuery for tables with RLS.
All layers	Covers paths that skip one layer.	Direct Lake on OneLake unions OneLake roles, then intersects the result with model roles. Every fix needs three rule sets.

FIG. 7.12 Follow each engine's path to see which rules apply; only OneLake security reaches every engine.

DEFAULT

Use OneLake security. Add model RLS with a fixed-identity connection only when consumers have no lakehouse access and need per-user dynamic rules.

SIGNS YOU CHOSE WRONG

One user sees different rows in a notebook and a report. Secured users' visuals run in DirectQuery. A rule change needs three tickets. The SQL endpoint still runs in delegated mode.

CHECK IT

A OneLake RLS rule is a single-table SELECT with a WHERE clause, at most 1,000 characters. Rules for one table across roles combine with OR.

```
SELECT * FROM sales.orders WHERE Region IN ('North', 'West')
```

RELATED

7.13 Secured users wait seconds for Direct Lake...

7.10 OneLake security hides the PII tables from...

7.14 You removed a group from a OneLake security role

7.13 Secured users wait seconds for Direct Lake visuals. Unsecured users on the same report get instant results.

ENGINES PIPELINES DATAFLOW SPARK **ONELAKE** WAREHOUSE SQL ENDPOINT DIRECT LAKE SECURITY CAPACITY

On **Direct Lake on SQL**, a table with SQL endpoint RLS sends the query to **DirectQuery**, so check the query path before the DAX. Then tune model RLS with a small security table and a one-hop filter.

Cost of failure **Slow reports**

Status **GA** · Verified **Sep 2026**

UNSECURED USER	SECURED USER
In memory	DirectQuery
Storage engine scans only	A SQL query per visual

FIG. 7.13 On the same visual, the secured user's query leaves memory and goes to SQL.

MEASURE FIRST

- 1 Run the visual in DAX Studio with Server Timings, connecting as the secured user. DirectQuery events mean fallback, so fix the path before tuning RLS.
- 2 Check whether the model is Direct Lake on SQL or on OneLake. Then open the SQL analytics endpoint Security tab and read the data access mode.
- 3 If the query stays in memory, compare storage engine time secured against unsecured. A large gap points at the role filter.

CAUSE

Direct Lake on SQL checks permissions through the SQL analytics endpoint. A table with SQL RLS, or a view, forces DirectQuery. When a lakehouse endpoint runs in User's identity mode, OneLake roles become SQL rules, and Direct Lake on SQL falls back on every query. Learn says new endpoints start in delegated identity mode, and the May 2026 GA post says User's identity, so read the mode on the Security tab. Direct Lake on OneLake never falls back.

FIX

Move the model to Direct Lake on OneLake and keep RLS in OneLake security. Or keep model RLS behind a fixed-identity connection with SSO off. Filter a flattened security table, one row per user, one hop from the facts.

```
// role filter on DimUserRegion
[UserEmail] = USERPRINCIPALNAME()
```

With a fixed identity, every viewer gets the OneLake roles of that identity. Per-user rules must then live in the model.

EXPECTED RESULT

Secured visuals run from memory, and Server Timings show no DirectQuery events.

RELATED [7.12](#) RLS sits in OneLake security, the SQL... [7.14](#) You removed a group from a OneLake security role

[7.15](#) You added column-level security in the Warehouse

7.14 You removed a group from a OneLake security role. The Direct Lake report still shows them the rows.

ENGINES PIPELINES DATAFLOW SPARK **ONELAKE** WAREHOUSE SQL ENDPOINT **DIRECT LAKE** **SECURITY** CAPACITY

Microsoft says to **refresh Direct Lake on OneLake models** after a OneLake security change, so build the refresh into the change. Group membership changes also take **about an hour** to propagate.

Cost of failure **Security**

Status **GA** · Verified **Sep 2026**

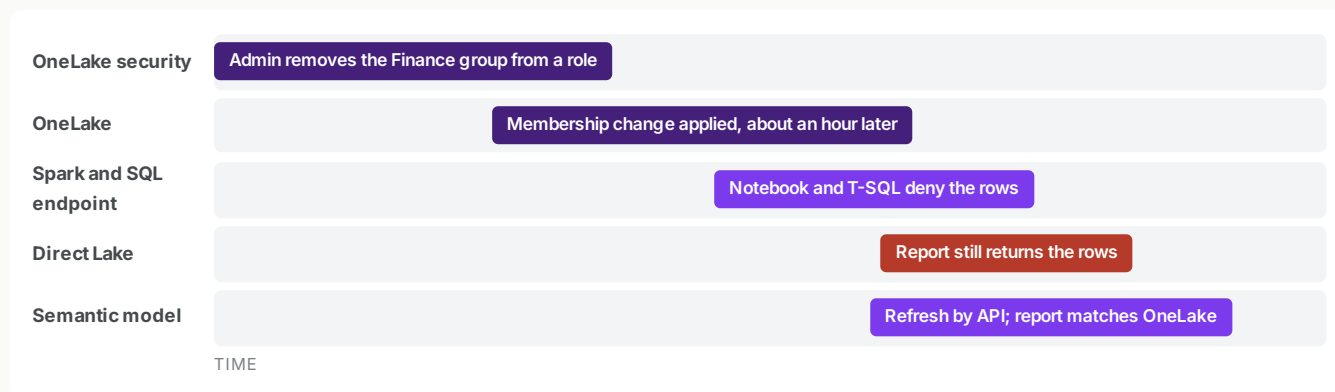


FIG. 7.14 OneLake and the SQL engines agree within the hour; the Direct Lake model lags until someone refreshes it.

SEEN IN

Direct Lake

CAUSE LIVES IN

OneLake

CAUSE

Direct Lake on OneLake resolves OneLake roles for the effective identity. Role definition changes take about 5 minutes in OneLake. Group membership changes take about an hour, and some engines cache for up to another hour. Learn says you must refresh these models manually or through the refresh APIs after a OneLake security change.

FIX

Add a model refresh to the security change procedure for every model that reads the item. After a group change, wait for propagation before refreshing.

```
import sempy.fabric as fabric
fabric.refresh_dataset("Sales model", workspace="Sales BI")
```

CONFIRM IN BOTH ENGINES

```
-- Test as the removed user in T-SQL
SELECT COUNT(*) FROM finance.gl_entries;
-- expect an access error
```

```
-- Refresh by REST
POST
https://api.powerbi.com/v1.0/myorg/groups/{workspaceId}/datasets/{datasetId}/refreshes
```

RELATED [7.12](#) RLS sits in OneLake security, the SQL... [7.13](#) Secured users wait seconds for Direct Lake...

[7.10](#) OneLake security hides the PII tables from...

7.15 You added column-level security in the Warehouse. Visuals slow down or fail, and SELECT * fails.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Warehouse CLS is a static **GRANT SELECT** on a column list, and restricted users get Msg 230 on **SELECT ***. Learn pages disagree on whether Direct Lake **falls back to DirectQuery** or returns an error, so test both.

Cost of failure **Slow reports**
Status **GA** · Verified **Sep 2026**

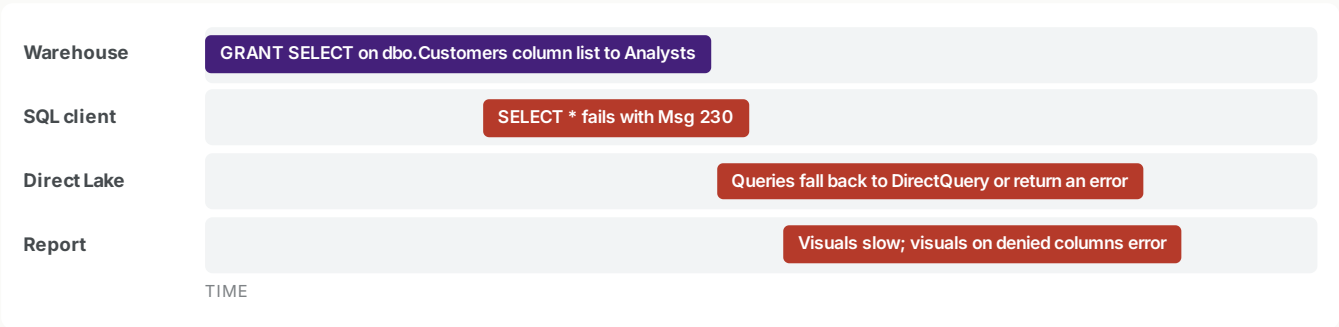


FIG. 7.15 Follow one Warehouse grant down the lanes to slow or failing visuals in the report.

SEEN IN

Direct Lake

CAUSE LIVES IN

Warehouse

CAUSE

CLS is a permission on named columns. The Warehouse CLS page says Direct Lake queries fall back to DirectQuery to abide by CLS. The Direct Lake security page says a query touching a CLS-restricted column returns an error.

FIX

Serve the model through a fixed-identity connection that can read every model column, and hide columns with model OLS. Give ad hoc SQL users a view of the permitted columns.

```
CREATE VIEW dbo.Customers_public AS
  SELECT CustomerID, Name, Region FROM dbo.Customers;
GRANT SELECT ON dbo.Customers_public TO [Analysts];
```

CONFIRM IN BOTH ENGINES

```
-- List column grants
SELECT OBJECT_NAME(p.major_id) AS tbl, c.name
AS col,
       USER_NAME(p.grantee_principal_id) AS
grantee, p.state_desc
FROM sys.database_permissions p
JOIN sys.columns c ON c.object_id =
p.major_id AND c.column_id = p.minor_id
WHERE p.class = 1 AND p.minor_id > 0;
```

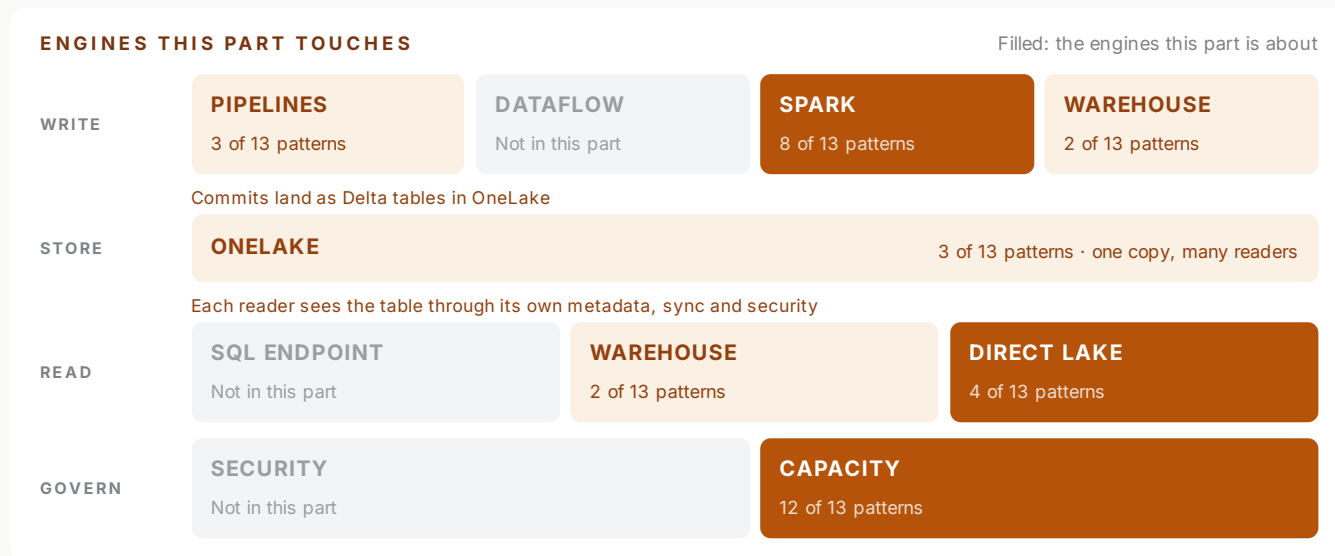
```
-- Performance Analyzer as the user
-- a 'Direct query' duration means fallback;
a visual error means CLS denial
```

RELATED [7.13](#) Secured users wait seconds for Direct Lake... [7.12](#) RLS sits in OneLake security, the SQL...

[7.10](#) OneLake security hides the PII tables from...

08 Capacity and Cost.

Capacity units, smoothing, throttling, queueing and what actually changes the bill.



NO.	PATTERN	PIPE	SPARK	ONELAKE	WH	DL	CAP	PAGE
8.01	Your scheduled notebook ran in 5 minutes...	●	●	●	●	●	●	142
8.02	Your interactive notebook fails with HTTP 430...	●	●	●	●	●	●	143
8.03	Everything is slow at 9am	●	●	●	●	●	●	144
8.04	One runaway job on the shared capacity	●	●	●	●	●	●	145
8.05	A critical workspace stopped working	●	●	●	●	●	●	146
8.06	You doubled the capacity SKU	●	●	●	●	●	●	147
8.07	You moved to a Fabric F SKU	●	●	●	●	●	●	148
8.08	The smallest SKU worked in development	●	●	●	●	●	●	149
8.09	You sized the capacity for the morning peak	●	●	●	●	●	●	150
8.10	Your capacity bill doubled after a new team...	●	●	●	●	●	●	151
8.11	OneLake storage grows every month	●	●	●	●	●	●	152
8.12	Nightly OPTIMIZE and VACUUM run on every table	●	●	●	●	●	●	153
8.13	Your most expensive pipeline runs every 15...	●	●	●	●	●	●	154

8.01 Your scheduled notebook ran in 5 minutes yesterday. Today, same code and data, it took 45.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

The job spent most of that time **queued for Spark cores**. Spark VCores belong to the capacity, so every workspace on it competes at job admission.

Cost of failure **Slow queries**Status **GA** · Verified **Sep 2026**

MEASURE FIRST

- 1 Open the run in the Monitoring hub. A queued job shows **Not started** until cores free up, then **In progress**. Compare submit time with start time.
- 2 Log when the Spark application started, so every run records its own queue delay.

```
import datetime
start = datetime.datetime.fromtimestamp(spark.sparkContext.startTime / 1000)
print(f"Spark app started {start:%H:%M:%S}")
```

- 3 In the Monitoring hub, filter by the same time window across all workspaces on the capacity. Count what else was running.

CAUSE

Each CU gives two Spark VCores, with bursting up to 3x. An F64 has 128 base VCores, 384 with burst. Spark admits a job if the pool's minimum nodes fit. Pipeline and scheduled jobs that do not fit join a FIFO queue, limited by SKU (64 on F64). Queued jobs expire after 24 hours and must be resubmitted.

FIX

Stagger heavy schedules across every workspace on the capacity. Give small jobs a pool with fewer minimum nodes so more fit. An admin can disable job-level bursting so one job cannot take all burst cores. For predictable Spark, enable Autoscale Billing for Spark, which runs Spark outside the capacity on a pay-as-you-go meter.

If the capacity is throttled, new Spark jobs are rejected instead of queued. Enabling, disabling or reducing the Autoscale Billing max CU cancels Spark jobs running under it.

EXPECTED RESULT

Queue time drops to near zero, so run duration reflects compute again.

Go deeper: PySpark Handbook 4.0, pool sizing.

RELATED **8.02** Your interactive notebook fails with HTTP 430... **8.04** One runaway job on the shared capacity

8.03 Everything is slow at 9am

8.02 Your interactive notebook fails with HTTP 430 at peak. The pipeline notebooks wait in the queue.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Spark queues only **pipeline, scheduler and job definition** runs. An interactive session that cannot get its minimum cores is **rejected** with `TooManyRequestsForCapacity`.

Cost of failure **Rework**
Status **GA** · Verified **Sep 2026**

	PIPELINE OR SCHEDULED NOTEBOOK	INTERACTIVE NOTEBOOK
Cores unavailable at submit	Queued FIFO, shows Not started	Rejected: HTTP 430 TooManyRequestsForCapacity
Waiting limit	Queue expires 24 hours after submission	No queue, retry by hand
Queue full	Rejected with HTTP 430	Rejected with HTTP 430
Capacity throttled	Rejected, queueing disabled	Rejected
Trial capacity	No queueing	No queueing

FIG. 8.02 The same core shortage queues batch runs and rejects interactive sessions.

CAUSE

Job admission checks the pool's minimum nodes against free Spark VCores on the whole capacity, burst included. Interactive sessions and notebooks run through the notebook public API are not queued. At peak, batch runs hold the cores and interactive users hit the limit first.

FIX

Cancel a running job in the Monitoring hub to free cores, or wait. Attach interactive work to a pool with a small minimum node count, one Medium node needs 8 VCores. Ask the capacity admin to disable job-level bursting so one batch job cannot hold all 3x burst cores.

PREVENT

Move heavy batch windows away from working hours. For Spark-heavy capacities, enable Autoscale Billing for Spark so Spark no longer competes with reports. When its max CU is reached, batch jobs queue and interactive jobs are throttled.

RELATED

8.01 Your scheduled notebook ran in 5 minutes...

8.04 One runaway job on the shared capacity

8.03 Everything is slow at 9am

8.03 Everything is slow at 9am. The only heavy work ran overnight and finished hours ago.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Background work is **smoothed over 24 hours**, so last night's load still occupies this morning's capacity. Morning spikes push it past 100% and the **throttling stages** start.

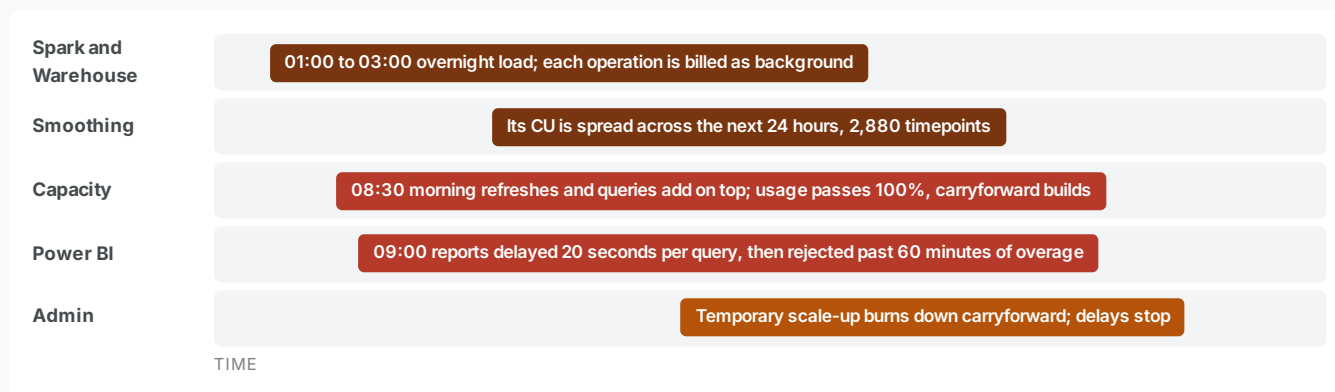
Cost of failure **Slow reports**Status **GA** · Verified **Sep 2026**

FIG. 8.03 The overnight load ends at 03:00, but its smoothed cost sits under the morning peak.

SEEN IN

Power BI reports and interactive queries

CAUSE LIVES IN

Background operations on the same capacity, smoothed from the night before

CAUSE

Interactive operations smooth over at least 5 minutes, background over 24 hours. Spark, Warehouse queries and scheduled refreshes are background. Once smoothed usage exceeds the SKU, overage accrues as carryforward. Up to 10 minutes of future capacity is overage protection. From 10 to 60 minutes, interactive requests are delayed 20 seconds. Between 60 minutes and 24 hours, interactive requests are rejected. Beyond 24 hours, background requests are rejected too. Carryforward only burns down in timepoints with idle capacity.

FIX

Short term, scale the F SKU up temporarily so each timepoint has more idle capacity to burn down. Pausing and resuming clears future usage but bills it at once and takes content offline. Long term, shrink or move the overnight load, or size the SKU for the 24-hour background baseline plus the morning peak.

```
# Azure CLI: temporary scale-up of an F SKU
az fabric capacity update --resource-group <rg> \
  --capacity-name <capacity> --sku "{name:F128,tier:Fabric}"
```

CONFIRM IN BOTH ENGINES

-- Capacity Metrics app, Compute page, Throttling tab
Interactive delay: 10 min Interactive % above 100
Interactive rejection: 60 min Interactive % above 100

-- Overages tab
Cumulative % rising overnight, Burndown % only when idle

Users see CapacityLimitExceeded or "Cannot load model due to reaching capacity limits" while rejected.

RELATED **8.04** One runaway job on the shared capacity **8.01** Your scheduled notebook ran in 5 minutes...
8.08 The smallest SKU worked in development

8.04 One runaway job on the shared capacity. Reports, notebooks and pipelines all slow down together.

ENGINES PIPELINES DATAFLOW SPARK ONELAKE WAREHOUSE SQL ENDPOINT DIRECT LAKE SECURITY CAPACITY

CUs, smoothing and throttling all work at the **capacity** level, shared by every workspace assigned to it. To isolate a workspace, use **surge protection**, separate capacities or Spark on its own meter.

Cost of failure **Outage**
Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
One capacity, no guardrails	Cheapest; idle CU in one workspace serves another.	One heavy workspace drives the whole capacity into delay and rejection stages.
One capacity with surge protection DEFAULT	Capacity-level rejection threshold on 24-hour background %; workspace-level CU limits auto-block the offender; critical workspaces marked Mission Critical.	In-progress jobs keep running. Interactive requests can still be delayed. Blocked teams stop until the block expires or an admin unblocks.
Separate capacities for reporting and engineering	Batch overload cannot throttle reports. Each capacity is sized and billed on its own.	Idle CU cannot be shared across capacities. Two bills to size and watch.
Autoscale Billing for Spark	Spark runs off the capacity on a pay-as-you-go meter, capped by a max CU setting.	Spark cost becomes variable. Reducing the max CU cancels running Spark jobs.

FIG. 8.04 Read the four options from no isolation to Spark billed separately; the default keeps one shared capacity and caps the worst offender.

DEFAULT
Keep one capacity and turn on both surge protection levels. Move to separate capacities, or Autoscale Billing for Spark, when Spark or batch load keeps tripping the limits.

SIGNS YOU CHOSE WRONG
Capacity Metrics app shows one item dominating CU (s). Reports slow while a pipeline runs. System events show SurgeProtectionActive or AllRejected.

CHECK IT

Capacity Metrics app, Compute page: sort Matrix by item and operation by CU (s), then by Overloaded minutes. The top rows are the workloads using the shared capacity.

RELATED **8.05** A critical workspace stopped working **8.03** Everything is slow at 9am

8.01 Your scheduled notebook ran in 5 minutes...

8.05 A critical workspace stopped working. Every operation fails, and the capacity is not full.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Workspace-level **surge protection** blocked it after its 24-hour CU use crossed the admin's limit. Mark it **Mission Critical** or set it back to Available.

Cost of failure **Outage**

Status **GA** · Verified **Sep 2026**

24 h

Workspace-level surge protection blocks a workspace when its CU use over a rolling 24-hour window exceeds a percentage of the capacity set by the admin, checked every 5 minutes.

Source: Microsoft Learn, Surge protection, Sep 2026

HOW YOU WILL NOTICE

All operations in the workspace are rejected. The capacity admin sees the state Blocked in Admin portal, Capacity settings, workspaces table. Admins can enable a banner so users see why.

GUARD

Before enabling the rule, mark workspaces that feed executive reports or regulated outputs as Mission Critical. They are exempt from workspace limits but not from capacity-level throttling.

WORKAROUNDS

Raising or deleting the rule does not unblock a blocked workspace. The admin marks it Mission Critical, sets it to Available after changing the limit, or waits for a timed block to expire. The rolling window does not reset when a block ends, so a heavy workspace can be blocked again at once. Autoscale compute is excluded from the calculation. Dataflows Gen1, paginated reports, scorecards, Graph model, Activator and Dataflow Gen2 editing are not covered. Workspace-level surge protection is GA for F SKUs.

NEIGHBOURING LIMITS

Capacity-level surge protection

Rejects new background operations when 24-hour background % passes the rejection threshold, until it falls below the recovery threshold.

Throttling stages

Delay at 10 minutes of overage, interactive rejection at 60 minutes, background rejection at 24 hours.

HTTP 430 for Spark

Spark admission failure when cores or the queue are full.

RELATED **8.04** One runaway job on the shared capacity **8.03** Everything is slow at 9am

8.02 Your interactive notebook fails with HTTP 430...

8.06 You doubled the capacity SKU. The slow Warehouse query is barely faster.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Warehouse already **bursts** up to 12x baseline on F8 to F2048, repaid through smoothing. A larger SKU raises the concurrency ceiling, but a query limited by its plan or data scanned stays slow.

Cost of failure **Cost**Status **GA** · Verified **Sep 2026**

MEASURE FIRST

- 1 Pull the query's history before and after the change. Compare elapsed time with CPU and data scanned.

```
SELECT TOP 20 start_time, total_elapsed_time_ms,
    allocated_cpu_time_ms, data_scanned_remote_storage_mb
FROM queryinsights.exec_requests_history
WHERE query_hash = '<hash>' ORDER BY start_time DESC;
```

- 2 Check wait before execution. A large gap between `submit_time` and `start_time` points to concurrency; a small one points to the query itself.

CAUSE

Warehouse compute scales per query and borrows future CU through bursting. F2 can burst to 32x, F8 to F2048 to 12x. By default, SELECT and non-SELECT work each get an isolated half of the SKU. A single query that scans too much or has a poor plan was never short of compute, so more CUs change little.

FIX

Fix the query first: filter earlier, cut columns, refresh statistics on join and filter columns. Keep the larger SKU only if Capacity Metrics shows throttling or queued concurrency.

```
UPDATE STATISTICS dbo.fact_sales (stat_customer_key);
```

Maximum burst appears only for seconds within a query, so do not size on it.

EXPECTED RESULT

Elapsed time falls with data scanned.

Go deeper: SQL Handbook, statistics and plans.

RELATED **8.08** The smallest SKU worked in development **8.03** Everything is slow at 9am

8.07 You moved to a Fabric F SKU

8.07 You moved to a Fabric F SKU. Semantic model refreshes still queue, fail or get throttled.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

F SKUs keep **refresh limits**: parallelism by SKU, 48 scheduled refreshes a day, 5 hours per refresh. Refreshes are **background** CU, smoothed over 24 hours with everything else.

Cost of failure **Slow reports**Status **GA** · Verified **Sep 2026**

40

An F64 runs at most 40 semantic model refreshes in parallel; F2 runs 1, F8 5, F32 20, F128 80.

Source: Microsoft Learn, What is Power BI Premium, semantic model SKU limitation, Sep 2026

HOW YOU WILL NOTICE

Refresh history shows long start delays or failures at the same clock times. Pull it per model.

```
GET
https://api.powerbi.com/v1.0/myorg/groups/{groupId}/datasets/{datasetId}/refreshes?
$top=20
```

GUARD

Trigger the refresh at the end of the load pipeline instead of on a fixed schedule, so refreshes run only when data changed and do not stack at the hour.

WORKAROUNDS

Stagger schedules so fewer than the parallelism limit start together. Convert large Import models to Direct Lake, where refresh is a metadata reframe with no full data reload. XMLA refreshes can exceed the 5-hour limit. API refreshes have no fixed daily count but queue when resources run short. If the Capacity Metrics app still shows rejection at peak after this, the SKU is too small.

NEIGHBOURING LIMITS

Max memory per model

25 GB on F64, 3 GB on F2 to F8.

Shared capacity

8 scheduled refreshes a day, 2 hours per refresh.

RELATED **8.03** Everything is slow at 9am **8.08** The smallest SKU worked in development

8.06 You doubled the capacity SKU

8.08 The smallest SKU worked in development. After go-live, reports time out and jobs stall all morning.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Size capacity for **24-hour smoothed load** plus peak interactive demand; an undersized SKU throttles instead of queueing. Below **F64**, every report viewer also needs Pro or PPU.

Cost of failure **Cost**

Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
Smallest SKU that runs the dev load	Low monthly cost.	Production background load fills the 24-hour smoothing window; mornings hit delay and rejection. Viewers need paid licences below F64.
Measure on a trial, then pick the SKU DEFAULT	Sizing comes from real timepoint peaks in the Capacity Metrics app. Trial capacity has 128 Spark VCores, as F64, but no burst.	Trial capacity does not queue Spark jobs, so batch behaviour differs from paid SKUs.
F64 or above for broad reporting	Free-licence users with a viewer role can read Power BI content.	Pays for CU that may sit idle if the user count is small.
Smaller SKU plus capacity overage	Pays 3x the pay-as-you-go rate only for usage that would be throttled, up to an admin-set rolling 24-hour CU-hour limit.	In preview. Does not add resources; sustained load still needs a larger SKU.

FIG. 8.08 Of the four sizing routes, only measuring first sizes for the 24-hour load.

DEFAULT

Run the production-shaped workload on a trial or temporary SKU and read the peak timepoints in the Capacity Metrics app. Choose the SKU that keeps smoothed usage under 100%. The SKU estimator (preview) gives a first guess.

CHECK IT

Capacity Metrics app, Compute page: drill to the highest timepoint and compare CU used per 30 seconds with SKU CU x 30, for example 1,920 on F64.

RELATED

8.03 Everything is slow at 9am

8.07 You moved to a Fabric F SKU

8.04 One runaway job on the shared capacity

SIGNS YOU CHOSE WRONG

Throttling tab crosses 100% on weekday mornings. Spark jobs rejected with HTTP 430. Overages tab carryforward never returns to zero. Viewers without Pro cannot open reports below F64.

8.09 You sized the capacity for the morning peak. It idles most of the day at full price.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Size the SKU for **sustained load**, because Fabric smooths background jobs over 24 hours and a short ETL window rarely needs a peak-sized SKU. Pausing bills **carried-forward usage** at once, so pause with care.

Cost of failure **Cost**

Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
Peak-sized SKU, running 24/7	No orchestration. A reservation lowers the hourly rate.	You pay the peak rate through every idle hour.
SKU sized for sustained load DEFAULT	Bursting runs the peak. Fabric smooths background CU over 24 hours, so idle hours absorb it.	Throttling starts if smoothed usage stays above the SKU. Interactive delay begins past 10 minutes of future usage.
Pause off-hours (pay-as-you-go)	Compute billing stops while paused. Billing is per second, one-minute minimum.	Content is unavailable. Pausing bills accumulated smoothed and carried-forward usage immediately. Reserved CU hours go unused. Storage is still billed.
Scale up for the window, down after	The large SKU runs only during the ETL window.	Needs automation. Crossing between F256 and F512, either way, can cancel running jobs.

FIG. 8.09 Four ways to pay for a daily peak.

DEFAULT

Start with the smallest SKU whose 24-hour smoothed usage stays under 100 percent. Reserve that baseline. Add pause or scaling only for capacities with a clear idle window and no scheduled work in it.

SIGNS YOU CHOSE WRONG

Capacity Metrics app shows utilisation far below 100 percent outside the ETL window. Background operations dominate the CU total. No throttling events in the last 14 days. A dev or test capacity has no users at night.

CHECK IT

Pause through the ARM API from a runbook, after the last job finishes.

POST

https://management.azure.com/subscriptions/{sub}/resourceGroups/{rg}/providers/Microsoft.Fabric/capacities/{name}/suspend?api-version=2023-11-01

resume: same path, /resume

RELATED

8.10 Your capacity bill doubled after a new team...

8.13 Your most expensive pipeline runs every 15...

8.04 One runaway job on the shared capacity

8.10 Your capacity bill doubled after a new team joined. Nobody can say which workload caused it.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

An F SKU bill is **fixed** until an SKU change, **Spark autoscale billing** or capacity overage (preview) moves it. New load shows up as throttling, and someone scaled the SKU to stop it.

Cost of failure **Cost**Status **GA** · Verified **Sep 2026**

3

Three things change an F SKU compute bill: a different SKU, Spark autoscale billing, or capacity overage (preview). Bursting, new workspaces and extra jobs do not.

Source: Microsoft Learn, Capacity overage overview, Sep 2026

HOW YOU WILL NOTICE

Open the Capacity Metrics app, not the Monitoring hub. The hub lists runs, status and duration with no CU. The app's compute page shows CU by workspace, item and operation for 14 days. Check the Azure activity log for an SKU change. Spark autoscale billing and overage appear as separate meters in Azure Cost Analysis.

WORKAROUNDS

Bursting borrows future capacity and repays it through smoothing: interactive over 5 to 64 minutes, background over 24 hours. If repayment falls behind, throttling starts. Overage (preview) buys off throttling at three times the pay-as-you-go rate, up to an admin-set 24-hour cap. Spark autoscale billing moves Spark off the capacity to its own pay-as-you-go meter. Both are opt-in, so a bill that rose without an SKU change points to one of them.

GUARD

Record a 14-day CU baseline per workspace before onboarding. Give the new team its own capacity, or have them read your tables through a shortcut in their workspace. Shortcut transactions count against the capacity of the workspace holding the shortcut. Queries they run on items in your workspace use your CU.

RELATED **8.09** You sized the capacity for the morning peak **8.13** Your most expensive pipeline runs every 15...

8.04 One runaway job on the shared capacity

8.11 OneLake storage grows every month. Your scheduled VACUUM never makes it smaller.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

VACUUM removes only files that no current version references, so it does **not enforce retention**. Delete old rows first, then VACUUM; soft delete keeps the removed files **billed for 7 more days**.

Cost of failure **Cost**
Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
Keep everything	Nothing to build. Any old load can be replayed.	The storage bill grows with every load, even while the capacity is paused.
DELETE past a window per layer, then VACUUM DEFAULT	Old rows leave the current version. VACUUM then removes their files once they pass the 7-day threshold.	Time travel before the VACUUM point is gone. Files stay billed 7 more days in soft delete.
Overwrite or drop by load date	Dropping a whole table or replacing a date range is cheaper than a row-level DELETE.	Needs date partitioning or one table per period.
Move cold data to your own ADLS Gen2, read by shortcut	Cool and archive access tiers cost less than hot storage.	Archive tier is offline. Reads cost extra transactions on the storage account.

FIG. 8.11 Retention needs two steps and two waits before storage drops.

DEFAULT

Set a window per layer from reprocessing and legal needs. Bronze keeps what replay requires, Gold keeps the reporting horizon. Run DELETE then VACUUM on a schedule and track size per layer.

SIGNS YOU CHOSE WRONG

No table has a DELETE or overwrite in its history. Bronze holds loads older than any replay would use. The summed DESCRIBE DETAIL size of a workspace's tables is far below its storage in the Capacity Metrics app. VACUUM runs but the storage line in Azure never drops.

CHECK IT

The 168-hour default protects concurrent readers.

```
DELETE FROM bronze.events WHERE load_date < date_sub(current_date(), 90);
VACUUM bronze.events RETAIN 168 HOURS;
-- files leave OneLake billing 7 days later
```

RELATED

8.12 Nightly OPTIMIZE and VACUUM run on every table

8.09 You sized the capacity for the morning peak

8.10 Your capacity bill doubled after a new team...

8.12 Nightly OPTIMIZE and VACUUM run on every table. Most never changed, and the job still burns capacity.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

OPTIMIZE is idempotent and does not rewrite unchanged tables, but the job still starts Spark, reads every log and lists files. Maintain only tables with **new commits**, and let **auto compaction** handle hot ones.

Cost of failure **Cost**Status **GA** · Verified **Sep 2026**

MEASURE FIRST

- 1 In the Capacity Metrics app, find the maintenance notebook or pipeline on the compute page. Note its CU seconds per night.
- 2 For each table, check whether anything but maintenance ran since last night.

```
DESCRIBE HISTORY silver.dim_product LIMIT 3;
-- only OPTIMIZE / VACUUM END rows: nothing to do
```

CAUSE

The job loops over hundreds of tables whatever their state. Each OPTIMIZE reads the Delta log and plans bins even when nothing needs compacting. Each VACUUM lists the table's files to find unreferenced ones. Weekly-updated dimensions pay that cost seven times for one change.

FIX

Skip tables whose latest commit is maintenance. Turn on fast optimize, which skips bins unlikely to improve. Set `delta.autoOptimize.autoCompact` on frequently written tables. It is off by default.

```
spark.conf.set("spark.microsoft.delta.optimize.fast.enabled", "true")
for t in tables:
    op = spark.sql(f"DESCRIBE HISTORY {t} LIMIT 1").first()["operation"]
    if op not in ("OPTIMIZE", "VACUUM END"):
        spark.sql(f"OPTIMIZE {t}")
```

Optimize write is off in the default writeHeavy resource profile, so frequent small writes create many small files.

EXPECTED RESULT

Maintenance CU now tracks the number of changed tables. VACUUM weekly for slow tables is enough, since the retention threshold is 7 days.

Go deeper: PySpark Handbook 4.0, Delta table maintenance.

RELATED **8.11** OneLake storage grows every month **8.13** Your most expensive pipeline runs every 15...

8.10 Your capacity bill doubled after a new team...

8.13 Your most expensive pipeline runs every 15 minutes. The report it feeds is opened twice a day.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Run frequency is a **cost multiplier**, so compare 96 runs a day with 2. Schedule to the moment people read the data, and agree that freshness with the business.

Cost of failure **Cost**

Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
Every 15 minutes	Data is never more than 15 minutes old.	96 runs a day. Each run's CU, writes and downstream refresh repeat. Small writes add maintenance work.
Scheduled before each read window	One or two runs a day.	A late source file waits for the next run.
Triggered by source arrival	Runs only when new data lands.	A chatty source can trigger more often than any schedule.

FIG. 8.13 The report's read times decide how fresh the data needs to be, and so which of the three schedules fits.

DEFAULT

Ask the report owners when they read the data. Schedule a run to finish before each read. Record the agreed freshness next to every schedule faster than daily.

SIGNS YOU CHOSE WRONG

The pipeline is a top background item in the Capacity Metrics app. Report usage metrics show opens clustered at one or two times a day. A semantic model refresh follows every pipeline run. Nobody can name who asked for the frequency.

RELATED

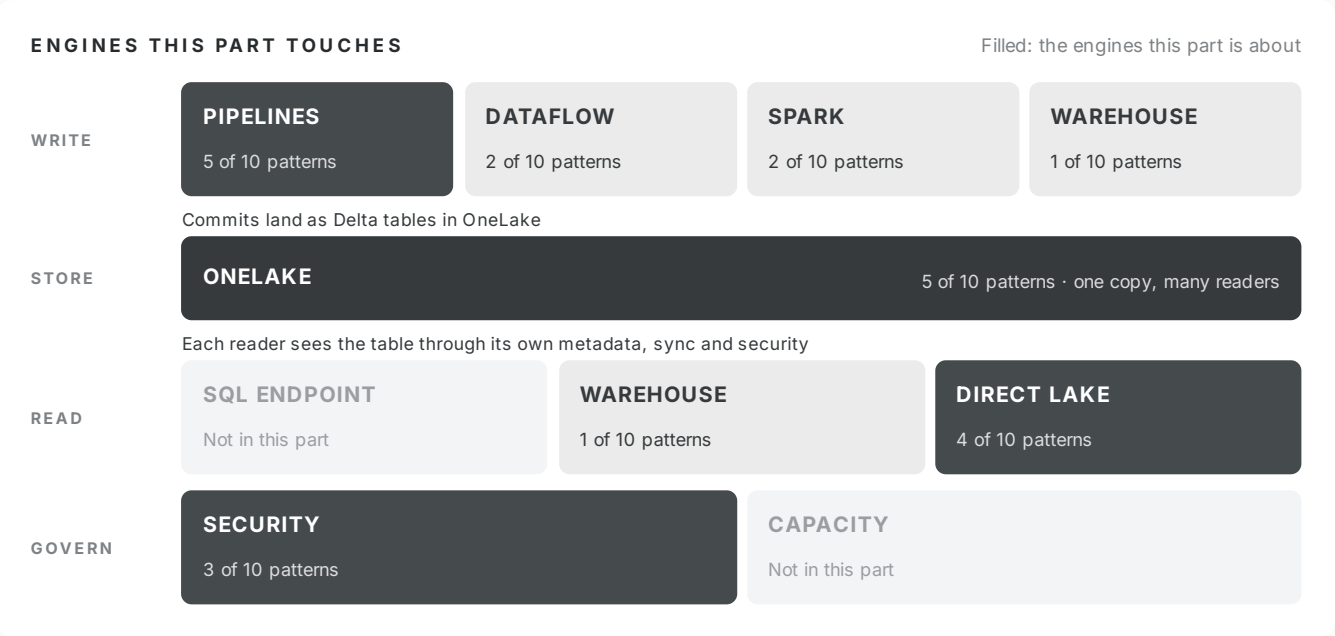
8.09 You sized the capacity for the morning peak

8.10 Your capacity bill doubled after a new team...

8.12 Nightly OPTIMIZE and VACUUM run on every table

09 CI/CD and Operations.

Deployment pipelines, Git, variable libraries, fabric-cicd and recovery. Promotion copies definitions, not data or bindings.



NO.	PATTERN	PIPE	DF	SPARK	ONELAKE	DL	SEC	PAGE
9.01	You edited a shared connection	●	●	●	●	●	●	156
9.02	Your deployment to production succeeded	●	●	●	●	●	●	157
9.03	You renamed columns in a shared semantic model	●	●	●	●	●	●	158
9.04	The deployment promoted the new model correctly	●	●	●	●	●	●	159
9.05	Your fabric-cicd deployment succeeded	●	●	●	●	●	●	160
9.06	Dev and prod share one workspace	●	●	●	●	●	●	161
9.07	Dev and prod have separate workspaces	●	●	●	●	●	●	162
9.08	Every promotion succeeds	●	●	●	●	●	●	163
9.09	A notebook was saved over in the workspace	●	●	●	●	●	●	164
9.10	A deployment damaged production warehouse tables	●	●	●	●	●	●	165

9.01 You edited a shared connection. Pipelines in several workspaces failed at their next run.

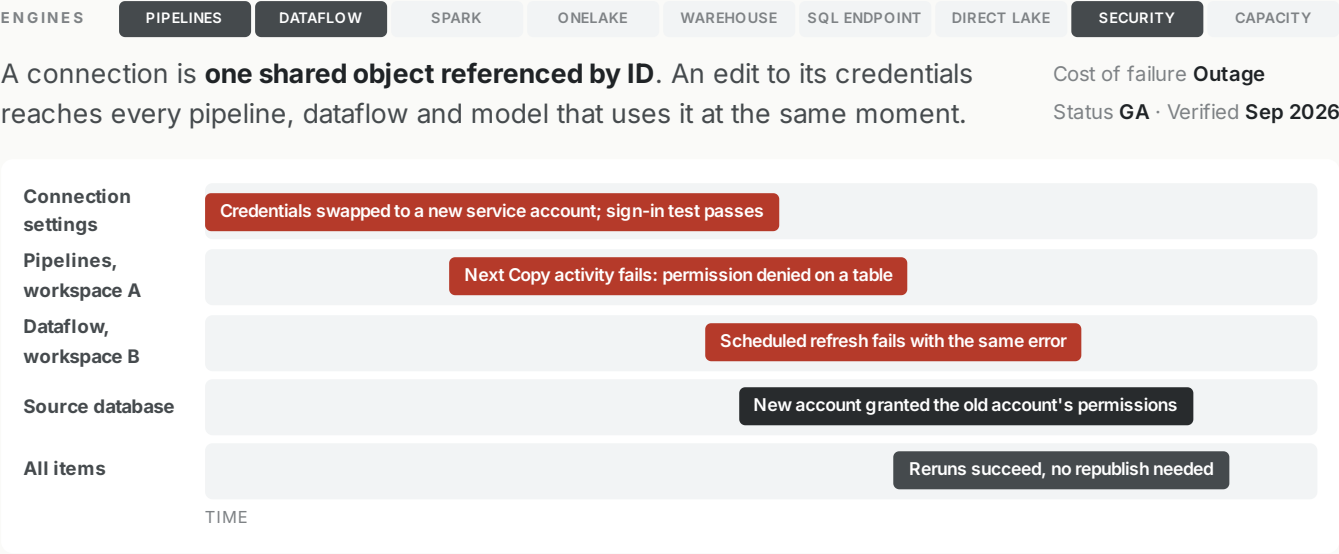


FIG. 9.01 Follow the lanes: one edit in connection settings fails items in several workspaces, and granting permissions at the source fixes all of them.

SEEN IN
Pipelines and dataflows in every workspace that uses the connection

CAUSE LIVES IN
The connection object in Manage connections and gateways

CAUSE
Items store the connection ID, so a credential change reaches all of them on their next run. A successful sign-in does not prove the account can read every table those items query. Republishing a pipeline changes nothing, because it still points at the same ID. The Update Connection API accepts credentials, display name and privacy level. It cannot change the server or database. A server move needs a new connection with a new ID; repoint every item to it.

FIX
Grant the new account the old account's permissions before swapping. For a server move, create a new connection, repoint one pipeline, test it, then move the rest. Keep connection IDs in pipeline parameters so one search finds every user.

```
# Find every item in the repo bound to the connection
git grep -l "<connection-id>" -- '*.json'
```

CONFIRM IN BOTH ENGINES

```
-- Check the new account's rights on the
source
EXECUTE AS USER = 'svc_fabric_new';
SELECT * FROM
fn_my_permissions('sales.orders', 'OBJECT');
REVERT;
```

```
-- List connections and their IDs
GET
https://api.fabric.microsoft.com/v1/connections
```

Copy the ID from the connection's Settings pane in Manage connections and gateways.

RELATED **9.02** Your deployment to production succeeded **9.05** Your fabric-cicd deployment succeeded

9.02 Your deployment to production succeeded. The production report still shows development data.

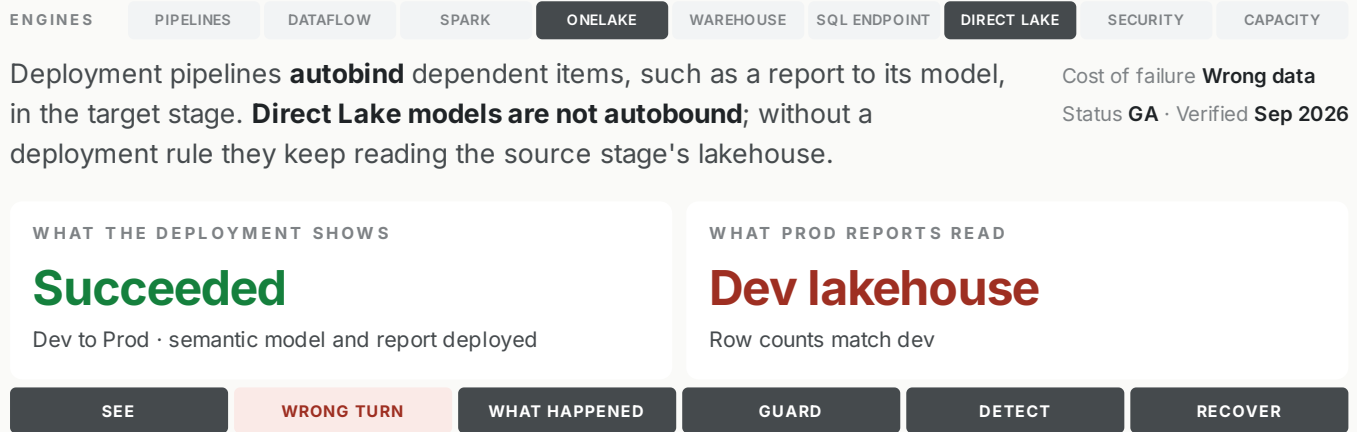


FIG. 9.02 The deployment shows Succeeded while the prod report still reads the dev lakehouse.

WRONG TURN

"The pipeline remaps connections for us. The prod lakehouse must be behind."

WHAT HAPPENED

Deployment copies item metadata, including data source definitions, and keeps the link between paired items. A report binds to the paired model in the target stage. A Direct Lake model does not, so the prod copy still binds to the dev lakehouse. No error appears because the dev lakehouse is valid and reachable.

1 GUARD

Set a rule on Test and Prod before the first deployment. Direct Lake on SQL: a data source rule mapping server and database to the target SQL analytics endpoint. Direct Lake on OneLake does not support data source rules. Community write-ups put the OneLake URL in a model parameter and set a parameter rule. Test it before relying on it. Only the item owner can create rules, and not in the development stage.

```
expression OneLakeURI = "https://onelake.dfs.fabric.microsoft.com/<ws-id>/<lh-id>" meta
[IsParameterQuery=true, Type="Text", IsParameterQueryRequired=true]
```

2 DETECT

In DAX query view on the prod model, read the source expressions. A dev workspace or endpoint ID means the rule is missing.

```
EVALUATE
SELECTCOLUMNS(INFO.EXPRESSIONS(), "Name",
[Name], "Source", [Expression])
```

3 RECOVER

Add the rule and deploy again. Rules take effect only on the next deployment. Then refresh the prod model, because a data source change needs a full refresh.

```
POST
https://api.powerbi.com/v1.0/myorg/groups/{workspaceId}/datasets/{datasetId}/refreshes
{"type": "Full"}
```

RELATED **9.04** The deployment promoted the new model correctly **9.01** You edited a shared connection
9.05 Your fabric-cicd deployment succeeded

9.03 You renamed columns in a shared semantic model. Reports built on it now show broken visuals.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Reports reference model fields **by table and field name**. A rename removes that name, and publishing the model does not check reports stored as separate items.

Cost of failure **Rework**
Status **GA** · Verified **Sep 2026**

	IN THE REPORT SAVED WITH THE MODEL	IN SEPARATE REPORTS ON THE MODEL
Rename a column	Visuals follow the rename	Visuals lose the field and show an error
Rename a measure	Visuals follow the rename	Visuals lose the field and show an error
Change a measure's DAX	New result	New result
Warning at publish	None	None

FIG. 9.03 The rename carries through in the report saved with the model and breaks visuals in separate reports.

CAUSE

Power BI Desktop rewrites field references only in the report it has open. Reports in other files or workspaces keep the old name. Their visuals break the moment the renamed model is published.

FIX

Run impact analysis on the model before the change. It lists downstream reports, dashboards and workspaces, with views and distinct viewers over the last 30 days. Items you cannot open appear as limited access. You need write permission on the model. Use **Notify contacts** to email the contact list of every affected workspace. Then update the dependent reports in the same release. Or add the new field, move reports to it, and remove the old name later.

PREVENT

Keep reports in PBIR format in Git beside the model, so a search for the old name finds every visual that uses it.

RELATED 9.04 The deployment promoted the new model correctly 9.02 Your deployment to production succeeded

9.04 The deployment promoted the new model correctly. Users still see figures from before it.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Deployment pipelines copy **metadata only**: no data and no refresh schedule. Cost of failure **Wrong data**
Refresh the target model after every deployment. Status **GA** · Verified **Sep 2026**

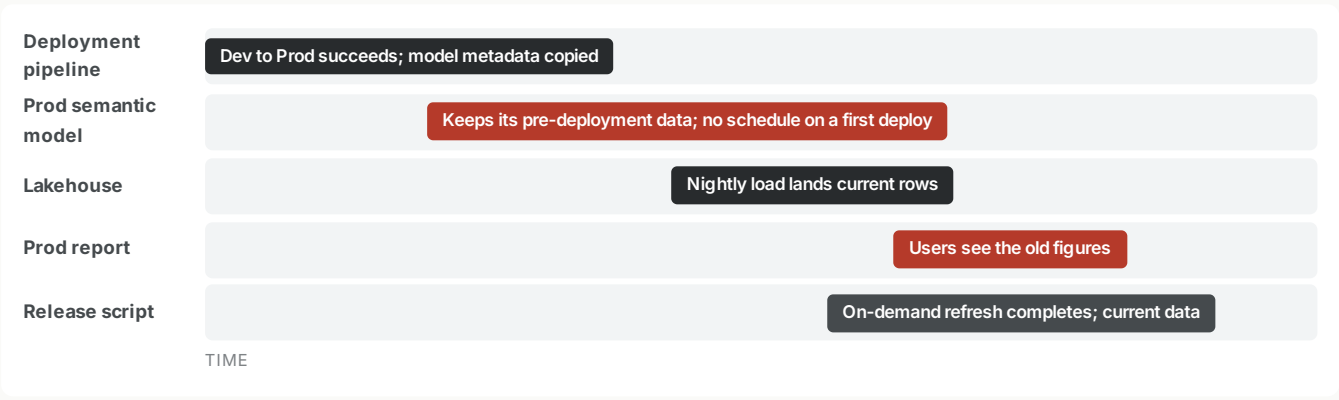


FIG. 9.04 Follow the lanes: the deployment succeeds, and the prod report shows old figures until the release script runs an on-demand refresh.

SEEN IN

Prod reports

CAUSE LIVES IN

The deployment step, which copies no data and no schedule

CAUSE

Deployment keeps the target's existing data when it can. Breaking schema changes or a data source change need a full refresh. Refresh schedule, data source credentials, role assignments and endorsement are not copied, so a newly created prod model never refreshes. Direct Lake models reframe on their own while **Keep your Direct Lake data up to date** is on, the default. A non-recoverable refresh error suspends that until an on-demand refresh succeeds.

FIX

Make a refresh the last step of every release, poll it, and only then announce the release. Set the refresh schedule and credentials once per stage.

```
POST https://api.powerbi.com/v1.0/myorg/groups/{workspaceId}/datasets/{datasetId}/refreshes
{"type": "Full"}
// Poll until status is Completed
GET https://api.powerbi.com/v1.0/myorg/groups/{workspaceId}/datasets/{datasetId}/refreshes?
$top=1
```

CONFIRM IN BOTH ENGINES

```
-- Does the prod model have a schedule?
GET https://api.powerbi.com/v1.0/myorg/groups/{workspaceId}/datasets/{datasetId}/refreshSchedule
```

The Direct Lake tab in refresh history lists failures and status changes, not every successful framing. If the numbers match dev, the model is still bound to the dev lakehouse (9.02).

RELATED 9.02 Your deployment to production succeeded 9.03 You renamed columns in a shared semantic model

9.05 Your fabric-cicd deployment succeeded. Prod notebooks and pipelines still point at dev items.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

fabric-cicd rewrites only **logical IDs of items in the repo** and the default workspace ID. Real GUIDs in definitions pass through unchanged unless `parameter.yml` maps them.

Cost of failure **Wrong data**
Status **GA** · Verified **Sep 2026**

	IN THE DEV WORKSPACE	AFTER FABRIC-CICD DEPLOYS
Pipeline runs a notebook in the same repo	Runs the dev notebook	Runs the prod notebook: logical ID replaced
Notebook default lakehouse	Dev lakehouse	Still the dev lakehouse unless find_replace maps it
Refresh Semantic Model activity	Dev model	Still the dev model unless parameterised
Item renamed in the repo	Same item	New item with a new GUID; the old one is orphaned

FIG. 9.05 Read the prod column: references stored as logical IDs follow the deployment, and dev GUIDs stay behind unless parameter.yml maps them.

CAUSE

fabric-cicd matches repo items to workspace items by type and display name. Before publishing, it replaces every logical ID from a repo `.platform` file with that item's deployed GUID. It also swaps the all-zero workspace ID for the target workspace. Anything else, such as a lakehouse GUID in notebook metadata, is copied as is. A rename breaks the name match, so the target gets a new item. `unpublish_all_orphan_items` then deletes the old one and anything outside the repo holding its GUID breaks.

FIX

Add a `find_replace` entry in `parameter.yml` for each dev GUID, with `$items.Lakehouse.Sales_LH.$id` or `$workspace.$id` as the per-environment value. Recreate a renamed item's external references, or rename in the target workspace first so the names match.

PREVENT

Never edit a `logicalId`. A copied item folder needs a new unique one, and fabric-cicd rejects duplicates. Before each release, list GUIDs in definitions that are not in a `.platform` file or `parameter.yml`.

RELATED 9.02 Your deployment to production succeeded 9.01 You edited a shared connection

9.06 Dev and prod share one workspace. A developer's test write truncated a production Gold table.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

A workspace role grants the same rights on **every item** in the workspace, including items named `dev_`. Give each stage its own workspace and promote with a **deployment pipeline**.

Cost of failure **Data loss**
Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
One workspace, dev_ and prod_ prefixes	One set of items and roles to manage.	Every Contributor can write, delete and run every prod item. A wrong table name hits prod.
One workspace per stage, deployment pipeline DEFAULT	Roles are set per workspace, so developers get Contributor in dev and Viewer or nothing in prod. The pipeline deploys metadata only, never data.	Connections need binding per stage (see 9.08). Someone must own promotion.
Workspace per stage on separate capacities	Adds compute isolation: a heavy dev job cannot throttle prod.	Costs a second capacity.

FIG. 9.06 Every item inside one workspace shares one permission set, whatever its name.

DEFAULT

One workspace per stage. Developers hold Contributor in dev only; prod writes come from the prod identity and from deployments.

CHECK IT

List who can write in the prod workspace.

```
GET https://api.fabric.microsoft.com/v1/workspaces/{prodWorkspaceId}/roleAssignments
# Admin, Member, Contributor can write all items
```

RELATED 9.07 Dev and prod have separate workspaces 9.08 Every promotion succeeds
2.27 Your pipeline run succeeded

SIGNS YOU CHOSE WRONG

Item names carry dev_ or prod_ prefixes. Developers hold Contributor or higher where prod tables live. Prod fixes are edited in place. Nobody can say which items feed the reports.

9.07 Dev and prod have separate workspaces. A dev pipeline still overwrote a table prod reads.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Both workspaces read one shared Lakehouse, and the dev identity held **Contributor** on it. A shortcut or a direct path writes to the **target data** whenever the caller can.

Cost of failure **Data loss**Status **GA** · Verified **Sep 2026**

WHAT THE DEV RUN SHOWS

SucceededTest load of `gold.customers` in the dev workspace

SEE

WRONG TURN

WHAT HAPPENED

WHAT PROD READS

Test rows

Prod reports now show the dev sample

GUARD

DETECT

RECOVER

FIG. 9.07 The dev run succeeded and prod now reads test rows: both workspaces write to one set of files.

WRONG TURN

"The shared Lakehouse is read-only for dev. Everyone knows that."

WHAT HAPPENED

OneLake authorises a shortcut with the caller's identity against the target. The dev pipeline wrote through the path, and the write landed in the prod table.

1 GUARD

Remove the dev identity's workspace role on the shared Lakehouse. Share the item instead with Read all with Apache Spark or Read all with SQL analytics endpoint. Sharing never grants write. Dev writes go to a dev Lakehouse; dev reads prod through shortcuts.

2 DETECT

The prod table history shows an operation from a run nobody scheduled in prod.

```
DESCRIBE HISTORY gold.customers;
-- check operation, timestamp,
operationParameters
```

3 RECOVER

Restore the last good version. It stays until VACUUM removes its files, 7 days by default.

```
RESTORE TABLE gold.customers TO VERSION AS
OF 57;
```

RELATED

9.06 Dev and prod share one workspace

9.08 Every promotion succeeds

2.27 Your pipeline run succeeded

9.08 Every promotion succeeds. Afterwards prod pipelines and models point at dev data again.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Deployment copies item definitions, including their connections. **Deployment rules** do not cover pipelines, which bind through **variable library** references with a value set per stage.

Cost of failure **Wrong data**
Status **GA** · Verified **Sep 2026**

	IN DEV	IN PROD AFTER DEPLOYMENT
Pipeline connection	Dev SQL connection	Same dev connection, copied with the definition
Lakehouse or workspace ID	Dev IDs hardcoded	Dev IDs, so writes land in dev
Semantic model source	Dev server	Prod server if a data source or parameter rule exists
Notebook default Lakehouse	Dev Lakehouse	Prod Lakehouse if a default lakehouse rule exists
Manual fix in prod	n/a	Overwritten by the next deployment

FIG. 9.08 Read the prod column: items with a deployment rule or variable library reference rebind per stage, and the rest keep dev's values.

CAUSE

Deployment rules exist for semantic models, Dataflow Gen1, paginated reports, mirrored databases and notebooks (default Lakehouse only). They cannot be set in the first stage. Pipelines, Copy jobs and Dataflow Gen2 carry whatever values their definition holds.

FIX

Create a variable library in each workspace with a value set per stage. Use String or Guid variables for IDs, and connection or item reference variables for connections. Reference them in the pipeline as `@pipeline().LibraryVariables.SourceLH`. In the prod stage, set the Prod value set as active. Number variables are not supported in pipelines; use Integer or String. Add deployment rules for the item types that support them.

PREVENT

Search every pipeline definition in Git for literal workspace, item and connection GUIDs before merge. Any hit is a binding that will not move.

RELATED **9.06** Dev and prod share one workspace **9.07** Dev and prod have separate workspaces

9.09 A notebook was saved over in the workspace

9.09 A notebook was saved over in the workspace. Weeks of logic need to come back.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

Workspace **version history** for notebooks is in preview and lives with the item. Cost of failure **Rework**

Git integration is the GA, durable history; every commit stays recoverable after Status **GA** · Verified **Sep 2026**

the notebook is deleted.

OPTION	GOOD AT	BREAKS WHEN
Workspace only	Nothing to set up. Version history (in preview) keeps a system checkpoint every 5 minutes for 1 year, with Restore and Save as copy.	History goes with the item. It has no review or branching, and it is in preview.
Git integration, feature branches DEFAULT	Each workspace syncs to a branch in Azure DevOps or GitHub. Commits give diffs, pull requests and history that survives deletion.	Uncommitted edits are unprotected. Commit is a manual step in the workspace.
Git plus deployment pipeline to prod	Prod changes only by promotion from a reviewed branch. Nobody edits prod notebooks.	Hotfixes must go through the same path.

FIG. 9.09 Checkpoints sit inside the item; Git history sits outside it and survives an overwrite or a delete.

DEFAULT

Connect every workspace that holds production code to Git. Develop on a feature branch workspace, merge by pull request, promote to prod.

SIGNS YOU CHOSE WRONG

No workspace shows a Source control status. Prod notebooks are edited in place. Recovery plans mention browser cache or copy and paste.

CHECK IT

Ask the Git API whether the workspace is connected.

```
GET https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}/git/connection
# gitConnectionState: NotConnected means no durable history
```

RELATED 9.08 Every promotion succeeds 9.06 Dev and prod share one workspace

9.10 A deployment damaged production warehouse tables. You need the data back, and fast.

ENGINES

PIPELINES

DATAFLOW

SPARK

ONELAKE

WAREHOUSE

SQL ENDPOINT

DIRECT LAKE

SECURITY

CAPACITY

A **zero-copy clone** AT a past time recovers one table, and **restore in place** rolls the whole warehouse back to a restore point. Delta **RESTORE** covers Lakehouse tables.

Cost of failure **Data loss**
Status **GA** · Verified **Sep 2026**

OPTION	GOOD AT	BREAKS WHEN
Clone table AT a timestamp DEFAULT	Metadata only, near instant. Any UTC time inside the retention window: 30 days by default, configurable up to 120 days (preview). The live table keeps running.	Same warehouse only. Cannot go back past a schema change on the table. Not on the SQL analytics endpoint.
Restore in place from a restore point	Rolls back every table, view and schema object together. System points every 8 hours; user-defined points on demand.	Overwrites the whole warehouse: good writes after the point are lost. Workspace Admin only.
Delta RESTORE on a Lakehouse table	One statement back to any version still on disk.	Lakehouse only. VACUUM removes old files, 7 days retention by default.

FIG. 9.10 Pick the option by scope: clone for one warehouse table, restore in place for the whole warehouse, Delta RESTORE for a Lakehouse table.

DEFAULT

Create a user-defined restore point before every warehouse deployment. On damage to a few tables, clone them AT the pre-deployment time and copy rows back. If the schema changed or damage is widespread, restore in place.

SIGNS YOU CHOSE WRONG

Row counts dropped right after a deployment. A column type or table definition changed in prod. Reports broke at the deployment timestamp.

CHECK IT

Clone the table as it stood before the deployment, then compare.

```
CREATE TABLE dbo.orders_pre AS CLONE OF dbo.orders
  AT '2026-09-28T06:00:00.000'; -- UTC
SELECT COUNT(*) FROM dbo.orders_pre;
```

RELATED **9.06** Dev and prod share one workspace **9.07** Dev and prod have separate workspaces

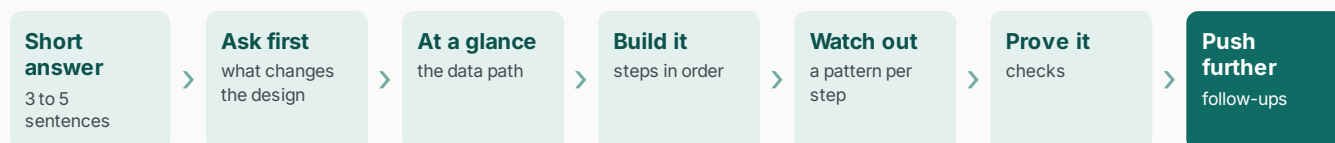
2.27 Your pipeline run succeeded

S

System Design Scenarios.

Thirty design briefs, from a nightly file load to a platform handover. Each answer runs the way I'd explain it to a colleague: the short answer, the questions I'd ask first, the build in order, and the trap at each step. The traps link to the patterns in Parts 1 to 9.

HOW EACH ANSWER RUNS



BUILDING ON FABRIC

S.01	Nightly store files over SFTP	168
S.02	Metadata-driven load, twelve sources	170
S.03	A 3 billion row fact read by date and region	174
S.04	Month-end figures frozen while loads run	177
S.05	Three business units, one customer list	180

AZURE DATA FACTORY AND FABRIC

S.11	300 ADF pipelines moved in a year	199
S.12	ADF lands the files, Fabric takes over	202
S.13	Runtime and gateway, same servers	205
S.14	Replacing ADF mapping data flows	209
S.15	ADF orchestrates Fabric jobs	212

SECURITY AND REGULATED DATA

S.21	Patient records from four systems	231
S.22	Branch-level row security for a bank	235
S.23	External partners read part of Gold	238
S.24	Every source behind a firewall	241
S.25	Who read the sensitive table last quarter	244

MIGRATING INTO FABRIC

S.06	Synapse dedicated SQL pool to Fabric	183
S.07	SQL Server and 150 SSIS packages	186
S.08	Databricks stays, Fabric reads in place	190
S.09	Premium import models to Direct Lake	193
S.10	Oracle ERP, 800 tables, no CDC	196

FRESHNESS AND REAL TIME

S.16	Ten minutes behind a busy table	215
S.17	No change tracking, no modified date	219
S.18	Records up to 30 days late	222
S.19	Operations dashboard in trading hours	225
S.20	Silver without half-finished loads	228

RUNNING THE PLATFORM

S.26	One capacity, three workloads	247
S.27	Dev, test and prod for two teams	250
S.28	The bill rose 60 percent	253
S.29	Recovering from a bad Monday release	256
S.30	Handover when the only engineer leaves	259

S.01 Design the nightly sales load for a retailer. Its 120 stores each send a CSV file to an SFTP server by 02:00, and dashboards must be ready by 07:00.

Short answer. I'd build one metadata-driven pipeline in **Data Factory** in Microsoft Fabric. A control table lists every store and where its file lands. The pipeline reads that table, copies each store's file from SFTP into the Lakehouse, logs what arrived, then runs a notebook that builds Bronze, Silver and Gold. The report refreshes only after Gold is complete.

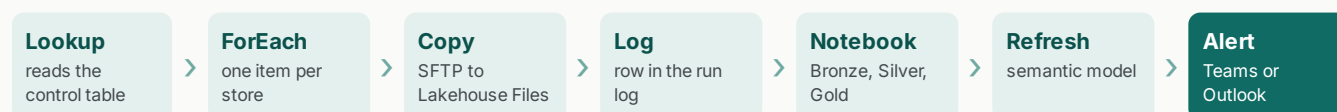
QUESTIONS TO ASK FIRST

- **Where does the SFTP server sit, and how do we log in?** This decides the gateway and the connection type.
- **Can a store send its file again, or late?** Then each load has to replace that store's day, not add to it.
- **If one store is missing, do we wait?** Usually not. Publish at 03:00 and show which store is missing.
- **Will stores be added or closed often?** If yes, adding a store must be a row in a table, not a pipeline change.

WHICH TOOL, AND WHY

- **Data Factory pipeline.** My choice. It gives a control table, a loop per store, logging, retries, a schedule and alerts in one place.
- **Copy job.** Good for straight table copies with little logic. This load needs per-store logging and missing-store checks, so it fits less well.
- **Dataflow Gen2.** Good for low-code shaping by analysts. For 120 files a night with a run log, a pipeline plus a notebook is easier to operate.
- **Notebooks only.** Possible, but you rebuild scheduling, retries and alerting that **Data Factory** already gives you.
- **Azure Data Factory.** Keep it if the retailer already runs ADF. It can land the files in OneLake, and everything after landing stays the same.

AT A GLANCE



One **Data Factory** pipeline, scheduled at 02:30. Data moves from SFTP to Files, then to Bronze, Silver and Gold, and finally to the Direct Lake report.

HOW I'D BUILD IT

1 Set up the SFTP connection

I create a **Data Factory** connection to the SFTP server with the host, port and a service account. Fabric's SFTP connector uses basic authentication, so it needs a user name and password. If the server only accepts SSH keys, that is a conversation with the store team before anything else. If the server sits behind a firewall, the connection runs through an on-premises or virtual network data gateway. I share the connection with the data team's group, not with one person.

Watch out. A connection owned by one engineer stops every load when that person leaves. **7.03**

2 Build the control table

A small table in the Lakehouse or a Warehouse lists each store: store ID, SFTP folder, file name pattern, expected arrival time, target table and an active flag. A second table, the run log, gets one row per store per night: file name, row count, status and load time. Adding a store means adding a row. Nobody edits the pipeline.

Watch out. Lookup returns at most 5,000 rows. 120 stores is fine, but a list that grows past that is cut off without an error. [2.10](#)

3 Copy the files, store by store

A Lookup activity reads the active stores. A ForEach loops over them and passes each row to a Copy activity as parameters: folder, file pattern and destination path. The Copy lands each file untouched in the Lakehouse Files area, under a folder for tonight's date. After each copy, a small step writes the result to the run log.

Watch out. ForEach runs 20 items at a time by default and 50 at most. Set the batch count on purpose. [2.08](#)

4 Load Bronze

A notebook activity reads tonight's files with a fixed schema and appends them to a Bronze table. Every row keeps the store ID, the file name and the load time.

Watch out. If types are guessed from the CSV, every column arrives as text. [2.05](#)

5 Clean into Silver

In Silver, the notebook replaces each store's day instead of adding rows. A store that sends its file again gets its old rows replaced, not doubled.

Watch out. A retry after a partial failure can add the same rows twice. [2.26](#)

6 Build Gold and refresh the report

Gold holds daily totals by store and product. The report reads Gold with Direct Lake. I turn off automatic updates on the model, and a Semantic model refresh activity at the end of the pipeline updates it once Gold is finished.

Watch out. A refresh can succeed and the report still show yesterday, if it ran before Gold finished. [6.04](#)

7 Handle missing stores and alerts

At 03:00 the pipeline compares the run log with the control table. Missing stores trigger an alert, and the report shows them as missing instead of as zero sales. I send it with the Outlook 365 Email activity or the Microsoft Teams activity. The legacy Teams and Outlook activities cannot be added to pipelines from 30 October 2026. Email connections belong to one user and are not shared. So the email lives in one small alert pipeline that other pipelines call with Invoke pipeline. Its connection uses the workspace identity or a service principal instead of a person's account.

Watch out. A night with no files can empty a table, and the run still says Succeeded. [2.27](#)

SECURITY IN ONE BREATH

Stores can only upload to the SFTP server. The pipeline reads it through a shared connection. Engineers work in the workspace, and report users get Gold through an app, so they never see Bronze or Silver.

HOW I'D KNOW IT WORKS

- The run log has one row per active store, every night.
- Rows copied per store match the rows in Bronze for that store and day.
- No store has the same receipt twice in Silver.
- After the pipeline finishes, the report shows tonight's date, and any missing store is named.

IF THEY PUSH FURTHER**What if there were 1,200 stores, each sending a file every hour?**

The control table design stays. Switch the start from a clock to a storage event when each file lands. That needs the files to arrive in OneLake or Azure Blob Storage. Stop one file from starting the pipeline twice. **2.14**

What if the SFTP server only allows SSH keys?

Fabric's SFTP connection uses basic authentication. Either the store team adds a password login for the service account, or an existing Azure Data Factory pipeline, which supports SSH keys, lands the files in OneLake first.

What if the budget were cut in half?

Pause or shrink the capacity outside the load window. Pausing charges for recent use straight away, so time it after the load. **8.09**

S.02 Design a metadata-driven ingestion framework that loads twelve source systems of mixed types into a Lakehouse. About one new source arrives each month, and onboarding it should take a connection and some configuration rows, with no pipeline edits.

Short answer. I'd build one **Data Factory** pipeline in Fabric, driven by a control table in a Warehouse. A Lookup reads the objects that are due, a ForEach loops over them, and a Switch picks the Copy activity for each source type. Each Copy takes its connection ID and watermark query from the control table and lands raw data in the Lakehouse. A notebook then merges everything into Bronze and Silver. A run log records every object on every run. Onboarding a source of a known type means one new connection and a few new rows.

QUESTIONS TO ASK FIRST

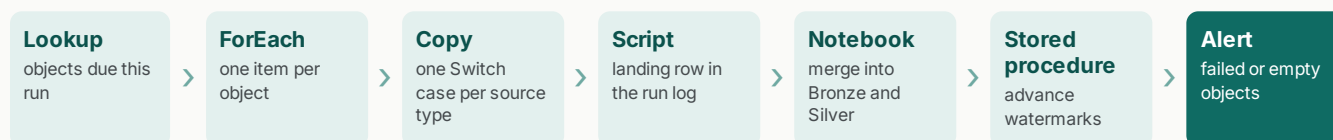
- **Which source types are in the twelve, and where do they sit?** Each type needs its own Copy branch and connection type. On-premises sources need an on-premises data gateway, and sources inside an Azure virtual network need a virtual network data gateway.
- **Does every table have a reliable change column?** Tables with a modified date or rowversion load incrementally with a watermark. Tables without one load in full, or through CDC where the source supports it.
- **Do deletes in the source need to reach the Lakehouse?** A watermark never sees deletes. If they matter, I add CDC, a periodic full key comparison, or soft delete flags from the source.
- **Who will add new sources, and what review do they need?** If several people add rows, the control table needs validation and history. I keep the rows in a seed script in Git and apply them through the release.

- **How fresh must each source be?** Most sources can share one daily schedule. A source that needs hourly data gets its own frequency value in the control table.

WHICH TOOL, AND WHY

- **Data Factory pipeline with a control table.** My choice. One pipeline handles every source type and keeps watermarks and the run log in one place. A new source of a known type needs no pipeline edits.
- **Copy job per source.** Copy job tracks its own watermark and supports incremental copy and CDC. Twelve Copy jobs mean twelve places to schedule and monitor. So I use one only where CDC is needed, and call it with the Copy job activity.
- **Mirroring.** Good for a supported database that needs a near real-time replica. It covers a fixed list of supported sources, mostly databases, so files and APIs still need a pipeline.
- **Dataflow Gen2.** Good when analysts own the shaping. It is harder to drive twelve mixed sources from one control table.
- **Notebooks only.** Possible for files and APIs. I would rebuild scheduling, retries, gateway access and alerting that **Data Factory** already gives me.

AT A GLANCE



One scheduled **Data Factory** pipeline. Every source follows the same path, and only the Copy branch inside the Switch differs by source type.

HOW I'D BUILD IT

1 Create the connections and gateways

I create one connection per source in Manage connections and gateways. On-premises SQL Server and Oracle go through an on-premises data gateway cluster with two members. Fabric pipelines need gateway version 3000.214.2 or later. Azure sources behind a private network use a virtual network data gateway, which is GA for pipelines and Copy job. Cloud sources that accept Microsoft Entra ID use a service principal or the workspace identity. Workspace identity is not supported on gateway connections, so gateway sources use a service account. The pipeline refers to connections by their ID, a GUID, so I copy each ID from the connection's settings. I share every connection with the data engineering group.

Watch out. A connection built on one engineer's OAuth sign-in stops every load that uses it when that account is disabled. [7.03](#)

2 Build the control table and run log

I keep the metadata in a Warehouse, because the pipeline writes to it and the Lakehouse SQL analytics endpoint is read-only. The object table has one row per source object. It holds source ID, source type, connection ID, and schema and object name or file path. It also holds load mode, watermark column, current watermark, key columns, target table, frequency, active flag and owner. The run log gets one row per object per run. It holds the run ID, rows read, rows copied, start and end time, status and error text. For example, twelve sources with 20 to 60 objects each give a few hundred rows.

Watch out. Lookup returns at most 5,000 rows and drops the rest without failing, and its output is capped at 4 MB. Filter the query to the objects due this run. [2.10](#)

3 Loop over objects with a Switch per source type

A Lookup activity queries the control table for active objects that are due. A ForEach passes each row on as @item(). I set its batch count on purpose, for example 8, so the gateway and sources are not overloaded. Inside the loop, a Switch activity branches on source type, with one case per connector, such as SQL Server, Oracle, REST and SFTP files. Each case holds one Copy activity whose connection uses dynamic content set to the connection ID from the row. The query, folder and file name come from the row too.

Watch out. Pipeline variables are global to the run, so a Set variable inside a parallel ForEach can hand one object's value to another. Pass values with @item() and activity outputs. [2.09](#)

4 Copy incrementally with watermarks

For incremental objects, the source query reads rows above the stored watermark. It reads up to and including a high mark fixed at the start of the run. I take the high mark once per run from the source, or from pipeline().TriggerTime. The Copy writes Parquet to Lakehouse Files under source, object and run date. Full-load objects skip the filter and replace their landing folder. The watermark itself does not move yet.

Watch out. Rows updated without touching the watermark column, or committed late with an older timestamp, fall behind it and never load. Overlap the window by a few minutes and let the merge handle the repeated rows. [2.01](#)

5 Merge the landed data into Bronze and Silver

After each Copy, a Script activity writes the landing row to the run log from the Copy output. When the loop ends, one Notebook activity runs with the run ID as a parameter. It reads that run's log rows and loads each landed folder with the schema declared in the control table. Then it merges into Bronze on the key columns. Every Bronze row keeps the source ID, run ID and load time. Silver applies types, removes duplicates and makes names consistent. One Spark session for all objects costs far less than a session per object.

Watch out. A retry after a partial failure appends the same rows again when the write is an append. Merge on keys so a rerun changes nothing. [2.26](#)

6 Advance watermarks only after a good merge

A stored procedure in the Warehouse takes the run ID. For each object the notebook marked as merged, it sets the watermark to that run's high mark and closes the run log row. Objects that failed keep their old watermark, so the next run reads the same window again. The procedure also compares the number of log rows with the Lookup count and fails the run when they differ.

Watch out. An empty Lookup, from a wrong filter or a cleared active flag, sends the run down a do-nothing path and every activity succeeds. Fail the run when zero objects are due on a normal day. [2.28](#)

7 Onboard a new source

A new source of a known type needs a connection, rows in the object table and its Silver mapping, which I also keep as rows. The rows go in the seed script in Git and reach test and production through the release, with each stage holding its own connection IDs. The first source of a new type, the first REST API for example, needs one new Switch case, once. I run a new source once in test, check the run log and row counts, then set it active in production.

Watch out. IDs copied from dev keep pointing at dev after deployment. Keep workspace, Lakehouse and Warehouse IDs in a variable library with a value set per stage. [9.08](#)

8 Schedule and alert

The pipeline runs on a schedule, for example 01:00 daily, and the frequency column decides which objects are due. A last step lists failed, skipped or empty objects from the run log. It emails the owners named in the control table with the Outlook 365 Email activity. Email connections belong to one user, so that connection uses the workspace identity. Production changes are published by a deployment identity so that runs do not depend on a person.

Watch out. A pipeline runs as the user who last modified it. When that person leaves, the schedule keeps firing and every run fails. [7.01](#)

SECURITY IN ONE BREATH

Source credentials live only in the shared connections, never in the control table. The pipeline identity can write to the metadata schema in the Warehouse, and engineers change the control table only through the release. Report users read Silver or Gold through the SQL analytics endpoint or a semantic model and never see the landing files.

HOW I'D KNOW IT WORKS

- Every run has one run log row per due object, and the count matches the Lookup output.
- Rows copied per object match rows merged into Bronze for that run.
- Watermarks move only for objects that merged, and rerunning the same window adds no rows.
- A test source added as rows only is loaded on the next run with no pipeline publish.

IF THEY PUSH FURTHER

What if there were 300 sources instead of twelve?

The design holds, but ForEach runs at most 50 items at once and each gateway has its own limits. I'd split by source type into child pipelines called with Invoke pipeline, and cap concurrency per gateway from the control table. [2.08](#)

What if a source has to replicate deletes?

A watermark sees inserts and updates only. A Copy job with CDC incremental copy also handles deletes. It needs CDC enabled on the source and a connector that supports it. The pipeline calls the Copy job with the Copy job activity.

What if one REST source answers slowly?

The REST source waits 100 seconds for each response by default, and one slow page fails the Copy. I store a timeout per source in the control table and pass it to the Copy. [2.17](#)

S.03 Design storage, layout and maintenance for a 3 billion row sales fact that grows by about 20 million rows a day. Most queries filter by date and region, and they come from Power BI reports and from SQL users.

Short answer. I'd keep the fact as one Delta table in a Lakehouse, written by Spark on Runtime 2.0. It has no partitioning and uses liquid clustering on sale date and region. V-Order is on for this table because Direct Lake reads from it, and a nightly OPTIMIZE keeps files and row groups large. SQL users query the table through the SQL analytics endpoint. Power BI reads a daily aggregate table through Direct Lake. The detail table already sits at the F128 guardrail of 3 billion rows and passes the F256 limit within months.

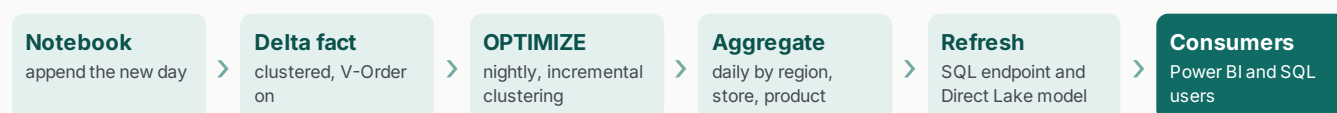
QUESTIONS TO ASK FIRST

- **Which capacity SKU is this on, and can it grow?** Direct Lake allows 1.5 billion rows per table on F64, 3 billion on F128, 6 billion on F256 and 12 billion on F512. That decides whether the detail table can be Direct Lake at all.
- **How far back do reports and SQL users look?** If Power BI needs only recent months at detail, the aggregate carries the history and SQL serves the rest.
- **Do old days get corrected after they load?** Corrections mean MERGE on old dates, which rewrites files and forces Direct Lake to reload those columns. Pure appends are cheapest.
- **Does anything else write to this table at the same time?** Concurrent writers on disjoint slices are the case where Learn still recommends partitioning. One daily writer does not need it.
- **Which Spark runtime can the workspace use?** Liquid clustering reclusters incrementally only on Runtime 2.0. On Runtime 1.3 it rewrites whole Z-cubes each run, so I'd pick another layout there.

WHICH TOOL, AND WHY

- **Liquid clustering on sale date and region.** My choice on Runtime 2.0. It skips files on both columns, reclusters incrementally during OPTIMIZE, and the SQL analytics endpoint and Direct Lake can read clustered tables.
- **Partition by day.** About 20 million rows a day may land below the 1 GB per partition that Learn targets. Years of days also give thousands of folders and small files.
- **Partition by month with Z-Order.** My fallback on Runtime 1.3. Partitions are large and few, and Z-Order by region inside OPTIMIZE helps region filters. Moving to liquid clustering later needs a full rewrite.
- **Fabric Warehouse with data clustering.** Data clustering in the Warehouse is in preview and must be set at CREATE TABLE. It fits when the loads are T-SQL, and here Spark writes the fact.

AT A GLANCE



A nightly **Data Factory** pipeline appends the day, compacts, rebuilds the aggregate, then refreshes the SQL analytics endpoint and frames the Direct Lake model once.

HOW I'D BUILD IT

1 Pick the runtime and write settings

I put the fact's workspace on Runtime 2.0, which is GA. Microsoft planned to make it the default for new workspaces in late September 2026. On an older workspace I select it in the workspace settings. On Runtime 2.0, liquid clustering reclusters only new or changed data. New workspaces default to the writeHeavy resource profile, which leaves V-Order off. Because Direct Lake reads this table, I set `delta.parquet.vorder.enabled` on it, or use the `readHeavyForPBI` profile for the Gold notebooks. V-Order costs about 15 percent on write time.

Watch out. Delta 4.2 specific features in Runtime 2.0 are experimental and work only in Spark. Turn one on for a shared table and the SQL analytics endpoint or Direct Lake can lose sight of it. [5.04](#)

2 Create the table with liquid clustering

I create the table with `CLUSTER BY (sale_date, region)` and no `PARTITIONED BY`. Learn now recommends liquid clustering for most tables and keeps partitioning for concurrent writers, with at least 1 GB per partition. Clustering columns can change later without rewriting data. The existing 3 billion rows move in with one `CTAS` into the new table, followed once by `OPTIMIZE FULL`.

Watch out. Partitioning and liquid clustering cannot be combined on one table. An existing partitioned table has to be rewritten to switch.

3 Load each day as an append

The daily notebook appends the new day. Appends leave existing Parquet files alone, so Direct Lake can frame incrementally and keep most columns in memory. Late corrections go through a `MERGE` that includes a `sale_date` filter, so only files for those dates are touched.

Watch out. An overwrite of the whole table erases the Delta history Direct Lake uses for incremental framing, and every column reloads cold on the next query. [6.08](#)

4 Compact and recluster every night

Auto compaction is off by default. I turn it on for this table, with file-level compaction targets so compacted files are not rewritten again. A nightly `OPTIMIZE` after the load clusters the new day and keeps files near the 1 GB target. `VACUUM` runs weekly with the default 7 day retention. Direct Lake also counts Parquet files and row groups per table, 5,000 of each on F64 to F256. At 3 billion rows that means at least 600,000 rows per row group on average.

Watch out. Nothing compacts a Fabric table unless you enable it. Daily small appends slowly push the table past the file and row group guardrails. [3.01](#)

5 Serve SQL users through the SQL analytics endpoint

SQL users query the fact through the Lakehouse SQL analytics endpoint, which reads the same Delta files. Filters on `sale_date` and `region` should skip most files, and I confirm that with a test query before go-live. After the load, the pipeline runs the Refresh SQL analytics endpoint activity so the endpoint sees the new day before users query it.

Watch out. The endpoint picks up Spark commits through a background sync, so a query that runs straight after the load can still see yesterday. [5.01](#)

6 Size Direct Lake against the guardrails

At 3 billion rows, the detail table sits at the F128 limit. At 20 million rows a day it adds about 7.3 billion rows a year. So it passes F256 within months and F512 in about fifteen months. The Direct Lake model therefore reads a daily aggregate by date, region, store and product, built by the same notebook. For example, that is a few hundred million rows. Detail questions go to the SQL analytics endpoint or to a DirectQuery drill-through report. I use Direct Lake on OneLake for the model.

Watch out. Direct Lake on OneLake has no DirectQuery fallback. A table past a guardrail fails refresh and queries until it is back under the limit. **6.01**

7 Frame the model once per load

I turn off automatic updates on the semantic model. The pipeline runs the semantic model refresh activity after OPTIMIZE and the aggregate finish, so users see one complete day. OPTIMIZE runs before framing because it replaces Parquet files, and Direct Lake has to transcode the affected columns again.

Watch out. Automatic updates reframe after every commit, so reports can show the detail appended and the aggregate still from yesterday. **6.05**

SECURITY IN ONE BREATH

Engineers write through the engineering workspace. SQL users get Read all with SQL analytics endpoint on the Lakehouse, and report users get the app. If some users may see only some regions, I put that rule in OneLake security or the model. SQL endpoint RLS would push Direct Lake on SQL to DirectQuery.

HOW I'D KNOW IT WORKS

- The fact's file count stays well under 5,000 and its average file size stays near the target after each nightly OPTIMIZE.
- A SQL query for one region and one week scans a small share of the data compared with a full-table query.
- Daily totals in the aggregate match the detail table for every day.
- Refresh history for the model shows one framing per load and no guardrail errors.
- After the pipeline ends, the SQL endpoint and the report both show the new day.

IF THEY PUSH FURTHER

What if the capacity is F64?

F64 allows 1.5 billion rows and 5,000 files per Direct Lake table. The aggregate fits and the detail never will, so I choose the aggregate grain to stay well under 1.5 billion rows after years of growth.

What if the workspace must stay on Runtime 1.3?

Liquid clustering there rewrites every file in a Z-cube on each OPTIMIZE, and Learn says not to combine it with auto compaction. I'd partition by month and Z-Order by region in the nightly OPTIMIZE.

What if Power BI users insist on detail rows?

A DirectQuery report over the SQL analytics endpoint, reached by drill-through from the aggregate report, keeps detail off Direct Lake memory. I test its speed on real filters before promising it.

S.04 Design a month-end close where finance figures are frozen once signed off, while daily loads keep writing to the same tables. Reports must show live numbers and the signed-off version of every closed month.

Short answer. I'd keep the finance Gold tables in a Fabric Warehouse and freeze each signed-off month into snapshot tables keyed by period and version. At sign-off, a **Data Factory** pipeline creates a zero-copy clone of each table at the recorded sign-off time. It then inserts that month's rows into the snapshot tables. During the close, a warehouse snapshot gives finance a stable, read-only view while loads continue. Month-end reports read the snapshot tables through Direct Lake, and time travel only covers the short gap between cutoff and freeze.

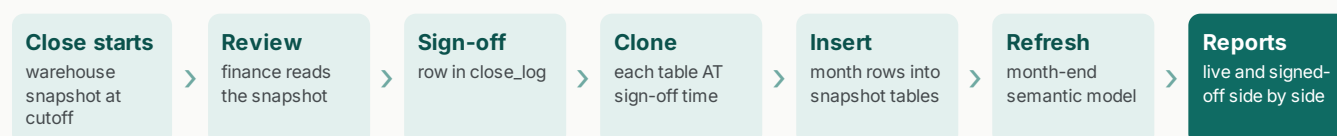
QUESTIONS TO ASK FIRST

- **What exactly is frozen? Is it the summary tables, or every detail row behind them?** Summaries freeze cheaply. Freezing detail too multiplies storage by the months kept, so I agree the grain with finance first.
- **How long must a signed-off month stay reproducible?** Audit usually means years. Warehouse history is kept for 30 days by default and at most 120, so anything longer needs real snapshot tables.
- **Can a closed month ever be reopened?** If yes, every freeze gets a version number and all signed-off versions stay. Reports show the latest one.
- **Are the finance tables in a Warehouse or a Lakehouse today?** Clones, warehouse snapshots and T-SQL time travel are Warehouse features. In a Lakehouse the same design uses Delta time travel in a notebook.
- **Which storage mode do the finance reports use?** Warehouse snapshots work with DirectQuery or Import only. Direct Lake reports must read ordinary tables.

WHICH TOOL, AND WHY

- **Snapshot tables by period and version.** My choice for the signed-off record. Rows are copied once per close into insert-only tables that stay forever, work with Direct Lake and do not depend on any retention setting.
- **Warehouse snapshot.** A read-only view of the whole warehouse at one timestamp that can be rolled forward. Ideal during the close, but limited to the retention period, deleted with the warehouse, and unavailable to Direct Lake.
- **Zero-copy table clone.** Instant, and it stays until someone drops it. One clone per table per month means many tables for reports to union, so I use clones as the exact source for the snapshot insert.
- **Time travel alone.** Fine for looking back a few days. Warehouse history ends at the retention period and Lakehouse history ends when VACUUM removes files, so it cannot be the record of a close.

AT A GLANCE



Daily loads never stop. The close reads a snapshot, and sign-off copies the approved state into tables that nothing else writes.

HOW I'D BUILD IT

1 Record each close in a control table

A `close_log` table in the Warehouse holds one row per period and version, with the period, cutoff time, sign-off time, approver, status and row counts per table. The freeze pipeline reads it and writes back to it. Nothing is frozen without a row here, and every row names who approved it. I store every time in UTC.

Watch out. Time travel and clones take timestamps in UTC. A sign-off time recorded in local time freezes the wrong moment.

2 Set retention to cover the close

The Warehouse keeps data history for 30 days by default. Configurable retention, which is in preview, allows 1 to 120 days through `ALTER DATABASE`. I set it long enough to cover the whole close plus a margin, for example 45 days. Retention governs time travel, clones at a past time, restore points and snapshot timestamps.

Watch out. A shorter retention silently limits how far back a clone or snapshot can point. Lowering it is blocked while an existing snapshot would fall outside the window.

3 Give finance a stable view during the close

When the close starts, I create a warehouse snapshot, a read-only child item of the Warehouse, with its timestamp at the cutoff. Finance reviews against it while daily loads keep writing to the parent. If the cutoff moves, `ALTER DATABASE` on the snapshot with `SET TIMESTAMP` moves it instantly, and the connection string stays the same. The review reports use DirectQuery or Import, because warehouse snapshots do not support Direct Lake.

Watch out. A snapshot disappears with its parent warehouse and can only point inside the retention period. It serves the review and can never be the archive.

4 Freeze at sign-off with clones

When finance signs off, the controller starts the freeze pipeline with the period as a parameter, and the pipeline records the sign-off time in `close_log`. A Script activity then creates a zero-copy clone of each finance table at that time. For example, `CREATE TABLE close.gl_2026_09 AS CLONE OF dbo.gl AT '2026-10-03T16:00:00'`. A clone copies only metadata, shares the Parquet files and stays until someone drops it. The pipeline therefore reads exactly what finance approved, even if loads ran after sign-off.

Watch out. Clones work only inside one warehouse. Cloning across warehouses or workspaces is not supported, so the close schema lives in the same Warehouse.

5 Write the snapshot tables

A stored procedure inserts the period's rows from each clone into a snapshot table such as `gl_monthend`, adding period, version and sign-off time. The snapshot tables are insert-only by design. The procedure writes row counts and totals back to `close_log` and compares them with the clone. I keep the clones for an agreed time for audit, then drop them.

Watch out. A freeze that runs twice for one period doubles the month. The procedure checks `close_log` and fails if that period and version already exist. [2.26](#)

6 Point reports at the frozen version

The month-end semantic model reads the snapshot tables in Direct Lake. It has a period dimension and a version filter that defaults to the latest signed-off version. The live model keeps reading the daily tables. One report page can put live and signed-off figures side by side, so late postings after close are visible. The pipeline refreshes the month-end model only after the insert has committed.

Watch out. A refresh that runs before the insert commits frames the old table, and the report shows last month as the latest close. [6.04](#)

7 Handle reopened months

If finance reopens a month, the next freeze writes version 2 for that period and version 1 stays untouched. Audit can compare versions. Reports show the highest version for each period.

Watch out. Updating signed-off rows in place destroys the record audit relies on. Corrections always arrive as a new version.

SECURITY IN ONE BREATH

Only the pipeline's identity writes to the close schema. Finance users read through the Viewer role or item sharing. Engineers do not hold Contributor on the production workspace, because a Contributor can alter or drop any table in it.

HOW I'D KNOW IT WORKS

- For each closed period, row counts and totals in the snapshot table match the clone taken at sign-off.
- A load after sign-off changes the live report and leaves the month-end report unchanged.
- Running the freeze twice for the same period and version fails with a clear message and adds no rows.
- Every period in the month-end model has a close_log row with a named approver.

IF THEY PUSH FURTHER

What if the finance tables are in a Lakehouse?

A notebook reads each table with VERSION AS OF the version current at sign-off and appends it to a snapshot Delta table. VACUUM keeps unreferenced files for 7 days by default, so the freeze has to run well inside that window.

What if monthly detail snapshots make storage grow too fast?

Freeze summaries at the grain finance reports on, and keep detail only for recent closes. A clone shares files with its source, so it also keeps old files alive after the source changes.

What if a bad load damages the live tables during the close?

The snapshot and the frozen tables are unaffected. A clone AT a time before the bad load, or a restore point, recovers the live table. [9.10](#)

S.05 Design the workspace, domain and data sharing layout for three business units that each run their own Fabric workspace but share customer records. One team should own customer data, and nobody should keep extra copies of it.

Short answer. I'd give customer data its own workspace and Lakehouse, owned by one customer data team and placed in a shared domain. Each business unit keeps its own domain and workspaces and reads the customer tables through OneLake shortcuts that point at the owning Lakehouse. OneLake security roles on that Lakehouse decide which rows and columns each unit sees, and the owning team endorses it as Master data. Units build reports with Direct Lake on OneLake, so each user's own identity is checked through the shortcut.

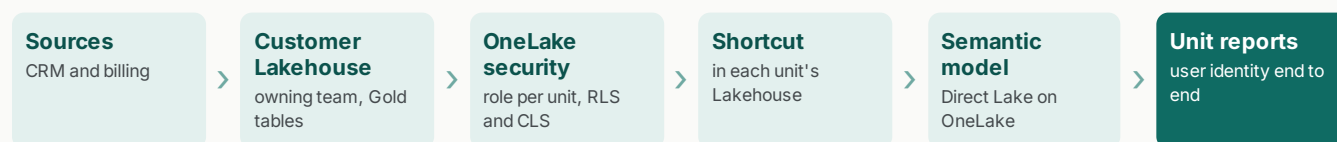
QUESTIONS TO ASK FIRST

- **Does every unit see every customer, or only its own?** If units see only their own customers, each unit's OneLake security role gets a row filter on a unit column.
- **Which customer fields are sensitive?** Fields such as date of birth or tax ID get column-level security, so most readers never see them.
- **Who fixes a wrong customer record today?** The owning team needs a route for corrections from the units, such as a request queue or a staging table it merges from.
- **Do the units share a capacity or pay separately?** Separate capacities keep one unit's heavy load from throttling the others and let each unit pay for its own use.

WHICH TOOL, AND WHY

- **OneLake shortcuts to one owning Lakehouse.** My choice. One physical copy, one owner, and each unit sees the tables in its own Lakehouse without a pipeline.
- **Copy customer tables into each unit.** Three copies load on different schedules and drift, and reports disagree about the same customer.
- **One shared workspace for all units.** Fewer items, but a workspace role grants rights on every item, so units could see and change each other's work.
- **External data sharing.** Built for users in another tenant. For users in the same tenant, Learn points to shortcuts.

AT A GLANCE



Customer data is stored once. Each unit reads it through a shortcut, and the rules on the owning Lakehouse apply in every engine that passes the user's identity.

HOW I'D BUILD IT

1 Set up the domains

A Fabric admin creates one domain per business unit and a shared domain, for example Enterprise data, for the customer workspace. Each domain gets a domain admin from its unit, who assigns workspaces and maintains the description. A default domain for each unit's security group places its new workspaces automatically. Certification settings and a default sensitivity label can be delegated to each domain. In the OneLake catalog, users filter by domain to find data.

Watch out. A domain groups workspaces and carries some settings. Data access still comes from workspace roles, item permissions and OneLake security.

2 Build the customer workspace and Lakehouse

The customer data team owns one workspace per stage, promoted with a deployment pipeline. Its production Lakehouse holds the main customer table and related tables such as addresses and contacts. A **Data Factory** pipeline builds them from the CRM and billing sources. Only that team holds Contributor or higher. Table names, keys and columns are a contract that changes only through a versioned release with notice to the units.

Watch out. Dropping or renaming a table breaks every shortcut to it, and the units find out when their reports fail. **1.06**

3 Grant the units item access

Each unit's readers sit in a Microsoft Entra security group. I share the customer Lakehouse with those groups with Read only, and data access then comes from OneLake security roles. I leave out Read all, because the DefaultReader role gives every table to users who hold ReadAll. Units never get a workspace role on the customer workspace.

Watch out. OneLake security restricts only Viewers and users with Read on the item. A unit engineer made Contributor on the customer workspace reads every row and can delete files through a shortcut. **7.10**

4 Define OneLake security roles

On the customer Lakehouse I create one OneLake security role per unit. Each role grants Read on the customer tables. Where units see only their own customers, the role adds a row filter. Column-level security hides fields such as date of birth. Role changes take about 5 minutes to apply. Group membership changes take about an hour, and engines with their own cache can take another hour.

Watch out. Permissions cannot be defined on a OneLake-to-OneLake shortcut itself. OneLake combines the folder's permissions with the target's, so the rules belong on the customer Lakehouse and nowhere else. **7.12**

5 Create shortcuts in each unit's Lakehouse

Each unit's Lakehouse gets OneLake shortcuts at the top level of Tables, one per customer table. A shortcut reads with the calling user's identity, so each user sees what their role allows. No pipeline copies customer data, and an update by the owning team reaches all units at once. Units keep their own attributes, such as a loyalty tier, in their own tables keyed by customer ID.

Watch out. A shortcut to another shortcut adds a hop, and chains stop at 5. Every unit points straight at the owning Lakehouse. **1.04**

6 Use query paths that pass the user's identity

A SQL analytics endpoint starts in delegated identity mode, which uses the item owner's identity. In that mode it blocks shortcuts to tables that carry OneLake row or column rules. I switch the units' SQL analytics endpoints to user identity mode. The switch cancels running and queued queries on every SQL analytics endpoint in the workspace, so I do it out of hours. Semantic models use Direct Lake on OneLake, which passes the user's identity, where Direct Lake on SQL would pass the model owner's.

Watch out. After a change to a OneLake security role, Direct Lake on OneLake models need a refresh before users see the new rules. [7.14](#)

7 Endorse and describe the shared data

The owning team asks a designated reviewer to mark the customer Lakehouse and its semantic model as Master data. Only people the Fabric admin names can apply that badge. Unit reports can be Promoted or Certified in their own domain. The Lakehouse description lists the tables, keys, refresh time and the contact for corrections. People then find it in the OneLake catalog before they build a copy.

Watch out. A unit that cannot find the master table loads its own from the CRM, and two customer counts appear. [1.03](#)

SECURITY IN ONE BREATH

Workspace roles stay inside each team. The customer team holds roles on its own workspace, and each unit holds roles on its own. Access across units is item Read plus OneLake security roles, always through groups. Pipelines in each workspace use that workspace's identity or its own service principal, so one leaked credential exposes one unit.

HOW I'D KNOW IT WORKS

- A user from one unit queries the customer table from a notebook, the SQL analytics endpoint and a Direct Lake report. The user sees the same rows each time.
- Restricted columns are hidden from unit users in every engine.
- OneLake holds one physical copy of each customer table, and the unit Lakehouses hold only shortcuts to it.
- The OneLake catalog shows the customer Lakehouse under the shared domain with the Master data badge.

IF THEY PUSH FURTHER

What if a unit needs customer data with its own extra fields?

It keeps those fields in its own table keyed by customer ID and joins them in its model. A field that every unit needs moves into the owning team's table.

What if a fourth unit joins next year?

It gets a domain, a group, a OneLake security role and a set of shortcuts. Nothing in the customer Lakehouse changes.

What if report viewers in a unit get errors on every visual?

Direct Lake on OneLake reads as the signed-in user. That user needs Read on the customer Lakehouse and membership of a OneLake security role, as well as access to the unit's own Lakehouse. [7.08](#)

S.06 Migrate an Azure Synapse dedicated SQL pool with 400 tables, hash-distributed fact tables and many stored procedures into Fabric. Choose the target item, move schema, code and data, validate the results and plan the cutover.

Short answer. I'd move to a Fabric Warehouse, because the workload is T-SQL with stored procedures that write data. The Migration Assistant for Fabric Data Warehouse is GA. It takes a DACPAC of the pool, converts schema and code, and lists what it could not convert. With Copilot enabled, it also suggests fixes. Large tables move with CETAS from Synapse to Parquet in ADLS Gen2, then COPY INTO the Warehouse. Distributions, indexes and resource classes are dropped, and a parallel run with table-by-table validation comes before the reports switch over.

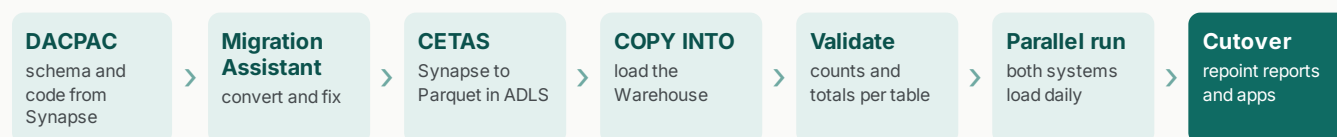
QUESTIONS TO ASK FIRST

- **Which objects use features Fabric Warehouse lacks?** Materialized views, triggers, recursive queries, synonyms and PREDICT are unsupported. Their count sets the rewrite effort.
- **How big are the largest tables, and how short is the cutover window?** Large facts go through CETAS and COPY INTO weeks ahead. A short window then only needs the latest changes.
- **Does the source compare strings case-insensitively?** A Fabric Warehouse is case-sensitive by default, and collation is fixed at creation. I decide before creating it.
- **Who connects today, and how?** SQL authentication does not carry over. Apps and service accounts move to Microsoft Entra identities, which may need changes on their side.
- **Will anyone need Spark to write these tables?** Spark can read Warehouse tables but cannot write them. Tables that Spark must write belong in a Lakehouse.

WHICH TOOL, AND WHY

- **Fabric Warehouse.** My choice. It runs the T-SQL, stored procedures and multi-table transactions the pool already uses.
- **Lakehouse with Spark.** Fits a team that wants to rewrite into notebooks. Its SQL analytics endpoint is read-only, so every procedure that writes data would need rewriting.
- **Warehouse plus a Lakehouse.** A Lakehouse for raw landing and Spark work beside the Warehouse for the modelled layer. I'd add it later if Spark work appears.

AT A GLANCE



Schema and code move first, data follows through ADLS, and both systems run side by side until validation is clean.

HOW I'D BUILD IT

1 Inventory and choose the collation

I list the 400 tables with row counts and sizes, the procedures, views and functions, and every consumer from Power BI models to apps. Then I create the Warehouse. Fabric Warehouse defaults to Latin1_General_100_BIN2_UTF8, which is case-sensitive. If the pool compares strings case-insensitively, I create the Warehouse as Latin1_General_100_CI_AS_KS_WS_SC_UTF8, through the workspace collation setting or the REST API. Collation cannot change after creation.

Watch out. A case-sensitive target makes joins and filters that matched in Synapse return fewer rows, with no error. [5.03](#)

2 Convert schema and code with the Migration Assistant

I extract a DACPAC from the pool, for example with SqlPackage or Visual Studio, and upload it to the Migration Assistant. It migrates tables, views, functions, stored procedures and security objects such as roles, permissions and dynamic data masking. Objects that fail land in a list to fix, and with Copilot enabled it suggests T-SQL fixes. Connecting the assistant directly to the source is an alternative to the DACPAC, and that option is also GA.

Watch out. The assistant leaves some hard cases to you. Multi-statement table-valued functions and external tables need manual rewrites. Scalar UDFs are in preview in Fabric, and ones that cannot be inlined may need rewriting too. [4.16](#)

3 Drop the Synapse layout options

DISTRIBUTION = HASH, ROUND_ROBIN and REPLICATE go, and so do clustered columnstore, heap and partition options. Fabric CTAS accepts only an optional CLUSTER BY for data clustering, which is in preview. Resource classes and workload groups go too, because the Warehouse splits SELECT and non-SELECT work into separate pools by itself. Keys stay as NONCLUSTERED NOT ENFORCED primary and unique keys, and foreign keys as NOT ENFORCED. Types change too. Money becomes decimal, datetime and smalldatetime become datetime2, nvarchar becomes varchar, and tinyint becomes smallint.

Watch out. Code tuned around distributions, such as a CTAS that redistributes before a join, now only costs time. Statistics and a small set of join hints are what is left to tune. [4.04](#)

4 Rewrite the procedures that hit gaps

I search the procedures for materialized views, triggers, recursive CTEs, synonyms, SET ROWCOUNT, SET TRANSACTION ISOLATION LEVEL, FOR XML and PREDICT. Temp tables, MERGE, TRUNCATE, IDENTITY and sp_rename for columns work. Variables are not allowed inside CTAS in Fabric, so a CTAS built on variables becomes dynamic SQL or INSERT with SELECT. Materialized views become tables rebuilt by the load, or aggregates in the semantic model.

Watch out. Fabric IDENTITY never reuses values and assigns them in no fixed order. A procedure that reloads a dimension and expects the same keys breaks every fact join. [4.14](#)

5 Move the data

For large tables, I run CETAS in Synapse to write each one as Parquet to an ADLS Gen2 account. Each table is one query on one concurrency slot, so the pool does not need scaling up. COPY INTO then loads each folder into the Warehouse with Microsoft Entra ID. If the storage account has a firewall, the workspace identity and a resource instance rule give access. Fabric loads many tables in parallel without contention, and small dimensions can go through the assistant's copy job. A **Data Factory** pipeline drives it all from a control table listing the 400 tables with status and row counts.

Watch out. COPY INTO into a column with a smaller decimal scale than the source truncates the extra digits without error, so counts match and totals drift. **4.10**

6 Validate table by table

For every table, a pipeline writes the row count, sums of key numeric columns, and minimum and maximum dates per month to a validation table. It does this once from Synapse and once from Fabric. Any mismatch marks the table as failed in the control table. Then I run the main procedures on both systems with the same inputs and compare their output tables. Report owners compare their top reports side by side.

Watch out. Statistics are created synchronously by the first query that needs them, so first test runs look slow. Update statistics as the last load step before timing anything. **4.05**

7 Run in parallel, then cut over

For a few weeks both systems load daily from the same sources, and validation runs every night. At cutover I stop the Synapse loads, run the last load into Fabric and validate once more. Then I repoint semantic models, apps and **Data Factory** connections to the Warehouse connection string. I pause the Synapse pool and keep it for an agreed rollback period before deleting it.

Watch out. A report or app missing from the inventory keeps pointing at Synapse and fails when the pool pauses. Check Synapse query history for remaining callers first.

SECURITY IN ONE BREATH

Fabric Warehouse uses Microsoft Entra ID, so SQL logins become Entra users or groups. Roles, permissions and dynamic data masking come across with the assistant. Column-level encryption does not, so those columns need masking or encryption in the application. Business users get item permissions or Viewer, because workspace Contributors can change every item.

HOW I'D KNOW IT WORKS

- Every one of the 400 tables has a validation row with matching counts and totals.
- Every procedure in the inventory has run in Fabric and produced the same output as in Synapse.
- In the last week of the parallel run, Synapse receives no queries apart from the loads.
- The top reports show the same figures on both systems for the same day.

IF THEY PUSH FURTHER

What if the cutover window is only a few hours?

History is loaded weeks ahead, and the daily loads keep Fabric current during the parallel run. Cutover is then a repoint of connections and one final validation.

What if the team wants Spark for new work?

Spark reads Warehouse tables through OneLake but cannot write them. New Spark pipelines write to a Lakehouse, and the Warehouse reads those tables with three-part names in the same workspace. **1.02**

What if a procedure is slower in Fabric than it was in Synapse?

I check statistics and column types first, because varchar(max) and oversized lengths distort estimates and block result set caching. There is no index or distribution to add, so query shape and statistics are what I tune. **4.03**

S.07 Plan the move of an on-premises SQL Server data warehouse to Fabric. It is loaded every night by 150 SSIS packages, and the business wants the same numbers in Fabric before the old server is switched off.

Short answer. I'd move the warehouse schema into a Fabric Warehouse with the Migration Assistant, then sort the 150 packages by what they actually do. Plain extracts become Copy jobs through an on-premises data gateway. Stored procedure logic becomes T-SQL in the Warehouse, and heavy transformations go to notebooks or Dataflow Gen2, all run by one **Data Factory** pipeline. The Invoke SSIS package activity is in preview and cannot reach on-premises sources, so it fits only a few cloud-only packages. Both systems load side by side until a reconciliation log shows matching numbers.

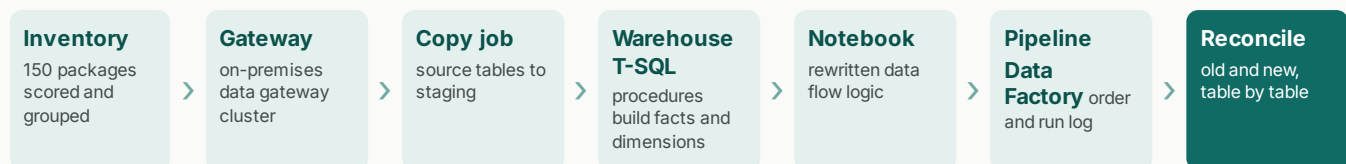
QUESTIONS TO ASK FIRST

- **Where do the packages read from, and will those sources stay on-premises?** If they stay, every extract needs an on-premises data gateway. The SSIS activity in Fabric is then ruled out for those packages.
- **How much logic sits in SSIS data flows, and how much in stored procedures?** Procedure-heavy estates move to Warehouse T-SQL quickly. Data-flow-heavy estates need rewrites into notebooks or Dataflow Gen2.
- **Do any packages use script tasks or third-party components?** Those have no direct target in Fabric. They become notebooks, and they usually set the timeline.
- **Do the source tables have CDC, change dates or row versions?** That decides between Copy job CDC, which is in preview, watermark incremental copy and full reloads.
- **Who reads the warehouse today, and how long can both systems run?** This sets the cutover waves and the length of the parallel run.

WHICH TOOL, AND WHY

- **Rewrite into Fabric items.** My choice for most packages. Each package lands on the item that fits its job, such as a Copy job, Warehouse T-SQL, a notebook or Dataflow Gen2. All of them run under one **Data Factory** pipeline and one run log.
- **Invoke SSIS package activity (preview).** Runs .dtsx files stored in OneLake on 4 vCores per workspace. It cannot connect to on-premises or private-network sources and does not run custom or third-party components.
- **Azure-SSIS integration runtime in Azure Data Factory.** Runs the packages almost unchanged, and a self-hosted integration runtime can pass on-premises data through. It keeps SSIS alive in a second service, so I'd use it only as a short-term bridge with an end date.

AT A GLANCE



Each package lands on one Fabric item. A **Data Factory** pipeline runs them in the old dependency order, and a reconciliation step compares every table with the old warehouse.

HOW I'D BUILD IT

1 Inventory and sort the packages

I pull execution history from the SSIS catalog. It shows which packages ran in the last three months, how long they took and how often they failed. A small script reads each .dtsx file and lists its connection managers, tasks, data flow components and script tasks. All of that goes into an inventory table in a Lakehouse. Each package gets a pattern: plain copy, SQL tasks, data flow transform, script or custom component, or retire. Each also gets an owner, a complexity score and a wave number. In many estates, for example, a third of the packages turn out to be plain copies.

Watch out. Packages nobody has run in months still get migrated unless execution history is checked first.

2 Move the warehouse schema

I extract a DACPAC from the SQL Server warehouse and load it into the Migration Assistant for Fabric Data Warehouse. It converts tables, views, functions and procedures, and it lists the scripts it could not convert. Copilot can suggest fixes if it is enabled for the tenant. A direct connection to the source system also works. Index, partition and trigger code has no place in the Warehouse, so I remove it instead of fixing it.

Watch out. The Warehouse runs a subset of T-SQL. Some SQL Server scripts fail there, and some run and behave differently. [4.16](#)

3 Set up the gateway and connections

I install an on-premises data gateway cluster with two members close to the source servers. The SQL Server connector accepts Basic, Organizational account and Windows authentication. Service principal works too, but only for SQL Server on Azure VMs. Windows works only through the on-premises gateway. I use a dedicated service account with read access to the source databases. Every connection is shared with the data engineering group and owned by a service identity. Gateway updates go through a change process that updates all members together.

Watch out. One cluster member on a different gateway build makes runs fail only when that member picks them up. [2.16](#)

4 Replace plain extracts with Copy jobs

Packages that only move tables become Copy jobs, one per source system. For on-premises SQL Server, Copy job supports full load, watermark incremental load and CDC. CDC in Copy job is in preview. It writes with append, overwrite or upsert. Tables with CDC enabled use CDC. Tables with a reliable change column use watermark incremental copy with merge on the key. Small reference tables are reloaded in full. Everything lands in a staging schema that the Warehouse procedures read.

Watch out. Editing the incremental column resets the Copy job to a full copy, and with append every row lands twice. [2.02](#)

5 Port SQL tasks to Warehouse T-SQL

Execute SQL tasks and the procedures they call become Warehouse stored procedures. A Script or Stored procedure activity in the **Data Factory** pipeline calls them in the old order. MERGE is generally available in the Warehouse, so upserts into dimensions stay set-based. I keep surrogate key logic explicit and test each dimension load against the old keys.

Watch out. Warehouse IDENTITY never reuses values. A truncate and reload of a dimension gives every member a new key, and the facts keep the old ones. [4.14](#)

6 Rebuild the data flow logic

Lookups, derived columns and conditional splits over large volumes become Spark notebooks. Small shaping jobs that analysts maintain become Dataflow Gen2. Script tasks become notebook code. The Invoke SSIS package activity stays an option only for packages whose sources and targets are all in the cloud. It runs packages stored in OneLake, such as a Lakehouse Files folder. It bills 1.35 CU hours per vCore of SSIS uptime. It keeps the runtime alive for 30 minutes after the last run. It can write package logs to OneLake.

Watch out. Packages saved with DontSaveSensitive carry no passwords, so the activity fails unless the Connection Managers tab supplies them at run time.

7 Orchestrate with a Data Factory pipeline

A parent pipeline reads a control table with a Lookup. The table lists each load step, its dependency group, its target item and an active flag. A ForEach per group calls child pipelines with Invoke pipeline, so each group waits for the one before. Every step writes rows read, rows written, status and duration to a run log table. The schedule matches the old SQL Agent job, and a failure path sends a Teams or Outlook alert.

Watch out. Pipelines run as the Last Modified By user, so a schedule stops when that person leaves or skips Fabric for 90 days. [7.01](#)

8 Run in parallel and reconcile

Both systems load the same nights. A reconciliation notebook copies control totals from the old warehouse through the gateway and compares them with Fabric, per table and business date. It checks row counts, sums of key measures and distinct key counts. Results go to a reconciliation table, and any difference raises an alert. A wave cuts over after an agreed number of clean nights, for example ten. The old packages are disabled, kept for rollback and deleted only after sign-off.

Watch out. A copy with skip incompatible rows switched on drops bad rows and still succeeds, so counts must come from both ends. [2.04](#)

SECURITY IN ONE BREATH

The gateway service account reads only the source databases it needs. Connections are owned by a service identity and shared with a group. Engineers build in a development workspace and promote with deployment pipelines, and report users read the Warehouse through its SQL permissions or an app.

HOW I'D KNOW IT WORKS

- Every package in the inventory has a target item, a wave and an owner, or a retire flag.
- The reconciliation table shows zero differences for every table for the agreed number of nights before a wave cuts over.
- The run log shows the nightly pipeline finishing inside the old SSIS window.
- After the last wave, no Fabric load reads from the old warehouse server.

IF THEY PUSH FURTHER

What if reports must move to Fabric before all packages are rewritten?

Mirror the old warehouse database into Fabric. Mirroring for SQL Server is generally available for SQL Server 2016 and later, through a gateway when the server is behind a firewall. Reports read the mirrored tables while the packages move wave by wave.

What if some sources move to Azure SQL Database during the project?

Their extracts switch to cloud connections without the gateway. The SSIS activity then becomes possible for those packages, but it is in preview with a fixed 4 vCores, so I still prefer the rewrite.

What if the nightly window is too short for full reloads?

Enable CDC on the source tables that allow it and switch those Copy jobs to CDC, which is in preview. Where CDC is refused, use a watermark column and check that every update path touches it. [2.01](#)

S.08 Design Fabric reporting for a company that keeps Azure Databricks writing Delta tables to ADLS Gen2. Reports must read those tables in Fabric without copying the data, and some storage accounts sit behind a firewall.

Short answer. I'd add a mirrored Azure Databricks catalog in Fabric. It syncs the Unity Catalog metadata and reads the tables through OneLake shortcuts, so no data moves. Tables outside Unity Catalog get ADLS Gen2 shortcuts in a Lakehouse. Semantic models read both with Direct Lake, and firewalled storage opens to Fabric through trusted workspace access and the workspace identity. Fabric never rewrites the Databricks files, so there is no V-Order, and file layout is tuned on the Databricks side.

QUESTIONS TO ASK FIRST

- **Are all the tables registered in Unity Catalog?** If yes, the mirrored catalog keeps the table list in sync. If not, I maintain ADLS shortcuts for the rest.
- **Do any tables use Unity Catalog row filters or column masks?** Mirroring does not support those tables, and a direct shortcut would skip the filters. Security for them is rebuilt in Fabric.
- **Is the storage account behind a firewall or private endpoints?** Then I need an F SKU capacity, a workspace identity and a resource instance rule for trusted workspace access.
- **Which Delta table features does the Databricks team turn on?** Some features, such as `TIMESTAMP_NTZ`, cannot be read by the SQL analytics endpoint or Direct Lake.
- **How fresh must the reports be after a Databricks job finishes?** This decides between automatic updates and a refresh started at the end of each job.

WHICH TOOL, AND WHY

- **Mirrored Azure Databricks catalog.** My choice for Unity Catalog tables. It mirrors metadata only, reads data through shortcuts, syncs new schemas and tables automatically and comes with a SQL analytics endpoint.
- **ADLS Gen2 shortcuts in a Lakehouse.** Works for tables outside Unity Catalog and gives full control per folder. I maintain every shortcut myself.
- **Copy job into OneLake.** Gives Fabric-managed layout and V-Order, but it copies the data, which the brief rules out.

AT A GLANCE



The data stays in ADLS Gen2. Fabric holds only metadata and shortcuts, and Direct Lake reads the Databricks Parquet files where they are.

HOW I'D BUILD IT

1 Prepare Unity Catalog for external reads

The Databricks admin enables external data access on the metastore. The principal Fabric uses gets the EXTERNAL USE SCHEMA privilege on each schema it reads. I confirm the Fabric capacity sits in a region that supports Azure Databricks mirroring. I also agree with the Databricks team which catalogues and schemas are in scope.

Watch out. Renaming a schema or table that sits in the mirroring inclusion or exclusion list is not supported, so renames need a plan.

2 Create the mirrored catalog

In the Fabric workspace I create a mirrored Azure Databricks catalog. The connection to the Databricks workspace uses a service principal, and organizational account is the other option. I pick the catalog and the schemas, and I leave automatic sync of future catalog changes on. Views, materialized views, streaming tables and tables with row filters or column masks do not appear. Changes in the data can take from seconds to several minutes to show in Fabric.

Watch out. Unity Catalog permissions do not carry over. Every Fabric user with access reads through the connection's credential.

3 Open the storage account to Fabric

The mirrored catalog also needs a connection to the ADLS Gen2 storage. I point it at the specific container folder and use the workspace identity, which Learn recommends. For firewalled storage I enable trusted workspace access. That needs an F SKU capacity and a workspace identity. The storage account needs a resource instance rule for the workspace. The identity needs the Storage Blob Data Reader role. Behind the firewall Fabric uses the workspace identity even if another auth type is chosen, so the identity must always be allowed.

Watch out. A deleted workspace identity cannot be restored, and its replacement has a new ID with no storage roles. [7.05](#)

4 Add shortcuts for tables outside Unity Catalog

In a Lakehouse, I create ADLS Gen2 shortcuts in the Tables area, one per Delta table folder, on the DFS endpoint. Shortcuts use delegated authorisation, so every reader goes through the credential set on the shortcut. I use a service principal or the workspace identity with Storage Blob Data Reader. SAS tokens and account keys also work, but only the Entra types work with trusted workspace access. A control table lists each shortcut, its path and its owner on the Databricks side.

Watch out. A shortcut placed in Files, or over plain Parquet, never becomes a table, so Direct Lake cannot see it. [5.06](#)

5 Check Delta feature compatibility

Databricks can turn on table features that Fabric engines read differently. Direct Lake reads deletion vectors, liquid clustering and column mapping. The SQL analytics endpoint does not read column mapping by ID, and neither it nor Direct Lake reads `TIMESTAMP_NTZ`. Tables with V2 checkpoints are not listed correctly by the Lakehouse and the SQL analytics endpoint. I agree a table property contract with the Databricks team and test each table in the endpoint after every change.

Watch out. A table property change in Databricks can hide columns or the whole table from the SQL analytics endpoint while the Databricks job succeeds. [5.04](#)

6 Build the Direct Lake models

I build each model with Direct Lake on OneLake, which reads the Delta tables directly and can use shortcut tables. It has no DirectQuery fallback, so problems show up as refresh or query errors. On F64 each table may have up to 5,000 Parquet files, 5,000 row groups and 1,500 million rows. I bind the model to a fixed-identity cloud connection so report viewers do not need read access on the item.

Watch out. One table over any guardrail stops Direct Lake for the whole model. [6.01](#)

7 Tune file layout on the Databricks side

Fabric does not rewrite or maintain files behind a mirrored catalog, and V-Order cannot be set there. V-Order is a Fabric write-time optimisation, so Databricks files arrive without it. Direct Lake works best with row groups between 1 million and 16 million rows. I ask the Databricks team to compact Gold tables and write larger files, and I check the result with Delta Analyzer. A V-Ordered copy in Fabric stays a documented exception for one hot table, if ever.

Watch out. Many small row groups make the first query after each load slow while Direct Lake loads the changed columns back into memory. [6.08](#)

8 Refresh after each Databricks job

Automatic updates are on by default and reframe the model when Delta tables change. For Gold tables written by several jobs, I turn them off. A **Data Factory** pipeline starts the Databricks job with the Azure Databricks activity, then runs a semantic model refresh activity when the job succeeds. The pipeline logs the run and sends a Teams alert on failure.

Watch out. With automatic updates on, reports can show a half-written set of Gold tables during a multi-table job. [6.05](#)

SECURITY IN ONE BREATH

Unity Catalog policies are not mirrored, and trusted workspace access does not apply row filters or column masks at the storage layer. I scope storage access to specific folders, keep sensitive tables out of the mirrored schemas, and rebuild row rules as semantic model RLS. Engineers hold workspace roles, and report users get an app.

HOW I'D KNOW IT WORKS

- The table count in the mirrored catalog matches Unity Catalog, minus the known unsupported table types.
- Every table returns the expected columns from the SQL analytics endpoint.
- Model refreshes succeed with no guardrail errors in the refresh history.
- After a Databricks job, the report shows that job's latest date within the agreed minutes.

IF THEY PUSH FURTHER

What if the storage account is in another Entra tenant?

Cross-tenant ADLS shortcuts work only with a service principal or SAS. Trusted workspace access does not work across tenants, so a firewalled account in another tenant needs a network decision first.

What if one large fact table is slow in Direct Lake?

Check its row groups and file count with Delta Analyzer. Ask Databricks to compact it with larger files. If that fails, a V-Ordered copy in Fabric is the last resort and needs sign-off. **5.08**

What if a table uses Unity Catalog row filters?

Mirroring skips it. I avoid a raw shortcut for wide audiences and ask Databricks to publish a filtered Gold table, or I rebuild the rule as model RLS.

S.09 Plan the move of 20 Premium import semantic models, each refreshed every 6 hours, to Direct Lake in Fabric. Users want fresher data, and the existing reports and measures must keep working.

Short answer. I'd first land each model's source data as Delta tables in a Lakehouse, because Direct Lake reads Delta directly and runs no Power Query. Most models become Direct Lake on OneLake, and Direct Lake on SQL endpoints is kept for models that need SQL views or SQL endpoint security. I convert each model with the Semantic Link Labs migration functions in a notebook. I check it against the SKU guardrails and frame it at the end of each **Data Factory** load. Calculated columns, calculated tables and Power Query steps move into the Delta tables.

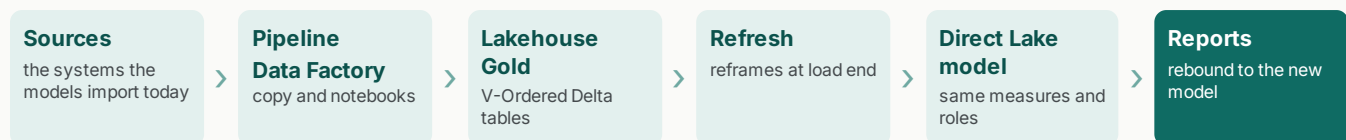
QUESTIONS TO ASK FIRST

- **Which capacity SKU hosts the models, and how big is the largest table?** On F64 or P1 a table may hold 1,500 million rows, 5,000 Parquet files and 5,000 row groups. A larger table needs aggregation or a larger SKU.
- **How much logic sits in Power Query, calculated columns and calculated tables?** All of it moves upstream into the Delta tables. This is most of the work.
- **Where does row security live today?** Model RLS carries over. SQL endpoint RLS would make Direct Lake on SQL fall back to DirectQuery.
- **How fresh is fresh enough?** Hourly loads with a reframe at the end, or reframing on every commit through automatic updates.
- **Do models mix sources, such as an Excel budget with warehouse data?** Direct Lake on OneLake can hold Import tables in the same model. Direct Lake on SQL cannot.

WHICH TOOL, AND WHY

- **Direct Lake on OneLake.** My default. Generally available, reads Delta from one or more items, and allows Import tables in the same model. It has no DirectQuery fallback, so problems show as errors.
- **Direct Lake on SQL endpoints.** Generally available, one source item, and it falls back to DirectQuery for SQL views and SQL endpoint security. It cannot mix Import or DirectQuery tables in the model.
- **Keep Import and refresh more often.** The smallest change. Every refresh reloads data and uses capacity, which scales poorly across 20 models.

AT A GLANCE



The transformation moves out of Power Query into the pipeline. The model only reframes, which takes seconds, so the load schedule sets freshness.

HOW I'D BUILD IT

1 Inventory each model

A notebook with semantic link lists every model's tables, Power Query expressions, calculated columns, calculated tables, roles and row counts. I add usage metrics to rank the 20 models. Everything goes into a migration tracker table in a Lakehouse, with an owner and a target date per model. I start with a small, well-used model as the pilot.

Watch out. Direct Lake does not support complex Delta column types, binary or GUID columns, strings over 32,764 characters or NaN values.

2 Move model logic upstream

Each Power Query step becomes part of the **Data Factory** pipeline. A Copy activity lands source data, and a notebook or Dataflow Gen2 shapes it into Gold tables. Direct Lake on SQL does not support calculated columns. Direct Lake on OneLake supports them in preview, for user context only. Calculation groups, what-if parameters, field parameters and calculated tables that do not reference Direct Lake data work in both flavors. Other calculated tables work only on OneLake, in preview. The Semantic Link Labs function `migrate_calc_tables_to_lakehouse` turns calculated tables into Delta tables.

Watch out. A SQL view makes a Direct Lake on SQL table fall back to DirectQuery, and on OneLake a view cannot be a Direct Lake table. [6.02](#)

3 Write Gold for Direct Lake

New workspaces have V-Order off by default. I write Gold with V-Order on, or with the `readHeavyForPBI` resource profile, because Direct Lake is the main reader. Direct Lake works best with row groups between 1 million and 16 million rows. I check layout with Delta Analyzer and schedule OPTIMIZE for tables that get many small writes.

Watch out. Gold written under the default `writeHeavy` profile has no V-Order and small row groups, and reports on it are slower. [5.08](#)

4 Choose the Direct Lake flavor and test fallback

Most models use Direct Lake on OneLake. A model that must keep SQL views or SQL endpoint security uses Direct Lake on SQL. For those, I set the `DirectLakeBehavior` property to `DirectLakeOnly` during testing, so any fallback shows as an error. In production it goes back to Automatic. Performance Analyzer shows a DirectQuery line when a visual falls back.

Watch out. With Automatic, fallback is silent, and pages just get slower while the model still says Direct Lake. [6.07](#)

5 Check the guardrails

On F64 or P1 each table is limited to 1,500 million rows, 5,000 Parquet files and 5,000 row groups. The model gets 25 GB of memory for paging data in. F128 doubles the row limit to 3,000 million. One table over any guardrail stops Direct Lake for the whole model. On SQL it then falls back, and on OneLake it fails. I record file and row group counts per table in the tracker after every load.

Watch out. Files grow with every append, so a table that passed in testing can cross the file guardrail months later unless it is compacted. [3.01](#)

6 Convert the model

For larger models I use the Semantic Link Labs migration functions in a notebook. They create a Power Query template, move tables and columns into a new Direct Lake model, then add measures, relationships, hierarchies and roles. `migration_validation` lists what moved and what did not. Small models I rebuild with live editing in Power BI Desktop. I keep the Import model untouched for rollback and rebind the reports to the new model once results match.

Watch out. Deployment pipelines do not autobind Direct Lake models, so production keeps reading the development Lakehouse without a deployment rule. [9.02](#)

7 Set framing and freshness

Framing is the refresh step that points the model at the latest Delta version, and it takes seconds. Automatic updates are on by default and reframe a table as soon as it changes. For models built from several Gold tables, I turn them off. A semantic model refresh activity runs at the end of each **Data Factory** load, for example every hour. The run log records when each model was framed.

Watch out. A refresh that runs before the load commits frames yesterday's data and still succeeds. [6.04](#)

SECURITY IN ONE BREATH

Model RLS works in both Direct Lake flavors. SQL endpoint RLS makes Direct Lake on SQL fall back. On OneLake the query succeeds and the SQL rule is not applied at all. So I keep row rules in the model. Direct Lake checks the model owner's access at every framing, so each model is owned by a service account.

HOW I'D KNOW IT WORKS

- `migration_validation` shows every measure, relationship and role as migrated.
- A fixed set of DAX queries returns the same results on the old and new model for the same data date.
- Refresh history shows no guardrail errors, and Performance Analyzer shows no DirectQuery lines on SQL models.
- Reports show data from the latest load within minutes of the pipeline finishing.
- The Capacity Metrics app shows the six-hourly import refresh load gone after cutover.

IF THEY PUSH FURTHER

What if one table is over the guardrail for our SKU?

Aggregate it in Gold for the main model and keep row detail in a separate model. A larger SKU raises the row limit, but the file and row group limits stay at 5,000 up to F256.

What if a model mixes an Excel budget with warehouse data?

Use Direct Lake on OneLake and keep the budget as an Import table in the same model. They form one source group with regular relationships. [6.03](#)

What if viewers get errors after cutover?

Direct Lake reads OneLake as the signed-in user by default, and Viewers cannot. Grant read on the item, or bind the model to a fixed-identity cloud connection. **7.08**

S.10 Design the daily ingestion of an Oracle ERP with 800 tables into Fabric. Change data capture is not available, many tables have no reliable change date, and a few tables are very large.

Short answer. I'd drive the load from a control table in a **Data Factory** pipeline, with the Oracle connection running through an on-premises data gateway. Tables with a trustworthy change column go to Copy jobs with watermark incremental copy and merge. Small tables are reloaded in full. Large tables without dates use a partitioned parallel Copy activity into staging, then a notebook compares row hashes to find inserts, updates and deletes. Mirroring for Oracle exists, but it needs LogMiner, archive log mode and supplemental logging, the same change logging that is ruled out here.

QUESTIONS TO ASK FIRST

- **Why is CDC unavailable: policy, licensing, or a read-only standby?** Mirroring and Copy job CDC both read change logs. LogMiner also needs a database in write mode, so a read-only standby rules both out.
- **Which tables have a change date that every ERP process updates?** Those get watermark loads. The rest get full loads or hash comparison.
- **How big are the largest tables, and are they partitioned in Oracle?** Physical partitions or an integer key decide how the copy splits into parallel reads.
- **Do reports need deleted rows removed?** Watermark loads cannot see deletes. Hash comparison against a full key list can.
- **What load window and how many sessions will the DBA allow?** This caps parallel copies and the ForEach batch count.

WHICH TOOL, AND WHY

- **Data Factory pipeline with a control table.** My choice. One pipeline handles watermark, full and hash loads, with a run log, dependencies and alerts in one place.
- **Copy job.** I use it inside the design for watermark tables. It supports Oracle watermark incremental reads and upsert, and it tracks the watermark itself.
- **Mirroring for Oracle.** Continuous replication of up to 1,000 tables with no pipelines. It needs LogMiner, archive log mode, supplemental logging and a primary key or unique index per table.
- **Dataflow Gen2.** Oracle is a supported source, but 800 tables a day are easier to run as copies and notebooks.

AT A GLANCE



One **Data Factory** pipeline runs daily through the gateway. Each table follows its pattern from the control table, and every table writes one row to the run log.

HOW I'D BUILD IT

1 Set up the gateway and Oracle connection

I install an on-premises data gateway cluster near the Oracle server. Each gateway machine needs the Oracle Client for Microsoft Tools, or pipeline copies through the gateway fail. The Oracle connection uses Basic authentication with a read-only ERP account. The connection is shared with the data engineering group and owned by a service identity.

Watch out. One gateway member on a different build makes runs fail only when that member picks them up. **2.16**

2 Profile the tables and build the control table

I read the Oracle data dictionary once to get row counts, columns, keys and partitions for all 800 tables. Each table gets a row in a control table. The row holds owner and name, load pattern, key columns, watermark column, partition option, load group and an active flag. The patterns are watermark, full and hash. A run log table gets one row per table per day with rows read, rows changed, duration and status.

Watch out. Lookup returns at most 5,000 rows. 800 tables fit, but a list that grows past that is cut off without an error. **2.10**

3 Load watermark tables with Copy jobs

Tables with a trustworthy change column go into Copy jobs, split by ERP module. Copy job supports watermark incremental reads from Oracle on datetime, date and integer columns. The first run copies everything, and later runs copy only newer rows. For date-only columns it re-reads the last day. I write with upsert on the table key into Bronze, so a re-read row updates instead of doubling.

Watch out. A column called LAST_UPDATE_DATE is not proof. Rows changed by a process that skips it are never read again. **2.01**

4 Reload small tables in full

Tables under an agreed size, for example one million rows, are reloaded every day. A Copy activity in a ForEach overwrites the Bronze table. Before the overwrite, the pipeline checks that the source returned rows. A table that suddenly returns zero rows fails the step instead of emptying Bronze.

Watch out. An overwrite from an empty source empties the table, and the run still succeeds. **2.27**

5 Copy large tables in parallel

Large tables use a Copy activity with a partition option. Partitioned Oracle tables use physical partitions. Others use dynamic range on an integer key, with bounds read from the control table. Degree of copy parallelism sets how many queries hit Oracle at once, multiplied by the ForEach batch count. I agree that total with the DBA. The source query casts NUMBER columns without precision, which otherwise arrive as a very wide decimal.

Watch out. With partition option None, the copy reads through one query whatever parallelism is set. **2.06**

6 Detect changes by hash

For tables without a change date, a notebook reads today's staging copy. It builds a SHA-256 hash over the business columns in a fixed order, with nulls, trimmed strings and decimals normalised first. One Delta MERGE compares by key with Silver. New keys insert, changed hashes update, and keys missing from the source get a deleted flag and date. The notebook writes insert, update and delete counts to the run log.

Watch out. A float or timestamp formatted differently between runs changes every hash and rewrites the whole table.

7 Schedule, log and alert

The pipeline runs once a day, for example at 01:00, owned by a service identity. The schedule's end date is set far in the future. After all groups finish, a check compares the run log with the active tables in the control table. Missing or failed tables raise a Teams or Outlook alert. Once a week a notebook compares Oracle row counts with Silver for every watermark table, to catch rows the watermark missed.

Watch out. A schedule stops when its owner has not signed in to Fabric for 90 days. [2.11](#)

SECURITY IN ONE BREATH

The Oracle account can only read the ERP schemas in scope. The gateway machines are patched and restricted like any server holding database credentials. Sensitive columns, such as bank details, are left out of the source queries or restricted in Silver, and only engineers see Bronze.

HOW I'D KNOW IT WORKS

- The run log has one row per active table every day.
- Weekly Oracle row counts match Silver's non-deleted rows for every table.
- A sample of keys updated in the ERP shows the new values in Silver the next day.
- The load finishes inside the window the DBA agreed.

IF THEY PUSH FURTHER

What if the DBA later enables archive log mode and supplemental logging?

Then Mirroring for Oracle becomes possible for up to 1,000 tables with a key or unique index, through the on-premises gateway. Copy job CDC from Oracle is another option. It is in preview, and SCD Type 2 is not yet supported for Oracle CDC.

What if someone changes the watermark column in a Copy job?

The next run resets to a full copy. With upsert this only costs time. With append every row would land twice. [2.02](#)

What if the ERP grows past 5,000 tables?

Split the control table by module and run one Lookup per module. Mirroring would also hit its 1,000-table limit per mirrored database. [2.10](#)

S.11 Plan the migration of 300 Azure Data Factory pipelines to Fabric Data Factory within one year. The pipelines use self-hosted integration runtimes, several trigger types and mapping data flows, and the loads must keep running throughout.

Short answer. I'd start with the migration assessment in Azure Data Factory, which is in preview. It sorts every pipeline into Ready, Needs review, Coming soon or Not compatible. I mount the factory in a Fabric workspace for visibility. Then I move pipelines in waves with the Migrate to Fabric experience, also in preview, or the PowerShell upgrade module. Linked services become Fabric connections, and self-hosted integration runtimes become on-premises data gateways. Mapping data flows, SSIS and most non-schedule triggers are reworked by hand. Each wave runs in both services until outputs match, then the ADF triggers are switched off.

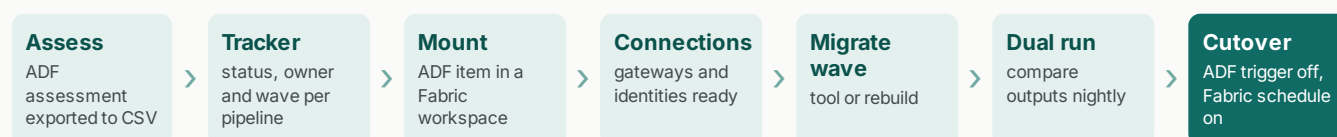
QUESTIONS TO ASK FIRST

- **Which integration runtimes do the pipelines use?** Self-hosted runtimes become on-premises data gateways, and managed virtual network runtimes become virtual network data gateways. SSIS runtimes have no equivalent.
- **How many pipelines use mapping data flows, SSIS or custom activities?** Those need rework, and their count sets the timeline more than the total of 300.
- **Which trigger types are in use?** Schedule triggers migrate disabled. Tumbling window, event and dependency triggers are rebuilt by hand.
- **Do linked services use parameters, global parameters or Key Vault?** Fabric connections take no dynamic properties, global parameters become variable libraries, and Key Vault references support fewer sources.
- **Which authentication do the linked services use?** Managed identity maps to workspace identity. Certificate auth and user-assigned managed identity are not handled by the migration experience.

WHICH TOOL, AND WHY

- **Migrate to Fabric experience (preview).** My choice for Ready pipelines. It runs from ADF Studio, creates Fabric connections where it can map auth safely, and migrates schedule triggers disabled.
- **PowerShell upgrade module.** Microsoft.FabricPipelineUpgrade is scriptable and repeatable for bulk waves. It covers a subset of activities and datasets and needs a resolution file that maps linked services to connections.
- **Mounted ADF item in Fabric.** Shows the factory inside a Fabric workspace for ownership and side-by-side review. Pipelines still run and bill in Azure Data Factory, so it is a staging step.
- **Manual rebuild.** For low-parity pipelines, such as those with mapping data flows. Slower, but it is the chance to simplify.

AT A GLANCE



Every pipeline goes through the same path. Nothing is switched off in ADF until the Fabric copy has matched it for an agreed number of runs.

HOW I'D BUILD IT

1 Assess and build the tracker

I run the assessment from the ADF authoring toolbar and export the result as CSV. It lists each pipeline and activity with a status and compatibility notes. I load it into a migration tracker table in a Lakehouse. I add the integration runtime, linked services, triggers, owner, last run date and business criticality. Pipelines with no runs in months are retired, and duplicates are merged. The tracker drives every wave and every status report.

Watch out. The assessment only scans configuration, so it cannot tell which pipelines are dead. Last run dates must come from ADF run history.

2 Mount the factory in Fabric

I add an Azure Data Factory item to a Fabric workspace. It needs Contributor on the factory, Member or Admin on the workspace, and both in the same Entra tenant. Teams can see and edit pipelines in Fabric and plan cutovers per domain. Global parameters do not show, and full monitoring and alerts stay in the ADF portal.

Watch out. Mounted pipelines still run on ADF integration runtimes and bill to Azure, so mounting does not reduce ADF cost.

3 Map connections and identities

Each linked service becomes a Fabric connection, created once per source and shared with the right group. Datasets disappear, because Fabric keeps their settings inside each activity. Managed identity maps to the workspace identity, and service principals cover the rest. Global parameters move to a variable library. Fabric connections take no dynamic properties, so metadata-driven pipelines parameterise the connection object instead.

Watch out. A connection owned by one engineer stops every pipeline that uses it when that account is disabled. [7.03](#)

4 Replace integration runtimes with gateways

Public Azure integration runtimes need no replacement. Each self-hosted runtime becomes an on-premises data gateway cluster, and each managed virtual network runtime becomes a virtual network data gateway. SSIS runtimes have no Fabric equivalent. The Invoke SSIS package activity is in preview and cannot reach on-premises data. During dual running, both the self-hosted runtime and the gateway hit the same sources, so I size and schedule for both.

Watch out. One gateway cluster member on a different build makes runs fail only when that member picks them up. [2.16](#)

5 Migrate pipelines in waves

Wave one is a pilot of about ten Ready pipelines from one domain. Later waves follow domains, Ready first, then Needs review, then rework. As an example plan, the first quarter covers assessment and the pilot. The next two quarters move 40 to 60 pipelines each month. The last quarter covers rework and decommissioning. Ready pipelines go through the Migrate to Fabric experience. Bulk waves can use the PowerShell module, which converts Execute pipeline into Invoke pipeline with a Fabric pipeline connection.

Watch out. The PowerShell module does not support global parameters, `pipeline().Pipeline` or a dynamic Web activity URL, so those pipelines need manual fixes.

6 Rework what does not map

Mapping data flows become Dataflow Gen2 for simple shaping, Warehouse T-SQL for set-based loads, or Spark notebooks for heavy work. Custom activities move to the Azure Batch activity. Tumbling window triggers become interval-based schedules, which are in preview and have no backfill yet. Storage event triggers are rebuilt as Fabric event triggers on file events. Custom event triggers cannot be migrated, so I check whether a Fabric event source covers each one. Chained trigger dependencies become Invoke pipeline calls in one parent pipeline.

Watch out. Two pipelines scheduled close together race, and the second can read the first one's stale output. [2.15](#)

7 Recreate schedules and alerts

Schedule triggers arrive disabled, which is deliberate. Fabric has no central trigger hub; each pipeline holds its own schedules, up to 20, each with a start and end date. I record every schedule in the tracker. Failure paths send Teams or Outlook alerts, and the Monitoring hub replaces the ADF monitor. Promotion to production uses deployment pipelines with variable library values per stage.

Watch out. An old schedule left enabled beside a new one fires too, and an append load doubles that day. [2.12](#)

8 Dual run and cut over

During dual running, the Fabric pipeline writes to a parallel target while ADF keeps writing production. A notebook compares row counts and checksums per table each night and writes a reconciliation table. I wait for an agreed number of clean runs, for example five business days. Then I switch off the ADF trigger and enable the Fabric schedule on the same day. The ADF pipeline stays disabled for 30 days for rollback, then it is deleted.

Watch out. If both services write the same production table, two copies of the truth appear and neither is authoritative. [1.03](#)

SECURITY IN ONE BREATH

Production connections use workspace identity or service principals, never personal accounts. Gateway service accounts get read access only to the sources they serve. People hold Viewer in production workspaces, and deployments run as a service principal.

HOW I'D KNOW IT WORKS

- The tracker shows every pipeline as migrated, retired or in rework, with a wave date.

- The reconciliation table shows zero differences for each wave before its cutover.
- After cutover, ADF monitoring shows no runs for migrated pipelines.
- The ADF bill for integration runtime hours falls wave by wave, and the Capacity Metrics app shows headroom.

IF THEY PUSH FURTHER

What if a connector the pipelines need does not exist in Fabric?

Check the connector parity page first. If it is missing, keep that pipeline in ADF, land the data in OneLake, and let Fabric take over from there.

What if dual running overloads a source system?

The Fabric copy and the ADF copy share the same source limits. Point the Fabric copy at a replica, or dual run only a validation subset of tables. [2.07](#)

What if schedules stop months after cutover?

Fabric pipelines run as the Last Modified By user. When that user leaves or skips Fabric for 90 days, runs stop, so deploy and edit as a service identity. [7.01](#)

S.12 Design the handoff between Azure Data Factory and Fabric. ADF keeps extracting from on-premises systems through a self-hosted integration runtime and lands files in ADLS Gen2. Everything after landing moves to Fabric, which must know when each landing is complete.

Short answer. I'd leave the landing zone in ADLS Gen2 and give Fabric a OneLake shortcut to it, so raw files exist once. ADF writes a small manifest file last, after every data file of a batch has landed. A storage event on that manifest starts a **Data Factory** pipeline in Fabric. The pipeline checks the batch and loads Bronze by batch ID with a notebook. Then it builds Silver and Gold. One run log, keyed by the ADF run ID, holds both halves of every batch.

QUESTIONS TO ASK FIRST

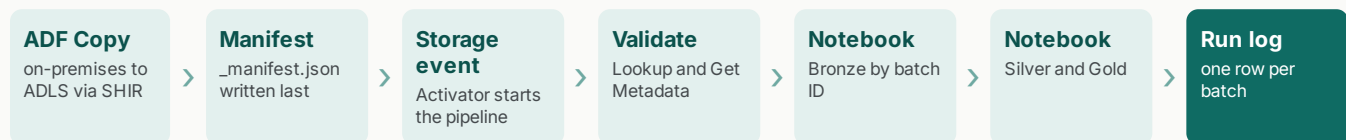
- **Can the ADF team write a manifest file at the end of each batch?** If yes, Fabric waits for that one file. If the ADF pipelines can't change, I fall back to a Fabric schedule that looks for complete folders.
- **Is the storage account behind a firewall?** Then the shortcut needs trusted workspace access with the workspace identity, and that needs a paid F SKU capacity.
- **How long are raw files kept, and who deletes them?** Retention stays with ADLS lifecycle rules. Bronze must hold its own copy of the data before any cleanup runs.
- **What file format does ADF land?** Parquet keeps column types. CSV needs a fixed schema when Bronze reads it.
- **How often does ADF rerun a batch, and can batches arrive out of order?** Reruns make a batch ID and replace-by-batch writes essential. Out-of-order arrival means Silver merges on the source timestamp.

WHICH TOOL, AND WHY

- **Manifest file plus storage event.** My choice. ADF writes the manifest last, and a Blob Storage event on it starts the Fabric pipeline. Each side owns its half, and neither needs rights in the other's service.

- **ADF calls Fabric at the end.** A Web activity in ADF calls the Fabric job scheduler API to start the pipeline. It couples the two tightly, and the ADF identity needs a Fabric workspace role. Good when ADF must also wait for Fabric to finish.
- **Schedule with a completeness check.** A Fabric schedule lists new manifests every 15 minutes, for example. Simple and safe, but it adds up to one interval of delay. I keep it as the nightly sweep.
- **Event on every data file.** Starts too early and too often. A batch of 40 files would start 40 runs, most of them on half a batch.

AT A GLANCE



ADF owns everything up to the manifest. Fabric owns everything after it, and both halves meet in the same run log.

HOW I'D BUILD IT

1 Agree the landing contract

I write down the folder layout with the ADF team. Each batch gets its own folder named by the ADF pipeline run ID, for example `landing/erp/orders/batch=<runId>/`. ADF writes the data files first, as Parquet where it can. Last, it writes `_manifest.json` into the same folder. The manifest lists every file, its row count, the extract window and the ADF run ID. A folder without a manifest is an incomplete batch.

Watch out. If ADF writes fixed file names into one fixed folder, a rerun overwrites files while Fabric is still reading them.

2 Connect OneLake to the landing zone with a shortcut

In the Bronze Lakehouse I create an ADLS Gen2 shortcut under Files, pointing at the landing container. The storage account needs hierarchical namespace, which ADLS Gen2 has. The shortcut authenticates with the workspace identity, which gets Storage Blob Data Reader on the account. If the account has a firewall, I add a resource instance rule for the workspace, which Fabric calls trusted workspace access. Notebooks then read the files through the shortcut as if they sat in OneLake, and nothing is copied twice.

Watch out. Shortcut caching doesn't cover ADLS, so every read goes back to the storage account. Bronze should read each batch once and keep its own Delta copy. [1.05](#)

3 Trigger on the manifest

On the Fabric pipeline I add a storage event trigger on the ADLS account, which creates an Activator item in the workspace. I filter `BlobCreated` events on the subject so only files named `_manifest.json` start a run. Built-in trigger parameters, such as `@pipeline()?.TriggerEvent?.FileName`, give the pipeline the file name and folder path. I check the capacity region first, because Blob Storage events aren't supported in a few regions, such as West US 2 and Central US.

Watch out. On ADLS Gen2, one file write raises a created event for `CreateFile` and another for `FlushWithClose`. The same manifest can then start two runs. [2.14](#)

4 Validate the batch before loading

A Lookup reads the manifest. A Get Metadata activity lists the batch folder, and an If Condition compares the two lists. A Script activity then inserts a row in the run log table in a small control Warehouse. The row holds the ADF run ID, the folder, status Started and the start time. If that run ID already shows Loaded, the pipeline ends there, which absorbs duplicate events. If a listed file is missing, the run fails and a Teams message names the batch.

Watch out. A manifest that lists zero files is still valid JSON. Without a check, the run succeeds and loads nothing. [2.28](#)

5 Load Bronze by batch ID

A Notebook activity gets the batch path and the ADF run ID as base parameters. The notebook reads the files with a fixed schema and adds batch_id, source_file and load time. It writes to the Bronze Delta table with replaceWhere on batch_id, so a rerun of the batch replaces its rows. It compares rows written with the manifest counts and writes both numbers to the run log.

Watch out. An Append after a retry writes the batch twice, and the run still reports success. [2.26](#)

6 Build Silver and Gold from the batch

A second notebook merges the batch into Silver on the business key. It keeps the latest version of each row by the source change timestamp, so an older extract rerun later can't overwrite newer data. Gold rebuilds only the business dates that the batch touched. At the end, a semantic model refresh frames the Direct Lake model once Gold has committed.

Watch out. A refresh that runs before Gold commits succeeds and still shows yesterday. [6.04](#)

7 Reprocess on purpose, and sweep for missed batches

Reprocessing is a manual run of the same pipeline with the batch path as a parameter. Every write replaces by batch ID, so running a batch twice gives the same result. A nightly schedule lists the manifests of the last two days and compares them with the run log. Any batch without a Loaded row starts the pipeline for that batch. That catches events lost during an outage.

Watch out. The sweep and a late event can start the same batch together. Both runs then write the same rows at the same time. [2.13](#)

8 Join the run log across ADF and Fabric

ADF sends pipeline and activity runs to Log Analytics through a diagnostic setting in resource-specific mode, into tables such as ADFPipelineRun. The manifest carries the ADF run ID into Fabric, so every Fabric run log row has it too. A daily report joins both sides on that ID. It shows when the extract finished, the manifest landed, Fabric started and Gold committed. A batch with an ADF success and no Fabric row after 30 minutes, for example, raises an alert.

Watch out. ADF logs can take up to 15 minutes to reach Log Analytics. Alert from the Fabric run log first.

SECURITY IN ONE BREATH

The workspace identity gets read access on the landing container only. A delete through a shortcut also deletes the target file when the caller has write rights there. Read-only access keeps Fabric from damaging the landing zone. ADF keeps its own managed identity with write access. Raw files can hold personal data, so Bronze sits in an engineering-only workspace and report users reach Gold through an app.

HOW I'D KNOW IT WORKS

- Every manifest in ADLS has exactly one run log row with status Loaded.
- Bronze rows per batch match the row counts in the manifest.
- Running the same batch twice leaves Bronze and Silver row counts unchanged.
- The time from manifest landed to Gold committed stays inside the agreed window, for example 30 minutes.

IF THEY PUSH FURTHER**What if ADF could write straight into OneLake?**

ADF's Microsoft Fabric Lakehouse connector works with the self-hosted integration runtime and authenticates with a service principal or a managed identity. The landing zone then moves into Lakehouse Files, and OneLake events replace Blob Storage events. The manifest, batch ID and run log stay exactly as they are.

What if an ADLS lifecycle rule deletes landing folders after 7 days?

Bronze already holds the data, so reports don't notice. The shortcut points at the container, so it keeps working. A shortcut to one batch folder would break as soon as that folder is gone. [1.06](#)

What if Fabric later takes over extraction too?

Microsoft's migration guidance maps self-hosted integration runtimes to on-premises data gateways. I'd recreate the connections on a gateway and replace ADF Copy with Fabric Copy activities or Copy jobs. The manifest contract stays, so the switch changes one side only.

S.13 Design on-premises connectivity where an Azure Data Factory self-hosted integration runtime and a Fabric on-premises data gateway must both reach the same firewalled servers. Cover machines, outbound network rules, service accounts, high availability, credentials, monitoring and sizing.

Short answer. I'd build two separate clusters on separate Windows servers. ADF gets a self-hosted integration runtime with two to four nodes. Fabric gets an on-premises data gateway cluster with at least two members. Microsoft's guidance says not to install both on one machine. Both open only outbound connections, so the edge firewall needs no inbound rule. Each runs under its own service account. Credentials sit in Key Vault for ADF and in shared Fabric connections for the gateway. Both clusters report into one alert channel.

QUESTIONS TO ASK FIRST

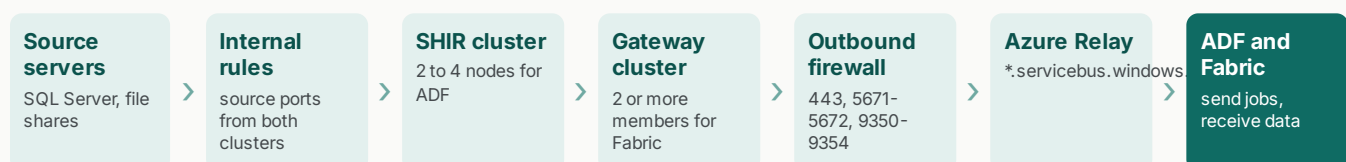
- **Which sources, how much data, and at what times?** Volume and concurrency set node size and count. If ADF and Fabric loads overlap, separate machines matter even more.
- **Do the sources use Windows authentication?** Then the servers must be domain-joined with a trust to the source domain, and the service account choice matters.

- **Does outbound traffic go through a proxy?** Both products need proxy settings. The gateway's port 1433 traffic to Fabric staging doesn't pass many proxies.
- **How long will ADF stay?** If ADF retires within a year, I size the gateway cluster for the final combined load now.
- **Who owns the servers, patching and the gateway recovery key?** Updates must be staggered across nodes, and a lost recovery key can't be recovered by Microsoft.

WHICH TOOL, AND WHY

- **Separate servers, two clusters.** My choice. The integration runtime's documentation says not to install it on the same machine as a Power BI or on-premises data gateway. Both lean on port 443. Separate machines also patch independently.
- **Both on one machine.** Fewer servers. It goes against Microsoft's guidance, and one reboot stops every ADF and Fabric load together.
- **Virtual network data gateway.** Fits when the servers are reachable from an Azure virtual network, for example over ExpressRoute. It removes gateway servers on the Fabric side. ADF still needs its own runtime.

AT A GLANCE



Two clusters sit side by side on separate servers. Both reach the sources inside the network, and every cloud connection opens outbound.

HOW I'D BUILD IT

1 Put each product on its own servers

I build four Windows Server machines close to the sources, two for the runtime and two for the gateway. None is a domain controller, because neither product supports one. The runtime's Learn page says not to install it on a machine with a Power BI gateway, since both use port 443 heavily. Separate machines also let me update one product without stopping the other. The runtime needs a Java runtime on each node if it reads or writes Parquet.

Watch out. One shared machine means one Windows update reboot stops every ADF and Fabric load at once.

2 Open outbound rules only

Runtime nodes need port 443 to the data factory endpoint or *.frontend.clouddatahub.net. They also need 443 to *.servicebus.windows.net for interactive authoring and to download.microsoft.com for auto-update. Add the Key Vault URL, and *.core.windows.net when they write to Azure storage. Gateway members need 443, 5671 to 5672 and 9350 to 9354 to *.servicebus.windows.net, plus the Entra sign-in and Power BI endpoints. For Fabric pipelines they need *.frontend.clouddatahub.net. For Dataflow Gen2 they need *.dfs.fabric.microsoft.com and port 1433 to *.datawarehouse.fabric.microsoft.com. Inside the network, both clusters reach the source servers on their ports, for example 1433. I run the gateway's network ports test under Diagnostics before any load.

Watch out. Dataflow Gen2 reaches its staging Lakehouse over TDS on 1433, which many proxies block. The connection test passes and the refresh fails.

3 Run the services under managed accounts

The runtime runs as NT SERVICE\DIAHostService by default, and the gateway as NT SERVICE\PBIEgwService. Both work for SQL logins. Sources may use Windows authentication, or the proxy may need a domain identity. Then I give each product its own domain account. For the gateway, Learn documents a group managed service account, so no password expires. For the runtime, Learn only asks for an account with the Log on as a service right. So I test a group managed service account there before I rely on it. For the gateway I change it through the gateway app, and that step asks for the recovery key.

Watch out. The gateway recovery key is needed to change the account, add members or restore a member. Microsoft can't retrieve a lost key.

4 Build high availability clusters

The runtime takes up to four nodes. First I turn on Remote access to intranet on the first node, so credentials can sync. Then I install the runtime on the second server with the same authentication key. The gateway cluster takes up to 10 members. I add the second member with the same recovery key and turn on Distribute requests across all active gateways in this cluster. In the gateway config file I set CPU and memory thresholds, for example 80 percent, so a busy member passes requests to another. Every gateway member must run the same version.

Watch out. An admin updates one gateway member and forgets the other. Runs then fail only when the old member picks them up. [2.16](#)

5 Store credentials once per product

ADF linked services read their passwords from Azure Key Vault, which also avoids credential sync problems across runtime nodes. In Fabric, I create connections under Manage connections and gateways, bound to the gateway cluster, with a SQL login or Windows account for each source. Workspace identity doesn't work through a gateway, so these connections use stored credentials. I give ADF and Fabric separate read-only source logins, so the DBA can tell them apart and revoke one. I share every Fabric connection with an engineering group and name at least two owners.

Watch out. A connection owned by one person stops working when that person leaves. [7.03](#)

6 Monitor both clusters from one place

For the runtime, ADF publishes Azure Monitor metrics such as `IntegrationRuntimeAvailableNodeNumber`, `IntegrationRuntimeCpuPercentage`, `IntegrationRuntimeAvailableMemory` and `IntegrationRuntimeAverageTaskPickupDelay`. I alert when available nodes drop below two or pickup delay passes a few minutes. For the gateway, performance monitoring is in preview and on by default. I load its logs with Microsoft's Power BI template. An hourly canary pipeline in each product copies one row through its cluster and alerts on failure. All alerts go to the same Teams channel.

Watch out. Gateway performance logs don't capture network bandwidth or latency, so a slow link looks like a slow query.

7 Size the nodes and scale on evidence

The runtime minimum is 4 cores, 8 GB of RAM and 80 GB of disk. The gateway recommendation is 8 cores, 8 GB and an SSD for spooling. I start every machine at 8 cores and 16 GB, as an example, and measure the busiest window. The runtime sets a concurrent job limit per node from its cores and memory. When jobs queue and CPU stays low, I raise that limit. When CPU is high, I add a node or a gateway member. I exclude the gateway spool folder from antivirus scanning.

Watch out. A full spool disk on a gateway member fails large copies with errors that look like source problems.

SECURITY IN ONE BREATH

Both clusters open every connection outbound, so the edge firewall needs no inbound rule. The servers sit in a network zone that reaches only the source ports they need. Source logins are read-only and separate per product. The gateway recovery key and the runtime authentication keys live in a vault, and only named admins can add nodes.

HOW I'D KNOW IT WORKS

- Stopping one runtime node and one gateway member leaves every load running.
- The gateway network ports test shows every port open.
- The available node count metric equals the node total, and an alert fires within minutes when it drops.
- Every Fabric connection on the gateway has at least two owners and a group share.
- The DBA can tell ADF and Fabric sessions apart by login name.

IF THEY PUSH FURTHER

What if the sources move to Azure virtual machines?

Fabric can then use a virtual network data gateway, which runs in Azure with no servers to patch. ADF can keep a runtime on an Azure VM in the same network.

What if ADF is retired?

Microsoft's migration guidance maps self-hosted integration runtimes to on-premises data gateways. I'd add gateway members for the combined load, move pipelines to Fabric, and remove runtime nodes once the last linked service is gone.

What if the firewall team refuses ports 5671 to 5672 and 9350 to 9354?

The gateway can run in HTTPS mode, and it falls back to HTTPS on its own when TCP fails. I test throughput with the heaviest load before go-live.

S.14 Plan the replacement of Azure Data Factory mapping data flows for a team moving to Fabric. They use the flows heavily, and Fabric offers Dataflow Gen2, notebooks, Warehouse T-SQL and Spark job definitions. Choose an engine per flow, and cover cost and debugging.

Short answer. I'd inventory every flow, score it on size, logic and maintainer, and move each one to the engine that fits. Low-code shaping goes to Dataflow Gen2, set-based loads into a Warehouse go to T-SQL procedures, and large or complex flows go to Spark notebooks. Mapping data flow transforms inside Dataflow Gen2, in preview, can carry some flows over unchanged while I rebuild them. Every replacement runs from a **Data Factory** pipeline and must match the ADF output before ADF is switched off.

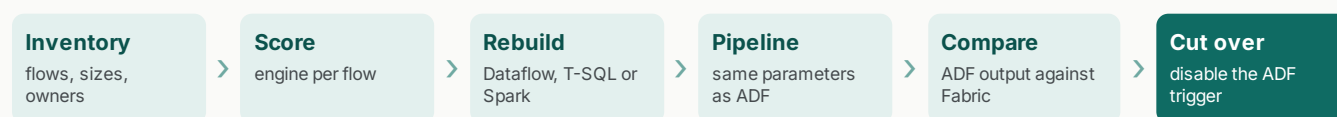
QUESTIONS TO ASK FIRST

- **How many flows, and how big is each one?** Row counts and run times decide which engine can carry each flow.
- **Who maintains the flows after the move?** Analysts who know Power Query get Dataflow Gen2. Engineers who read Python or SQL get notebooks or T-SQL.
- **Where does each flow write?** Flows that end in a Warehouse suit T-SQL. Flows that end in Lakehouse tables suit Spark.
- **Do any flows use flowlets, UDFs or data flow libraries?** The preview mapping data flow transforms support none of these, so those flows must be rebuilt.
- **Is there a date by which ADF must be off?** A near date favours the preview lift-and-shift for some flows, with a planned rebuild after it.

WHICH TOOL, AND WHY

- **Spark notebook.** My default for large or complex flows. The code lives in Git, can be tested, and every mapping data flow transform has a PySpark or Spark SQL equivalent.
- **Dataflow Gen2.** Low-code Power Query with fast copy and many destinations. Good for small to mid volumes with simple joins and filters. It slows down badly when a step stops folding.
- **Warehouse T-SQL.** Best when a flow lands in a Warehouse and is mostly joins, aggregates and MERGE. No Spark startup, and the logic sits next to the data.
- **Spark job definition.** For packaged Python, Scala or R code with command-line arguments and a retry policy. It can't be debugged cell by cell, so I keep it for stable, shared code.
- **Mapping data flow transforms in Dataflow Gen2.** In preview. ADF's built-in migration, also in preview, converts flows into them, and they run on Spark from a Fabric pipeline. A bridge while I rebuild.

AT A GLANCE



Each flow moves through the same path on its own, and ADF keeps running it until the comparison passes.

HOW I'D BUILD IT

1 Inventory the flows

I mount the factory as an Azure Data Factory item in a Fabric workspace, which gives a live view of its pipelines. I also export its ARM template to list every mapping data flow. For each flow I record sources, sinks, transforms, parameters, rows read, run time and owner, from recent ADF runs. I flag flowlets, UDFs, data flow libraries, Alter Row, Window, Assert and Stringify, because those limit the target. Everything goes into a migration control table with one row per flow.

Watch out. One parameterised flow can run for 40 tables. Count runs as well as definitions, or the work is underestimated.

2 Choose the engine per flow

I apply simple rules and record the result in the control table. Flows with filters, joins and lookups on a few million rows, owned by analysts, go to Dataflow Gen2. Flows that land in a Warehouse and mainly upsert go to T-SQL. Flows with large volumes, window logic, slowly changing dimensions or Alter Row go to notebooks. Code that many pipelines share goes into a Spark job definition. The row counts are examples, and I adjust them after the first few rebuilds.

Watch out. A Dataflow Gen2 that handles 1 million rows can stall at 10 million when one step stops folding. [2.22](#)

3 Use the preview transforms as a bridge

ADF's built-in migration can convert a mapping data flow into mapping data flow transforms inside a Dataflow Gen2 item, in preview. They run on Spark and start only from a Dataflow activity in a pipeline, where I set the compute size. Flowlets, data flow libraries, UDFs and managed virtual network aren't supported. Learn says customers aren't charged for this Spark compute yet and that pricing will be published before billing starts. I use it only for flows with a rebuild date in the control table.

Watch out. A cost comparison made while the transforms are free will be wrong once billing starts.

4 Rebuild complex flows in notebooks

Each notebook gets a parameter cell that matches the ADF data flow parameters, so the pipeline passes the same values. Derived Column becomes withColumn, Conditional Split becomes filtered writes to two tables, and Alter Row with its sink becomes a Delta MERGE. Window becomes Spark window functions. Each notebook writes idempotently, by replacing a date range or merging on a key. Shared helper functions go into a library in an Environment.

Watch out. Runtime 2.0 runs Spark 4.1. With ANSI mode on, casts that returned null in the old flow now fail the job. [3.07](#)

5 Rebuild set-based flows in T-SQL

Flows that end in a Warehouse become stored procedures. Each one loads a staging table with COPY INTO, or reads Lakehouse tables through three-part names, then runs a MERGE into the target. A Stored procedure activity in the pipeline calls it with the same parameters the data flow had.

Watch out. MERGE with no change test updates every matched row, changed or not. [4.11](#)

6 Rebuild low-code flows in Dataflow Gen2

Learn's mapping guide translates the transforms. Join becomes Merge queries, Aggregate becomes Group by, and Derived Column becomes Custom column. I keep folding steps early, turn on fast copy for large sources, and set Lakehouse or Warehouse destinations. New dataflows are Dataflow Gen2 (CI/CD) by default. Their public parameters, in preview, let the Dataflow activity pass values from the pipeline.

Watch out. The editor preview shows the first rows only. Duplicates and nulls appear only in the full refresh. **2.20**

7 Prove each flow matches before cutover

The new flow runs next to ADF for a week, for example, and writes to a shadow table. A comparison notebook checks row counts, key sets, sums of numeric columns and a hash of each row. Differences go into a run log with the flow ID and date. After a set number of clean days, I disable the ADF trigger and point consumers at the new table.

Watch out. Decimal scale often differs between engines. Row counts match while sums drift. **4.10**

8 Measure cost and plan debugging

In Fabric, every engine draws from the capacity. Dataflow Gen2 (CI/CD) standard compute costs 12 CU per second for the first 10 minutes of a query and 1.5 after that. Fast copy adds 1.5 and staging compute adds 6. Spark counts 2 VCores per CU and bills while the session runs. I measure each rebuilt flow in the Capacity Metrics app over a week and compare it with the ADF data flow charges. Debugging differs too. Notebooks run cell by cell, and Dataflow Gen2 has refresh history per query. T-SQL runs in the query editor, and Spark job definitions leave only run logs.

Watch out. Many small notebooks each start their own Spark session. A session tag with high concurrency mode lets notebooks in one pipeline share one.

SECURITY IN ONE BREATH

New connections use a workspace identity or a service principal. Notebook activities that run as the workspace identity need Contributor on the workspace and the tenant setting Service principals can call Fabric public APIs. Analysts who own dataflows get Contributor in development and Viewer in production, and deployment promotes their work.

HOW I'D KNOW IT WORKS

- Every flow in the control table has a target engine, an owner and a comparison status.
- Counts, key sets and column sums match between ADF and Fabric output for the agreed number of runs.
- The Capacity Metrics app shows the migrated flows inside the planned CU budget.
- No ADF trigger still starts a flow marked as cut over.

IF THEY PUSH FURTHER

What if 30 flows share one flowlet?

I'd rebuild the flowlet once as a Python function in an Environment library, or as a Spark SQL view, and call it from each notebook. The preview transforms don't support flowlets, so there is no shortcut.

What if analysts won't maintain PySpark?

Their flows stay in Dataflow Gen2. I move only the heavy step upstream into a notebook that writes a prepared table. The dataflow then reads a small, clean input and keeps folding. **2.21**

What if a dataflow runs far past its window?

Activity timeout and cancellation reach only Dataflow Gen2 (CI/CD) items. New dataflows are CI/CD by default, so I check that no migrated item is an older non-CI/CD one. **2.23**

S.15 Design orchestration where Azure Data Factory stays the orchestrator and must start Fabric notebooks and pipelines. Cover authentication, starting jobs, polling status, passing parameters, and showing failures from both platforms in one place with alerts.

Short answer. ADF's activity list has no activity that runs Fabric items. So I'd build one reusable ADF child pipeline around the Fabric job scheduler REST API. A Web activity starts the job with the factory's managed identity. An Until loop polls the job instance, and a Fail activity stops the run with Fabric's failure reason. Parameters travel in the request body, together with the ADF run ID. Every Fabric failure then shows up as an ADF failure, so one Azure Monitor alert and one run log cover both platforms.

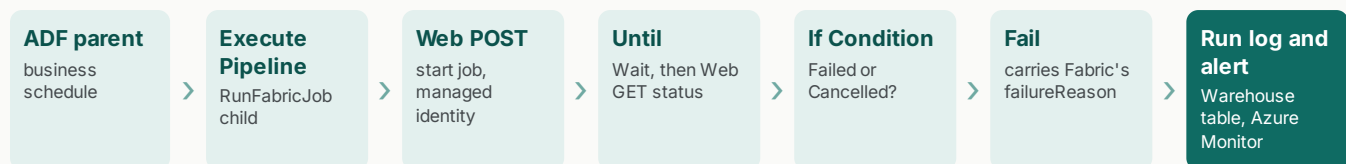
QUESTIONS TO ASK FIRST

- **Which Fabric items does ADF start, and how many runs a day?** Calling a few Fabric pipelines is simpler than calling dozens of notebooks. High volumes need care with API rate limits.
- **Will the Fabric admin let service principals call Fabric APIs?** Without the tenant setting Service principals can call Fabric public APIs, every managed identity call is refused. It is on by default for new tenants, so I confirm it isn't turned off.
- **Does ADF need anything back from Fabric, such as a notebook exit value?** Then the poller tries to read the exit value from the job instance and return it as output. I test that first, because Learn shows it only on a beta call.
- **Where does the operations team look today?** If they work in Azure Monitor, alerts start there. If they work in Teams, both platforms post to one channel.

WHICH TOOL, AND WHY

- **Web activity and the job scheduler API.** My choice. It works for pipelines, notebooks and Spark job definitions, uses ADF's managed identity, and I control polling and failure handling.
- **Web activity with its built-in async polling.** Less to build. Learn doesn't document how that polling treats Fabric's status response, so I can't rely on it to report a Fabric failure.
- **Azure Function or Logic App wrapper.** Hides tokens and polling in code. It adds one more service to deploy, secure and monitor.
- **File drop and a Fabric event trigger.** Loose coupling. ADF can't see whether Fabric succeeded, so failures live in two places.

AT A GLANCE



Every Fabric job goes through one child pipeline, so starting, waiting, failing and logging behave the same everywhere.

HOW I'D BUILD IT

1 Set up the identity

I use the data factory's system-assigned managed identity, so there is no secret to rotate. I put it in an Entra security group. The Fabric admin adds that group to the tenant setting Service principals can call Fabric public APIs. In each Fabric workspace that ADF must run, I give the group the Contributor role. Learn's notebook API page lists Admin, Member or Contributor for service principals. The job scheduler API accepts users, service principals and managed identities. A service principal is the fallback if the factory identity can't be used.

Watch out. One identity across dev, test and prod means one mistake reaches production. Use a separate factory and identity per environment. [7.04](#)

2 Build the start call

The child pipeline RunFabricJob takes workspaceId, itemId, jobType and a parameters object. A Web activity sends POST to <https://api.fabric.microsoft.com/v1/workspaces/{workspaceId}/items/{itemId}/jobs/instances?jobType=...> Learn's pipeline page uses Pipeline, and the notebook page uses RunNotebook. The API reference now puts the job type in the path, and the query form still works. Authentication is managed identity with the Fabric API as the resource. I check the exact resource value in a test call. I turn off the Web activity's async behaviour, so it returns at once with 202 Accepted. The API returns the job instance URL in the Location header, with a Retry-After value such as 60 seconds. I check that the Web activity output shows that header. If it doesn't, I list the item's job instances and take the newest.

Watch out. Workspace and item IDs differ between environments. Keep them in a control table per environment, and out of the pipeline JSON.

3 Pass parameters and the ADF run ID

For a notebook, Learn says the request can carry parameters, session settings and the default lakehouse. For a pipeline, Learn shows no parameter example. The API reference shows a parameters list, and some item types reject it with FeatureNotAvailable. So I test the body shape once for each item type. Every call also passes adf_run_id with the ADF run ID, so Fabric can log it.

Watch out. A parameter name that doesn't match may be ignored, and the item then runs with its default value. I check the run's inputs in the Monitoring hub after the first test. [2.29](#)

4 Poll until the job ends

An Until activity loops until the status is Completed, Failed, Cancelled or Deduped. Inside it sit a Wait of 60 seconds, a Web GET on the job instance URL and a Set variable with the status. I set the Until timeout a little above the job's longest normal run, for example 3 hours. Learn lists a limit of 100 job submissions and 100 Get Job requests per minute per user. 60-second polls leave room for many parallel jobs. The Web GET has a retry policy, so an HTTP 429 doesn't end the loop.

Watch out. A job can wait for minutes while Spark queues for cores. The timeout must include queue time.
8.01

5 Fail loudly with Fabric's reason

After the loop, an If Condition checks the final status. For Failed or Cancelled, a Fail activity ends the child pipeline with a message built from failureReason and the job instance ID. The parent sees an ordinary ADF failure with Fabric's message inside. For notebooks, Learn shows the status call returning the exit value in an exitValue field, on a call marked beta. If my test confirms it, I pass the value back as an output.

Watch out. A Fabric pipeline that catches its own error on a failure path can finish as Completed. It must end with a Fail activity when its work failed.

6 Write one run log across both platforms

The run log lives in a Fabric Warehouse table. The ADF child writes a Started row and an Ended row through a Stored procedure activity. It uses ADF's Microsoft Fabric Warehouse connector with the same managed identity. The Fabric notebooks and pipelines write their own step rows into the same table, with adf_run_id on each. One query then shows an ADF run, the Fabric job it started, how long it queued, how long it ran and the row counts.

Watch out. If Fabric runs its own jobs by schedule too, those rows have no adf_run_id and look orphaned. Tag them with the trigger type.

7 Alert from one place

ADF sends pipeline and activity runs to Log Analytics in resource-specific mode, into ADFPipelineRun and ADFActivityRun. Because every Fabric failure surfaces as an ADF Fail activity, one Azure Monitor alert on failed runs covers both platforms. The alert text carries the Fabric job instance ID, so the engineer can open the right run in the Monitoring hub. Fabric job events in Real-Time hub cover Fabric jobs that run outside ADF. An Activator rule on ItemJobFailed posts those failures to the same Teams channel.

Watch out. Log Analytics can lag by up to 15 minutes, so alerts arrive later than the failure.

SECURITY IN ONE BREATH

The factory identity holds Contributor only on the workspaces it runs, and only production's factory reaches production. No personal account starts production jobs. The Warehouse run log is readable by operations and writable only by the factory identity and the Fabric workspace identity.

HOW I'D KNOW IT WORKS

- A Fabric notebook that raises an error makes the ADF parent fail, with Fabric's message in the error text.
- Cancelling a Fabric job from the Monitoring hub fails the ADF child within one poll interval.

- Every ADF run that called Fabric has matching rows from both platforms in the run log.
- A test failure raises one alert in the agreed channel within the expected delay.

IF THEY PUSH FURTHER

What if Fabric becomes the orchestrator later?

Fabric's Invoke pipeline activity can call ADF and Synapse pipelines through a connection. The run log and `adf_run_id` convention stay, with the roles reversed.

What if a Fabric pipeline started this way fails with a token error months later?

Fabric pipelines can depend on the Last Modified By user for their connections. Moving connections to a workspace identity or service principal removes that dependency. [7.01](#)

What if ADF must start 200 notebooks each night?

I group them into a few Fabric pipelines and let ADF start those. That keeps API calls and polling low and lets Spark share sessions inside Fabric. [2.07](#)

S.16 Design near-real-time loading for a 40 million row operational table that receives about 50,000 changes an hour. The business wants Fabric data and reports at most 10 minutes behind the source, and wants that lag measured.

Short answer. If the source is supported, I'd mirror the database into Fabric, which replicates changes within seconds and doesn't charge capacity for replication. Change data feed on the mirrored table, in preview, lets a notebook merge only changed rows into Gold every five minutes. The Direct Lake model picks up each commit through automatic updates. A heartbeat row in the source measures the true end-to-end lag. CDC into Eventstream and Eventhouse is my choice only when seconds matter, and a Copy job is the fallback when mirroring isn't possible.

QUESTIONS TO ASK FIRST

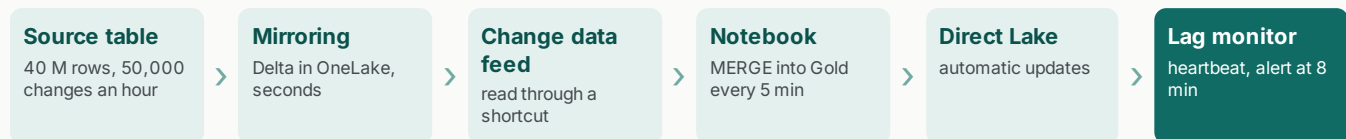
- **Which database holds the table, and where does it run?** Azure SQL Database, SQL Server, Oracle, Azure Database for PostgreSQL and Snowflake can be mirrored. An on-premises server adds a gateway.
- **May the DBA enable CDC on the source?** Mirroring of SQL Server 2016 to 2022 uses CDC, and so do the Eventstream and Copy job CDC paths.
- **Where must the 10 minutes hold, at the table, at Gold or at the report?** Each layer adds delay. I give each layer a budget and measure at the report.
- **Do reports need current state or full history?** Current state is a MERGE. History means keeping every change row in Silver.
- **Are there hours when freshness doesn't matter?** A slower Gold schedule at night and on weekends saves capacity.

WHICH TOOL, AND WHY

- **Mirroring.** My choice when the source is supported. Replication compute is free, storage is free up to 1 TB per capacity unit, and changes land as Delta tables within seconds.
- **CDC into Eventstream and Eventhouse.** Best when the business wants seconds and live dashboards. DeltaFlow, in preview, turns Debezium change events into tables. Eventhouse bills up time, and with changes all day it rarely idles.

- **Copy job with CDC.** Merges inserts, updates and deletes into a Lakehouse table on a schedule as short as one minute. Each run bills incremental data movement, and a 5-minute cadence means 288 runs a day.
- **Pipeline with a watermark Copy.** Most control and the most parts to operate. A watermark misses deletes.

AT A GLANCE



Mirroring carries changes into OneLake within seconds. A five-minute notebook cycle carries them into Gold, and every Gold commit reaches the report.

HOW I'D BUILD IT

1 Split the 10 minutes into budgets

I give each layer a share. For example, replication gets up to 2 minutes, and Gold gets 5 minutes between runs plus 2 minutes of run time. The report gets under a minute. That leaves slack for a slow cycle. 50,000 changes an hour is about 14 a second, which is a light load for all three options.

Watch out. Running every layer as often as it can go multiplies cost, and users can't see the difference. [8.13](#)

2 Mirror the source database

I create a mirrored database item and select the table. Azure SQL Database connects directly. An on-premises SQL Server goes through an on-premises data gateway, and versions 2016 to 2022 need CDC on the source. The first run snapshots all 40 million rows, which adds CPU and IO on the source, so I start it off-hours. After that, Learn says changes can publish as often as every 15 seconds. The table appears as Delta in OneLake and on the SQL analytics endpoint.

Watch out. A long-running source transaction stops log truncation, and the source log grows while mirroring waits.

3 Read only the changes

I turn on delta change data feed in the mirrored database settings. It's an extended capability in preview, billed only for real changes, while core mirroring stays free. The feed must be read through a Lakehouse shortcut, so I create one to the mirrored table in the Silver Lakehouse. A notebook reads table_changes from the last commit version it processed, which it keeps in a small watermark table.

Watch out. The notebook may stop for longer than the feed retention, which is one day in Learn's example. Then the versions it needs are gone, and Gold must be rebuilt in full.

4 Merge changes into Gold in one commit

The notebook keeps the last change per key in the batch and runs one MERGE into Gold that updates, inserts and deletes. If Gold is an aggregate, it recomputes only the groups those keys touch. One MERGE is one commit, so readers see a whole cycle or none of it. Gold has deletion vectors on and is clustered on the key, so each MERGE rewrites few files. I don't use materialized lake views here. They handle source updates and deletes incrementally only with a refresh hint, which is in preview, and otherwise fall back to a full refresh.

Watch out. Without deletion vectors, MERGE rewrites every file that holds a matched row, and 14 changes a second touch most files. [3.04](#)

5 Schedule the cycle and keep it healthy

A **Data Factory** pipeline runs every 5 minutes with one Notebook activity, and the activity runs as the workspace identity. The notebook first sets a lock row in the watermark table and exits if the previous cycle is still running. Auto compaction is on for Gold, and a nightly OPTIMIZE handles the rest. Each run writes rows read, rows merged, start and end time to a run log.

Watch out. A schedule expires when its owner hasn't signed in to Fabric for 90 days, and the loads stop quietly. [2.11](#)

6 Keep the report current

The semantic model reads Gold with Direct Lake on OneLake. Automatic updates reframe the model after each Gold commit, so no refresh schedule is needed. Because each cycle is one commit, users never see half a cycle. I keep Gold narrow, with only the columns the report uses.

Watch out. Each reframe evicts the changed columns, so the first query after a commit transcodes them again. [6.08](#)

7 Measure lag end to end

In the source I add a one-row heartbeat table, and I mirror it too. A SQL Agent job updates it every minute with the UTC time. Each Gold cycle copies the heartbeat into a Gold lag table with its own commit time, so lag is commit time minus heartbeat time. Every 5 minutes a small pipeline also calls the Get Tables Mirroring Status API, which returns status and lastSyncLatencyInSeconds per table, and logs the result. An alert fires when lag passes 8 minutes, before the promise breaks. A card on the report shows the data as-of time from the heartbeat.

Watch out. Mirroring can stall on one table while reports keep serving the last data. Only a heartbeat shows the gap. [2.03](#)

8 Compare the running cost

Mirroring replication uses no capacity units, and the change data feed bills only when rows change. The Gold cycle is the main cost, with 288 short Spark runs a day, visible in the Capacity Metrics app. For comparison, Eventstream bills 0.222 CU hours per active hour and 0.342 CU hours per GB. Its connectors bill 0.611 CU hours per vCore hour, and Eventhouse bills up time on top. A Copy job bills incremental copy at 3 CU hours per unit of throughput used. I run the chosen design for a week and check it against the budget before go-live.

Watch out. Background work is smoothed over 24 hours, so a small job every 5 minutes never looks big at one moment and still adds up. **8.03**

SECURITY IN ONE BREATH

Mirroring connects with a source login that has only the rights mirroring needs, and the gateway connection is shared with the data team's group. Report users reach Gold through an app and never see the mirrored database. The heartbeat and lag tables are readable by operations.

HOW I'D KNOW IT WORKS

- Over a week, the lag table shows no cycle over 10 minutes, and most under 7.
- `lastSyncLatencyInSeconds` for the table stays under the replication budget, for example 120 seconds.
- Gold's row count matches the source count, give or take the last cycle's changes.
- The Capacity Metrics app shows the Gold cycle inside its CU budget.

IF THEY PUSH FURTHER

What if the business now wants 10 seconds?

I'd move to CDC into Eventstream and land the changes in Eventhouse. There an `arg_max` materialized view keeps the latest row per key, and a Real-Time Dashboard shows it. On Azure SQL Database, Learn says not to enable mirroring when using the Eventstream CDC source. The Mirrored Database Change Feed connector, in preview, is the alternative that keeps mirroring.

What if the source can't be mirrored?

A Copy job with CDC-based incremental copy merges inserts, updates and deletes into a Lakehouse table every 5 minutes. Watermark mode is weaker, because it misses deletes and rows updated without touching the watermark column. **2.01**

What if Eventhouse data must also reach a Direct Lake model?

OneLake availability writes Eventhouse tables as Delta, but by default it can wait up to 3 hours to build large files. I'd set the target latency to its minimum of 5 minutes, or query Eventhouse directly.

S.17 Design an incremental load for a large SQL Server table that has no change tracking, no CDC and no modified date column. Inserts, updates and deletes must reach the Lakehouse every night without reloading everything downstream.

Short answer. With no change marker in the source, the only reliable way to find changes is to copy the whole table and compare it. I'd run a **Data Factory** pipeline that copies a full snapshot into a Lakehouse staging table. A notebook then hashes every row and compares the hashes with Silver by business key. It MERGEs only new, changed and missing rows into a Delta table. A control table and run log stop a partial extract from deleting good rows. In parallel I ask the source team for a rowversion column, so a Copy job can later switch to watermark-based incremental copy.

QUESTIONS TO ASK FIRST

- **How big is the table, and how much of it changes each day?** For example, 40 million rows with 1 percent changing a day. Size decides whether a nightly full extract fits the window and what it costs.
- **Can the source team add a rowversion column, or enable CDC or change tracking?** A rowversion column gives a real watermark for inserts and updates. CDC also gives deletes. Either one turns this design into a fallback.
- **Are rows really hard-deleted, and do consumers need to know?** If yes, I keep deleted rows with a flag and a date. If rows are never deleted, I drop the delete branch and its safety checks.
- **Does the table have a stable primary key?** The compare needs one. Without a key, a changed row can only be seen as a delete plus an insert.
- **When can we read the whole table without hurting the application?** A full scan competes with users. The extract runs in the quiet window, with parallel reads agreed with the DBA.

WHICH TOOL, AND WHY

- **Data Factory pipeline with Copy activity and a notebook.** My choice. The Copy activity pulls the full snapshot with partitioned parallel reads, and the notebook does the hash compare and MERGE. The pipeline adds the control table, run log, retries and alerts.
- **Copy job, full copy.** Copy job can copy the whole table on a schedule into a Lakehouse table, which accepts Append, Overwrite and Upsert writes. I'd test Merge with full copy before relying on it. Merge on a key inserts and updates rows, but nothing in it notices a row that vanished from the source.
- **Copy job, incremental on ROWVERSION.** The target state once the source team adds a rowversion column. Copy job keeps the watermark itself. Watermark copies still can't see deletes.
- **Mirroring.** Mirroring for SQL Server 2016 to 2022 runs on CDC in the source database. SQL Server 2025 needs Azure Arc and the Fabric change feed. Both are source changes the team has ruled out for now.

AT A GLANCE



One **Data Factory** pipeline, nightly at 01:00. The whole table lands in staging, and only new keys, changed hashes and missing keys reach Silver.

HOW I'D BUILD IT

1 Ask for a change marker first

Before building anything, I ask the source team for a rowversion column on the table. SQL Server sets a new rowversion value on every insert and update, so the application code does not change. Copy job accepts ROWVERSION as a watermark column for incremental copy. CDC would give deletes as well, and Copy job can replicate them. CDC replication in Copy job is in preview. I still build the full compare, because it works today and stays useful later as a weekly delete check.

Watch out. A modified date filled by application code is a weak substitute. Any update path that forgets to set it is never copied. **2.01**

2 Connect to the source

The table sits on an on-premises SQL Server, so the connection runs through an on-premises data gateway, installed as a cluster of two machines. The SQL Server connector supports basic SQL authentication, and Windows authentication through the on-premises gateway. I use a read-only service account with SELECT on the source tables and nothing else. The connection is shared with the data engineering group.

Watch out. A connection built on one engineer's own login stops the load the day that account is disabled. **7.03**

3 Build the control table and run log

A control table in the Lakehouse holds one row per source table. Each row has the schema, table name, key columns and columns excluded from the hash. It also has the partition column, staging table, target table and a delete threshold. For example, the threshold might allow at most 2 percent of rows to be deleted in one run. The run log gets one row per table per run: snapshot row count, inserted, updated, deleted, unchanged, duration and status. A Lookup activity reads the control table, and a ForEach passes each row to the copy and the notebook as parameters.

Watch out. If the list of hashed columns is hard-coded, a new source column is silently left out of change detection.

4 Copy the full snapshot into staging

A Copy activity reads the whole table and overwrites a staging Delta table in the Lakehouse, with the extract time in a column. For a large table I set Partition option to Dynamic range on the primary key. I also set the degree of copy parallelism, so several queries read slices of the table at once. I agree that number with the DBA, because every slice is a query on their server. Fault tolerance stays off, so a row that fails conversion fails the copy.

Watch out. With Partition option None, one query reads the whole table, whatever parallelism you set. **2.06**

5 Hash and compare in a notebook

The notebook computes a hash for each staging row. For example, it runs sha2 over the business columns cast to strings, joined with a separator and an explicit marker for nulls. Silver stores the hash from the last load, so the compare never goes back to the source. A full outer join on the business key puts every row in one of four groups: new key, changed hash, missing key or unchanged. Only the first three go forward. With 40 million rows and 1 percent change, about 400,000 rows reach the MERGE.

Watch out. Spark's concat_ws skips nulls, so ('A', null) and (null, 'A') produce the same string and the same hash unless nulls are replaced first.

6 Guard the deletes

Before any write, the notebook checks the snapshot. The row count must be within an expected range of the last run. The number of missing keys must stay under the threshold in the control table. If either check fails, the notebook raises an error, the pipeline stops and Silver stays as it was. A cut-short extract then shows up as a failed run instead of a mass delete.

Watch out. An empty or partial snapshot is a valid input. Without a guard, the MERGE deletes every row that did not arrive, and the run succeeds. [2.27](#)

7 MERGE into Silver with soft deletes

One Delta Lake MERGE applies all three groups in a single commit. WHEN MATCHED with a different hash updates the row and its stored hash. WHEN NOT MATCHED inserts. WHEN NOT MATCHED BY SOURCE is an open-source Delta Lake clause. With AND is_deleted = false, it sets is_deleted and deleted_at, so the row stays visible to audits and downstream jobs. Change data feed on the Silver table lets Gold read only the rows this MERGE touched.

Watch out. Changed keys are spread across the whole table, so without deletion vectors the MERGE rewrites nearly every file. [3.04](#)

8 Schedule, alert and plan the switch

The pipeline runs nightly in the source's quiet window, for example 01:00. On failure, a Microsoft Teams activity posts the run log row to the data team's channel. The run log also shows the cost trend. Extract minutes, gateway throughput and Spark time all grow with the table, even on quiet nights. When the rowversion column arrives, I move the table to a Copy job with incremental copy and Merge. I keep the full compare as a weekly job that only looks for deletes.

Watch out. Changing the incremental column of a Copy job resets it to a full copy, and an Append destination then holds every row twice. [2.02](#)

SECURITY IN ONE BREATH

The service account can read only the tables in the control table. The staging table is a raw copy of the source, so only engineers can read it, and consumers read Silver or Gold. The gateway machines sit in the server network, and only the platform team can manage the gateway.

HOW I'D KNOW IT WORKS

- The snapshot row count in the run log matches COUNT(*) on the source at extract time.
- Inserted, updated, deleted and unchanged add up to the number of keys compared.

- A test update to one source row shows up the next morning as exactly one updated row in Silver.
- A deliberately truncated staging table makes the run fail before Silver changes.
- Each business key appears once in Silver.

IF THEY PUSH FURTHER

What if the table grows too large to extract every night?

Split the compare by key range across the week, so each night reads a slice and a full pass completes every seven days. Deletes then show up with a delay, which the business must accept. It also makes the case for rowversion or CDC much stronger. [3.12](#)

What if the source has no stable primary key?

Use the row hash as the key. A changed row then appears as a delete plus an insert, so Silver loses row history. I'd write that down and ask the source team for a key.

What if the source team enables CDC later?

Copy job can then replicate inserts, updates and deletes with Merge, with CDC replication still in preview. Mirroring becomes an option too. Keep a monthly full compare, because a stalled replication doesn't fail anything downstream. [2.03](#)

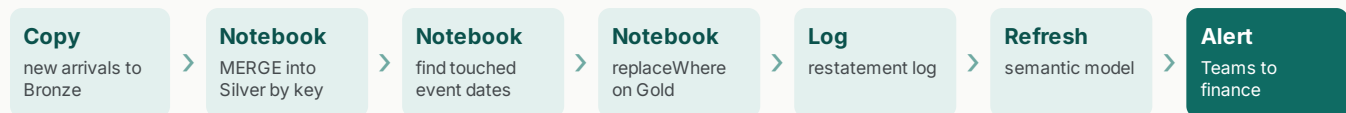
S.18 Design a load where records can arrive up to 30 days after the event they describe. Each record must land in the correct reporting period, totals must never double count, and reports must show when a past period has changed.

Short answer. I keep event time and load time on every row. Bronze appends every arrival. Silver MERGES on the business key, so a late or repeated record updates one row. A notebook in a **Data Factory** pipeline then rewrites only the Gold dates each batch touched, with a replaceWhere overwrite. A restatement log records which periods changed and by how much. The Direct Lake report reads that log and flags restated periods, and the model refreshes only after Gold and the log are both written.

QUESTIONS TO ASK FIRST

- **Which date defines the period: event date, posting date or load date?** This sets the key for Gold. Finance often reports by posting date and operations by event date.
- **May closed periods change?** If a month is locked after close, late records for it go into an adjustment in the open period instead of rewriting the closed one.
- **Can the same record arrive twice, or arrive again as a correction?** Then Silver needs a business key plus a sequence number or source timestamp, so an older copy never overwrites a newer one.
- **Is 30 days a hard limit?** If yes, a 30-day lookback is a safe fallback. If not, the rebuild must follow the dates in each batch, with an alert for anything older than expected.
- **Who needs to know that a period changed?** Finance may want an alert with old and new figures. Dashboard users usually need only a marker on the period.

AT A GLANCE



A **Data Factory** pipeline runs hourly. Bronze keeps every arrival, Silver keeps one row per record, and Gold is rewritten only for the event dates that changed.

HOW I'D BUILD IT

1 Keep event time and load time on every row

Every record keeps its `event_date` from the source. At landing, the pipeline adds `load_ts` and a `batch_id`. Reports group by `event_date`. Operational questions, such as what arrived yesterday, use `load_ts`. I only partition Silver and Gold by event month when volume justifies it, for example billions of rows.

Watch out. Grouping by load date puts a sale from 3 September into October when it arrives on 2 October.

2 Land every arrival in Bronze

A Copy activity appends each batch to Bronze unchanged, with `batch_id` and `load_ts`. Bronze is the audit trail. If a record arrives three times, Bronze has three rows, and I can show when each version came in. The copy selects new records by the source's arrival or insert time, kept as a watermark in a control table.

Watch out. A watermark on `event_date` skips every late record, because its date is older than the last value already loaded. [2.01](#)

3 MERGE into Silver on the business key

The notebook MERGEs each batch into Silver on the business key, for example `source_system` plus `transaction_id`. A matched row is updated only when the incoming sequence number is higher, so a delayed older copy can't overwrite a correction. Duplicates inside one batch are removed first with a window function. In Delta Lake, a MERGE can fail when two source rows match and try to update the same target row.

Watch out. A retry that appends instead of merging doubles every row in the batch. [2.26](#)

4 Work out which dates the batch touched

After the MERGE, the notebook collects the distinct event dates of the rows it inserted or changed. It reads them from Silver's change data feed or from the batch itself. It writes them to an `affected_dates` table with the `batch_id`. For example, a batch that is mostly today's events might touch today plus six older dates. That list drives the Gold rebuild.

Watch out. Rebuilding a fixed 30 days every hour spends capacity on 29 days that did not change. [3.12](#)

5 Rewrite only those dates in Gold

The notebook rebuilds Gold daily aggregates for the affected dates from Silver. It writes them with overwrite mode and a replaceWhere predicate, for example event_date IN the affected list. Delta Lake replaces those rows in one atomic commit, so readers never see a date half rebuilt. Once a night, a second run rebuilds the full 30-day lookback window as a safety net.

Watch out. In Delta Lake, if the rebuilt data holds a row outside the replaceWhere predicate, the write fails by default. Filter the new data with the same predicate.

6 Record restatements

Before overwriting a date in a period that was already reported, the notebook reads the old total. It writes a row to a restatement log: period, event date, batch_id, old value, new value, difference and restated_at. A period counts as reported once it has closed and appeared in a published report. If the change passes a threshold, for example 0.5 percent of the month, a Microsoft Teams activity alerts finance.

Watch out. If the old value isn't captured at the moment of change, nobody can later show what the report said before.

7 Show restated periods in the report

The semantic model reads Gold and the restatement log with Direct Lake, which reads the Delta tables directly. A measure puts a marker next to any month with a restatement, with the last restated date and the change. I turn off automatic updates on the model. A semantic model refresh activity at the end of the pipeline refreshes it after Gold and the log are both written. The new totals and the marker then appear together.

Watch out. With automatic updates on, users can see new totals before the restatement marker exists. **6.05**

SECURITY IN ONE BREATH

Finance owns the restatement log and the threshold. Report users see Gold and the log through an app, and only engineers can read Bronze and Silver.

HOW I'D KNOW IT WORKS

- Each business key appears once in Silver.
- For any event date, the Gold total equals the sum of Silver rows for that date.
- A test record dated 25 days ago lands on the right day in Gold after the next run, with a row in the restatement log.
- Replaying the same batch changes nothing in Silver or Gold.
- A record older than 30 days loads and raises an alert.

IF THEY PUSH FURTHER**What if closed months must never change?**

Keep the true event_date but add an accounting period. Late records for a closed month go into the first open period as flagged adjustments, and the restatement log records the move.

What if Gold were built with Dataflow Gen2 incremental refresh?

It only reprocesses buckets inside its 'extract data from the past' window, and only when the maximum of the change column in a bucket moves. Set the window beyond 30 days and use an arrival timestamp as the change column. **2.25**

What if the hourly run and a manual rerun rebuild the same date at once?

Both overwrite the same rows. One fails with a concurrency conflict, or the last commit wins. Run the Gold rebuild from one pipeline only, and have it refuse to start while a run is in progress. **2.13**

S.19 Design an operations dashboard for a trading business that refreshes every few minutes during trading hours. Order and trade events stream in continuously, operations staff need alerts when something goes wrong, and capacity use outside trading hours should stay low.

Short answer. I'd stream the events through an Eventstream into a KQL database in an Eventhouse, the Real-Time Intelligence store built for querying fresh event data. A Real-Time Dashboard queries it with KQL and uses live refresh, so tiles update when new data lands. Activator rules on the dashboard tiles alert operations in Teams or email. Outside trading hours the Eventstream stops charging once events stop, and the Eventhouse scales down. OneLake availability exposes the same tables as Delta for Power BI and history.

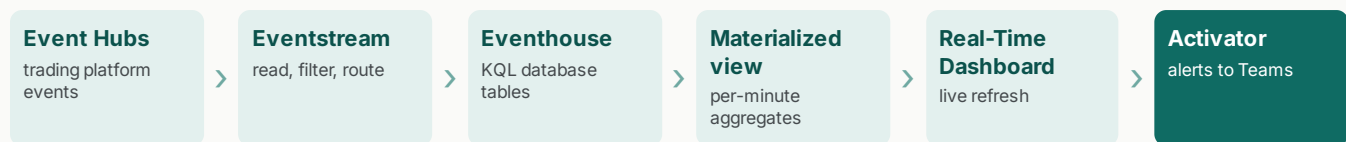
QUESTIONS TO ASK FIRST

- **Where do the events come from today?** If the trading platform already publishes to Azure Event Hubs or Kafka, an Eventstream reads from there. If events only land in a database, a CDC source is the fallback.
- **How quickly must a problem be seen?** If a stuck order must be seen within a minute, the alert interval matters more than the dashboard refresh.
- **What are the trading hours, and is there any overnight or weekend activity?** This sets the Eventhouse minimum capacity schedule and when alert rules should be active.
- **Who views the dashboard, and should each desk see only its own orders?** This decides between the viewer's own identity and the editor's identity, and whether I need row filtering in the database.
- **Does finance also need this data in Power BI or a Lakehouse?** If yes, I turn on OneLake availability for those tables and accept its batching delay.

WHICH TOOL, AND WHY

- **Real-Time Dashboard.** My choice for operations. Tiles are KQL queries against the Eventhouse, live refresh updates them when new data is ingested, and each tile can raise an Activator alert.
- **Power BI in DirectQuery on the KQL database.** Good when the page needs Power BI formatting or joins to an existing semantic model. Automatic page refresh works on DirectQuery, with a minimum interval the capacity admin sets, five minutes by default.
- **Power BI in Direct Lake on OneLake availability.** Reads the Delta copy of the Eventhouse tables. That copy is batched and can lag by up to 3 hours by default, so it suits history more than live operations.
- **Lakehouse loaded by a pipeline every 5 minutes.** Possible, but each run has start-up cost, and small appends every few minutes leave many small Delta files.

AT A GLANCE



Events flow continuously from the trading platform to the dashboard. Nothing runs on a schedule, and quiet periods cost little because nothing flows.

HOW I'D BUILD IT

1 Connect the event source

In this example, the trading platform publishes order and trade events to Azure Event Hubs. I add Event Hubs as a source in an Eventstream and give it its own consumer group. The connection uses a shared access key with listen rights only, stored in a Fabric connection that the data team group can use. If the event hub sits in a protected network, the Eventstream reaches it through a managed private endpoint.

Watch out. The connection holds the key. When the Event Hubs team rotates it, ingestion stops until someone updates the Fabric connection.

2 Land the events in an Eventhouse

An Eventhouse holds one or more KQL databases, which store event data and query it with KQL. I add the Eventhouse as the Eventstream destination in direct ingestion mode, into a table such as TradeEvents with an explicit schema. I may need to drop test messages or split orders from trades. Then I use event processing before ingestion and route each type to its own table. I set retention and caching on purpose, for example 90 days in cache and two years retention.

Watch out. Both policies default to 3,650 days, so every event is kept and cached for ten years unless someone changes them.

3 Pre-aggregate with KQL

Most operations tiles need counts and sums per desk per minute. A materialized view in the KQL database keeps those aggregates current as data arrives: orders, fills, rejects and notional per desk per minute. Tiles query the view, and only drill-down tiles scan raw events. A tile's query then reads the view for the last two hours and summarises by desk.

Watch out. Tiles that summarise raw events for the whole day rescan millions of rows on every refresh, all day long.

4 Build the Real-Time Dashboard

The dashboard has tiles for order flow, reject rate, fill latency and stuck orders, each a KQL query with a time range parameter. I keep live refresh on. It runs one light check per tile based on ingestion time and refreshes a tile only when new data arrived. In Refresh settings I set the refresh rate limit, for example 1 minute, and a fallback interval for tiles that can't use ingestion detection.

Watch out. A refresh rate of a few seconds on 20 tiles, with 50 people watching, becomes a steady stream of queries all day. [8.13](#)

5 Decide how viewers reach the data

By default the dashboard uses pass-through identity, so each viewer needs read access to the KQL database. For a wide operations audience I switch to the dashboard editor's identity through a cloud connection, so viewers see tiles without database access. I share the dashboard with View permission. If each desk may see only its own orders, pass-through identity with row-level security in the database is the safer choice.

Watch out. With the editor's identity, everyone who can view the dashboard sees everything that connection can read.

6 Alert with Activator

Activator is Fabric's rules engine for acting on data. Set alert on a tile creates an Activator rule on that tile's query. The query runs every 5 minutes by default, and I shorten it for urgent rules. For example, one rule fires when rejects for a desk pass 2 percent in a minute. Another fires when no trade has arrived for 3 minutes during trading hours. Actions post to the operations Teams channel or send email, and a rule can also start a pipeline or a Power Automate flow.

Watch out. A no-data rule that stays active after the close fires every check until the next open.

7 Expose the data in OneLake for history

Finance and data science want the same events as Delta tables. I turn on OneLake availability for the tables they need, which writes a Delta copy to OneLake with no extra storage charge. A Lakehouse shortcut points at it, and Power BI can read it with Direct Lake. The copy is batched into efficient Parquet files and can lag by up to 3 hours by default. The target latency can go down to 5 minutes, at the price of many small files.

Watch out. While OneLake availability is on, you can't rename a table, change a column type, apply row-level security or delete data in it.

8 Keep capacity use low outside trading hours

Eventstream flat, traffic and processor charges stop once no data has flowed for two hours. By default the Eventhouse suspends when unused and takes a few seconds to wake. If that wake-up is unacceptable at the open, Capacity Planner mode keeps it active. Its minimum capacity schedule, in preview, sets a floor for trading-hour blocks and no minimum overnight. Live refresh only queries when data changes, so a closed market sends few queries. I confirm all of this in the Capacity Metrics app under Eventhouse UpTime.

Watch out. Pausing the capacity overnight also pauses every Eventstream source and destination on it, and everything else that runs there. [8.09](#)

SECURITY IN ONE BREATH

Operations staff get View on the dashboard and no role in the workspace. Engineers own the Eventstream and Eventhouse. The Event Hubs key has listen rights only, and key rotation is in the runbook together with the Fabric connection update.

HOW I'D KNOW IT WORKS

- A test event sent to Event Hubs appears on the dashboard within the refresh rate limit.
- Per-minute counts in the KQL table match the counts the trading platform sent for the same minute.

- A forced reject spike fires the Activator rule and posts to Teams.
- The Capacity Metrics app shows Eventhouse UpTime near its floor overnight and Eventstream charges stopping after the close.
- The .show table mirroring operations command reports OneLake latency within the agreed limit.

IF THEY PUSH FURTHER

What if operations insist on Power BI?

Build the report in DirectQuery on the KQL database and use automatic page refresh, or change detection on a single measure. Keep the report light, because every visual sends KQL on every refresh.

What if regulators need ten years of trade history?

OneLake availability follows the KQL retention policy, so data removed from the Eventhouse also leaves OneLake. Copy the OneLake data into a Lakehouse table on a daily schedule and keep the Eventhouse retention short.

What if the capacity also runs the nightly batch?

Ingestion and dashboard queries then compete with smoothed batch load, and throttling can delay both. Give operations its own capacity, or mark its workspace mission critical in workspace-level surge protection, which is in preview. Mission critical exempts it from surge protection rules, but capacity throttling still applies. **8.04**

S.20 Design a Silver layer that an hourly job and a daily job both read, where neither job may ever see a half-finished load. Each load writes several related Silver tables that must stay consistent with each other.

Short answer. Delta makes each table's commit atomic, but a load that writes five tables makes five commits, and a reader can start between them. I'd stamp every row with a batch id and record each completed batch, with each table's Delta version, in a control table. Both reader pipelines in **Data Factory** start with a Lookup on that table and read only the last completed batch. They use a batch id filter for append-only tables and Delta time travel for merged ones. If Silver is built in T-SQL, a Warehouse with one explicit transaction across the tables is the simpler alternative.

QUESTIONS TO ASK FIRST

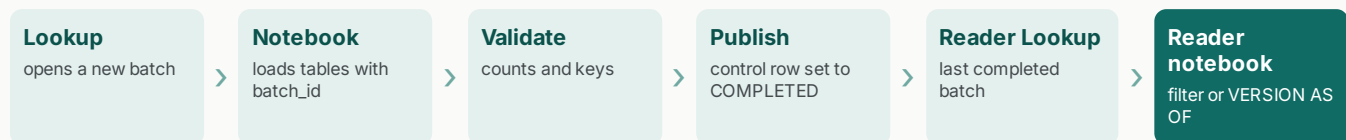
- **How many tables does one load write, and are they append-only or merged?** Append-only tables can hide unfinished rows with a batch id filter. Tables with updates need readers pinned to a table version.
- **Which engines read Silver: Spark, T-SQL or Direct Lake?** Spark can read a table as of a version. The SQL analytics endpoint and Direct Lake need a filter, a Warehouse or a controlled refresh.
- **Can a load still be running when a reader starts?** With an hourly reader, assume yes. If loads always finished long before readers, orchestration alone would be enough.
- **Is the team more at home in Spark or T-SQL?** A T-SQL team can move Silver into a Warehouse and use multi-table transactions.
- **How long does the longest reader run?** Pinned versions must survive VACUUM for longer than that.

WHICH TOOL, AND WHY

- **Lakehouse with a batch control table.** My choice for a Silver layer built in Spark. Each table keeps its atomic Delta commits, and the control table turns a whole load into one published batch.

- **Warehouse with explicit transactions.** BEGIN TRAN and COMMIT across the Silver tables makes the load all or nothing for T-SQL readers. Spark readers see Warehouse tables through the Delta logs it publishes.
- **Warehouse snapshot.** A read-only view of a Warehouse at one point in time, rolled forward after each load. It suits report readers, but it doesn't support Direct Lake and has no Lakehouse equivalent.
- **Load-then-swap with two table sets.** The loader fills an idle copy of every table and flips one pointer. It works, but it doubles storage and every reader must follow the pointer.

AT A GLANCE



The loader publishes a batch only after every table has committed and passed checks. Both reader jobs start from the control table, so they only ever see complete batches.

HOW I'D BUILD IT

1 Start from what Delta guarantees

Every write to one Delta table is one atomic commit, so a reader sees that table before or after the write. A load that writes orders, order_lines and customers makes three commits. A reader that starts between them sees new orders without their lines. Fabric lists multi-table transactions as a Warehouse capability that the Lakehouse doesn't have.

Watch out. Putting the three writes in one notebook cell does not make them one transaction. [4.13](#)

2 Create the batch control table

A small Delta table, silver_batch, gets one row per load: batch_id, started_at, completed_at, status and the Delta version of each table at commit. The loader pipeline opens a batch with status RUNNING. Only the loader writes this table, and readers read it before they touch Silver.

Watch out. Two overlapping loads can each publish rows the other wrote halfway. The loader refuses to start while a RUNNING row exists. [2.13](#)

3 Stamp every row and load the tables

Each Silver table has a batch_id column. The notebook writes dimensions first, then facts. Append-only tables get plain appends. Tables with updates use MERGE, which sets batch_id on every row it inserts or changes. After each write, the notebook reads the table's new version from DESCRIBE HISTORY and keeps it.

Watch out. A retried append writes the same rows again under the same batch id. [2.26](#)

4 Validate, then publish the batch

Before publishing, the notebook compares row counts per table with the source, and checks that every fact key has a dimension row. If a check fails, the batch is marked FAILED and readers keep using the previous one. If all checks pass, one update sets status COMPLETED, completed_at and the table versions. That single commit to the control table is the moment the whole batch becomes visible.

Watch out. A failed batch leaves its rows in the append-only tables. Readers are safe, but the next load must remove rows with that batch id first.

5 Make readers use only completed batches

Both reader pipelines start with a Lookup activity on `silver_batch` that returns the latest COMPLETED batch and its table versions. The values go to the reader notebooks as parameters. For append-only tables, the reader filters on `batch_id` up to that batch. For merged tables, the reader uses time travel, for example `SELECT` from the table `VERSION AS OF` the recorded version. The hourly and daily jobs then read one consistent state, even while a new load runs.

Watch out. On a merged table, a `batch_id` filter hides rows the running load has just updated, because the old version of each row is gone.

6 Protect the pinned versions

Time travel needs the old data files. By default `VACUUM` keeps unreferenced files for seven days, and the Delta log keeps 30 days of history. I never set retention below the longest reader run plus a margin. Readers always take the latest completed batch, which is at most a few hours old.

Watch out. A `VACUUM` with a one-hour retention can delete files a long reader is still using, and that version can no longer be read.

7 Schedule and watch freshness

The loader runs from its own **Data Factory** pipeline, for example every hour at :10. The hourly reader runs at :40 and the daily reader at 02:00, but neither depends on the timing, because both read the latest completed batch. A run log records which batch each reader used. A Microsoft Teams activity alerts when the newest completed batch is older than two hours.

Watch out. When no new batch exists, the hourly reader finds nothing new and succeeds, so a broken loader goes unnoticed without the freshness alert. [2.28](#)

8 Know when a Warehouse is the better answer

If Silver is built in T-SQL, a Warehouse removes most of this. The loader wraps all writes in `BEGIN TRAN` and `COMMIT TRAN`, and readers under snapshot isolation see every table before or after the load. Two transactions that update the same table conflict at commit, even on different rows, so only one loader may write at a time. For Spark readers, I pause Delta Lake log publishing during the load and set it back to `AUTO` afterwards. They keep seeing the previous state until the load finishes.

Watch out. DDL such as `TRUNCATE` inside a long transaction holds a schema lock that blocks every reader of that table until commit. [4.08](#)

SECURITY IN ONE BREATH

Only the loader pipeline writes Silver and the control table. Reader jobs have read access to both. A manual edit that marks a batch COMPLETED publishes whatever is in it, so status changes go through the pipeline only.

HOW I'D KNOW IT WORKS

- During a running load, a test reader returns the same counts as before the load started.
- Stopping the loader halfway leaves the batch `RUNNING` or `FAILED`, and both readers keep using the previous batch.
- The versions stored for each batch match `DESCRIBE HISTORY` for each table.
- Every fact row in a completed batch has its dimension row.

- The run log shows which batch each reader run used.

IF THEY PUSH FURTHER

What if analysts query Silver through the SQL analytics endpoint?

The endpoint picks up Delta commits through a background metadata sync, so it can briefly lag Spark, and it can't read by batch version. Give analysts views filtered on completed batches for append-only tables, or point them at Gold. **5.01**

What if a Direct Lake model reads Silver?

Turn off automatic updates, so the model doesn't reframe on every commit. The loader refreshes the model right after it marks the batch COMPLETED. **6.05**

What if a reader also wants to write back into Silver?

Then two writers modify one Delta table, and a MERGE can fail with a concurrency exception. Keep Silver single-writer and let readers write their own downstream tables. **3.08**

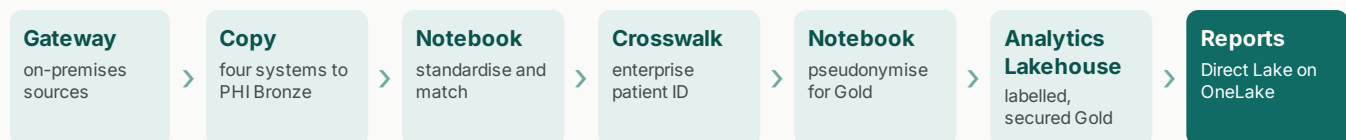
S.21 Design a Fabric platform for a hospital group that combines patient records from four source systems. Patients must be matched across systems, protected health information must be secured and audited, and the data must stay in the group's own country.

Short answer. I'd keep identifiable patient data in a separate PHI workspace on a capacity in the group's own region, encrypted with a customer-managed key. Only pseudonymised Gold data goes to an analytics workspace. A matching notebook links records from the four systems to one enterprise patient ID through a crosswalk table. Microsoft Purview sensitivity labels and protection policies mark and guard the PHI items. OneLake security, or Warehouse row-level security, column-level security and dynamic data masking, narrows what each role sees. Purview audit and OneLake diagnostics record who touched what.

QUESTIONS TO ASK FIRST

- **Is there a national patient identifier, and do all four systems hold it?** If yes, most matching is deterministic. If not, I need probabilistic matching on name, date of birth and address, with human review.
- **Which laws and contracts apply, and must data stay in the country?** This sets the capacity region, and whether Multi-Geo metadata in the home region is acceptable.
- **Who needs identifiable data, and who can work with pseudonymised data?** Clinical quality teams may need names. Most analytics does not, and that split drives the workspace design.
- **Does the security team require customer-managed keys?** Then every item in the PHI workspace must support workspace CMK, which moves some item types elsewhere.
- **How are the four source systems reached?** On-premises systems need an on-premises data gateway. Systems in Azure may allow a virtual network data gateway or mirroring.

AT A GLANCE



Identifiable data stays in the PHI workspace. Only pseudonymised Gold tables reach the analytics workspace, and labels follow the data into reports.

HOW I'D BUILD IT

1 Choose the region and capacity

I create a dedicated F SKU capacity in the group's country, for example UK South for a UK hospital group. Workspaces on that capacity store their content, including OneLake data, in that region. If the tenant's home region is elsewhere, this is Multi-Geo. Some tenant metadata then stays in the home region, such as permissions, report tile metadata and Purview Data Map metadata. The privacy officer signs that off. When the geography has more than one region, data at rest is also replicated to another region in it for disaster recovery.

Watch out. Moving a workspace to a capacity in another region can leave a copy in the old region for up to 30 days.

2 Separate PHI and turn on customer-managed keys

I create two production workspaces on that capacity. The PHI workspace holds Bronze, Silver and the matched patient tables. The Analytics workspace holds pseudonymised Gold and reports. Only the platform team has a role in the PHI workspace. I turn on workspace customer-managed keys for the PHI workspace. They use a versionless RSA key in Azure Key Vault, with soft delete and purge protection on. CMK is generally available for Lakehouse, Warehouse, notebooks, pipelines and dataflows, and in preview for Eventhouse. Dev and test use synthetic data.

Watch out. OneLake security doesn't restrict Admin, Member or Contributor. Giving an analyst Contributor on the PHI workspace opens every table. [7.10](#)

3 Connect the four source systems

Each source gets its own connection. The on-premises EHR, lab and patient administration systems go through an on-premises data gateway cluster inside the hospital network, each with a read-only service account. A cloud source that supports it uses a service principal or workspace identity. A **Data Factory** pipeline reads a control table with one row per source table and copies each into PHI Bronze. Every row gets the source system and load time.

Watch out. Workspace identity isn't supported on gateway connections, so on-premises sources need a service account. [2.16](#)

4 Match patients across systems

I build matching as a notebook, so the rules are visible and testable. It first standardises names, dates of birth, postcodes and phone numbers. A deterministic pass links records that share a national identifier and date of birth. A probabilistic pass scores pairs on name, date of birth, sex and postcode, compared only within blocks such as postcode district and birth year. Pairs above a high threshold link automatically. Pairs in the gray zone go to a review queue for the health records team, and the rest stay separate.

Watch out. Auto-linking gray-zone pairs merges two real patients, which is worse for care and privacy than a missed match.

5 Keep a crosswalk and pseudonymise Gold

The matching writes a crosswalk table: source system, source patient ID, enterprise patient ID, match method, score, reviewer and valid_from and valid_to dates. The enterprise ID is a random surrogate, never derived from PHI. Merges and unmerges add new rows, so I can say what a record was linked to on any date. The Gold notebook replaces the enterprise ID with a keyed hash, drops names and contact details, and writes Gold into the Analytics workspace's Lakehouse.

Watch out. Overwriting the crosswalk on each run loses the reason two records were linked, and a later unmerge can't be explained.

6 Label and protect the PHI items

Microsoft Purview sensitivity labels mark Fabric items, for example 'Highly Confidential PHI' on every item in the PHI workspace. Labels flow downstream to items built from a labelled item, such as semantic models and reports. They stay with data exported from reports to Excel, PDF or PowerPoint. A Purview protection policy tied to that label keeps access for named groups and blocks everyone else, even after an accidental share. Purview DLP policies scan lakehouses and warehouses for sensitive information types and alert when PHI turns up where it shouldn't.

Watch out. A new protection policy can take up to 24 hours to start protecting labelled items, and it doesn't support guest users.

7 Control rows and columns for readers

Gold in the Analytics workspace is secured with OneLake security. It enforces table, row and column rules in Spark, the SQL analytics endpoint in user identity mode and Direct Lake on OneLake. For example, a hospital role sees only its own site, and the postcode district column is hidden from most roles. Analysts get Viewer or item access only. The curated model may live in a Warehouse instead. Then I use a Warehouse security policy for rows, GRANT on columns, and dynamic data masking for fields like date of birth. Masking only hides values in results, and ad hoc queries can infer them, so it never replaces row or column security.

Watch out. A user granted ReadAll on the Warehouse can read its Parquet files from Spark, around the T-SQL row and column rules.

8 Audit and monitor access

Fabric user activities are logged in the Microsoft Purview audit log, which the compliance team searches in the Purview portal. For data access, I turn on OneLake diagnostics in the PHI and Analytics workspaces. It streams access events as JSON into a lakehouse on the same capacity, and I make those logs immutable. Warehouse query insights keep 30 days of query history. A nightly notebook flags unusual access, such as one user reading many patients across sites.

Watch out. For Fabric engines, OneLake diagnostics records that access was granted. The detail of what was queried sits in engine logs such as query insights.

SECURITY IN ONE BREATH

PHI workspaces can use workspace private links for inbound traffic, and outbound access protection blocks outbound connections except those allowed by rule. Connections use service accounts, service principals or workspace identity, never personal logins. Guests never get access to the PHI workspace. Revoking the Key Vault key makes reads and writes to the CMK workspace fail within 60 minutes. Key access is therefore part of the incident runbook.

HOW I'D KNOW IT WORKS

- A test patient present in all four systems resolves to one enterprise ID. Two different patients with the same name and birth date stay separate.
- An analyst account opens Analytics reports but is denied every PHI item, including through a direct OneLake path.
- A site role sees only its own site's rows in Spark, the SQL analytics endpoint and the Direct Lake report.
- The test access appears in the Purview audit log and in the OneLake diagnostics lakehouse.
- Workspace settings show CMK on the PHI workspace, and the capacity region matches the approved region.

IF THEY PUSH FURTHER

What if researchers need patient-level data without identities?

Publish a research Lakehouse per study, with a study-specific pseudonym, shifted dates and suppressed small counts. Grant it per project, with its own label and an end date.

What if a Direct Lake report must enforce the site filter?

Keep the rule in OneLake security and build the model as Direct Lake on OneLake. SQL endpoint row-level security would push a Direct Lake on SQL model into DirectQuery. [5.07](#)

What if every workspace role were Contributor to save admin work?

Contributors bypass OneLake security and can change or delete every item. Keep readers on Viewer and deploy with a service principal. [7.09](#)

S.22 Design branch-level row security for a bank, where each branch manager sees only their own branch's rows. The same rule must hold in SQL queries, Spark notebooks and Power BI reports, with one place to maintain it.

Short answer. I'd keep Gold in a Lakehouse and define the branch rule once, as OneLake security roles with row rules, which are generally available. Spark notebooks, the SQL analytics endpoint in user's identity mode and a Direct Lake on OneLake model with single sign-on all enforce them. A mapping table of branch access drives a notebook that creates one role per branch through the OneLake security API. Branch managers get Viewer or item Read access only, because Admin, Member and Contributor bypass the rules in Spark.

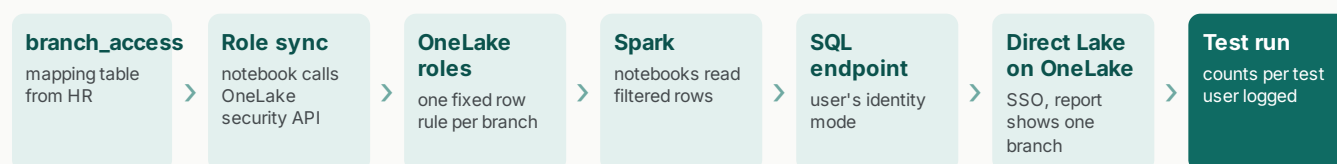
QUESTIONS TO ASK FIRST

- **How many branches are there, and do some people cover several branches?** I write each OneLake row rule as a fixed filter, so I need one role per branch. An item holds 250 roles by default. Regional managers join several roles, and the rules combine with OR.
- **Is Gold a Lakehouse or a Warehouse?** OneLake security row rules cover Lakehouse tables. A Warehouse needs T-SQL row-level security instead, and the Power BI side changes with it.
- **Do managers also need columns hidden, such as national ID numbers?** Then the column rule goes into the same branch role. A user with row rules in one role and column rules in another gets query failures.
- **Who writes the notebooks, the managers or analysts working for them?** Anyone with Contributor on the Gold workspace sees every row in Spark. Notebook users work in their own workspace and read Gold through a shortcut.
- **How fast must a transfer between branches take effect?** A role change applies in about 5 minutes. A group membership change takes about an hour. If that is too slow, I put users directly into roles.

WHICH TOOL, AND WHY

- **OneLake security row-level security.** My choice. One rule, enforced by Spark, the SQL analytics endpoint in user's identity mode and Direct Lake on OneLake. Each rule is a fixed filter, so I generate one role per branch.
- **T-SQL row-level security in the Warehouse or SQL endpoint.** Dynamic, with a predicate function that joins the mapping table on USER_NAME(). It only covers T-SQL queries, and Direct Lake on SQL falls back to DirectQuery for those tables.
- **Semantic model row-level security.** Dynamic with USERPRINCIPALNAME() and a mapping table. It only filters inside that model, so SQL users and notebooks still see every branch.
- **One table or workspace per branch.** Simple to explain, but a few hundred copies of every Gold table is a maintenance and cost problem.

AT A GLANCE



The branch rule lives once in OneLake security. Every engine reads through it, and a mapping table plus a sync notebook keep the roles in step with HR.

HOW I'D BUILD IT

1 Put Gold in a Lakehouse and fix the workspace roles

Gold sits in a schema-enabled Lakehouse in the production workspace. Engineers and the deployment service principal are the only Members and Contributors. Branch managers get Viewer on the workspace, or Read on the Lakehouse item, and nothing higher. I delete or edit the DefaultReader role, because it gives every user with ReadAll all the rows in the Lakehouse.

Watch out. A manager made Contributor to fix one notebook bypasses the roles in Spark and sees every branch. [7.10](#)

2 Build the branch access mapping table

A table called security.branch_access holds one row per grant. Each row has the branch ID, an Entra group object ID or user UPN, valid from, valid to and a change ticket. I prefer one Entra security group per branch, for example sg-branch-0412, filled from HR joiners and leavers. Individual rows cover regional managers and temporary cover. A **Data Factory** pipeline with a Copy activity loads the HR extract into this table every night at 01:00.

Watch out. A UPN stored with different case or an old domain matches nobody, and that manager silently sees nothing. [7.07](#)

3 Generate one OneLake security role per branch

I write each OneLake row rule as a fixed filter, such as `SELECT * FROM gold.fact_transactions WHERE branch_id = '0412'`. The OneLake security guides say dynamic and multitable rules aren't available. Each rule is at most 1,000 characters. So a notebook reads branch_access and calls the OneLake data access security REST API to create or update a role per branch, for example branch_0412. Each role grants read on every Gold table that carries branch_id, with the row rule on each, and lists the members. A head office role has no row rule. The notebook runs as a service principal with Member on the workspace, and writes security.role_sync_log with roles added, changed and removed.

Watch out. An item holds 250 roles by default, up to 1,000 on request, and 500 members per role. For example, 180 branches fit, and 1,200 do not.

4 Set the SQL analytics endpoint to user's identity mode

In user's identity mode the endpoint passes the signed-in user to OneLake, and the OneLake roles decide what they read. In that mode SQL GRANT on tables is ignored, and T-SQL row security can't be defined. That leaves one rule, which is what I want. I check the mode on each Gold endpoint and set it to user's identity. Every table with branch rows carries a branch_id column, and a table left out of a role is invisible to that role's members.

Watch out. An endpoint left in delegated mode reads as the owner and ignores the OneLake rules, so SQL users see every branch.

5 Let notebooks read through the OneLake Spark catalog

Spark enforces OneLake row and column rules when users read tables by catalog name, such as `gold_lh.gold.fact_transactions`. Branch analysts run notebooks in their own workspace, where they are Contributor. They read Gold through a shortcut, so the filter applies at the Gold Lakehouse. Direct abfss path reads are blocked for them. The filtering runs in a Microsoft-managed security context that shows up as `SparkSecurityControl` jobs in the Monitoring hub.

Watch out. A user with row rules in one role and column rules in another gets failed queries. So both rules for a branch live in one role.

6 Build the report on Direct Lake on OneLake with single sign-on

The semantic model uses Direct Lake on OneLake, which resolves each viewer's OneLake roles and applies the row rules. The data connection keeps single sign-on on, so every query runs as the viewer. The model has no RLS of its own. The model owner is an engineering account with full read, because framing checks the owner's access. Direct Lake on OneLake has no `DirectQuery` fallback, so I test every visual before release.

Watch out. If someone binds a fixed identity with SSO off, every viewer sees what that identity sees, and the branch rule is gone for Power BI. [7.12](#)

7 Test the rule the same way in all three engines

I keep test accounts for three sample branches, one regional manager and head office. After every role sync and every release, a tester signs in as each account and runs the same three checks. They are a row count by `branch_id` through the SQL endpoint, the same count in a notebook and a card on a test report page. The results go into `security.rls_test_log` with the expected counts from `branch_access`. Any mismatch fails the release.

Watch out. Testing as an engineer proves nothing, because Members and Contributors see every branch in Spark.

8 Audit role changes and plan for the delay

Purview audit records role changes as `CreateOrUpdateDataAccessRoles`. An alert fires when a role changes outside the sync notebook's run window. I tell the business that a group membership change takes about an hour to apply. After a role change, the pipeline also refreshes the Direct Lake model, so the report and the roles agree.

Watch out. A manager moved out of a branch can still see its rows for up to an hour, and longer in reports until the model refreshes. [7.14](#)

SECURITY IN ONE BREATH

Branch managers hold Viewer or item Read only. The engineering group and the sync service principal see all rows by design, so that group stays small and its sign-ins are reviewed. Customer columns such as national ID get column rules inside the same branch roles.

HOW I'D KNOW IT WORKS

- A branch 0412 test user gets the same row count from the SQL endpoint, a notebook and the report, and it matches the expected count.
- The regional test user sees exactly its branches and nothing else.

- The role sync log shows one role per active branch, and no role exists without a matching row in `branch_access`.
- A test user's direct abfss read of a Gold table fails.
- A manager removed from a branch group loses access within about an hour, after the model refresh.

IF THEY PUSH FURTHER

What if Gold lives in a Warehouse?

I'd use T-SQL row-level security. A predicate function joins `branch_access` on `USER_NAME()`, and a security policy binds it. Direct Lake on SQL falls back to DirectQuery for those tables. The faster option is Direct Lake on OneLake with a fixed identity and model RLS on the same mapping table. That means keeping two copies of the rule in step. [5.07](#)

What if the bank has 1,500 branches?

That passes the role limit even after a support request. I'd write the rule as one dynamic T-SQL policy in a Warehouse and one model RLS role. Both read the same mapping table, and I test them together.

What if a manager covers another branch for two weeks?

I add a row to `branch_access` with a `valid_to` date. The next sync adds the manager to that branch role, and the sync after `valid_to` removes them.

S.23 Design read-only access for external partner companies to a defined subset of Gold tables. Each partner may see only the tables agreed in its contract, every access must be auditable, and access must be revocable quickly.

Short answer. I'd build a partner layer in a separate workspace, with one Lakehouse per partner. A notebook writes only the agreed tables into it from Gold. Partners on Fabric get an external data share, which is generally available and gives them a live, read-only shortcut in their tenant with no copy. Partners without Fabric join as Microsoft Entra B2B guests with item Read, and read through the SQL analytics endpoint or a report. Purview audit and SQL audit logs record what happens, and I revoke per share or per guest group.

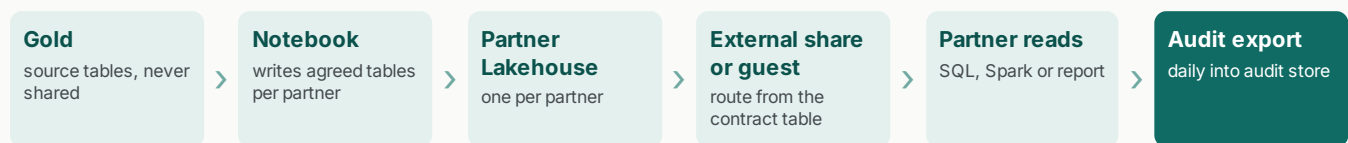
QUESTIONS TO ASK FIRST

- **Does each partner have its own Fabric tenant and capacity?** Yes means an external data share. No means B2B guest access into our tenant.
- **Is the subset whole tables, or also certain rows and columns?** An external share covers whole tables or folders. Row and column subsets become per-partner tables that I build.
- **Must we know which people at the partner read the data?** With an external share the partner decides who reads it, and we can't control that. If we must know, the partner comes in as named guests.
- **Are there data residency rules?** Shared data might be accessed across geographic boundaries in the partner's tenant, so legal signs this off first.
- **How fresh must the partner's data be?** A share is live on the partner table. I set the freshness by how often the pipeline rebuilds that table.

WHICH TOOL, AND WHY

- **External data sharing.** My choice for partners on Fabric. Data stays in our OneLake, the partner gets a read-only shortcut, and revoking is one action.
- **B2B guest access.** For partners without Fabric. We control the identities and see their queries, but we manage guest accounts, licences and Conditional Access.
- **Shortcuts from the partner workspace into Gold.** No copy, but guests then need OneLake access on Gold itself. Shortcuts inside a shared folder also don't resolve for an external share.
- **File export with a Data Factory pipeline.** A Copy activity drops files into the partner's storage. It fits partners who want files, but delivered data can't be revoked.

AT A GLANCE



Gold feeds a partner Lakehouse per partner. Each partner reads only that Lakehouse, through an external share or as a guest.

HOW I'D BUILD IT

1 Create the partner workspace

I create a workspace called partner-share-prod on the production capacity. It holds one schema-enabled Lakehouse per partner, for example lh_partner_acme. Engineers are Members. No partner identity ever gets a workspace role. The Gold workspace doesn't change.

Watch out. Outbound access protection isn't compatible with external data sharing, so this workspace stays out of that setting.

2 Keep a partner contract table

A table called partner_contract holds one row per partner. Each row has the partner ID, tenant ID, contact email, route, table list, column list, row filter, valid-to date and share ID. After the Gold load, a **Data Factory** pipeline runs a notebook that reads this table. For each partner it writes the listed columns and rows into that partner's Lakehouse as Delta tables, with overwrite. A run log gets one row per partner and table with the row count.

Watch out. A notebook that selects all columns passes every new Gold column to partners, so column lists are explicit.

3 Share with Fabric partners through external data sharing

A Fabric admin turns on the External data sharing tenant setting for a small group, sg-partner-publishers. The partner's admin turns on Users can accept external data shares on their side. An engineer with Read and Reshare on lh_partner_acme creates the share, selects the tables and enters the recipient email. The invitation is valid for 90 days. The partner accepts into a Lakehouse in their tenant, which creates a shortcut they can query with SQL, Spark or a semantic model. I automate this with the external data shares REST API under a service principal and store the share ID in partner_contract.

Watch out. Shortcuts inside a shared folder don't resolve in the consumer tenant, which is why partner tables are real Delta tables.

4 Bring in partners without Fabric as B2B guests

The Guest users can access Microsoft Fabric tenant setting is scoped to a group. Each partner's guests sit in an Entra group, such as sg-guest-acme, under a Conditional Access policy that requires MFA. I share lh_partner_acme with that group, so its members can query the SQL analytics endpoint. Reports go through an app. On F64 or larger, guests with a free licence can view Power BI content, and below F64 each guest needs Pro or PPU. Guests can't use Analyze in Excel or publish from Power BI Desktop.

Watch out. A guest's UPN can look different in Power BI, so any rule on guest identity is tested while signed in as that guest. [7.07](#)

5 Lock down the partner Lakehouse with OneLake security

I delete the DefaultReader role in each partner Lakehouse. A role per partner grants read on its own tables, with the partner's group as members. The SQL analytics endpoint runs in user's identity mode, so the same roles apply to SQL. If one Lakehouse ever serves two partners, OneLake row rules split the rows.

Watch out. OneLake security targets Viewers and Read users, so a guest given Contributor can read every table. [7.10](#)

6 Audit shares and guest reads

Purview audit records CreateExternalDataShare and AcceptExternalDataShare, and guest activity such as ViewReport and connections to the SQL analytics endpoint from external apps. SQL audit logs on each partner Lakehouse's SQL endpoint record every guest query. OneLake diagnostics on the partner workspace records API and file access with the caller's UPN. A daily pipeline copies all three into the audit store. Reads made inside the partner's tenant are theirs to log, so the contract requires them to keep access logs.

Watch out. Purview keeps Fabric events for 180 days by default, and the activity events API only reaches back 28 days. So the daily copy can't skip days.

7 Revoke quickly and on purpose

For a share, I open Manage permissions on the partner Lakehouse, go to the External data shares tab and revoke it. The API call is Revoke External Data Share. Revoking cuts all access at once and can't be undone, and everything the partner built on it stops working. For guests, I remove them from the partner group and block sign-in in Entra, then remove the item share. partner_contract records the date and the reason, and a quarterly review checks every live share against it.

Watch out. A revoked share can't be restored, so a pause for an investigation means a new share and a rebuild on the partner side.

SECURITY IN ONE BREATH

Partners only ever see partner Lakehouses we build on purpose. Our governance controls, including sensitivity labels and DLP, don't cross the tenant boundary with an external share. So labelled or personal data goes through the guest route or not at all. Share creation raises an alert, and only sg-partner-publishers can create shares.

HOW I'D KNOW IT WORKS

- Each partner's accepted share or guest session lists only the tables in its contract row.

- A test guest who queries a Gold table by name gets an access error.
- Every share listed in Manage permissions matches a partner_contract row, and the weekly reconciliation finds none extra.
- Yesterday's share events and guest queries are in the audit store.
- After a test revocation, the test consumer's shortcut returns no data.

IF THEY PUSH FURTHER

What if a partner may see only its own customers in a shared table?

The notebook writes a filtered table into that partner's Lakehouse. Our OneLake security doesn't travel with an external share, so filtering must happen before the data reaches the shared table.

What if the security team turns on outbound access protection for every workspace?

External data sharing isn't compatible with it. I'd ask for an exception for partner-share-prod and keep Gold, which has no shares, fully protected. [7.06](#)

What if a partner wants daily files instead?

A **Data Factory** pipeline copies the partner tables to their storage or SFTP over a connection they own. Delivered files can't be revoked, so the contract covers deletion.

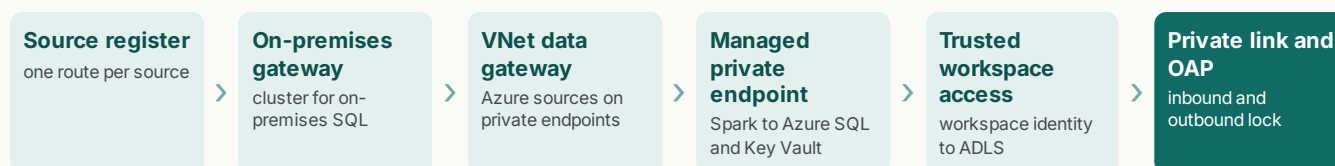
S.24 Design connectivity for a Fabric platform where every source sits behind a firewall or a private endpoint. Sources include on-premises SQL Server, Azure SQL Database and firewalled ADLS Gen2, and pipelines, dataflows, notebooks and the Warehouse must all load data.

Short answer. I'd give every source one documented route. On-premises sources go through an on-premises data gateway cluster for **Data Factory** pipelines, Copy jobs, Dataflow Gen2 and mirroring. Azure sources on private endpoints use a virtual network data gateway for those items, and managed private endpoints for Spark notebooks. Firewalled ADLS Gen2 uses trusted workspace access with the workspace identity for shortcuts, pipelines and Warehouse COPY INTO. Workspace-level private links and outbound access protection then close the workspace itself.

QUESTIONS TO ASK FIRST

- **Which sources are on-premises, which are Azure with private endpoints, and which are other clouds?** Each network location maps to a different route and a different team to approve it.
- **Which Fabric items must reach each source?** Spark can't use gateways, and trusted workspace access doesn't cover Spark. A notebook needs a managed private endpoint.
- **Must users reach Fabric only from the corporate network?** Then I add workspace-level private links. Workspaces that hold semantic models can't block public access, so reports sit in a separate workspace.
- **Is the capacity a purchased F SKU?** Trusted workspace access, workspace-level private links and outbound access protection all need an F SKU. Trial capacities don't work.
- **Who owns Azure networking and approvals?** Managed private endpoints wait for the resource owner's approval, and the virtual network gateway needs a delegated subnet. That team is on the plan from day one.

AT A GLANCE



The routes run side by side. The source register says which one each source and each Fabric item uses, and the workspace controls come last.

HOW I'D BUILD IT

1 Build the source register

A table called `ops.source_route` holds one row per source. Each row has the host, network location, auth type, route, connection ID, owner and the Fabric items that use it. No connection gets created without a row. Each route gets a small canary copy every morning, and its result goes to the run log.

Watch out. A source read by a pipeline and a notebook needs two routes, and the second one is often never opened on the firewall.

2 Install an on-premises data gateway cluster

I install the on-premises data gateway on two Windows servers in the data centre as one cluster. Pipelines need version 3000.214.2 or later and outbound access to `*.frontend.clouddatahub.net`. Connections to SQL Server use a SQL login or a Windows service account, and are shared with the data team's group. Pipelines, Copy jobs, Dataflow Gen2 and semantic model refresh all use this cluster. Workspace identity doesn't work on gateway connections.

Watch out. Members updated one at a time run different builds, and runs fail depending on which member picks them up. [2.16](#)

3 Create a virtual network data gateway for Azure sources

The subscription owner registers the Microsoft.PowerPlatform resource provider. The network team adds a subnet delegated to Microsoft.PowerPlatform/vnetaccesslinks in a VNet that reaches the private endpoints. It is sized at 5 addresses plus one per gateway member. I create the gateway in Manage connections and gateways. Dataflow Gen2, pipelines, Copy jobs, mirroring and semantic models can run on it. Connections to Azure SQL Database use a service principal.

Watch out. The VNet data gateway doesn't refresh expired OAuth2 tokens, so a long load on an OAuth connection fails after about an hour. [7.02](#)

4 Add managed private endpoints for Spark

Notebooks can't use either gateway. A workspace admin opens Workspace settings, Network security, and creates a managed private endpoint with the Azure SQL server's resource ID and subresource. The resource owner approves it in the Azure portal. I add another one for Key Vault, where the notebook reads its secrets. For on-premises SQL from Spark, the pattern is a Private Link Service in front of a load balancer that forwards over ExpressRoute.

Watch out. The first managed private endpoint creates a managed virtual network, starter pools stop, and each Spark session takes about 3 to 5 minutes to start. [3.11](#)

5 Use trusted workspace access for firewalled ADLS Gen2

I create a workspace identity and give it Storage Blob Data Reader on the storage account. The storage team adds a resource instance rule for the workspace's resource ID, using ARM or PowerShell. After that, ADLS Gen2 shortcuts, pipeline Copy activities, Warehouse COPY INTO and import semantic models reach the account with workspace identity authentication. Spark reads the data through the shortcut. A storage account holds at most 200 resource instance rules.

Watch out. A deleted workspace identity can't be restored, and its replacement has a new ID with no storage roles. [7.05](#)

6 Feed the Warehouse from OneLake

The Warehouse can't use gateways or managed private endpoints. Pipelines land data in a Lakehouse, and the Warehouse reads those tables with cross-database queries. Files in firewalled ADLS arrive through COPY INTO with trusted workspace access. Every Warehouse load starts from OneLake or from that storage account.

Watch out. COPY INTO through trusted workspace access fails on a trial capacity, because the feature needs a purchased F SKU.

7 Close inbound access with workspace-level private links

Workspace-level private link is generally available. A tenant admin enables it in the admin portal. Then I map each data workspace to the VNet through a private endpoint and restrict inbound public access. Lakehouses, Warehouses, pipelines, notebooks, Copy jobs and Dataflow Gen2 (CI/CD) are supported. Semantic models aren't, so reports live in their own workspace.

Watch out. A workspace assigned to a deployment pipeline can't block public access, so these workspaces deploy with Git and fabric-cicd.

8 Turn on outbound access protection last

Outbound access protection blocks every outbound connection from the workspace. Pipelines, Copy jobs, Dataflow Gen2 and mirrored databases then need data connection rules that allow named connectors and endpoints. Spark items need managed private endpoints. It is generally available for **Data Factory** items, mirrored databases, Spark items, OneLake shortcuts and semantic models. I switch it on in a test workspace first and rerun every pipeline.

Watch out. Switched on in production without rules, it fails every copy at the next run. [7.06](#)

SECURITY IN ONE BREATH

Every connection uses a service principal, a workspace identity or a service account, never a person. Gateway admins are a named group, and the gateway servers get patched with the rest of the data centre. Storage accounts keep public access disabled, and the only allow rules are resource instance rules and private endpoints.

HOW I'D KNOW IT WORKS

- Every row in the source register has a working connection, and each route's morning canary succeeds.
- From a machine outside the approved VNet, the locked workspace refuses connections.
- A notebook's Azure SQL host name resolves to a private IP address.
- The storage firewall has no public IP rules, only resource instance rules and private endpoints.

IF THEY PUSH FURTHER

What if a notebook must read on-premises SQL Server directly?

It needs a managed private endpoint to a Private Link Service with a load balancer and forwarding over ExpressRoute or VPN. Unless the logic must live in Spark, I'd land the data with a pipeline through the gateway and read it from OneLake.

What if the team wants workspace identity on every connection?

It works for ADLS shortcuts, pipelines, semantic models and Dataflow Gen2, but never through a gateway. Gateway connections keep a service principal or a service account. **7.03**

What if both gateway servers go down during the night load?

Every on-premises copy fails. I'd put the members on different hosts, alert on gateway status, and have the pipeline retry once after a delay.

S.25 Design the audit trail a Fabric platform needs for a sensitive customer table. When an auditor asks who read it during the last quarter, the team must answer with names, times and access paths.

Short answer. No single log answers this, so I'd switch on one log per read path well before any audit and copy them all into one Eventhouse. Purview audit records Fabric and Power BI activity, such as report views and external connections, for 180 days by default. SQL audit logs on the Warehouse and SQL analytics endpoint record each query and the objects it used. OneLake diagnostics records API and file access to the Lakehouse, and workspace monitoring (in preview) records semantic model queries. A daily **Data Factory** pipeline copies each source so we set the retention.

QUESTIONS TO ASK FIRST

- **Which paths reach the table today, such as the SQL endpoint, notebooks, reports, the OneLake API or external shares?** Each path writes a different log. Fewer paths means fewer logs to trust.
- **How far back must we answer, and how quickly?** This sets the retention in the audit store, for example 13 months, and whether the answer is a saved query.
- **Does 'read' include opening a report that shows totals?** If yes, report views and model queries count. If only row access counts, SQL and OneLake logs carry the answer.
- **Which Purview audit licensing do we have?** Standard keeps records 180 days. Longer retention inside Purview needs Audit Premium licensing and a policy created before the events happen.
- **Who may read the audit data itself?** The audit store gets its own workspace, with compliance and a few platform leads only.

WHICH TOOL, AND WHY

- **Eventhouse in an audit workspace.** My choice. KQL searches months of events fast, and a retention policy per table keeps them as long as we decide.
- **Lakehouse Delta tables.** Cheaper to keep and fine for occasional audits, but slower to search across mixed JSON and XEL sources.
- **Purview audit retention policies only.** Needs no pipeline, but it only covers Purview events, and SQL queries and OneLake reads aren't there.
- **Azure Log Analytics or Sentinel.** Good if the security team already works there. The same exports feed it, and the security team runs the search.

AT A GLANCE



Four logs cover four read paths. One daily pipeline copies them into an Eventhouse where one saved query answers the auditor.

HOW I'D BUILD IT

1 Map every read path to a log

I list how the table can be read and which log records it. Report views go to Purview audit. DAX queries against the model go to workspace monitoring. SQL queries on the Warehouse or SQL endpoint go to SQL audit logs. API and file access to OneLake goes to OneLake diagnostics. Then I cut the paths down with OneLake security. Business users reach the table only through the SQL endpoint and one semantic model. Spark reads stay with the engineering group.

Watch out. Purview's 'Create an SQL query' events are about saved queries in the editor, and they say nothing about who read which rows.

2 Collect Purview audit every day

Purview audit is on by default and keeps Fabric events for 180 days on Audit Standard. A notebook calls the Power BI admin activity events API with a service principal that has Tenant.Read.All. The API returns the last 28 days, one UTC day per call, at most 200 requests an hour. I keep ViewReport, AnalyzeInExcel, ConnectFromExternalApplication, ConnectWarehouseAndSqlAnalyticsEndpointLakehouseFromExternalApp, ExportArtifactDownload and CreateOrUpdateDataAccessRoles, and in practice the whole day.

Watch out. A collector broken for a month can't catch up from the API, and the gap must be filled from a Purview portal search.

3 Switch on SQL audit logs

SQL audit logs are off by default. In the Warehouse settings, on the SQL audit logs page, I turn on Save events to SQL audit logs. I choose DATABASE_OBJECT_ACCESS_GROUP, SCHEMA_OBJECT_ACCESS_GROUP, BATCH_COMPLETED_GROUP and the login success and failure groups, and set retention; the default is nine years. The same applies to the Lakehouse's SQL analytics endpoint. Logs are XEL files in the item's Audit folder, read with sys.fn_get_audit_file_v2. The audit reader gets VIEW DATABASE SECURITY AUDIT. Predicate filtering, now generally available, drops the noisy ETL service principal.

Watch out. Deleting the Warehouse deletes its audit files too, so the daily export is the copy that survives.

4 Turn on OneLake diagnostics

A workspace admin opens Workspace settings, OneLake settings, and turns on Add diagnostic events to a lakehouse. The target is an audit Lakehouse on the same capacity. Events arrive as hourly JSON files with the caller's UPN, operation, item and whether a shortcut was used. Every UI and ADLS or Blob API operation is logged. For Fabric engines it records that temporary access was granted, and the engine's own log holds the detail. The first events can take an hour to appear, and I make the files immutable for the retention period.

Watch out. OneLake diagnostics has no default retention, so without a cleanup job the files grow forever, and without immutability they can be deleted.

5 Capture semantic model queries

Purview shows who opened a report, but not which data the visuals asked for. Workspace monitoring, in preview, writes semantic model operation logs into an Eventhouse in the workspace. The Azure Log Analytics integration is the alternative, and it records query events with the user and the DAX text. A workspace can have one or the other, never both. I use workspace monitoring and copy its tables out every day.

Watch out. Workspace monitoring keeps data for 30 days, so a quarter is gone unless the daily copy runs.

6 Copy everything into one audit Eventhouse

An audit workspace holds the Eventhouse, with access for compliance and platform leads only. A **Data Factory** pipeline runs at 02:00. It calls the activity events notebook, reads yesterday's SQL audit files with `sys.fn_get_audit_file_v2`, loads the OneLake diagnostics JSON and pulls the monitoring tables. Each source writes a row to the run log with its event count. The Eventhouse tables get a retention policy, for example 13 months.

Watch out. A day with zero events looks like a quiet day, so zero rows from any source raises an alert. [2.27](#)

7 Build and rehearse the auditor query

A KQL function `who_read(table, start, end)` combines four sources. They are SQL audit events on that object and OneLake diagnostics on its path. They also include model queries against models that contain it and report views on those models. A small mapping table links reports to models and models to tables. The output gives user, time, path and client application. A lookup table turns service principal IDs into readable names. Each quarter I read the table as a test user and check that the query finds it.

Watch out. ETL service principals appear as app IDs, and an auditor won't accept an unexplained GUID.

SECURITY IN ONE BREATH

The audit workspace sits apart from the data workspaces, and data engineers have no write access to it. OneLake diagnostics files are immutable for the retention period. Access to the audit Eventhouse is itself logged.

HOW I'D KNOW IT WORKS

- A test read through each path appears in the audit Eventhouse within a day, with the test user's UPN.
- The run log shows events from every source for every day of the quarter.
- The oldest record in each audit table meets the retention target.

- The quarterly rehearsal answers who_read in under an hour.

IF THEY PUSH FURTHER

What if the auditor asks next week and none of this exists yet?

Purview keeps 180 days, so report views and external connections can still be answered. Query insights keeps 30 days of SQL queries with login names, which covers part of the last month. Older SQL queries and OneLake reads from before the logs were on can't be recovered, and I say that plainly.

What if the table is also shared with a partner tenant?

Our audit shows the share being created and accepted. Reads inside the partner's tenant are logged there, so the contract must require that log.

What if seven years of history is required?

The Eventhouse retention policy covers it. Inside Purview it needs Audit Premium, the 10-year add-on and a retention policy created before the events happen.

S.26 Size and protect a single Fabric capacity for nightly ETL, 200 Power BI report users from 08:00 and ad hoc SQL during the day. No workload may starve the others.

Short answer. I'd start on F64, because on a smaller SKU each of the 200 viewers needs Pro or PPU. Then I'd size from the Capacity Metrics app after two weeks of real load. ETL and Warehouse queries are background work smoothed over 24 hours, so the night load still occupies the capacity at 08:00. I'd move Spark to Autoscale Billing, switch on capacity-level surge protection to reject new background jobs before reports suffer, and cap the ad hoc workspace. If that isn't enough, the ETL moves to its own capacity.

QUESTIONS TO ASK FIRST

- **How many CU-hours does the nightly ETL use, and in which engine?** Spark-heavy ETL can leave the capacity through Autoscale Billing. Pipeline and Warehouse work stays on it.
- **Do the report users already have Pro licences?** Below F64 every viewer needs Pro. That often decides the SKU before any CU math.
- **When must the ETL finish, and can it start earlier?** An earlier start gives smoothed usage more hours before 08:00.
- **Does ad hoc SQL hit the same Warehouse the reports read?** Then heavy analyst queries compete with report queries, and they need their own workspace and limits.
- **Is the capacity reserved or pay-as-you-go?** This decides whether scaling or pausing saves money at all.

WHICH TOOL, AND WHY

- **One F64 with surge protection and Spark Autoscale Billing.** My choice to start. It covers free-licence viewers, takes Spark off the shared CUs and protects reports from background overload.
- **Two capacities, ETL and reporting.** Hard isolation, because each capacity smooths and throttles on its own. It costs more and is my step two if tuning is not enough.
- **A bigger SKU only.** It raises the ceiling for everyone, but a runaway job still hits all workloads together.
- **Scheduled scale or pause.** It fits dev and test capacities. Pausing a production capacity serving 200 users bills the carried-forward usage and takes reports offline.

AT A GLANCE



Night work runs early and Spark bills separately. Surge protection and the Capacity Metrics app keep the 08:00 report load safe.

HOW I'D BUILD IT

1 Pick the starting SKU

F64 is the smallest SKU where viewers with a free licence can open Power BI content. Below it, each of the 200 users needs Pro or PPU. I run one week of the real nightly ETL on F64 and read the CU-hours from the Capacity Metrics app. For example, F64 gives 1,536 CU-hours a day. An ETL that uses 600 CU-hours takes about 39 percent of every timepoint for the next 24 hours.

Watch out. The SKU that worked in development with five users throttles all morning with 200. [8.08](#)

2 Plan for smoothing and bursting

Interactive work, such as report queries, is smoothed over 5 to 64 minutes. Background work, such as ETL, refreshes and Warehouse queries, is smoothed over 24 hours. Fabric counts almost all Warehouse work as background, ad hoc SQL included. Bursting lets a job run faster than the SKU allows. Spark gets 3 times its VCores, 384 on F64, and the Warehouse can burst up to 12 times on F64. The capacity pays the burst back through smoothing.

Watch out. A 01:00 load that finished hours ago still takes its share at 09:00, and the morning peak pushes the total over 100 percent. [8.03](#)

3 Know the throttling stages

Up to 10 minutes of future capacity is overage protection, with no throttling. Between 10 and 60 minutes, interactive jobs are delayed by 20 seconds. Between 60 minutes and 24 hours, interactive jobs are rejected. Past 24 hours, background jobs are rejected too. Carried-forward usage burns down only in timepoints below 100 percent.

Watch out. Report users feel throttling first, even when background work caused it, because the interactive stages start earlier. [8.04](#)

4 Move Spark to Autoscale Billing

Autoscale Billing for Spark is generally available, and newer Learn pages call it on-demand billing for Spark. A capacity admin turns it on in Capacity settings and sets a maximum CU. Spark jobs then run on their own pay-as-you-go meter and don't use the capacity's CUs, with no smoothing or bursting. At the limit, background Spark jobs queue and interactive ones are throttled. Costs show in Azure Cost Analysis and the Capacity Metrics app. A higher limit needs an Azure quota request.

Watch out. Enabling, disabling or lowering the maximum CU cancels running Spark jobs, so I change it outside the ETL window.

5 Protect reports with surge protection

Capacity-level surge protection is generally available. In Capacity settings I turn Background Operations on, with a rejection threshold of, for example, 80 percent and a recovery threshold of 60 percent. Above the threshold, new background jobs are rejected and running jobs continue. Autoscale-billed Spark isn't blocked. Workspace-level surge protection, in preview, caps the ad hoc workspace at, for example, 20 percent of the capacity over 24 hours. I mark the reporting workspace Mission Critical, so the workspace limit can't block it.

Watch out. A workspace that crosses its limit is blocked for every operation, interactive ones included. **8.05**

6 Schedule and trim the night work

The ETL starts at 01:00 and finishes by 05:00, and Direct Lake models are framed once at the end. OPTIMIZE and VACUUM run only on tables with new commits. Analysts get their own workspace for ad hoc SQL, reading Gold through shortcuts, so their load has a boundary for surge limits. Heavy one-off queries go after 18:00.

Watch out. Every extra run a day multiplies cost, so schedules follow when people read the data. **8.13**

7 Watch the Capacity Metrics app

I install the Capacity Metrics app. Each week I read the Compute page. I look at background and interactive percent over time, the throttling tab, carryforward on the overages tab, and the system events for Overloaded and SurgeProtectionActive. The matrix by item and operation shows the top CU users. Surge protection sends email to the platform team when it activates.

Watch out. A daily average of 60 percent can hide an hour at 140 percent, so I read timepoints and the throttling tab.

8 Split, scale or pause only with evidence

If reports still see delays after tuning, the ETL workspaces move to a second capacity, for example F32, and reports stay on F64. Each capacity then smooths and throttles on its own. Resizing across the F256 and F512 boundary briefly interrupts the capacity and can cancel running jobs. Pausing adds all smoothed and carried-forward usage to the bill at once, so I only pause dev and test.

Watch out. Pausing to stop throttling clears it, but the carried-forward usage lands on the bill the same hour. **8.09**

SECURITY IN ONE BREATH

Only two named people are capacity admins. Pause and resume rights come from a custom Azure role with the read, write, suspend and resume actions on the capacity. An automation account uses it for dev and test.

HOW I'D KNOW IT WORKS

- Over two weeks, the Capacity Metrics app shows no interactive delay between 08:00 and 18:00.
- The 24-hour background percent stays under the surge rejection threshold on normal nights.
- The ETL run log shows completion before 05:00 every day.
- Spark Autoscale Billing cost in Azure Cost Analysis stays within the monthly budget.

IF THEY PUSH FURTHER

What if month-end ETL is three times the normal size?

I'd start it earlier and scale up the day before if the numbers call for it. Capacity overage, in preview, bills excess usage at three times the pay-as-you-go rate, against a 24-hour threshold the admin sets. I'd check whether it's switched on for this capacity first.

What if one analyst query slows everything at 10:00?

Warehouse queries are background work, so the workspace limit on the ad hoc workspace catches it. Query insights shows the query text and the login name for the last 30 days, so I can talk to the analyst the same morning. **4.07**

What if the notebooks queue at night even with Autoscale Billing?

They have hit the maximum CU. Raise it within the Azure quota, or spread the notebooks so fewer start at once. **8.01**

S.27 Plan release management for 40 Fabric items spread over 5 workspaces. Each workspace needs dev, test and prod, two teams release every week, and every stage must point at its own connections and lakehouses.

Short answer. I'd keep the code in one Git repository in Azure DevOps, with one folder per workspace. Each team builds in branched workspaces and merges to main through pull requests. An Azure DevOps pipeline then runs fabric-cicd, a Python library that deploys item definitions from Git. It deploys into test and prod as a service principal, with an approval before each stage. Per-stage values live in a variable library in each workspace and in fabric-cicd's parameter.yml file, so no dev ID ever reaches prod.

QUESTIONS TO ASK FIRST

- **Which Git provider does the company already use?** Fabric supports cloud Azure DevOps, GitHub and GitHub Enterprise. Learn documents service principal Git integration for Azure DevOps, and GitHub has a 50 MB limit per commit.
- **Which item types are in the 40?** I check each against the Git supported items list. Reports and semantic models are still in preview there. Unsupported items are ignored and never committed.
- **Do both teams change items in the same workspace?** A workspace connects to one branch, so shared workspaces need clear folder ownership and required reviewers on those paths.
- **Which sources and lakehouses differ between dev, test and prod?** That list becomes the variables and parameter.yml entries. If nothing differs, the design gets much simpler.
- **Who approves a production release, and when may it run?** The approvers become an Azure DevOps group on the prod environment. The window becomes a schedule or a manual gate.

WHICH TOOL, AND WHY

- **fabric-cicd from Azure DevOps.** My choice. Git stays the single source of truth, parameter.yml rewrites IDs per stage, and Azure DevOps gives approvals, history and rollback by redeploying an older commit.
- **Fabric deployment pipelines.** Native and low code, with 2 to 10 stages and a compare view. Deployment rules cover only semantic models, notebooks' default lakehouse, dataflows Gen1, paginated reports and mirrored databases. I'd also need five pipelines here.

- **Git integration with a branch per stage.** Each stage workspace syncs from its own branch through the update from Git API. It works for Gitflow teams, but promotion becomes cherry-picked pull requests and nothing rewrites IDs.
- **Bulk Import Item Definitions API.** A Microsoft API, in beta, that creates or updates many items in dependency order. Good if you want to write your own tooling, but you rebuild what fabric-cicd already does.

AT A GLANCE



One repository and one release pipeline. A merged change moves from main to test, waits for an approver, then goes to prod with prod values.

HOW I'D BUILD IT

1 Lay out workspaces and the repository

I create 15 workspaces, with dev, test and prod for each of the 5 areas. The repository has one folder per area. Each dev workspace connects to its folder on main, because a Git connection can point at a folder. Test and prod stay disconnected from Git and only receive deployments. Each team owns named folders, and a branch policy on main requires a reviewer from the owning team.

Watch out. If dev and prod share a workspace, a developer's test write lands in production tables. [9.06](#)

2 Give each team its own branches

Developers use Branch out to another workspace from the dev workspace. It creates a feature branch from main and a private workspace synced to that branch. They commit there and open a pull request to main. Each team works on short-lived branches and merges before its weekly cut. Nobody edits the shared dev workspace by hand.

Watch out. A workspace can connect to only one branch, so two people switching branches in one workspace overwrite each other's items.

3 Check what Git can carry

I list the 40 items against the supported items page. Pipelines, notebooks, lakehouses, warehouses, Dataflow Gen2, Copy jobs, environments and variable libraries are supported. Reports and semantic models are in preview. For a Lakehouse, Git holds only metadata and shortcut definitions, so tables and files never move with a release. Items Git does not support are ignored, so I keep a short list of them with a manual step for each.

Watch out. The Teams activity goes inactive under CI/CD. A Teams or Outlook activity on user authentication stays inactive in each higher stage until someone creates a new user connection there.

4 Parameterise connections and lakehouse references

Each workspace gets a variable library, a Fabric item that holds variables with one value set per stage. Pipelines read connection IDs and lakehouse IDs from it, and notebooks read it through NotebookUtils. Each stage workspace sets its own active value set. The item reference and connection reference variable types are in preview. Everything the library cannot reach goes in parameter.yml. Its find_replace entries map dev GUIDs to test and prod values. Dynamic values such as \$items.Lakehouse.<name>.\$id resolve to the deployed item's ID. I turn on Lakehouse auto-binding in Git in every notebook, because it is off by default.

Watch out. The semantic model refresh activity and a Direct Lake model's connection string keep workspace-specific IDs, so they point at dev until parameter.yml replaces them. [9.05](#)

5 Build the release pipeline

An Azure DevOps service connection signs in as a service principal. The tenant setting Service principals can call Fabric public APIs is on, and the service principal is Member on each test and prod workspace. The pipeline has a test stage and a prod stage, each linked to an Azure DevOps environment. The prod environment has an approval check for the release managers group and a branch control check that allows only main. Each stage runs fabric-cicd per workspace folder with the stage name, and unpublishes items removed from Git.

Watch out. Connections never auto-bind across workspaces, so a pipeline copied to prod keeps its dev connection unless a variable points it elsewhere. [9.08](#)

6 Run two weekly release trains

Team A releases on Tuesday and Team B on Thursday, for example. Each release deploys only that team's folders and tags the commit with a release number. A shared run log table records release number, workspace, items deployed, approver and time. Rolling back means deploying the previous tag through the same pipeline.

Watch out. A reference from one team's workspace to another's, such as a shortcut to the other team's Gold, uses an object ID. It never rebinds, so it needs its own variable. [9.08](#)

7 Finish each deployment

A deployment copies definitions only, with no data and no refresh. The last stage runs the pipelines that load new tables and refreshes the semantic models. Then a small smoke-test notebook checks row counts in Gold. Failures post to the release channel and fail the stage.

Watch out. A model deployed without a refresh shows old numbers, and the release still looks green. [9.04](#)

8 Lock down production

People get Viewer on test and prod workspaces. Only the release service principal and a small break-glass group hold Contributor or above. Changes reach prod through a pull request and an approval, and every change has a commit behind it.

Watch out. Contributor can create, change and delete every item in a workspace, so one wrong click in prod bypasses the whole process. [7.09](#)

SECURITY IN ONE BREATH

The service principal secret lives in Azure Key Vault and reaches the pipeline through an Azure DevOps variable group linked to that vault. Each area can use its own service principal, so one leaked secret opens one area. Branch policies stop direct pushes to main.

HOW I'D KNOW IT WORKS

- A one-line change goes from pull request to prod with one approval and no manual step.
- A search of prod item definitions finds no dev workspace ID, lakehouse ID or connection ID.
- Every prod workspace shows the prod value set as active in its variable library.
- Redeploying the previous tag to test restores the previous behaviour.

IF THEY PUSH FURTHER**What if the teams prefer Fabric deployment pipelines?**

It works, with one pipeline per area. Notebooks get default lakehouse rules and models get data source or parameter rules. Pipelines still need variable libraries. Direct Lake models need a rule because they are not autobound. Approvals come from calling the deployment pipelines API inside an Azure DevOps stage. **9.02**

What if a hotfix is needed on a Wednesday?

Branch from the last prod tag, fix, and open a pull request. The same pipeline deploys it with the same approval. Merge the fix back to main so the next train keeps it.

What if the company uses GitHub instead?

The design stays. GitHub Actions runs fabric-cicd with the same parameter.yml, and GitHub environments hold the approvals. Commits from Fabric are capped at 50 MB, so large changes go in several commits.

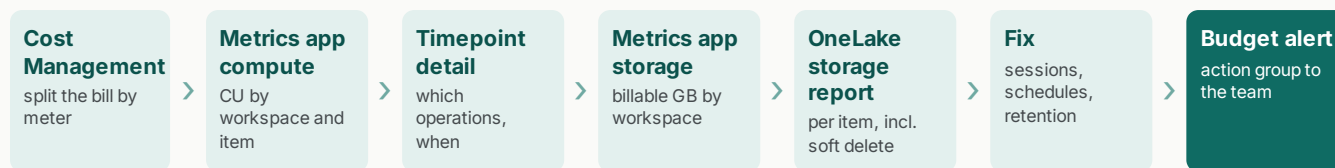
S.28 Find why the Fabric bill rose 60 percent in one quarter, and fix it. Finance wants to know which meters, workspaces and items drove the rise, and how to stop it happening again.

Short answer. An F SKU capacity charges a fixed hourly rate, so a bill that rises needs a meter that moved. I split the bill by meter in Azure Cost Management first. That shows whether a larger SKU, on-demand billing for Spark, capacity overage or OneLake storage moved. Then the Capacity Metrics app shows which items used the compute, and the OneLake storage report shows which items hold the storage. The usual fixes are shorter Spark sessions, high concurrency, fewer refreshes, no Direct Lake fallback, sane retention, and a budget alert.

QUESTIONS TO ASK FIRST

- **Did anyone scale the capacity or add a capacity this quarter?** A scale-up bills the new size at pay-as-you-go hourly rates, and it often stays after the reason is gone.
- **Is on-demand billing for Spark or capacity overage turned on?** Both bill on top of the SKU. Overage bills excess usage at three times the pay-as-you-go rate.
- **Is the capacity reserved or pay-as-you-go?** Scaling below a reservation does not change the bill. Capacity above it bills at pay-as-you-go rates.
- **What new teams, workspaces or projects arrived this quarter?** New work often explains most of it. Then I check whether that cost was planned.
- **How long must deleted data stay recoverable?** That sets VACUUM, item recovery and warehouse retention, which all hold billable storage.

AT A GLANCE



I follow the money from the invoice meters down to the items that used compute or storage. Then I fix the causes and put an alert on the budget.

HOW I'D BUILD IT

1 Split the bill by meter

In Azure Cost Management I group the Fabric capacity's cost by meter and compare the three months. Compute Pool is the base capacity. Autoscale for Spark, Capacity Overage and the OneLake Storage meters sit beside it. If Compute Pool rose, the SKU changed, and I find out who scaled it, when and why. If a side meter rose, I know which workload to chase.

Watch out. A capacity scaled up to stop throttling one busy week often stays at the larger size for the rest of the quarter. [8.10](#)

2 Find the items behind the compute

The Capacity Metrics app compute page shows the last 14 days. I sort the matrix by item and operation on CU, then filter by workspace. The top ten items usually hold most of the use. I drill into the timepoint pages to see which operations run at which hours. Every week I export the matrix to a Lakehouse table, so I can compare a whole quarter later.

Watch out. The compute page covers only 14 days, so last quarter's peak is gone unless someone exported it.

3 Rein in Spark

Interactive sessions bill while they are active, and the default session expiry is 20 minutes. I shorten the Spark session timeout in the workspace's Jobs settings for teams that leave notebooks open. I turn on high concurrency, where later notebooks share the first one's session and only the first is billed. I check custom pools for large node counts nobody needs. If on-demand billing for Spark is on, I set its maximum CU limit to a figure finance agrees.

Watch out. A pipeline that runs 30 notebooks in standard mode starts 30 sessions, one per activity.

4 Match schedules to readers

I list every pipeline, dataflow and semantic model refresh schedule with its frequency. Then I ask who reads each output and when. A pipeline every 15 minutes runs 96 times a day for a report opened twice. I move schedules to the times people read the data, and I delete schedules left over from old versions.

Watch out. An old schedule left enabled beside the new one doubles the runs and the cost. [8.13](#)

5 Stop Direct Lake fallback

Direct Lake on SQL falls back to DirectQuery through the SQL analytics endpoint for views, SQL endpoint row-level security or tables past the SKU guardrails. Those queries are slower and add SQL work to the capacity. Performance Analyzer shows which visuals fall back. I replace views with Delta tables or materialized lake views, and move RLS into the model or OneLake security.

Watch out. A model that falls back still says Direct Lake, and nothing reports an error. [6.07](#)

6 Trim OneLake storage

OneLake storage bills per GB and does not use CUs. The Metrics app storage page shows 30 days by workspace. The OneLake storage report in workspace settings shows each item, including hidden system data and soft-deleted data. Soft-deleted files stay billable for 7 days. Deleted items stay billable for the item recovery period of 3 to 90 days, and warehouses keep 30 days of history by default. I delete old rows first, then run VACUUM with at least the 7-day default retention.

Watch out. VACUUM removes only files that no current version needs, so a table that keeps every row never shrinks. [8.11](#)

7 Look at overage and throttling

The Overages tab in the Metrics app shows carryforward and billed overage. Capacity overage is on by default for new capacities and bills excess usage at three times the pay-as-you-go rate, up to a rolling 24-hour threshold. That threshold is not a hard cap. If overage fires daily, the capacity is too small or a job is badly timed, and I fix that first.

Watch out. Pausing a capacity to save money bills all the smoothed and carried-forward usage at once. [8.09](#)

8 Keep it from coming back

I set an Azure budget on the capacity's resource group with forecast alerts at 80 and 100 percent. The budget calls an action group that emails the platform team's group. A monthly review compares meters and the top items against last month. Each workspace has a named owner, so a new cost has someone to ask.

Watch out. A budget only alerts; it never stops the capacity.

SECURITY IN ONE BREATH

Capacity admins and the finance contact get read access to the Capacity Metrics app. Only capacity admins can change SKU, overage and on-demand Spark settings, and those changes go through a ticket.

HOW I'D KNOW IT WORKS

- Cost Management by meter and the Metrics app agree for the same dates, within a few percent.
- Every item in the top ten by CU has an owner and a written reason for its schedule.
- Billable storage stops rising once the next 7-day soft delete window passes.
- The budget's forecast alert fires in a test with a low threshold.

A WORKED EXAMPLE

Situation. For example, a monthly Fabric bill of \$12,000 grew to \$19,200 over a quarter, a 60 percent rise. The team had also noticed some throttling on Monday mornings.

Task. Find which meters and items caused the extra \$7,200 a month and bring the bill back near the old level without breaking any report.

Action. The meter split showed, as an illustration, \$3,600 from Autoscale for Spark, \$2,400 from Capacity Overage and \$1,200 from OneLake Storage. A data science team had turned on on-demand Spark billing and left notebooks open overnight. So I set a 10-minute session timeout, turned on high concurrency and set a maximum CU limit. Overage came from a refresh every 15 minutes, which moved to hourly during business hours. Storage came from a Lakehouse keeping every daily snapshot, so I deleted snapshots older than 90 days and ran VACUUM.

Result. In this example the next month's bill was \$13,100. The Monday throttling stopped, and a budget alert now reaches the team at 80 percent of forecast.

IF THEY PUSH FURTHER

What if each team must pay for its own use?

Use the weekly Metrics app export to split CU by workspace for chargeback. For hard separation, give a team its own capacity. **8.04**

What if finance wants a hard spending cap?

The F SKU rate is already fixed. Turn capacity overage off or set a low threshold, and set a maximum CU limit on on-demand Spark. Storage has no cap, so retention policy is the control.

What if the rise came from one runaway workspace?

Workspace-level surge protection, in preview, caps how much compute one workspace can use in 24 hours. Fix the job first, then use the cap as a guard. **8.05**

S.29 Plan recovery for a bad release that goes out on Monday at 09:00 and corrupts three Gold tables. Design it before it happens, so the team restores correct data and working code quickly and can prove the reports are right.

Short answer. I'd make sure history exists before any release. That means VACUUM retention of at least 7 days and Delta log retention of 30 days. A release step also records each Gold table's current Delta version. When the bad release lands, I pause schedules, revert the commit in Git and redeploy, then RESTORE the three tables to their pre-release versions. Loads that ran after 09:00 rerun from Silver, the semantic model is refreshed, and checks compare Gold with Silver. For a Warehouse, restore points and table clones play the part of RESTORE.

QUESTIONS TO ASK FIRST

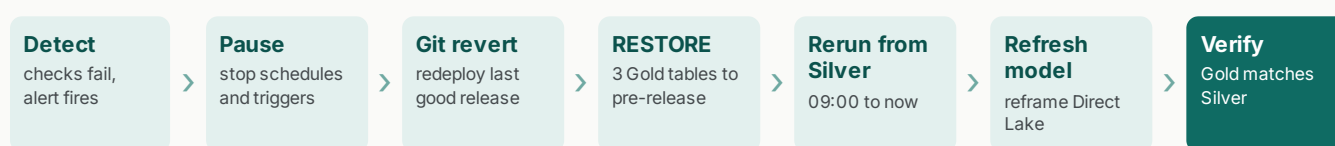
- **Are the Gold tables in a Lakehouse or a Warehouse?** Lakehouse tables use Delta RESTORE per table. A Warehouse uses a clone of one table at a past time or a restore in place of the whole warehouse.
- **How long could corruption go unnoticed?** Retention must be longer than that, or the good version is already vacuumed.
- **What runs after the release on a Monday?** Hourly loads, exports and shortcuts in other workspaces all carry bad rows and need replay or a warning.
- **Is Silver intact and can it be replayed for a time window?** If yes, rerunning from Silver fills the gap after the restore. If Silver was also touched, recovery starts one layer lower.

- **What recovery time and data loss has the business accepted?** That decides whether restore points every 8 hours are enough or I need my own checkpoint just before each release.

WHICH TOOL, AND WHY

- **Delta RESTORE on Lakehouse tables.** My choice for Lakehouse Gold. It makes an earlier version current in one command and records itself as a new version, so it can be undone.
- **Rebuild Gold from Silver.** Always possible if Silver is good, and needed for the hours after 09:00. For whole tables it is slower and costs more capacity than RESTORE.
- **Warehouse restore in place.** Rolls the whole warehouse back to a restore point. Right when Gold lives in a Warehouse and many tables broke; too wide for three tables among many.
- **OneLake soft delete.** Keeps deleted files for 7 days. A last resort for files that were deleted, with no notion of table versions.

AT A GLANCE



Stop the damage, put the code back, put the data back, fill the gap, then show users correct numbers.

HOW I'D BUILD IT

1 Set retention before the first release

VACUUM keeps unreferenced files for 7 days by default, and Fabric maintenance requests under 7 days fail unless someone turns the safety check off. I keep Gold at 14 days, because corruption can take a week to notice. Delta log retention, `delta.logRetentionDuration`, defaults to 30 days, and both limits must cover the version I want back. For a Warehouse, history defaults to 30 days, and configurable retention of 1 to 120 days is in preview.

Watch out. A maintenance notebook with the retention check turned off and a retention of 0 hours wipes every older version.

2 Checkpoint at every release

The first step of the release pipeline runs a notebook that reads `DESCRIBE HISTORY` for each Gold table. It writes the release number, table name, current version and timestamp into a release log table. For a Warehouse, the same step creates a user-defined restore point, which Admin, Member or Contributor can create. The Git commit gets the release tag.

Watch out. System restore points come every 8 hours, so the latest one can miss the morning's loads. [9.10](#)

3 Detect fast and stop the damage

After each load, a check notebook compares Gold row counts and totals with Silver and writes to the run log. A failure sends an alert to the on-call group. The first action in the runbook is to disable the pipeline schedules, so the bad code does not run again at 10:00. I keep automatic updates off on the Direct Lake model, so users stay on the last framed version.

Watch out. With automatic updates on, reports reframe to each bad commit as soon as it lands. [6.05](#)

4 Revert the code

I revert the release commit in Git, which creates a new commit, and push it through the same release pipeline. That puts the previous notebooks and pipelines back in prod. Git holds Lakehouse metadata only, so this changes code and leaves the tables alone.

Watch out. Restoring data before the code is reverted lets the next scheduled run corrupt it again.

5 Restore the three tables

For each table I read the pre-release version from the release log and check it with DESCRIBE HISTORY. Then I run `RESTORE TABLE gold.<name> TO VERSION AS OF <version>`. Each RESTORE adds a new version, so the corrupted state is still there if I need to inspect it. For a Warehouse table, I create a clone AT the pre-release time and swap the names with `sp_rename`. If most tables broke, I restore the whole warehouse in place instead. Only a workspace Admin can restore a warehouse.

Watch out. Running VACUUM with a short retention before the restore can delete the files the good version needs, and then RESTORE fails.

6 Replay the gap from Silver

Loads ran at 10:00 and 11:00 with the bad code, and they also moved the watermarks in the control table. I set those watermarks back to the release checkpoint and rerun Silver to Gold for that window with the reverted code. The Gold writes replace by key and date, so a rerun cannot double any row.

Watch out. An append load rerun after a partial restore writes the same rows twice. [2.26](#)

7 Reframe the model and verify

A semantic model refresh reframes the Direct Lake model on the restored versions. The check notebook runs again and compares Gold with Silver for the affected days. I post the result in the incident channel with the row counts. Then I turn the schedules back on.

Watch out. The refresh can succeed before the replay finishes, and the report then misses the late hours. [6.04](#)

8 Write the runbook and practice it

The runbook names the on-call owner and lists each step with the exact notebook, parameters and approver. It includes how to recover deleted files from OneLake soft delete through Azure Storage Explorer within 7 days. We practice it in test once a quarter by breaking a table on purpose. The drill's duration becomes the recovery time we promise.

Watch out. A runbook nobody has run fails on a missing permission during the real incident.

SECURITY IN ONE BREATH

Only the release service principal writes to prod Gold. The recovery notebooks run through a pipeline under that identity, so the on-call engineer needs permission to start the pipeline and read results. Warehouse restore in place is limited to workspace Admins, so one Admin is always on the rota.

HOW I'D KNOW IT WORKS

- Every release has one row per Gold table in the release log with its version.
- The quarterly drill restores a table and replays a window within the agreed time.

- After recovery, Gold totals match Silver for every affected day.
- DESCRIBE HISTORY shows RESTORE as the newest operation on each restored table.

A WORKED EXAMPLE

Situation. For example, a release at 09:00 on a Monday changed a join in the Gold notebook. It duplicated customers in gold.dim_customer and inflated gold.fact_sales and gold.sales_daily. Hourly loads ran at 10:00 and 11:00.

Task. Bring the three tables back to correct numbers, remove the bad code, and keep reports honest while we worked.

Action. The Gold versus Silver check failed at 10:05 and alerted the on-call group. By 10:20 the schedules were disabled and the Git revert was deployed. I restored the three tables to versions 412, 88 and 301 from the release log, as illustrative numbers. Then I reset the watermarks to 09:00 and replayed the two hourly loads from Silver.

Result. In this example Gold matched Silver again at 11:40, and the model refresh at 11:45 showed correct sales. Users saw Friday's framed data during the incident instead of wrong numbers.

IF THEY PUSH FURTHER

What if the corruption is found 20 days later?

With 14-day VACUUM retention the files are gone, so RESTORE fails. Rebuild the three tables from Silver for the full period. If late discovery is likely, raise retention and accept the storage cost.

What if the release also damaged Silver?

Restore Silver first, then rebuild Gold from it. Bronze kept as append-only raw data is what makes this possible. [3.10](#)

What if Gold lives in a Warehouse?

Clone each table AT the pre-release time with zero-copy cloning, then swap names with sp_rename. Restore in place only when most of the warehouse is wrong, because it rolls back every table. [9.10](#)

S.30 Plan the handover of a Fabric platform whose only builder leaves next month. Many items, connections, schedules and alerts run under that engineer's personal account, and nobody else knows how the platform runs day to day.

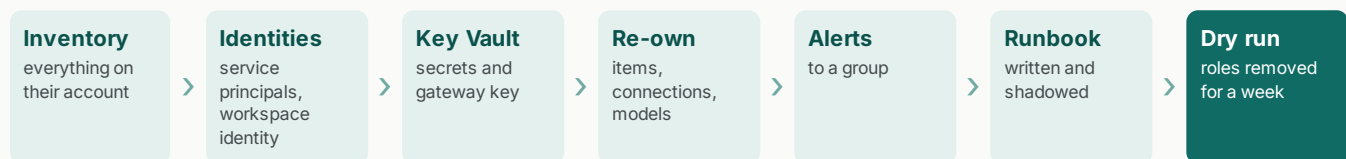
Short answer. I'd spend the first week on an inventory of everything tied to that account. That covers items, connections, schedules, the gateway, Key Vault access and alert recipients. Then I move production to service principals and workspace identities and put secrets behind Azure Key Vault references. I take over or re-own every item and send alerts to a group. The engineer writes the runbook while a colleague shadows them. Before the last day, I remove their workspace roles for a week and watch what breaks.

QUESTIONS TO ASK FIRST

- **May we create service principals, and is the tenant setting for Fabric APIs on?** Service principals can call Fabric public APIs must be on. Learn pages disagree on its default, so I check it. Without it, the notebook activity cannot run as a workspace identity.

- **Which sources does the platform read, and how does each authenticate?** Workspace identity works for a set list of sources. The rest need a service principal or a Key Vault reference.
- **Is there an on-premises data gateway, and where is its recovery key?** Without the recovery key nobody can migrate or take over the gateway later.
- **Who owns the platform from next month?** A named lead and a security group. Every owner field I change points at one of them.
- **Is the work in Git already?** If not, connecting the workspaces to Git is the first task, because it is the only full history we will have.

AT A GLANCE



In four weeks I find everything, move it to identities that do not leave, write it down, then prove it runs without the person.

HOW I'D BUILD IT

1 Inventory what hangs on the account

A Fabric admin calls the admin API that lists every item a user can access, which is in preview. From Manage connections and gateways I list the connections they own, with the authentication type of each. I record every pipeline's Last Modified By, every schedule, every semantic model owner, gateway admins, Key Vault access policies and app registrations they own. Everything goes into a handover table in a Lakehouse, with columns for item, dependency, new owner and status.

Watch out. Schedules expire when their owner does not sign in to Fabric for 90 days, and the pipelines just stop. [2.11](#)

2 Create identities that do not leave

I create one service principal per workload area, so one leaked secret opens one area. Each production workspace gets a workspace identity, a service principal that Fabric manages for the workspace. A tenant admin turns on Service principals can call Fabric public APIs. The identities get Contributor on their own workspaces and read access on their sources.

Watch out. A single service principal for everything turns one leaked secret into access to every workspace. [7.04](#)

3 Move the connections

For each OAuth connection under their account, I create a new shared connection with workspace identity or a service principal. Sources that need a password or key use an Azure Key Vault reference, created in Manage connections and gateways. Fabric then stores only a pointer to the secret. The reference reaches the vault with its creator's OAuth sign-in, so someone who stays creates it. I switch each pipeline, dataflow and model to the new connection and make the platform group an Owner of it. Connections I cannot recreate in time go through the admin connection role API, in preview, to add a new owner.

Watch out. An OAuth connection keeps the engineer's refresh token, and it dies when the account is disabled. [7.03](#)

4 Take over items

A Contributor or above can take over a Fabric item from its settings, and child items move with it. For pipelines, I make a small edit after the takeover so Last Modified By changes too, because pipelines run as that user. Better still, the release service principal updates each pipeline through the REST API and becomes Last Modified By. Notebook activities use a workspace identity connection. A warehouse can be given to a service principal with PowerShell. Mirrored databases cannot change owner, so I recreate them.

Watch out. Taking over a pipeline leaves the notebooks it calls owned by the old account. [7.01](#)

5 Re-own the semantic models

Every Direct Lake refresh checks the model owner's access to the data, whoever starts it. I take over each model and bind it to a fixed-identity cloud connection that uses a service principal or workspace identity. Then I refresh each model once and confirm the report still shows today's data.

Watch out. When the owner's account is disabled, every Direct Lake refresh fails with access denied, even for workspace admins. [6.09](#)

6 Move secrets into Key Vault

Every secret the engineer knows gets rotated and stored in Azure Key Vault, including the service principal secrets and the gateway recovery key. Notebooks read secrets with `notebookutils.credentials.getSecret`, which authenticates as the identity running the notebook, so that identity needs read access to the vault. I add two more gateway admins and write down where the gateway runs.

Watch out. Secrets pasted into notebook cells are in Git history, so rotation has to include them.

7 Send alerts to a group

I add the platform group to the Failure notifications of each scheduled item. Legacy Outlook activities move to the Outlook 365 Email activity with workspace identity or service principal authentication, and they send to a shared mailbox. From 30 October 2026, new legacy Teams or Outlook activities can't be added to pipelines. The Microsoft Teams activity does not support workspace identity or service principal yet, so Teams alerts come from Activator. Workspace monitoring, in preview, writes job logs to an Eventhouse. One Activator rule on a KQL query alerts on any failed pipeline in the workspace. Surge protection notifications go to the same group.

Watch out. An Outlook activity on user authentication sends mail from that user's account, so it stops when they leave.

8 Document, shadow, then run a dry week

The engineer writes a runbook for daily checks, reruns, backfills, capacity and releases, and a colleague follows it while they watch. Two weeks before the last day, I remove their workspace roles and connection access but keep the account active. Anything that breaks that week is still fixable with their help. On the last day, an Entra access review on the platform groups confirms who keeps access.

Watch out. Leaving an ex-employee's Admin role on production gives a disabled account a place to come back to. [7.09](#)

SECURITY IN ONE BREATH

No production item, connection or schedule stays on a personal account. Service principal secrets live in Key Vault with an expiry and a rotation date in the runbook. People reach workspaces through Entra groups, and access reviews on those groups need Microsoft Entra ID Governance or Entra Suite licensing.

HOW I'D KNOW IT WORKS

- The handover table shows a new owner and a Done status for every row.
- The admin access list for the engineer returns no production items.
- During the dry week, every scheduled run succeeds without the engineer's accounts.
- A forced test failure sends an alert to the group mailbox and the Teams channel.

A WORKED EXAMPLE

Situation. For example, one engineer built 5 workspaces with about 60 items. The inventory found 14 connections, 11 of them OAuth on their account, 23 schedules, 6 semantic models they owned and one on-premises data gateway.

Task. Move everything off the personal account within four weeks and show the platform runs without them.

Action. In week 1 we built the handover table and created three service principals and five workspace identities. In week 2 we recreated the 11 connections, 7 with workspace identity and 4 with Key Vault references. In week 3 we took over the items, re-owned the models and moved alerts to a group. In week 4 we removed their roles for the dry week.

Result. In this example the dry week surfaced two failures, a notebook still owned by the engineer and a forgotten Outlook activity. Both were fixed the same day, and the month after they left had no failed runs caused by the handover.

IF THEY PUSH FURTHER**What if the engineer already left and the account is disabled?**

Take over the items and repair connections, starting with the ones behind the most important reports. Use the admin role API to add an owner to orphaned connections. Recreate mirrored databases, and reinstall the gateway if nobody has its recovery key. [7.03](#)

What if a source supports neither workspace identity nor a service principal?

Create a dedicated service account for that source and store its password in Key Vault. If the connector supports Key Vault references, the connection uses one, so rotation happens in one place. [7.02](#)

What if the platform lead leaves next year?

Nothing should depend on them. A quarterly access review and a check of the handover table keep it that way.

From five books to one.

The five Fabric Engineering Patterns books held 250 scenarios, 50 each. The Fabric Practitioner's Handbook 4.0 keeps 103 of them, rewrites each one, adds 20 new patterns, and regroups all 123 by layer into nine parts.

FIVE BOOKS · 250 SCENARIOS

FABRIC PRACTITIONER'S HANDBOOK 4.0 · 123 PATTERNS

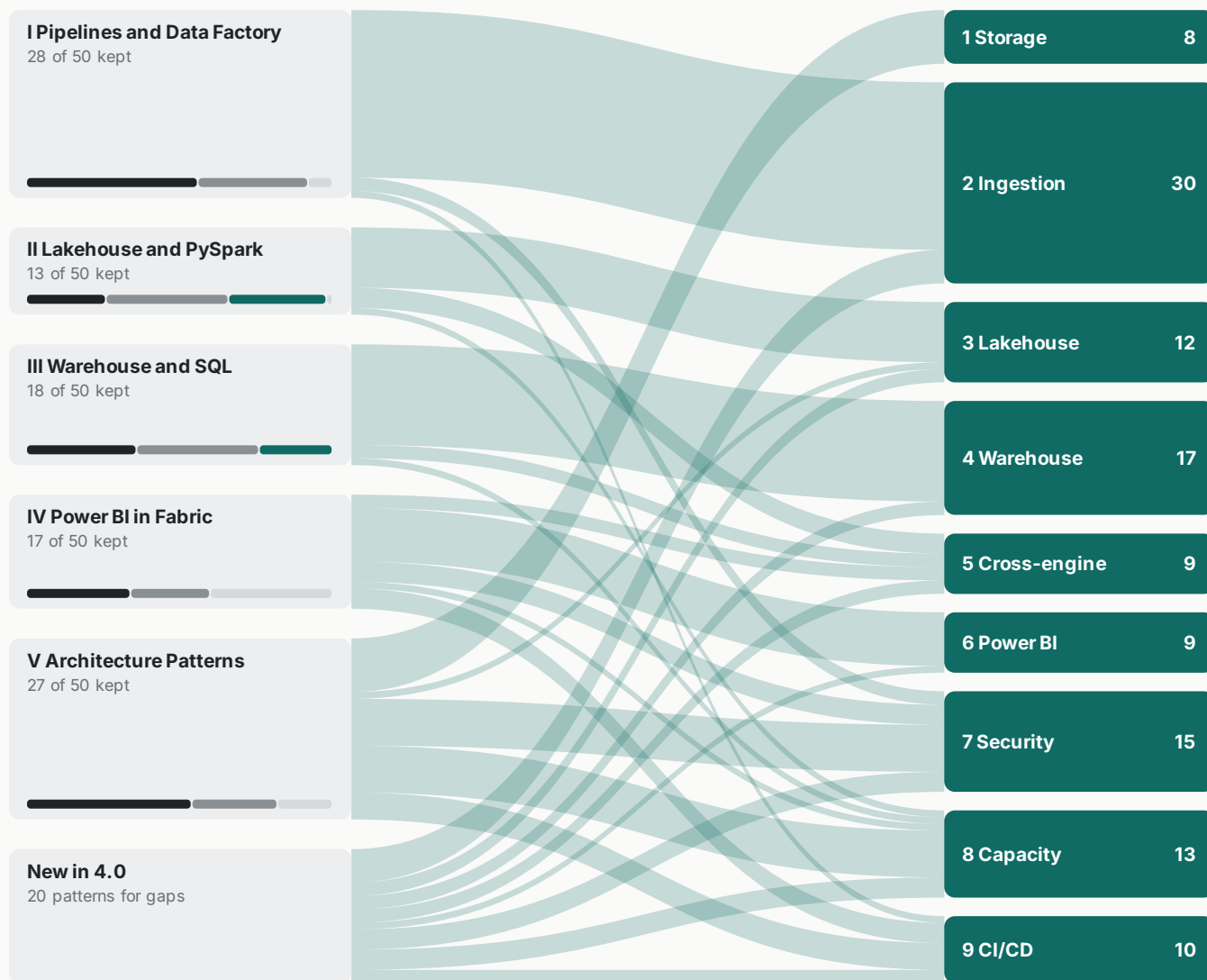


FIG. Ribbons show where each book's kept scenarios landed. The bar under each book splits its 50 scenarios.

Kept and rewritten
 Merged into another pattern
 Moved to the PySpark Handbook 4.0 or the SQL Handbook
 Cut: not Fabric-specific, out of scope, or wrong

Of the 250: 103 kept, 85 merged, 28 moved to the Handbooks and 34 cut. Merged scenarios live on inside the pattern that absorbed them.